

O'REILLY®



RAG with Python Cookbook

Practical Recipes from
Data Preprocessing to LLM Agents

Dominik Polzer

RAG with Python Cookbook

As businesses race to unlock the full potential of large language models (LLMs), a critical challenge has emerged: How do you connect these tools to real-time, external data to solve real-world problems? Retrieval-augmented generation (RAG) is the answer. By combining LLMs with information retrieval, RAG empowers you to build everything from intelligent chatbots to autonomous, task-solving agents.

Packed with over 70 practical recipes, this go-to guide tackles a wide range of GenAI applications through structured hands-on learning. Author Dominik Polzer provides the tools you need to design, implement, and optimize RAG systems for your unique use cases. Whether you're working with simple data retrieval or designing cutting-edge autonomous agents, this cookbook will help you stay ahead of the curve.

- Learn core RAG components including embedding, retrieval, and generation techniques
- Understand advanced workflows like semantic-aware chunking and multiquery prompting
- Build custom solutions such as chatbots and autonomous agents for specific data challenges
- Continuously evaluate and optimize systems for accuracy, relevance, and performance

Dominik Polzer is an experienced machine learning engineer who has worked at leading companies like Siemens. Specializing in forecasting, anomaly detection, and generative AI, he leads initiatives leveraging foundation models and RAG systems to automate business processes. He also shares ML insights on his popular Medium blog.

DATA

US \$79.99 CAN \$99.99

ISBN: 979-8-341-60056-0



5 7 9 9

O'REILLY®

RAG with Python Cookbook

*Practical Recipes from Data
Preprocessing to LLM Agents*

Dominik Polzer

O'REILLY®

RAG with Python Cookbook

by Dominik Polzer

Copyright © 2026 Dominik Polzer. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Nicole Butterfield

Development Editor: Jeff Bleiel

Production Editor: Jonathon Owen

Copyeditor: Sharon Wilkey

Proofreader: Dave Awl

Indexer: BIM Creatives, LLC

Cover Designer: Susan Brown

Cover Illustrator: José Marzan Jr.

Interior Designer: David Futato

Illustrator: Kate Dullea

April 2026: First Edition

Revision History for the First Edition

2026-04-27: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9798341600560> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *RAG with Python Cookbook*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-60056-0

[LSI]

Table of Contents

Preface.....	vii
1. Getting Started with RAG.....	1
1.1 Identifying High-Value RAG Use Cases for Your Organization	3
1.2 Choosing Your IDE and Coding Agent Setup	6
1.3 Getting Started with Jupyter Notebooks in VS Code	8
1.4 Storing Secrets and API Keys with .env Files	10
1.5 Building Your First RAG App	13
1.6 Choosing the Frameworks and Libraries for Your RAG Applications	18
1.7 Running the Code Examples in the Book Repository	21
2. Foundation Models.....	23
2.1 Defining a Suitable Prompt Template	24
2.2 Selecting the Right Language Model for Your Task	26
2.3 Generating Content with the OpenAI API	29
2.4 Generating Content with Google's Gemini Models	33
2.5 Generating Content with the Anthropic API	35
2.6 Running Open Source Models Locally with Ollama	37
2.7 Creating Structured Outputs with the OpenAI SDK and Pydantic	41
3. Loading Data.....	45
3.1 Loading Word Files in Python	47
3.2 Loading PDF Files	50
3.3 Loading and Handling Tabular Data from Excel and CSV Files	52
3.4 Loading Structured Data from a PostgreSQL Database	57
3.5 Loading Audio Files via Speech-to-Text Models	59
3.6 Extracting Text from Images and PDFs via Tesseract OCR	61
3.7 Extracting Text from Images via Multimodal Models	65

3.8 Generating Text Description for Images via Multimodal Models	68
3.9 Generating Text Summaries for Embedded Tables via Multimodal Models	70
3.10 Parsing PDFs with Multimodal Content	73
3.11 Loading Videos via Speech-to-Text and Multimodal Models	77
4. Data Preparation.....	83
4.1 Adding Metadata to Enable Metadata Filtering	84
4.2 Enhancing Data Quality by Replacing Abbreviations and Technical Terms	90
4.3 Improving Search Accuracy by Creating Hypothetical Questions for Text Chunks	95
4.4 Splitting Documents via Character Splitting	99
4.5 Splitting Documents with Recursive Text Splitters	101
4.6 Chunking Documents with Document-Aware Splitting	105
4.7 Splitting Text with Semantic-Aware Chunkers	108
4.8 Splitting Text with Agentic Chunkers	111
5. Embeddings.....	117
5.1 Mapping the Linguistic Meaning of Text Chunks to a Numerical Representation	118
5.2 Visualizing Semantic Relationships Between Text Chunks via Dimensionality Reduction Techniques	123
5.3 Calculating the Distance Between Embeddings	127
5.4 Choosing the Right Embedding Model	131
5.5 Generating Embeddings for Images and Text with CLIP	133
5.6 Performing Text Classification with Embeddings	137
5.7 Improving Search Results with a Hybrid Search Approach	141
6. Vector Databases and Similarity Searches.....	147
6.1 Choosing the Right Vector Database	148
6.2 Storing and Searching Embeddings with FAISS	151
6.3 Storing and Working with Embeddings in a Chroma Vector Database	155
6.4 Storing Embeddings in PostgreSQL with the pgvector Extension	159
6.5 Performing Similarity Search in PostgreSQL	164
6.6 Accelerating Vector Searches in PostgreSQL with Indexing Techniques	167
6.7 Combining Keyword and Similarity Search to Improve Retrieval Accuracy with PostgreSQL	173
7. Retrieval.....	177
7.1 Optimizing Query Results via Metadata Filtering in PostgreSQL	180
7.2 Enhancing Retrieval Accuracy with HyDE	184
7.3 Improving Search Results with Multiquery Retrieval	188
7.4 Addressing Complex Requests by Designing a Query Routing System	192

7.5 Enhancing Retrieved Documents by Designing an Auto-Merging Retriever	197
7.6 Retrieving More Complete Text Chunks with a Sentence Window Retriever	200
7.7 Improving Retrieval Relevancy with Reranking Methods	203
7.8 Decomposing Complex Queries into Multiple Subqueries	206
8. Agentic RAG.....	211
8.1 Designing a Custom Tool in Python	215
8.2 Using Workflow Patterns in Multiagent Systems	216
8.3 Choosing an Agentic Framework	221
8.4 Building an Agentic System via Function Calling	224
8.5 Accelerating Agents with asyncio	231
8.6 Building a Sales Negotiation Agent with OpenAI's Agents SDK and Chroma	236
8.7 Enriching Your Agent's Capabilities with MCP Tools	245
8.8 Building an Agentic System with LangGraph	250
9. Graph RAG.....	261
9.1 Creating Your First Neo4j Knowledge Graph and Feeding It with Text from Documents	263
9.2 Extending the Knowledge Graph with Structured Data	271
9.3 Building Your First Cypher Query	275
9.4 Enabling Semantic Search on a Neo4j Knowledge Graph	278
9.5 Optimizing the Knowledge Graph for RAG Systems	281
10. Evaluating RAG Systems.....	285
10.1 Choosing the Right Evaluation Metrics for RAG Systems	289
10.2 Evaluating RAG Systems by Humans	293
10.3 Creating Synthetic Data for Automated Testing	296
10.4 Evaluating the Retriever Step by Calculating Context Precision@k	301
10.5 Evaluating Faithfulness During Generation with LLM-as-a-Judge	308
10.6 Evaluating the Response Relevancy of Your RAG System	316
11. RAG Web Apps.....	323
11.1 Building Your First Streamlit App	324
11.2 Building a Chatbot App with Streamlit	326
11.3 Adding PDF Analyzer Functionality to Your Chatbot	336
11.4 Connecting Your RAG App to a SQL Database	341
11.5 Deploying Your Streamlit App with Docker and AWS	347
Index.....	351

Preface

In 2017, Google researchers published the paper “Attention Is All You Need” and introduced the Transformer architecture, a breakthrough that reshaped modern AI. Over the following years, large foundation models demonstrated what happens when these ideas are scaled. The models began to write coherent text, answer complex questions, and generate working code. For the first time, software systems could interact with language in ways that felt broadly useful in real applications, not just impressive in research demos.

Yet these models had an important limitation. They were powerful but isolated, prone to hallucinating facts, lacking access to up-to-date information, and unable to work with private company data. Retrieval-augmented generation (RAG) addresses these gaps by coupling language models with external knowledge sources. I see RAG and agentic RAG as a key step toward AI systems that emulate human problem-solving. They actively gather new information, interpret it in context, and continuously plan their next steps based on their findings. By connecting foundation models to external knowledge sources, RAG grounds model outputs in verifiable data and enables systems to reason over trusted information when handling complex tasks.

This book is about building production-ready RAG systems. Each recipe focuses on a concrete engineering challenge that appears when moving from prototype to dependable application and explains the trade-offs behind key design decisions. You’ll learn how to design pipelines, select retrieval strategies, evaluate system quality, and operate RAG systems at scale. The goal is to help you build solutions that are accurate, scalable, and maintainable—systems that perform reliably not only in demos but in real-world environments.

Who This Book Is For

This book is written for developers and data scientists who want to build practical generative AI applications. You should be comfortable with Python and familiar with APIs, data processing, and general software development. A deep machine learning

background is not required. The necessary concepts are introduced as they appear, but you should be ready to write, run, and debug code along the way.

What You'll Learn and How the Book Is Organized

The 11 chapters follow the natural progression of building a RAG system, from core concepts to production deployment:

- Chapters 1 and 2 cover foundations. You'll identify high-value use cases, set up your environment, work with prompts, and select appropriate foundation models.
- Chapters 3 and 4 focus on the data pipeline. You'll load data from documents, databases, images, audio, and video, then prepare it through cleaning, chunking, and metadata enrichment.
- Chapters 5 and 6 explain embeddings and storage. You'll learn how embeddings represent semantic meaning, choose embedding models, and select vector databases based on your requirements.
- Chapter 7 introduces advanced retrieval techniques such as metadata filtering, multiquery retrieval, HyDE, reranking, and query decomposition to improve accuracy.
- Chapters 8 and 9 explore intelligent systems. You'll build agentic workflows with tools and learn how knowledge graphs preserve relationships between entities.
- Chapters 10 and 11 address production readiness. You'll evaluate RAG systems with automated metrics and human judgment, generate synthetic test data, and deploy applications with Streamlit, Docker, and Amazon Web Services.

Each chapter contains practical recipes with working code that you can adapt and extend in your own projects.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/polzerdo55862/RAG-with-Python-Cookbook>.

If you have a technical question or a problem using the code examples, please send email to support@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you’re reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O’Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product’s documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*RAG with Python Cookbook* by Dominik Polzer (O’Reilly). Copyright 2026 O’Reilly Media, 979-8-341-60056-0.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning



For more than 40 years, **O'Reilly Media** has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
141 Stony Circle, Suite 195
Santa Rosa, CA 95401
800-889-8969 (in the United States or Canada)
707-827-7019 (international or local)
707-829-0104 (fax)
support@oreilly.com
<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata and any additional information. You can access this page at <https://oreil.ly/rag-with-python-cookbook>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

Writing this book has been one of the most challenging and rewarding projects I have taken on. Although I have enjoyed writing for years, shaping a complete manuscript taught me more than I expected. If this book helps you build better RAG systems or solve real problems, I would love to hear about it. Please feel free to reach out via [LinkedIn](#) or [GitHub](#). Your feedback and stories are incredibly motivating.

I am deeply grateful to the technical reviewers who dedicated their time and expertise to improving this book. Vishwesh Ravi Shrimali, Leonie Monigatti, Laura Uzcategui,

Prashanth Josyula, Louis-François Bouchard, Nicolas Bièvre, and Susan Shu Chang provided thoughtful feedback, caught errors, clarified explanations, and helped ensure that the examples work as intended. Any remaining mistakes are my own.

This project would not exist without the outstanding team at O'Reilly. I am especially indebted to Jeff Bleiel and Jill Leonard for their editorial guidance, and to Nicole Butterfield for initiating and supporting this project from the beginning. Thanks also to the production team for turning the manuscript into a polished book.

I owe special thanks to Mirko Einert, who has been both a manager and a mentor. His drive to challenge the status quo helped evolve RAG systems from prototypes into core systems that transform our procurement processes at Siemens Energy.

Finally, I am deeply grateful to my girlfriend, Luisa. Writing this book required many late nights and weekends. She gave me the space to focus, encouraged me when progress felt slow, and reminded me to rest when needed. Her support made this book possible. Thank you, Luisa.

Getting Started with RAG

Large language models (LLMs) have transformed how we approach complex cognitive tasks—from writing production code to analyzing financial reports to translating dozens of languages. Complementary foundation models now handle vision, speech synthesis and recognition, audio processing, image generation, and multimodal reasoning, all built on similar transformer architectures that can process and generate human-like content across multiple domains.

Despite these capabilities, these models have fundamental structural limitations. They don't have access to your private or confidential data unless you provide it. Their context windows limit how much information they can consider at a time, which makes long documents expensive or impossible to analyze in a single pass. And when these models lack the right information, they tend to hallucinate rather than admit uncertainty.

Retrieval augmented generation (RAG) addresses all three problems at once. It gives the model controlled access to external knowledge, lets it work with far more information than fits into a single prompt, and grounds its answers in retrieved evidence instead of guesswork. When a user asks a question, the system first retrieves relevant information from a knowledge source and then passes that context to the model to generate a response.

Figure 1-1 shows the simplest form of a RAG system. A retriever searches a knowledge source such as a vector store or database and returns relevant passages. Think of the vector store as a specialized search engine that indexes your specific documents and knowledge base, rather than the entire web like Google. A generator, typically an LLM, uses that retrieved material to produce a grounded answer.

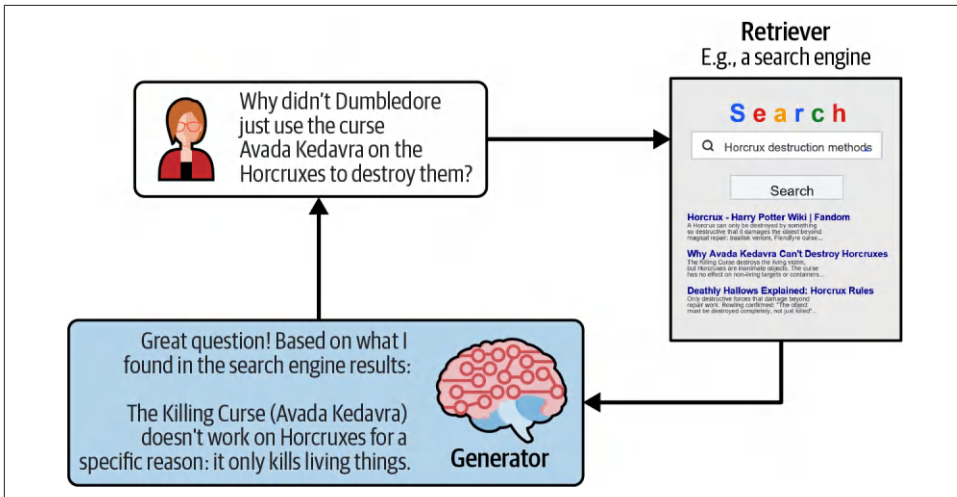


Figure 1-1. A RAG chatbot retrieves relevant information and generates contextual responses

RAG is most useful when knowledge is spread across many documents, when answers require synthesis rather than simple lookup, or when the data is too large or too diverse for a human to review. Modern systems often use multiple retrievers. One may search documents, another may query structured databases, and others may call external APIs. The model coordinates these tools to decide which information to use for each question.

Figure 1-2 shows exactly this approach—a RAG system with multiple API endpoints that can search a Notion database, search the web, perform calculations, and access both SQL and vector databases to search the knowledge base. The model decides which retriever to call based on the question and combines the retrieved information to generate an answer. This architecture allows the system to handle a wide variety of questions.

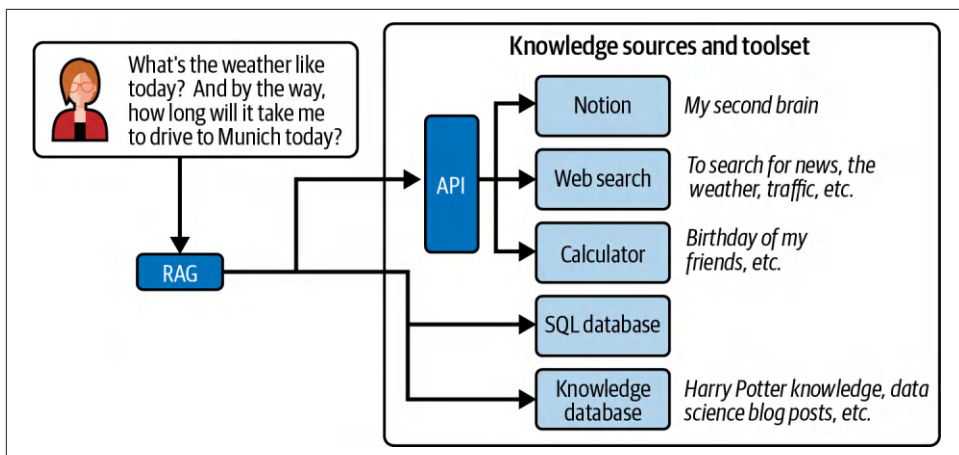


Figure 1-2. A RAG system with multiple retrievers answering questions about Harry Potter books

While many people associate RAG primarily with chatbots, chat interfaces aren't always the optimal solution. Chat works well for exploratory questioning and initial user interactions, but often the best approach combines multiple interfaces. A typical workflow might start with a conversational interface to understand user intent, then transition to a dashboard, form-based application, or automated report once the system understands the user's needs. Many RAG systems also run entirely in the background, delivering results through APIs, scheduled reports, or integration with existing business applications.

This chapter walks you through setting up your environment and building your first basic RAG application.

All code examples are available in the [book's GitHub repository](#).

1.1 Identifying High-Value RAG Use Cases for Your Organization

Problem

Your organization is exploring RAG and needs to identify use cases that deliver the most value.

Solution

Start by identifying use cases that solve concrete problems. Many RAG pilot projects fail because they're too complex, too expensive, or don't address real user pain.

Focus on two high-value areas where RAG delivers measurable impact:

Data extraction

Converting unstructured data (contracts, invoices, technical drawings, meeting notes) into structured, searchable information

Process automation

Handling repetitive tasks that require decision making and adaptation

Table 1-1 provides some guidance on how to evaluate different scenarios in terms of RAG fit and potential value. A value of 1 indicates a RAG fit of no or little value, and 5 indicates the best fit.

Table 1-1. Evaluating RAG use cases

RAG fit (1–5)	Use case	Reasoning
1	“I want to chat with my quarterly report.”	Single document, low volume. Reading it directly or analyzing with general apps (e.g., ChatGPT or Claude) can do this well.
2	“Summarize this one document for me.”	General models (Claude, ChatGPT) already do this well. Custom RAG adds little value.
2	“Automate tasks that require high-stakes decisions.”	Possible, but too risky without strong controls (human in the loop, auditing, fail-safes). LLMs still make mistakes.
4	“Meeting recordings pile up, but insights never make it into our knowledge base.”	Converts inaccessible data (audio, videos, long unstructured notes) into searchable form. Strong ongoing value, but quality depends on transcription.
4	“You must check technical drawings against specification documents.”	Multimodal comparison that humans find tedious. Automation saves time, but requires strong evaluation and exception handling.
5	“We have 10,000 contracts but don’t know which ones have auto-renewal clauses.”	High volume makes manual review impractical. Clear extraction task with verifiable output.
5	“Our customer support tickets contain product issues, but we can’t aggregate them.”	Cross-document pattern detection. Humans can’t spot trends at scale.
5	“You get hundreds of customer inquiries daily and need to route them to the right team.”	Repetitive classification/routing with clear success criteria and measurable return on investment.

Many use cases combine data extraction, analysis, and automation. **Figure 1-3** shows an example from engineering and quality management. Manufacturers receive thousands of quality documents daily from suppliers. RAG applications extract key information, compare it against ISO standards in a vector database, check compliance, and generate reports. If information is missing or contains noncompliant parameters (like welding thickness in this example), the system can automatically retrieve the supplier’s contact information and generate a draft follow-up email.

This automation saves hours of manual review and speeds up issue resolution, an example for a high-value RAG use case.

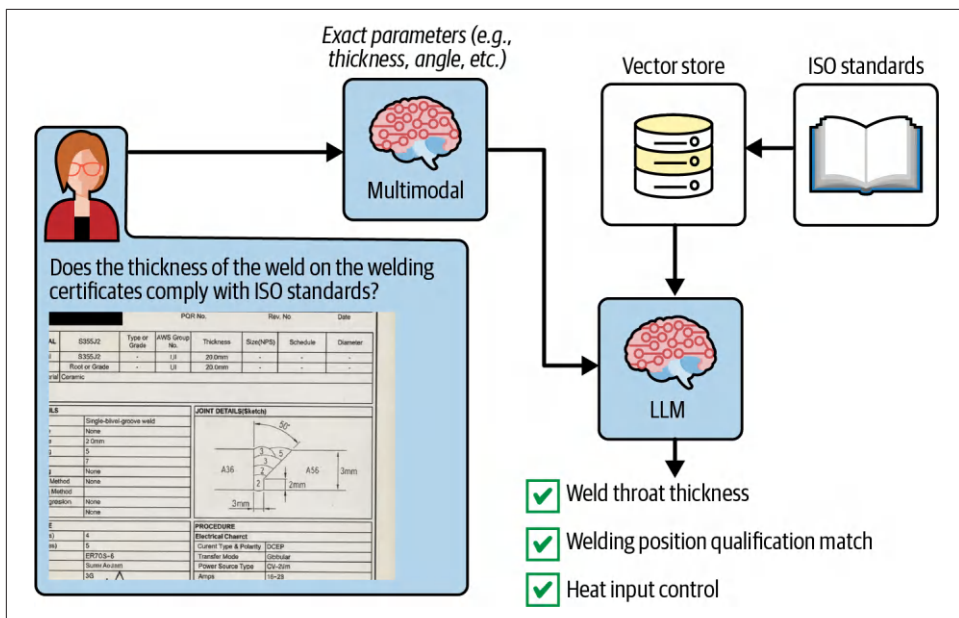


Figure 1-3. Extracting structured data from technical quality documents

Discussion

RAG works for automation because LLMs can interpret intent and context, not just match patterns. Traditional automation requires explicit rules for every scenario. When a supplier email says “delayed due to weather” versus “shipment rescheduled,” rule-based systems need separate handlers for each phrase. RAG-based systems understand that both express the same operational impact.

This capability matters most when data is unstructured and variable. Use RAG when each input differs slightly but requires similar handling—customer emails, contract clauses, or incident reports. Don’t use RAG for simple lookups, fixed-format data extraction, or tasks based on unchanging rules. If you can write a regular expression (regex) or SQL query that handles 95% of cases, RAG adds unnecessary complexity and cost.

RAG trades simplicity for flexibility. Traditional automation flows that rely on rule-based decision steps run faster, cost less per operation, and fail in predictable ways. LLM-powered RAG systems, on the other hand, handle edge cases better but require model hosting, prompt engineering, and evaluation frameworks. Factor in both token costs and engineering effort when comparing approaches.

Start with high-volume tasks where manual review creates bottlenecks. A system that processes 10 documents daily may not justify RAG development costs, but one that processes 1,000 documents daily does. Each implementation teaches you about

prompt patterns, evaluation metrics, and failure modes that apply to subsequent projects.

Avoid complex multisystem integrations as first projects. Success requires both RAG expertise and organizational buy-in. After demonstrating value on focused use cases, expand to scenarios requiring cross-functional coordination or multiple data sources.

See Also

- Dasha Maliugina’s blog post “[10 RAG Examples and Use Cases from Real Companies](#)” provides an overview of practical RAG implementations from organizations such as LinkedIn, Harvard Business School, Vimeo, and Pinterest.
- The open source repository [GenAI & LLM System Design: 500+ Production Case Studies](#) collects real-world examples of production generative AI (GenAI) systems, including many architectures and deployment patterns based on RAG.

1.2 Choosing Your IDE and Coding Agent Setup

Problem

You want to set yourself up to quickly develop and share RAG use cases in Python.

Solution

Popular Python IDEs for RAG development include the following:

Visual Studio (VS) Code

Lightweight, extensible, with strong Python support

PyCharm

Full-featured Python IDE with advanced debugging

Jupyter Notebook/Lab

Interactive development with inline visualization

Spyder

Scientific computing focus

Vim/Neovim

Terminal-based, highly customizable

Popular LLM-powered coding assistants include the following:

GitHub Copilot

IDE integration, chat and agent modes

Cursor

Similar to Copilot with IDE-native experience

Claude Code

Command-line tool, excels at understanding large codebases

Figure 1-4 shows VS Code with GitHub Copilot, displaying the chat window with ask, edit, and agent modes.

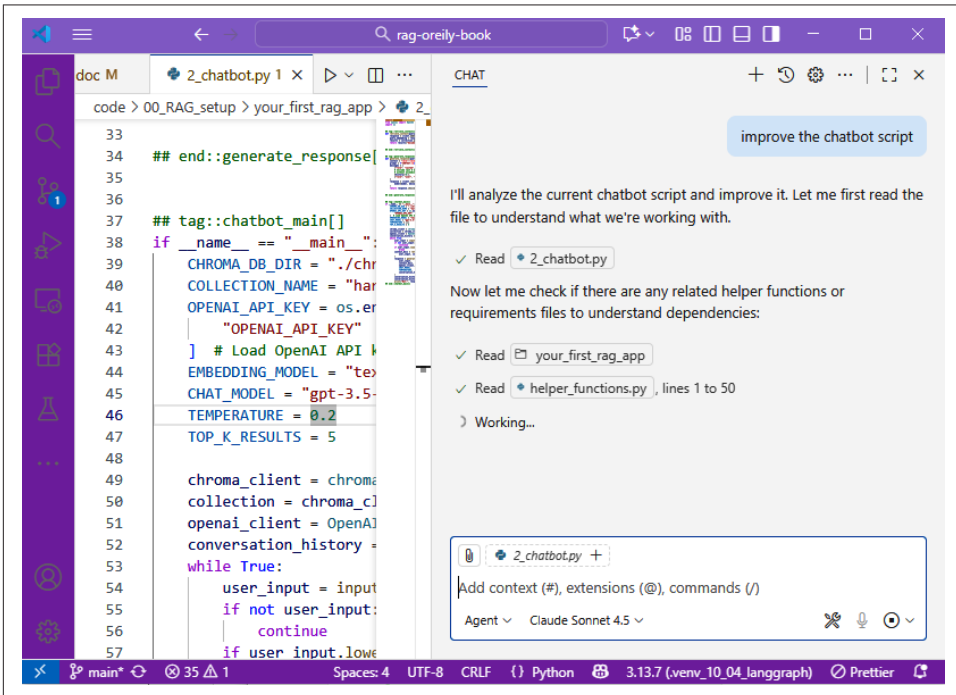


Figure 1-4. GitHub Copilot in VS Code

Beyond IDEs, you need to set up a Python environment with the following requirements to run the book's code examples:

- Python 3.9 or later (code examples were tested with Python 3.10)
- Virtual environment for each project (see [Recipe 1.3](#))
- Chapter-specific dependencies (see [Recipe 1.7](#))

Discussion

GitHub Copilot and Cursor integrate directly into your IDE and show code changes in real time. You can review each suggestion before accepting it. Claude Code operates from the command line and handles large codebases with complex module dependencies more effectively than IDE-based assistants. Most developers find value in both IDE-integrated and command-line approaches for different tasks.

All coding agents have different strengths, weaknesses, and pricing models. Try free tiers to evaluate which tools fit your workflow before committing to subscriptions. Some teams standardize on one tool for consistency, while others let developers choose based on personal preference.

See Also

- [The GitHub Copilot documentation](#) explains setup and usage for AI-powered code completion in your IDE.
- [Cursor](#) provides an AI-first code editor with integrated chat and inline editing features.
- [Claude](#) offers API access for building custom coding assistants and workflows.

1.3 Getting Started with Jupyter Notebooks in VS Code

Problem

You want to quickly get started building your own RAG workflows. You want to experiment a lot and keep your code, visualizations, and documentation all in one place.

Solution

Jupyter notebooks let you write and run code in separate cells, test parts independently, and display outputs, tables, and charts inline. This makes them ideal for RAG experimentation.

To set up Jupyter notebooks in VS Code, follow these steps:

1. Install VS Code from the [official website](#).
2. Install the Python extension (which includes Jupyter support).
3. Create a new notebook file (*.ipynb* extension).
4. Create and activate a virtual environment:

```
python3 -m venv .venv
# Windows: .\.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
```

5. Select the virtual environment as your Jupyter kernel (at the top top right in VS Code).

Figure 1-5 shows a Jupyter notebook in VS Code with the correct environment selected.

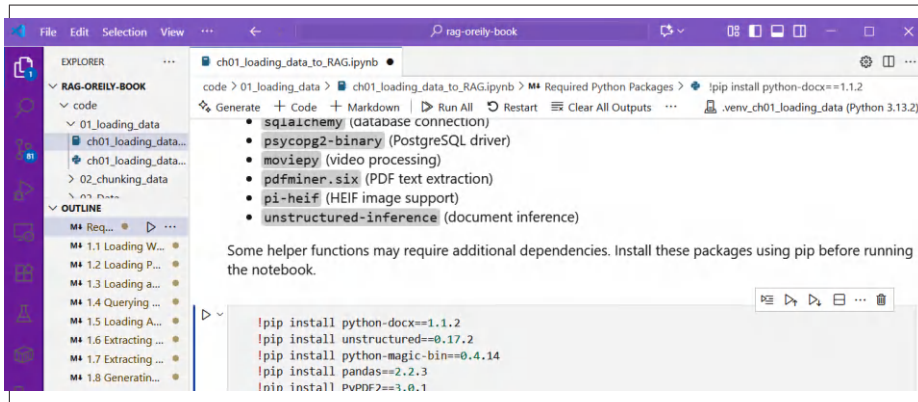


Figure 1-5. A Jupyter notebook in VS Code

6. Install packages either in the terminal or directly in notebook cells:

```
!pip install openai chromadb
```

7. Export the dependencies for sharing:

```
pip freeze > requirements.txt
```

8. Install the dependencies on another system:

```
pip install -r requirements.txt
```

Discussion

Notebooks keep data and models in memory. You can load data once and run experiments repeatedly without restarting.

Notebooks work well for exploration but have limitations for production systems. Cells can run in any order, creating hidden dependencies that aren't obvious from top-to-bottom reading. Notebooks store content as JSON, making diffs and merge conflicts difficult to resolve. These factors make notebooks unsuitable for production deployments that require robust testing and version control.

Use notebooks for exploration and visualization. Extract reusable logic into separate Python modules. Import these modules into notebooks. This keeps notebooks focused on experimentation while reusable logic stays testable and maintainable.

Use the IPython `autoreload` extension to automatically reload changed modules:

```
%load_ext autoreload
%autoreload 2 # Reload all modules before running code

import my_module
```

Don't use notebooks when collaborating with teams who are unfamiliar with Jupyter. The execution order dependencies confuse developers accustomed to linear Python scripts. Similarly, avoid notebooks for code that must meet compliance requirements or pass through formal review processes, because the JSON format makes auditing difficult.

See Also

- [The VS Code Jupyter documentation](#) covers notebook features, debugging, and variable inspection.
- [The Jupyter installation guide](#) provides setup instructions for standalone Jupyter Notebook and JupyterLab environments.

1.4 Storing Secrets and API Keys with .env Files

Problem

You want to keep secrets like API keys out of your source code while still reusing them across multiple projects.

Solution

Use an `.env` file to store secrets. This keeps API keys out of your codebase and prevents accidental commits to version control.

Add the `.env` file to your `.gitignore`. The `.gitignore` file lists files that Git should not track, ensuring that secrets stay local and never reach remote repositories.

This recipe demonstrates loading secrets from `.env` files via the `python-dotenv` library. See [Figure 1-6](#) for the complete architecture.

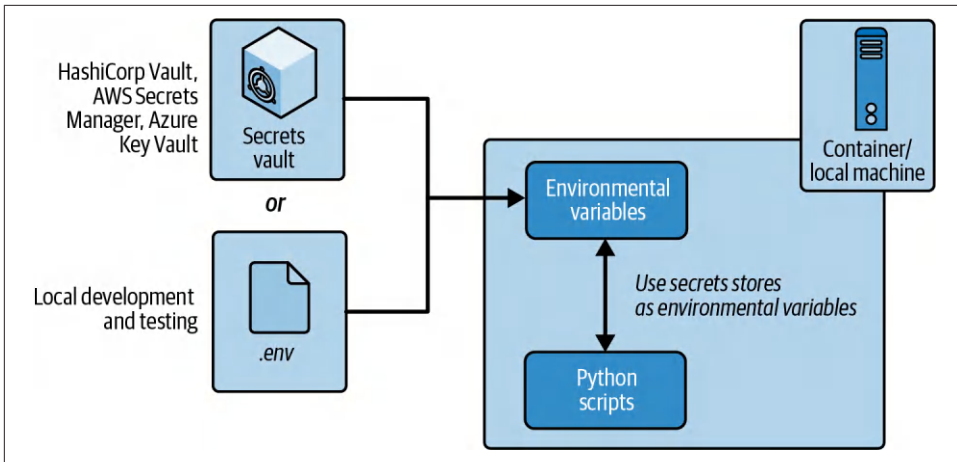


Figure 1-6. Loading secrets from `.env` in development and from secret management systems in production



Treat `.env` files as a local development convenience only. For production deployments, always use a managed secret store provided by your cloud platform.

Start by installing the `python-dotenv` library:

```
pip install python-dotenv
```

Most software development kits (SDKs) expect specific environment variable names, so always check the documentation. For example, OpenAI looks for `OPENAI_API_KEY`. Put each secret into your `.env` file using the format `KEY=_VALUE`:

```
HUGGINGFACEHUB_API_TOKEN=hf_...
OPENAI_API_KEY=sk-...
```

Make sure your `.env` file is listed in your `.gitignore`. This prevents Git from tracking it. When you create a new repository on GitHub, a default `.gitignore` is often generated for you. Add the `.env` file if it isn't already included. The following snippet shows an example `.gitignore` file with common file types that are typically ignored:

```
# Environments
.env
.venv
.vscode
env/
venv/
```

To load secrets in your Python code, call `load_dotenv` once at startup. You can then access values via `os.getenv`, as shown here:

```
from dotenv import load_dotenv
import os

load_dotenv()

openai_key = os.getenv("OPENAI_API_KEY")
```

Discussion

Environment variables are key-value pairs that your program reads from the operating system at runtime. They provide a standard interface for accessing configuration and secrets without hardcoding values in source code.

The `.env` approach works well for local development because it keeps secrets out of version control without requiring cloud infrastructure. However, `.env` files are plain text stored on disk, providing no encryption or access control.

For production systems, use managed secret stores like Amazon Web Services (AWS) Secrets Manager, Microsoft Azure Key Vault, or HashiCorp Vault. These services provide encryption at rest and in transit, audit logging, automatic rotation, and fine-grained access controls. Most cloud platforms inject secrets directly as environment variables at runtime, so your application code remains unchanged—it still uses `os.getenv`.

Don't share `.env` files via email, Slack, or shared drives when multiple team members need access to the same secrets—this defeats the security benefits. Instead, use a shared development secret store or credential management tool like 1Password or AWS Secrets Manager with developer access.

The key principle is that secrets should never be hardcoded in source code or committed to version control. Whether you use `.env` files locally or a managed service in production, environment variables provide a consistent interface for accessing secrets.

See Also

- [The python-dotenv documentation](#) explains how to load environment variables from `.env` files into your Python applications.
- [GitHub's guide on removing sensitive data](#) shows how to clean up accidentally committed secrets from Git history.

1.5 Building Your First RAG App

Problem

You want to build your first RAG-based conversational knowledge chatbot.

Solution

Every RAG system is built around three essential components:

- An embedding model that represents text as vectors
- A vector store that searches those vectors
- An LLM that generates answers from retrieved context

The workflow has two main parts: data ingestion and query processing, which are described next.

First, for data ingestion, you'll follow these steps:

1. Split documents into smaller chunks.
2. Generate embeddings for each chunk.
3. Store vectors in a vector database.

Then, for query processing, you'll use a complete RAG pipeline, which is illustrated in **Figure 1-7**:

1. Convert the user's question into an embedding vector.
2. Search the vector database for the closest matching vectors.
3. Select the most relevant chunks.
4. Send chunks together with the user's question to the LLM.

In the last step, the LLM generates an answer using this context.

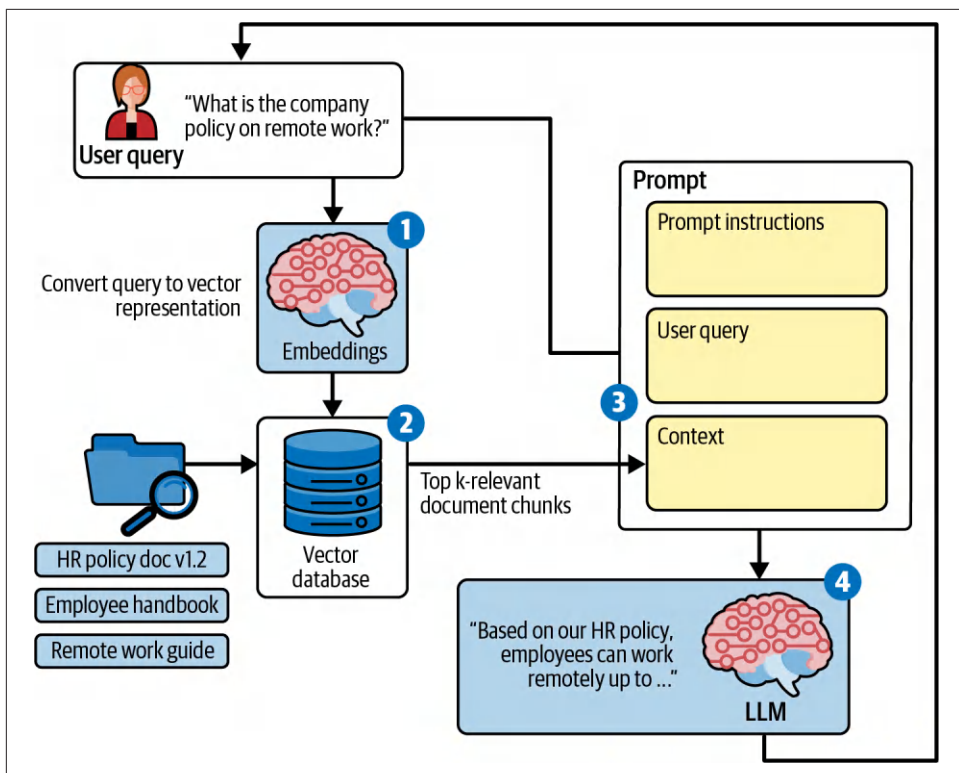


Figure 1-7. A basic RAG pipeline

Before you begin, make sure you have the following:

- An OpenAI account and an API key from the [OpenAI Platform](#) website
- Chroma for vector storage (for vector store concepts, see [Chapter 6](#))

This recipe demonstrates the workflow by building a Harry Potter knowledge chatbot. [Figure 1-8](#) illustrates the overall architecture this recipe implements. The retriever searches a Chroma vector store built from the Harry Potter books, and the generator uses OpenAI models to answer questions based on retrieved context.

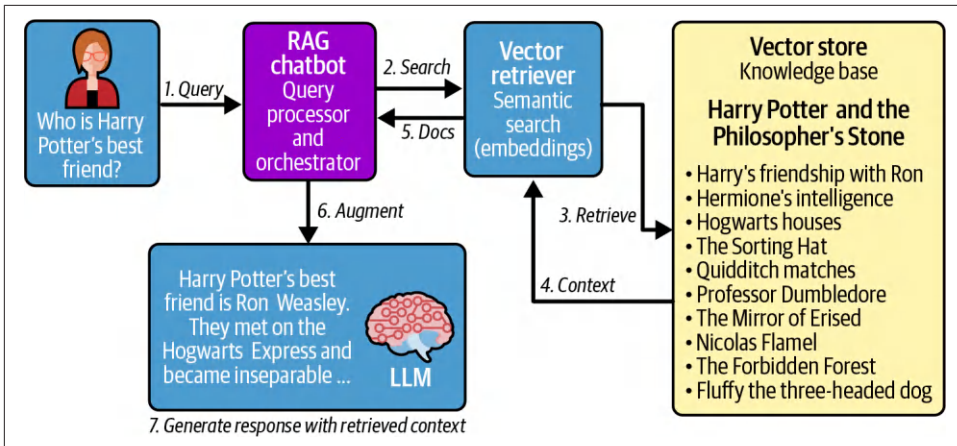


Figure 1-8. Harry Potter chatbot RAG architecture

Step 1: Text chunking

This recipe builds a RAG system from scratch without frameworks to demonstrate how simple the core concepts are. RAG orchestration frameworks like LangChain and LlamaIndex provide ready-to-use chunking functions, but this example avoids those dependencies to keep the implementation lightweight and the core concepts clear.

In this first step, you chunk text at natural boundaries, such as paragraphs or sentences, with overlap to preserve context across chunks:

```
from pathlib import Path
import chromadb
from openai import OpenAI

def chunk_text(text, size=1000, overlap=200):
    chunks, start = [], 0
    while start < len(text):
        end = min(start + size, len(text))
        if end < len(text):
            bp = text.rfind("\n\n", start, end)
            if bp == -1:
                bp = text.rfind(".", start, end)
            if bp > start:
                end = bp + 1
        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
        start = end - overlap if end < len(text) else end
    return chunks
```

```

file_path = Path("datasets/text_files/harry_potter_knowledge_base.txt")
text = file_path.read_text(encoding="utf-8")
chunks = chunk_text(text)

```

Step 2: Embed and store in Chroma

Next, you generate embeddings and store them in a Chroma collection. The code checks whether a collection with the same name already exists in the directory and creates one if it doesn't. If you run this cell multiple times and get ID conflicts, delete the *chroma_db* folder or use a new collection name:

```

client = OpenAI()
embedding_model = "text-embedding-3-small"

def embed_and_store(chunks, db_path, collection_name):
    chroma = chromadb.PersistentClient(path=str(db_path))
    collection = chroma.get_or_create_collection(
        name=collection_name,
        metadata={"description": "Harry Potter knowledge base"},
    )

    embeddings = []
    for i in range(0, len(chunks), 100):
        batch = chunks[i : i + 100]
        res = client.embeddings.create(model=embedding_model, input=batch)
        embeddings.extend([x.embedding for x in res.data])

    collection.add(
        ids=[f"chunk_{i}" for i in range(len(chunks))],
        documents=chunks,
        embeddings=embeddings,
        metadatas=[{"chunk_index": i} for i in range(len(chunks))],
    )
    return collection

chroma_db_dir = Path("chroma_db")
collection = embed_and_store(chunks, chroma_db_dir, "harry_potter_kb")

```

Step 3: Test retrieval

With data stored, test that the retrieval system finds relevant chunks for a given question:

```

def retrieve(question, top_k=3):
    q_emb = client.embeddings.create(
        model=embedding_model,
        input=question,
    ).data[0].embedding

    res = collection.query(

```

```

        query_embeddings=[q_emb],
        n_results=top_k,
        include=["documents"],
    )

    return res["documents"][0]

question = "Why did Uncle Vernon take the family to the hut by the sea?"
docs = retrieve(question)

```

Step 4: Test generation

Once retrieval works correctly, test the generation step by feeding the retrieved documents and question to the LLM to generate an answer:

```

def answer(question, docs):
    context = "\n\n---\n\n".join(docs)
    prompt = f"""Answer the question using only the context below.

Context:
{context}

Question:
{question}

Answer: """

    res = client.chat.completions.create(
        model="gpt-5-mini",
        messages=[{"role": "user", "content": prompt}],
    )

    return res.choices[0].message.content

question = "Why did Uncle Vernon take the family to the hut by the sea?"
docs = retrieve(question)
answer_text = answer(question, docs)

```

With the retriever and generation working, you have your first basic RAG system.

Discussion

This basic RAG pattern works when your data fits in a single vector store and users ask direct factual questions. This approach fails when queries require multistep reasoning (“compare X and Y, then recommend Z”) or when different data types need different retrieval strategies. For example, combining structured database queries with document search requires orchestration beyond simple vector similarity.

Test retrieval independently before debugging generation. If the retriever returns irrelevant chunks, fix retrieval before adjusting prompts or LLM parameters. If answers feel generic or unrelated to your documents, the problem is usually retrieval quality, not the LLM.

As you progress through this book, you'll learn a variety of techniques for both retrieval and generation that can be combined to build more sophisticated RAG systems tailored to your specific use cases.

See Also

- [The LangChain RAG tutorial](#) walks through building a complete question-answering application with source code.
- [The LlamaIndex starter tutorial](#) demonstrates indexing and querying documents in under 5 minutes.
- [The LlamaIndex RAG examples repository](#) contains production-ready RAG patterns for various use cases.

1.6 Choosing the Frameworks and Libraries for Your RAG Applications

Problem

You're looking for a set of libraries and frameworks to help you build RAG applications.

Solution

RAG applications use both high-level orchestration frameworks like LangChain and LlamaIndex and low-level libraries like Facebook AI Similarity Search (FAISS) and NumPy for data processing.

[Figure 1-9](#) shows the main library categories used in this book.

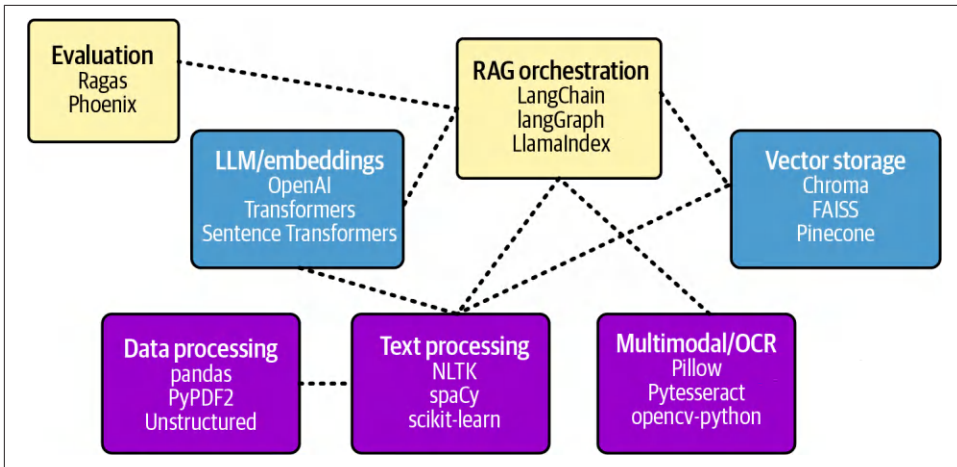


Figure 1-9. An overview of RAG libraries

Table 1-2 provides an overview of a selection of important libraries and frameworks for RAG applications.

Table 1-2. Common libraries and frameworks for RAG applications

Category	Example libraries	Description
RAG and agentic RAG orchestration	LangChain, LangGraph, LlamaIndex	Orchestration frameworks bundle common RAG components like retrievers, generators, prompts, and memory into unified abstractions. They handle the plumbing for vector stores, LLMs, and databases, letting you focus on application logic rather than integration code.
LLM/embeddings	OpenAI, Anthropic, Transformers, Sentence Transformers	Foundation models and embedding services form the core of your RAG system, providing the intelligence for both understanding queries and generating responses.
Vector storage	Chroma, faiss-cpu, Pinecone, Milvus, Weaviate	Vector databases enable efficient similarity search by storing and retrieving embeddings.
Data processing	pandas, NumPy, PyPDF2, pypdf, python-docx, Unstructured, openpyxl, scikit-learn	Libraries for data manipulation, file reading, and preprocessing tasks. Help load and preprocess unstructured data before connecting it to your RAG app.
Multimodal and media	Pillow, Pytesseract, MoviePy, pdf2image, opencv-python	Mature libraries for loading and processing various file formats including videos, podcasts, Word docs, PowerPoint presentations, and images.
Text processing and natural language processing (NLP)	NLTK, Transformers, Rank-BM25, Beautiful Soup 4	Traditional NLP libraries that help process text efficiently and perform text analysis tasks without relying solely on LLMs.
Evaluation and monitoring	Ragas, Phoenix, LangSmith, Prometheus-Eval	Frameworks providing predefined metrics to evaluate RAG app components (retriever, generator) and overall application performance.

Category	Example libraries	Description
Web frameworks and deployment	Streamlit, Gradio, Flask, Django	Frameworks for building user interfaces and deploying RAG applications. Streamlit and Gradio are ideal for rapid prototyping, while Flask and Django work well for production-grade apps.
Database and storage	SQLAlchemy, Psycopg 2, SQLite3	Connectors and object-relational mapping libraries for working with traditional SQL databases as data sources behind RAG apps.

Discussion

Orchestration frameworks like LangChain, LangGraph, and LlamaIndex (from [Table 1-2](#)) provide prebuilt components for common RAG tasks—retrieval chains, prompt templates, memory management, and agent loops. This accelerates development when building standard patterns. However, frameworks bundle many functions and pull in substantial dependencies. [Figure 1-10](#) shows the dependency tree for the LangChain Text Splitters library.

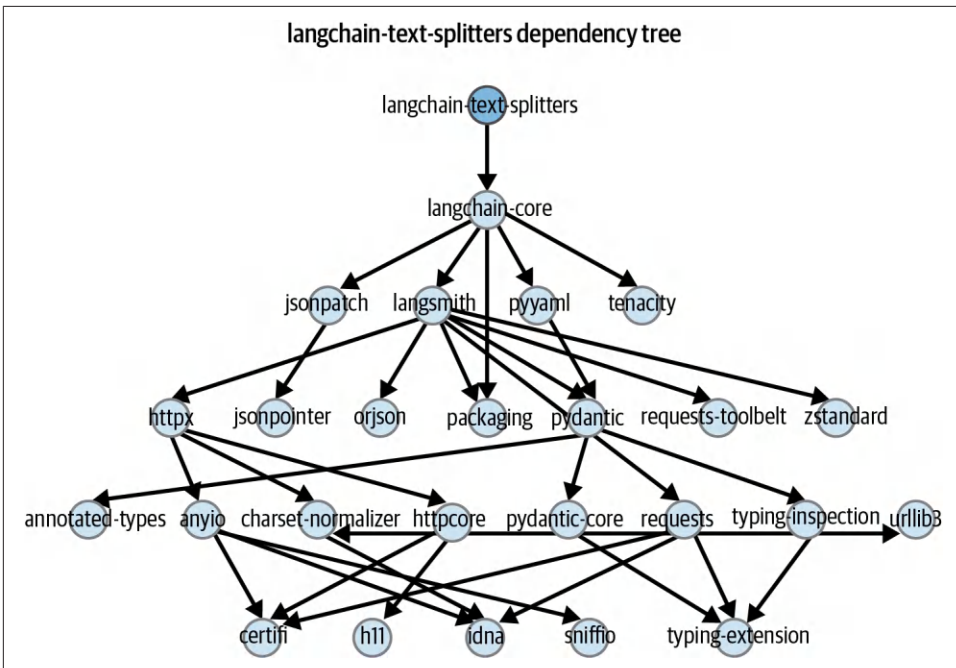


Figure 1-10. A dependency tree for a framework component

Each dependency introduces potential breaking changes, version conflicts, security updates, and incompatibilities. Projects with 100+ dependencies (common with full framework adoption) require significantly more troubleshooting than those using 5–10 core libraries.

Use frameworks when building applications that need multiple RAG patterns—conversational memory, multistep retrieval, tool calling, and reranking. Frameworks handle the integration work between these components. Don't use frameworks for simple applications that only need vector search and generation. A direct implementation with OpenAI and Chroma requires 2–3 dependencies instead of 50+.

Consider starting simple and adopting frameworks when complexity justifies the trade-off. You can migrate a basic retriever-generator implementation to a framework later if requirements expand. Going in the opposite direction—simplifying from framework to direct implementation—is much harder.

1.7 Running the Code Examples in the Book Repository

Problem

You want to clone the book repo and install the dependencies needed for each chapter.

Solution

First, clone the repository to your local machine. Then, set up a virtual environment and install only the dependencies required for the chapter you're working on.

Here's how to clone the repository and navigate into it:

```
git clone git@github.com:polzerdo55862/RAG-with-Python-Cookbook.git
cd rag-oreilly-book
```

Each chapter has its own *requirements_chXX.txt* file. Create a virtual environment and install dependencies, based on your operating system.

On Windows (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements_ch08.txt
```

On macOS/Linux (bash):

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements_ch02.txt
```

The code directory structure organizes examples by chapter (e.g., *01_loading_data/*, *02_chunking_data/*), with shared datasets, utility functions, and chapter-specific requirements files:

```
code/
├─ 01_loading_data/
├─ 02_chunking_data/
```

```
| 03_text_embedding/  
| ...  
| 11_building_rag_apps/  
| datasets/  
| results/  
| util/  
| helper_functions.py  
| requirements.txt  
| requirements_ch01.txt  
| requirements_ch02.txt  
| ...  
└ requirements_ch08.txt
```

Discussion

Installing dependencies per chapter rather than globally reduces conflicts and keeps environments lightweight. Each *requirements_chXX.txt* file contains only the libraries needed for that chapter's recipes. Alternatively, you can find the installation commands at the top of each Jupyter Notebook file in the book's repository.

Virtual environments isolate project dependencies from system Python packages. This prevents version conflicts when different projects require incompatible library versions. Activate the environment each time you work on the book examples.



Most code examples in this book are also available as Google Colab notebooks. Google Colab is a free cloud-based Jupyter notebook environment that requires no local setup—you can run Python code directly in your browser with access to GPUs. This makes it easy to experiment with the recipes without installing dependencies locally. Visit the book's repository to find the Colab links for each chapter's examples in the README.

See Also

- [The LangChain documentation](#) covers architecture, integrations, and API references for RAG applications.
- [The LlamaIndex documentation](#) provides guides on indexing strategies, query engines, and advanced retrieval patterns.
- [The Haystack documentation](#) explains pipeline-based RAG design and production deployment.

Foundation Models

Foundation models—including LLMs and multimodal models—form the backbone of modern RAG systems. These models are used both to generate answers for users and to prepare content before it's stored and retrieved.

In the generation step, foundation models analyze retrieved context and user questions to produce grounded responses. In the preparation step, foundation models extract text from images, transcribe audio, summarize long documents, and enrich content with metadata that improves retrieval quality.

This chapter focuses on the language models and multimodal models used in both the preparation step (also called the ingestion phase)—where content is processed, transformed, and prepared for storage—and the generation step—where models analyze retrieved information and generate answers for users.

Figure 2-1 shows a typical multimodal workflow for processing video content:

1. Use a vision model to analyze video frames.
2. Use speech-to-text to transcribe audio.
3. Embed the resulting text by using an embedding model.
4. Retrieve relevant context when users ask questions.
5. Generate answers with a language model.

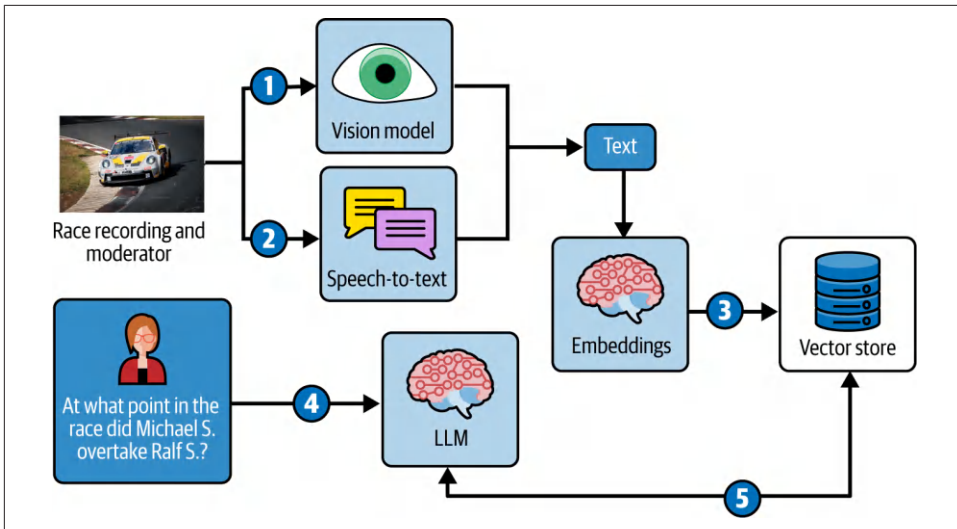


Figure 2-1. Multimodal models can interpret and generate text, images, audio, and video

Every RAG system needs a generation model that interprets the retrieved content and generates the required output—whether that’s answering a user question, producing a report, or completing another specific task. The following recipes show how to design effective prompts, choose appropriate models, and generate structured outputs.

You can find all the code examples for this chapter in the [book’s GitHub repository](#).

2.1 Defining a Suitable Prompt Template

Problem

You need to define an effective prompt template for the generation step of a RAG application.

Solution

For RAG applications, prompts follow a structured pattern. A suitable prompt template consists of four key components:

- A role that defines how the assistant should behave
- An instruction that describes the task
- The retrieved context the model must rely on
- Output requirements that define how the answer should look

This structure guides the model toward grounded responses. By explicitly instructing the model to use only the retrieved context, users can later review the sources and verify that the answer is based on real information rather than hallucinated facts.

Figure 2-2 shows the high-level structure of such a prompt.

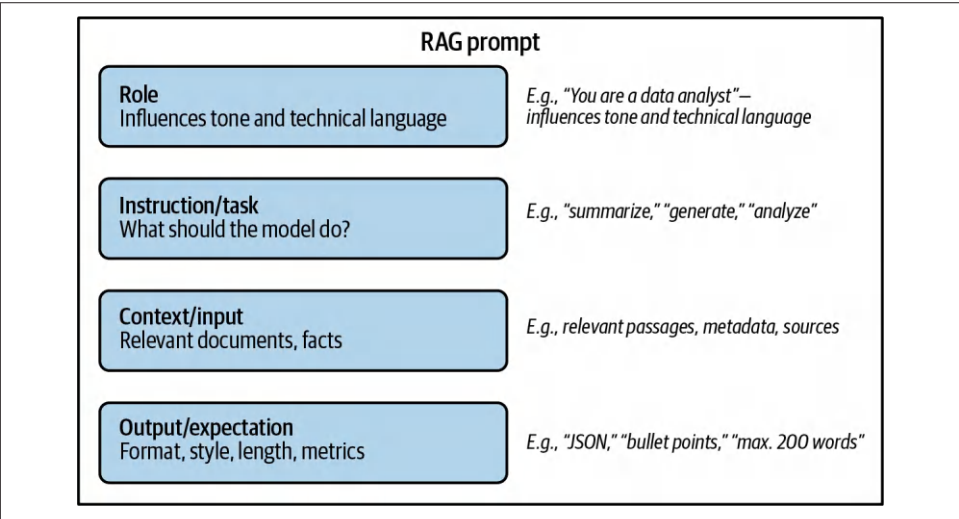


Figure 2-2. Prompt structure in RAG applications

To make this concept tangible, imagine an AI assistant that supports an internal purchasing team. The assistant has access to internal policy documents, contract clauses, and supplier notes. All this information is stored in a vector database and retrieved at query time to answer questions in a grounded way.

Here's a simple prompt template for this use case. The Context section contains a placeholder because, in a real RAG system, retrieved passages are inserted there dynamically at runtime:

```
[Role]
You are a procurement analyst who explains internal policies in a
simple and reliable way.

[Instruction]
Read the retrieved context and answer the user's question using
only the information from the retrieved context.

If the answer is not covered in the context, say that the
information is not available in the provided context.

[Context]
<<context>>

[User Question]
```

"Do our standard supplier contracts include a minimum warranty period?"

[Output Requirements]

Short answer. Clear wording. Up to 120 words.

This prompt instructs the model to analyze the retrieved context and generate an appropriate answer based solely on that information. This is a classic chatbot setup, where the assistant is connected to a vector store containing domain-specific knowledge. Because the model is explicitly instructed to stick to the context, every answer remains traceable to real source material.

Discussion

Prompt templates work by creating consistent structure that constrains model behavior. The role field primes expectations about expertise level and communication style. Instructions explicitly define success criteria instead of relying on implicit model assumptions. Context provides grounding in retrieved facts. Output requirements prevent format errors that break downstream processing.

You should balance flexibility and control. When templates are too loose, models inject training data and fabricate citations. When templates are too strict, models refuse to answer edge cases that fall partly outside retrieved context. Test with real queries and adjust based on failure patterns.

Use structured templates when downstream systems parse model output or when consistency matters more than creativity. Don't use them for exploratory conversations where rigid structure limits useful responses. Templates add token overhead; every retrieval call adds to this cost, so keep templates concise.

See Also

- The [DAIR.AI Prompt Engineering Guide](#) provides a comprehensive, community-maintained collection of prompt engineering techniques, papers, and practical examples for RAG and AI agents.
- [Anthropic's Prompt Engineering Interactive Tutorial](#) teaches core prompt engineering techniques through hands-on exercises applicable to most LLMs.

2.2 Selecting the Right Language Model for Your Task

Problem

You need to choose a suitable generation model for your RAG system. Picking a model that is too weak leads to unreliable answers. Picking a model that is too powerful can make your system slow and expensive.

Solution

All major providers offer model families in multiple sizes. They range from ultra-efficient models to reliable everyday models, and up to powerful flagship and reasoning models for demanding tasks.

Table 2-1 summarizes each tier’s strengths and struggles as well as the models that typically fall into each category.

Table 2-1. Language model selection framework (as of January 2026)

Tier	Accuracy/typical use cases	Examples
Ultra-efficient	• Simple tasks, lightweight workloads • High-throughput, real-time responses • Classification, quick Q&A • Edge or embedded applications	Gemini 3 Flash, Claude Haiku 4.5, GPT-5 nano
Efficient/everyday	• General productivity • Text generation, summarization • Chatbots and support assistants	Gemini 2.5 Flash, GPT-5 mini, GPT-4.1
Flagship	• Complex or creative tasks • Deep analysis and structured reasoning • Advanced code generation • Customer-facing automation	Gemini 3 Pro, GPT-5, Claude Sonnet 4.5
Reasoning/frontier	• Multistep reasoning and logical problem-solving • Scientific and mathematical tasks • Long-context planning or proofs • Autonomous agentic workflows	GPT-5, Claude Opus 4.5

Cost is an important factor when using LLMs. Commercial LLMs charge separately for input and output tokens. Output tokens are often significantly more expensive because the model must generate each token one at a time while considering all previous tokens. Input tokens are processed in parallel during a single forward pass.

When choosing a provider, compare similar model tiers across vendors. For example, GPT-5 and Gemini 3 Pro both fall in the flagship tier, while GPT-5 mini and Gemini 3 Flash compete in the efficient tier. Price differences within the same tier are usually marginal compared to the substantial cost variation between tiers.

Figure 2-3 shows typical input token price ranges across model tiers as of January 2026:

Ultra-efficient
~\$0.05–\$0.20 per one million (1M) input tokens

Efficient
~\$0.25–\$0.60 per 1M input tokens

Flagship

~\$1–\$5 per 1M input tokens

Reasoning/frontier

~\$2–\$30 per 1M input tokens

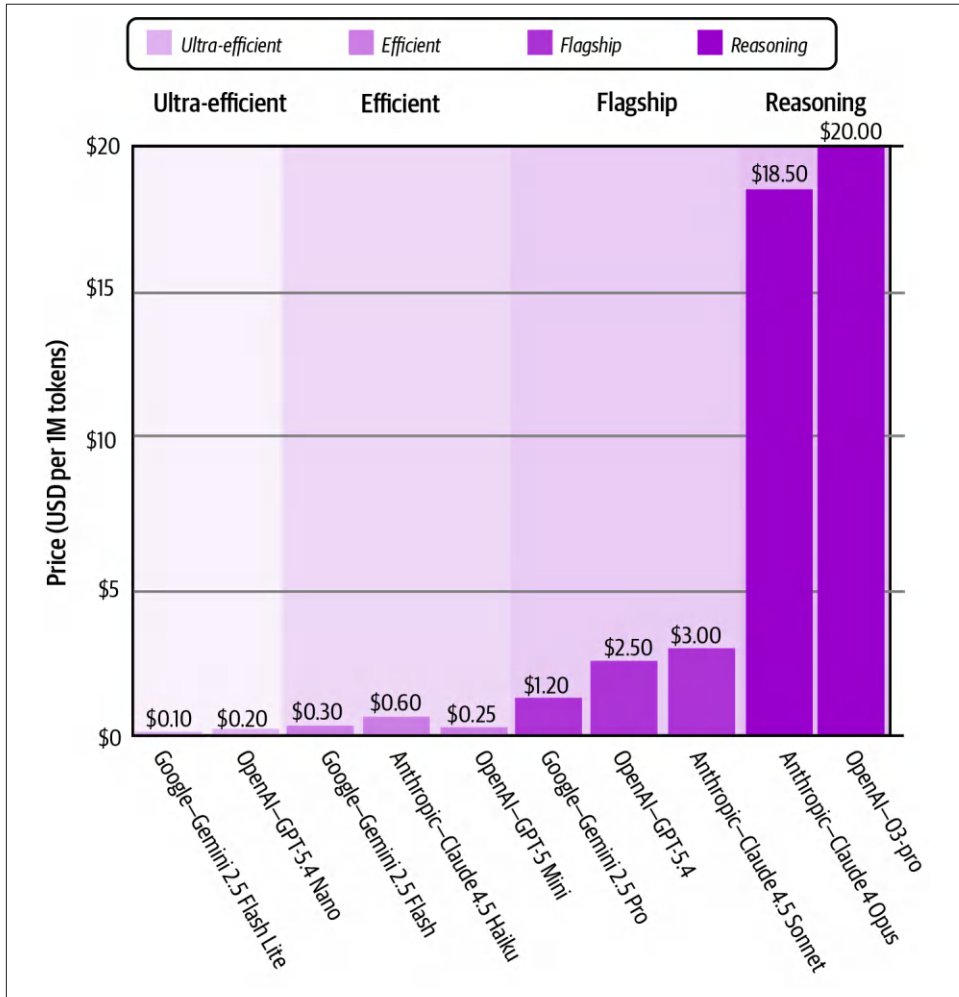


Figure 2-3. LLM model pricing comparison—input token costs across model tiers (January 2026)

Discussion

Model selection balances cost, latency, and accuracy. Your goal is to pick the smallest model that still meets requirements for each pipeline step.

Start with latency constraints. Real-time chatbots need responses within seconds. Users rarely wait more than 10 seconds unless answers provide exceptional value. For standard chatbot interactions with straightforward questions and manageable context—typically, three to four retrieved chunks—ultra-efficient or efficient models often deliver adequate quality while responding faster than powerful models. In these scenarios, larger models don't provide enough additional value to justify the increased latency.

Batch jobs tolerate longer latency and can run for minutes or hours, so improved accuracy may justify using slower, more expensive models. A pipeline that reviews complex contracts can save a significant amount of money if slightly better extraction quality prevents costly errors, making reasoning models worth the extra cost. Simple tasks like extracting contract parties or expiry dates work well with cost-efficient models.

Don't use large reasoning models for simple classification or extraction tasks that efficient models handle reliably. Test at each tier; you may find that ultra-efficient models work for 80% of queries, reserving expensive models for ambiguous cases only.



For complex workflows, break tasks into smaller steps and select the right model for each step. This lets you solve multistep problems with efficient models because each call handles one clearly defined task instead of requiring reasoning across the entire workflow.

See Also

- The [OpenAI model selection guide](#) provides structured criteria for choosing GPT models based on task complexity and cost.
- The [Arena leaderboard](#) ranks language models based on blind user preference votes from side-by-side comparisons.

2.3 Generating Content with the OpenAI API

Problem

You want to use remotely hosted LLMs from OpenAI to analyze and generate content, without deploying and operating models yourself.

Solution

OpenAI hosts its models on managed infrastructure and exposes them through API endpoints. This allows you to use powerful language models with minimal setup.

OpenAI offers model families for various task profiles. Flagship models such as GPT-5 are suitable for complex reasoning tasks and are supported by smaller and faster variants like GPT-5 mini and GPT-5 nano. All of these models can process both text and images to generate responses.

Figure 2-4 shows the release of the main GPT model series.

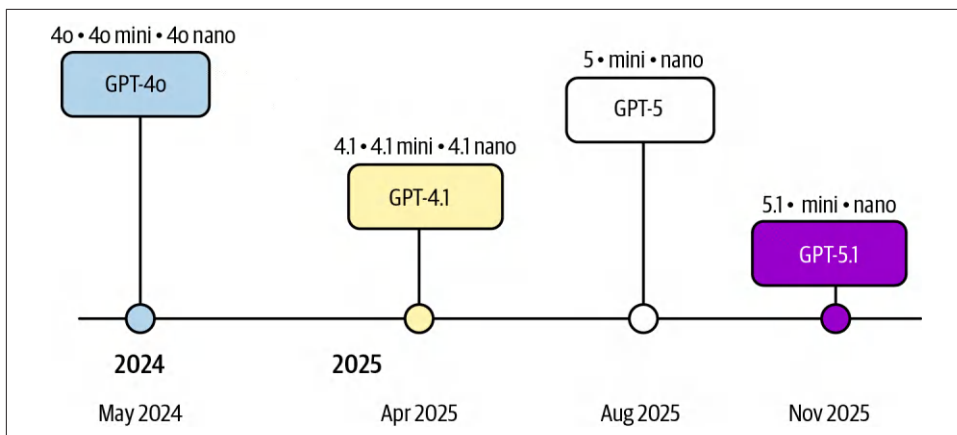


Figure 2-4. OpenAI GPT model series

You can find your API key in your OpenAI account settings. Search for “OpenAI API key” to create a new one.

Store the key in an environment variable named `OPENAI_API_KEY`. This is the default place for the OpenAI SDK to load the API key from. Install the dependencies as follows:

```
pip install openai httpx
```



If you work for a larger company with a strong Microsoft Azure presence, you might not use OpenAI’s public API directly. Instead, you can access the same OpenAI models through Microsoft’s Azure OpenAI service. Azure hosts models such as GPT-5, GPT-5 mini, GPT-5 nano, and text-embedding-3-small within the Azure ecosystem.

Communicating with OpenAI models follows a simple pattern. First, you create a client that connects to OpenAI’s servers. Then you send messages with your prompt and receive a response with generated text.

Each message consists of two required fields. The `role` field indicates who is speaking (for example, `system`, `user`, or `assistant`). The `content` field contains the actual message text.

The following code snippet shows how a message dictionary looks. You can add additional key-value pairs, but `role` and `content` are mandatory:

```
{
  "role": "user",
  "content": "What is RAG?"
}
```

Now you're ready for a complete example. You send messages as a list of dictionaries. The first entry is usually a system message, followed by the first user message. After the prompt is sent, the model generates a response, which is appended to the message list:

```
from openai import OpenAI

def ask_with_context(context, question):
    client = OpenAI()

    messages = [
        {
            "role": "system",
            "content": "Answer based only on the provided context."
        },
        {
            "role": "user",
            "content": f"Context:\n{context}\n\nQuestion:\n{question}"
        },
    ]

    response = client.chat.completions.create(
        model="gpt-5.2",
        messages=messages
    )

    return response.choices[0].message.content

# Usage
context = "RAG stands for Retrieval-Augmented Generation."
question = "What does RAG stand for?"
answer = ask_with_context(context, question)
print(answer)
```

In addition to GPT models, OpenAI provides speech-to-text and video generation models. Speech-to-text is relevant for RAG when you need to transcribe audio recordings, meeting notes, or customer calls before indexing them.

To transcribe audio, you upload an audio file and receive a transcript in return. You can use any file from the [LJ Speech Dataset](#) as an example. LJ Speech is a collection of audio recordings from a single speaker reading various sentences. This example uses the `gpt-4o-transcribe` endpoint to transcribe audio:

```

import httpx
from openai import OpenAI

client = OpenAI()

with open(
    "..\\datasets\\audio_files\\LJ037-0171.wav",
    "rb",
) as audio_file:
    transcript = client.audio.transcriptions.create(
        model="gpt-4o-transcribe",
        file=audio_file,
    )

print(transcript.text)

```

Discussion

Using hosted foundation models requires minimal setup—set the API key and HTTP client, and you're generating text. Your choice of provider matters when you have specific constraints around cost, context windows, or regulatory requirements.

OpenAI's rate limits and throughput typically exceed those of competitors for high-volume production deployments. Switch to Google or Anthropic when you need longer context windows, cheaper high-volume inference, or specific multimodal capabilities.

GPT-5 handles complex reasoning, structured outputs, and tool use. For latency-sensitive workloads where speed matters more than maximum reasoning depth, consider GPT-4.1 or smaller variants like GPT-5 mini. GPT-4.1 optimizes for low latency and high throughput, making it better for real-time chat and customer support.

If you work in a company with strict cybersecurity requirements and already use Azure, you can use Azure OpenAI services to process data within the Azure context without OpenAI capturing it for use on its own platforms.



When choosing a provider, focus on model capabilities and context window requirements rather than pricing. While cost differences between model tiers—from ultra-efficient to reasoning models—can be substantial (ranging from \$0.05 to \$30 per 1M input tokens), the price differences among providers within the same tier are marginal. The impact of selecting the right model tier for your use case matters far more than choosing OpenAI, Google, or Anthropic based on small price variations.

See Also

- The [OpenAI Chat API reference](#) documents the request and response format, including streaming, function calling, and parameter options.
- The [OpenAI embeddings guide](#) explains how to use embedding models for similarity search and retrieval in RAG systems.

2.4 Generating Content with Google's Gemini Models

Problem

You want to use Google's Gemini models to generate and analyze text and images.

Solution

Google aligned its API with OpenAI client library conventions, so you can reuse the same SDK. You change only the base URL so that the client sends requests to Google's generative language endpoint instead of OpenAI's.

Figure 2-5 shows the main Gemini model releases so far.

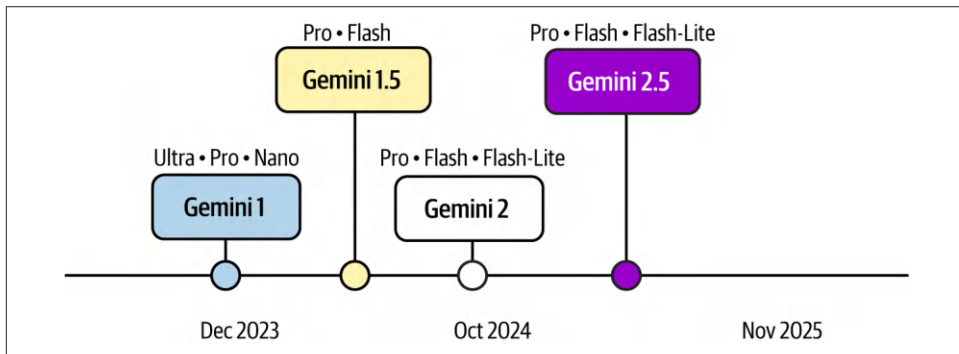


Figure 2-5. Google Gemini model timeline

Install the OpenAI client library:

```
pip install openai
```

To use Gemini, you need an API key from Google AI Studio. Create a key with a few clicks and store it in an environment variable named `GOOGLE_API_KEY`. The following example reads the key from this variable.

Next, create an OpenAI client that connects to Google instead of OpenAI. The structure is identical to using an OpenAI model. The only difference is the base URL defined when creating the client:

```

import os
from openai import OpenAI

client = OpenAI(
    api_key=os.getenv("GOOGLE_API_KEY"),
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)

resp = client.chat.completions.create(
    model="gemini-3-flash",
    messages=[
        {"role": "user", "content": "What is the capital of France?"}
    ],
)

print(resp.choices[0].message.content)

```

Once this works, you can switch to other Gemini models by replacing the model name.

Discussion

As of early 2026, Gemini models are among the strongest options available, especially for multimodal tasks. They can interpret text, images, charts, and screenshots within a single reasoning chain and handle very large documents. Gemini 2.5 Pro supports context windows of around one million tokens and is designed for long-context reasoning across text, audio, images, video, and even source code repositories.

For everyday workloads, smaller models like Gemini 2.5 Flash and Gemini 2.5 Flash-Lite provide similar multimodal capabilities at much lower cost and higher speed, which makes them a strong choice for many RAG pipelines.

In practice, Gemini can act as a drop-in replacement for OpenAI models. The main reason to choose Gemini today, particularly Gemini 2.5 Pro, is its strong multimodal performance combined with robust long-context handling. Choose Gemini over OpenAI when processing documents larger than 100,000 tokens or when combining images, audio, and text in a single request.

For low-latency, short-context interactions such as chat, RAG retrieval steps, and agent loops, OpenAI models typically deliver faster time-to-first-token than Gemini. Gemini's architecture is optimized for long-context and multimodal workloads, which can introduce additional overhead for small requests.



Use Gemini 2.5 Pro for strong multimodal reasoning or very long context, and Gemini 2.5 Flash or Flash-Lite for cost-sensitive, high-throughput RAG pipelines.

See Also

- The [Gemini API Quickstart guide](#) shows how to use Google’s Gemini SDK directly with model selection, text generation, and multimodal inputs.
- The [Gemini model documentation](#) lists available models, capabilities, context windows, and pricing for each variant.

2.5 Generating Content with the Anthropic API

Problem

You want to use Anthropic’s Claude models to analyze content and generate text.

Solution

Anthropic provides a strong alternative to OpenAI and Google, especially when you need precise reasoning, reliable coding support, or mathematically accurate outputs.

[Figure 2-6](#) shows the main Claude models that have been released.

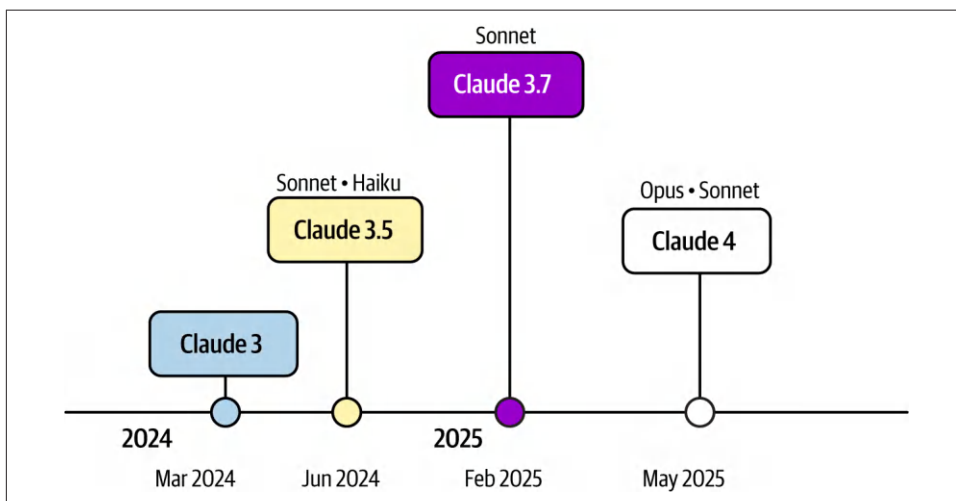


Figure 2-6. Anthropic Claude model release timeline

To use the Anthropic API, create an account on the Anthropic website and generate an API key. You can generate the key in the [Claude Console](#) under the API section. Purchasing a small number of credits is required. Five dollars’ worth is usually enough to run the examples in this book. Store the key as an environment variable named `ANTHROPIC_API_KEY` so the Anthropic SDK can locate it.

While Google provides an OpenAI-compatible endpoint, Anthropic uses its own protocol, which does not match what the OpenAI SDK expects. Therefore, you need to use the Anthropic SDK, which you can install as follows:

```
pip install anthropic
```

Use the official Anthropic Python SDK to interact with Claude models. In this example, you'll use a simple request-response flow to send a prompt and retrieve generated text. In a RAG setup, this call typically sits after the retrieval step, where retrieved documents are injected into the user message.

Let's use the `messages.create` method, which is the standard interface for communicating with Claude models. The model returns its response in the `content` field:

```
from anthropic import Anthropic
import os

client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

response = client.messages.create(
    model="claude-sonnet-4.5",
    max_tokens=200,
    messages=[
        {
            "role": "user",
            "content": "Explain how vector databases work in "
                       "simple terms.",
        }
    ],
)

print(response.content[0].text)
```

Discussion

Claude models emphasize explicit reasoning and edge-case handling. In practice, Claude tends to produce longer explanations and explicitly flags ambiguities rather than picking a plausible answer. This behavior makes Claude suitable for tasks where conservative reasoning matters more than speed.

Choose Claude when explicit edge-case handling matters; legal document review, compliance analysis, and code generation benefit from this approach. Claude's extended thinking capability also helps with complex mathematical reasoning and multistep logical problems.

Don't use Claude for high-throughput chatbots where users expect instant replies to simple queries, or when you need the fastest possible inference for batch jobs. Claude's deliberate reasoning trades speed for thoroughness.

Anthropic offers models in different sizes. Claude Haiku efficiently handles high-volume workloads. Claude Sonnet provides strong all-around performance. Claude Opus delivers deep reasoning for complex tasks. This structure parallels OpenAI's and Google's model families.

See Also

- The [Anthropic Messages API reference](#) provides complete documentation for request/response format, streaming, tool use, and system prompts.
- The [Anthropic quickstart guide](#) is a step-by-step tutorial for implementing Claude API calls with the Python SDK.
- The [Anthropic prompt engineering guide](#) indicates best practices for writing effective prompts for Claude models.

2.6 Running Open Source Models Locally with Ollama

Problem

You need to run language models locally because your data is too sensitive to send to external providers, or because self-hosting is more cost-effective at your API call volume.

Solution

A simple way to run models privately is Ollama. This free open source tool lets you run open source LLMs directly on your own machine. Once installed, Ollama starts a local service that behaves like the remote API endpoints from OpenAI and Google.

Before you can call Ollama from your Python code, you need two things:

- The Ollama runtime installed on your machine
- At least one local model downloaded and ready to use

Ollama is not a Python package. It is a system tool that runs a small local server in the background. This server loads the model, performs inference, and exposes an API on `localhost:11434` that your Python code can call.

To install Ollama, follow the instructions at the [Download Ollama page](#). Installers are available for macOS, Windows, and Linux.

Once Ollama is installed, you still need to download the models you want to run. You do this with simple `ollama pull` commands. These commands fetch the model weights and store them locally so the model is ready to run.

In this example, you'll pull three popular models: Llama 2, Mistral, and Code Llama. You can choose other models as well. A full list is available at the [Ollama Models web page](#). Heres the code:

```
# Pull models (run in terminal)
ollama pull llama2
ollama pull mistral
ollama pull codellama
```

Once Ollama is running, you can call it like any other API. The simplest approach is to reuse the same OpenAI SDK used for the OpenAI and Gemini APIs. Ollama exposes an OpenAI-compatible endpoint at <http://localhost:11434/v1>, so your existing code barely needs to change.

Next, you'll use the OpenAI client to call a local Ollama model. You create a client that points to the local endpoint instead of OpenAI's servers, then send your prompt exactly as before. The only differences are the `base_url` and the model name:

```
from openai import OpenAI

# Point the client to your local Ollama server
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama", # Ollama does not require a real key,
                    # but the SDK expects one
)

response = client.chat.completions.create(
    model="qwen3:4b",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {
            "role": "user",
            "content": "What is retrieval augmented generation?"
        },
    ],
)

print(response.choices[0].message.content)
```

This keeps your code structure consistent across providers. If you later switch from Ollama to OpenAI or Gemini, you need to change only the `base_url` and `api_key` values.



You can also call Ollama directly, using `curl` or the `requests` library if you prefer not to use an SDK:

```
curl http://localhost:11434/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "llama2",
  "messages": [
    {
      "role": "user",
      "content": "What is RAG?"
    }
  ]
}'
```

Once you have the basic setup working, you can experiment with different models to compare their performance and behavior.

In the next example, you evaluate multiple local models at once. The code loops over a list of installed Ollama models, sends them the same prompt, and collects their answers. This allows you to compare quality, speed, and consistency across models:

```
from openai import OpenAI

models = ["llama2", "mistral", "codellama"]
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama"
)

for model in models:
    print(f"\n--- Testing {model} ---")
    response = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "user", "content": "Explain RAG in one sentence."}
        ]
    )
    print(response.choices[0].message.content)
```

You can find more popular open source models in [Table 2-2](#).

Table 2-2. Popular open source language models

Model (Developer)	Release	Parameter count	Performance	Popularity
Llama 2 (Meta AI)	July 2023	7B, 13B, 70B	MMLU ~68.9% for the 70B chat model	Very high
GPT OSS (OpenAI)	August 2025	20B, 120B	Near GPT-3.5 performance	High
Mistral 7B (Mistral AI)	September 2023	7B	Strongest 7B class open model	Very high
Qwen 7B and 14B (Alibaba Cloud)	September 2023	7B, 14B	Strong multilingual performance	High
Falcon 40B (TII)	May 2023	40B	Early open benchmark leader	Strong community interest

Discussion

Ollama handles model downloads and runs them on your CPU or GPU. Running models locally has limits: without a strong GPU or dedicated inference hardware, your laptop can handle only smaller models, and even those respond noticeably slower than API calls. For privacy-sensitive workflows, this trade-off is often acceptable.

Very large models in the 70 billion (70B) parameter range, such as Llama 3.1 70B, are usually too large for most laptops. More realistic choices are smaller variants like Llama 3.1 8B or similar midsize models. They offer solid quality while keeping hardware requirements manageable. Expect 8B models to require at least 8 GB of virtual random access memory (VRAM); 13B models need 16 GB or more.

Run models on your own hardware when you process regulated health or financial data that cannot leave your network, or when you make millions of API calls per month and hosting costs drop below API fees.

Don't use local models when your use case requires the latest reasoning capabilities or when development speed matters more than cost. Local model setup, fine-tuning, and maintenance consume engineering time that could go toward application features.

See Also

- The [Ollama documentation](#) covers installation, model management, and the local API for running open-weight models.
- The [Ollama model library](#) lists supported models with pull commands and hardware requirements for each model variant.
- The [Open LLM Leaderboard](#) provides standardized benchmark scores for comparing open-weight language models.



Treat public leaderboards as screening tools, not as a final decision criterion. Common limitations include the following:

Benchmark leakage or data contamination

Some benchmark questions and answers are publicly available and may have been included (directly or indirectly) in training data, inflating scores.

Goodhart's law or over-optimization

Once a benchmark becomes a target, model developers may tune specifically for that benchmark, improving the score without improving general usefulness.

Mismatch with real-world usage

Your production setup (prompt templates, retrieval quality, tool use, long context, multilingual content, quantization, and latency constraints) can materially change results. A model that “wins” on a leaderboard may still perform worse on your own RAG queries.

2.7 Creating Structured Outputs with the OpenAI SDK and Pydantic

Problem

You want to extract insights from text and images while ensuring that the output from LLM and multimodal models follows a predefined schema with the correct data types.

Solution

The OpenAI SDK lets you use a Pydantic model to define a response schema. This schema describes the fields and their data types that the model should return. Instead of working with raw JSON, the SDK validates the model output and returns a typed Python object.

Figure 2-7 shows a simple multimodal invoice analysis flow. The model extracts the invoice date, supplier, receiver, line items, currency, and total value. A Pydantic model describes these fields and enforces the expected data types.

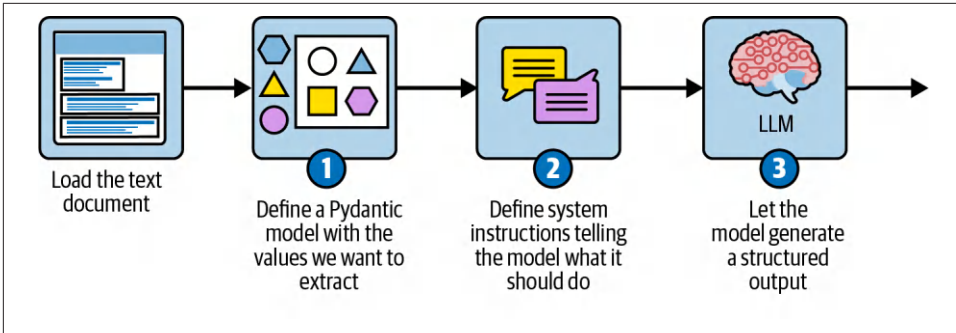


Figure 2-7. Structured output with Pydantic models

To use structured outputs, you need to install the OpenAI SDK and Pydantic:

```
pip install openai pydantic
```

You start by defining the structure you want the model to fill. In this example, you use Pydantic’s `BaseModel` class to define Python objects for invoice extraction with nested structures. This class creates a `LineItem` to represent individual items on an invoice, and `Invoice` to represent the complete invoice with multiple line items. Every Pydantic model must extend `BaseModel` because it provides automatic type validation and parsing methods that convert raw model output into clean Python objects. Here’s the code:

```
# pip install openai pydantic

from openai import OpenAI
from pydantic import BaseModel
from datetime import date
from typing import List

class LineItem(BaseModel):
    description: str
    quantity: int
    total: float

class Invoice(BaseModel):
    invoice_number: str
    invoice_date: date
    supplier: str
    items: List[LineItem]
    total_due: float
```

Once the structure is defined, the next step is to ask the model to populate it. You send an invoice image along with instructions to the OpenAI endpoint, and the OpenAI SDK returns a parsed `Invoice` object that already passed type validation:

```
result = client.responses.parse(
    model="gpt-5-mini",
```

```

input=[{
    "role": "user",
    "content": [
        {"type": "input_text", "text": "Extract the invoice data."},
        {"type": "input_image", "image_url": "invoice.png"},
    ],
}],
response_format=Invoice,
)

invoice: Invoice = result.output_parsed
print(invoice)

```

The result is a validated `Invoice` object. Because the data already has the correct types, you can store it directly in a database or pass it to downstream workflow steps without additional parsing or manual checks.

Discussion

Structured outputs enforce a schema at inference time, ensuring that the model returns validated fields rather than unstructured text. This works because the API constrains model generation to produce only valid JSON matching your Pydantic schema. Typical use cases include the following:

- Extracting contract details from emails in a sales system
- Automatically parsing invoices or order confirmations
- Reading metadata from technical drawings to label them consistently
- Making decisions in automated workflows whose output must be constrained to specific value ranges

Pydantic validation catches type mismatches or missing fields before they propagate downstream, making failures immediately visible instead of letting incorrect values silently flow through the system.

Use structured outputs when downstream systems depend on the data. Database inserts, API calls, decision logic, and multistep agent workflows all benefit from guaranteed field types. Don't use structured outputs for exploratory chat, creative writing, or when you want the model to choose its own output format.

Structured outputs add latency because the model must generate valid JSON conforming to your schema. They can also constrain useful answers when the schema doesn't match what the model wants to explain. Forcing a binary answer when the correct response is "it depends" loses nuance. Test whether your use case benefits more from structure or flexibility.

See Also

- The [Anthropic tool use documentation](#) explains how to enforce JSON schemas with Claude models for structured outputs.
- The [Google Gemini JSON mode guide](#) describes how to enable structured output support in Gemini models.

Loading Data

About **80% of enterprise information is unstructured** and distributed across presentations, documents, emails, and media files. **Figure 3-1** shows some of the common data types.

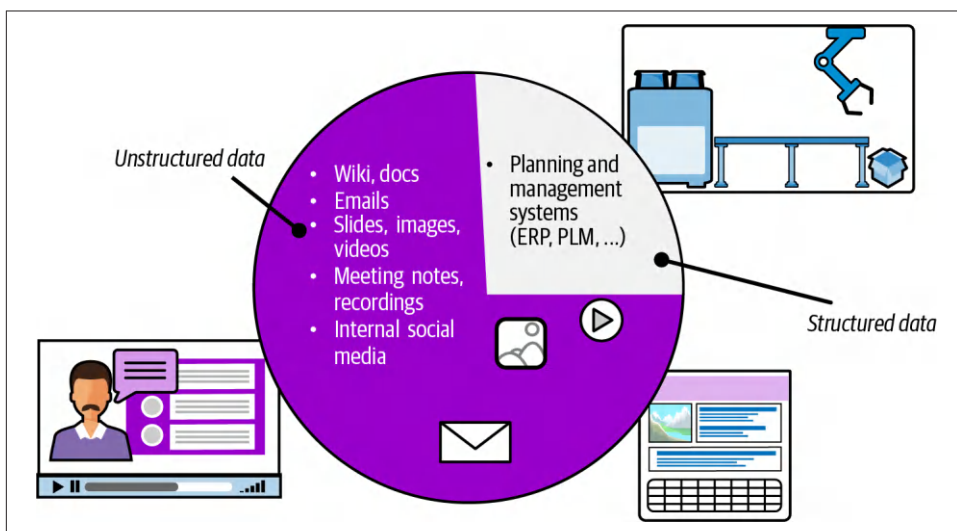


Figure 3-1. A common distribution of data in companies

This chapter shows how to turn common data sources into text that you can embed and retrieve. Most RAG retrievers work with text embeddings, so a practical first step is to convert different formats into a consistent text representation.

Figure 3-2 summarizes the main components of a RAG system, showing both the indexing pipeline (loading and processing documents) and the runtime retrieval flow (retrieving relevant chunks and generating answers).

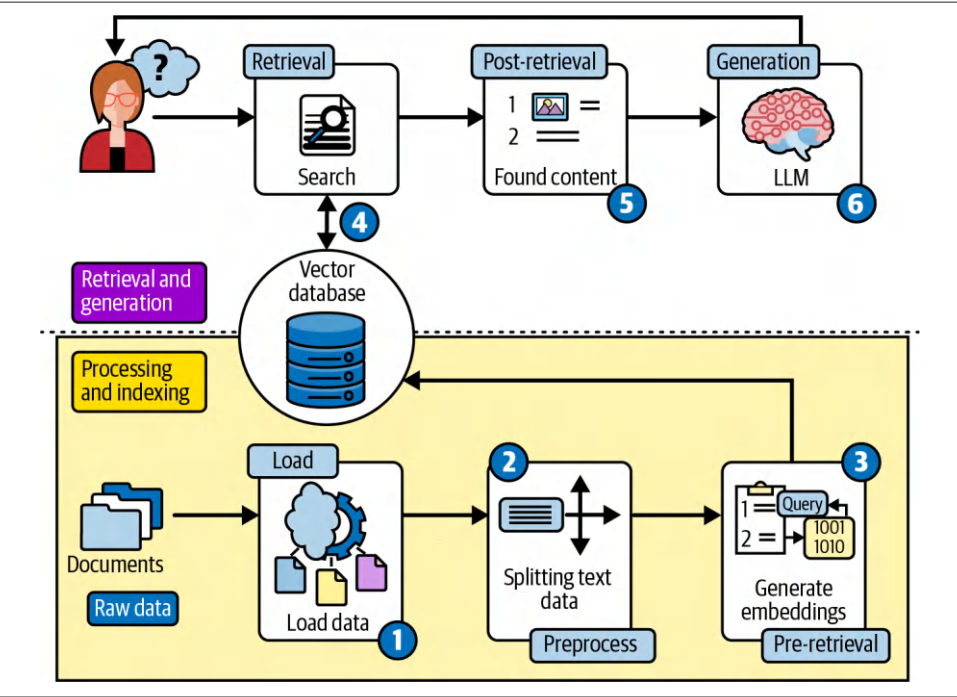


Figure 3-2. The components of a RAG system

This chapter also explains the loading process for various document types.



This book builds core components from scratch to clarify the underlying concepts. In production, orchestration frameworks such as LangChain or LlamaIndex can accelerate development, but they also introduce moving parts like frequent breaking changes, fast-evolving APIs, and additional abstractions.

If you use these frameworks, pin dependency versions, follow their upgrade guides, and isolate framework-specific code behind small adapters. For the most stable foundation, rely on mature general-purpose libraries instead (for example, pandas, NumPy, SQL-Alchemy, Pydantic, and Requests) and keep the RAG logic testable without framework coupling.

You can find all the code examples for this chapter in the [book's GitHub repository](#).

3.1 Loading Word Files in Python

Problem

You want to load Microsoft Word files into your RAG system.

Solution

Use `python-docx` for simple text extraction when you don't need to distinguish element types. Use `Unstructured` when you need to preserve document structure (headings, paragraphs, lists, images) for targeted processing of the elements.

Figure 3-3 shows the basic process when loading documents with `Unstructured`. It breaks the document into its structured elements, like a text paragraph, headline, listing, or image.

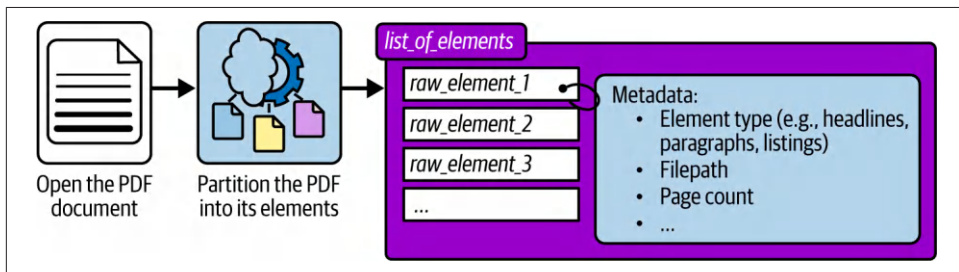


Figure 3-3. Loading Word files via `python-docx` or `Unstructured`

Both approaches are demonstrated next. The first uses `python-docx`; the second uses `Unstructured`. Both examples use `pandas` for further data processing. Install the required libraries:

```
pip install python-docx unstructured pandas
```

Now, let's use `python-docx` to load text from a sample Word file and concatenate all paragraphs into a single string:

```
import os
from docx import Document

file_path = (
    "../datasets/word_files/"
    "2023_Jan_7_Feature_Engineering_Techniques.docx"
)

doc = Document(file_path)

text = []
for paragraph in doc.paragraphs:
    text.append(paragraph.text)
```

```
full_text = "\n".join(text)
```

Figure 3-4 shows the output with line breaks preserved as `\n` characters.



```
[5] full_text
... '\n7 of the Most Used Feature Engineering Techniques\nHands-on Feature Engineering
```

Figure 3-4. The output of the loaded Word file as a string

This approach works for continuous narrative text but loses structural information. To preserve element types (titles, narrative text, lists), split and label them during loading.

Now let's try an alternative approach, which uses the `partition_docx` function from the `Unstructured` library to load the Word file, storing each element with its metadata (filepath, category, text content, last modified) in a dictionary before converting it to a pandas `DataFrame`:

```
from unstructured.partition.docx import partition_docx
import os
import pandas as pd

elements = partition_docx(filename=file_path)

list_of_elements = []

for element in elements:
    element_dict = {
        "element_id": element.id,
        "file_path": file_path,
        "category": element.category,
        # e.g., "Title", "NarrativeText", "ListItem"
        "text": element.text,
        "last_modified": element.metadata.last_modified,
    }

    list_of_elements.append(element_dict)

elements_df = pd.DataFrame(list_of_elements)
```

Figure 3-5 shows each element with its category and metadata.

elements_df.head()

[7]

Python

element_id	file_path	category	text	last_modified
56d92e2b9d10bc	../datasets/word_files/2023_Jan_7_Feature_Engi...	Title	7 of the Most Used Feature Engineering Techniques	2025-03-09T15:07:48
2a87d4b428299b	../datasets/word_files/2023_Jan_7_Feature_Engi...	Title	Hands-on Feature Engineering with Scikit-Learn...	2025-03-09T15:07:48
56a6cc20884f661	../datasets/word_files/2023_Jan_7_Feature_Engi...	NarrativeText	7 of the most used Feature	2025-03-09T15:07:48

Figure 3-5. Structured Word file output with element categories and metadata

Discussion

Structured loading enables targeted processing strategies. Titles can be assigned higher retrieval weights or indexed separately for hierarchical search. List items can be flagged for Q&A pair extraction. Metadata fields like `last_modified` enable time-based filtering during retrieval, preventing outdated information from appearing in results.

When documents contain distinct sections (like book chapters or policy domains), hierarchical retrieval reduces noise. The system first identifies relevant sections through title matching, then searches only within those sections. This two-stage approach outperforms flat text search when user queries map to specific document segments.

Avoid this approach when documents are short or homogeneous. The overhead of element-level processing pays off only when different elements require different handling or when retrieval benefits from structural filtering.



Most loading libraries like **Unstructured** or **Docling** are especially mature in loading PDFs. When dealing with various file formats, convert all to PDF first. **Recipe 3.10** covers PDF loading and processing approaches.

See Also

- The [python-docx Quickstart guide](#) covers creating and modifying Word documents programmatically.
- The [Unstructured partitioning documentation](#) explains element types, metadata extraction, and supported file formats.

3.2 Loading PDF Files

Problem

You want to load Portable Document Format (PDF) files into your RAG system.

Solution

The [PyPDF2](#) library is an open source library for splitting, merging, cropping, and transforming PDF pages. It uses [Pillow](#) to extract images.

[Figure 3-6](#) shows the pipeline: load the file, iterate over pages, extract text, and store it as metadata.

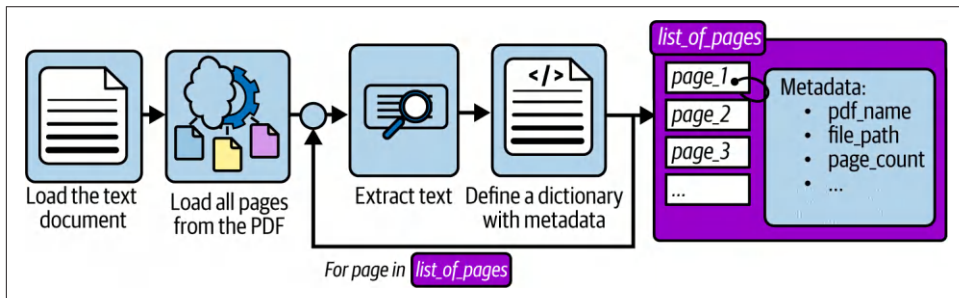


Figure 3-6. Loading PDF files and storing the text and metadata

First, you use `PyPDF2.PdfReader` to load the PDF and extract text and metadata from each page. This example uses a sample PDF document about feature engineering techniques, which is available in the book's code repository:

```
import PyPDF2
import os
import pandas as pd

file_path = (
    "../datasets/pdf_files/"
    "2023_Jan_7_Feature_Engineering_Techniques.pdf"
)

with open(file_path, "rb") as file:
```

```

reader = PyPDF2.PdfReader(file)

list_of_pages = []
page_counter = 1

for page in reader.pages:
    page_dict = {
        "file_name": reader.metadata.get("/Title"),
        "producer": reader.metadata.get("/Producer"),
        "page_number": page_counter,
        "text": page.extract_text(),
        "images": page.images,
    }

    list_of_pages.append(page_dict)

    page_counter += 1

pages_df = pd.DataFrame(list_of_pages)

```

Figure 3-7 shows a printout of the resulting DataFrame.

<pre># Display the first few rows of the DataFrame pages_df.head()</pre>					Python
file_name	producer	page_number	text	images	
Skia/PDF m131 _Techniques	Google Docs Renderer	1	7\nof\nthe\nMost\nUsed\nFeature\nEngineering\n...	[File(name=X7.png, data: 1.8 kB)]	
Skia/PDF m131 _Techniques	Google Docs Renderer	2	3.2\nBucketizing\nusing\nTensorflow\n3.3\nBuck...	[]	
Skia/PDF m131 _Techniques	Google Docs	3	A\nstandard\nMachine\nLearning\npipeline—Ins...	[File(name=X17.png, data: 699 Byte)]	

Figure 3-7. The output of the loaded PDF file as a DataFrame

Discussion

PyPDF2 extracts text from digitally generated PDFs containing text that exists as selectable characters. The library fails on scanned PDFs or image-based documents with text that appears as pixels, not characters. For those cases, use optical character recognition (OCR) engines (see [Recipe 3.6](#)) or multimodal models (see [Recipe 3.10](#)).

Page-level metadata enables filtered retrieval. You can exclude pages by date, author, or section before running semantic search, reducing computation and improving relevance. The metadata also provides source attribution, allowing users to verify information by navigating to specific pages in the original document.

Use PyPDF2 when your PDFs are digitally generated (exported from Word, LaTeX, or web browsers). Its simplicity and speed make it ideal for processing large document collections where OCR overhead would be prohibitive.

See Also

- The [PyPDF2 text extraction guide](#) shows methods for extracting text from digitally generated PDFs.
- The [PyPDF2 image extraction guide](#) covers extracting embedded images and their metadata.
- The [Unstructured partitioning documentation](#) explains splitting PDFs into structured elements with metadata.

3.3 Loading and Handling Tabular Data from Excel and CSV Files

Problem

You want to load tabular data from Microsoft Excel or comma-separated values (CSV) files into your RAG system.

Solution

This recipe demonstrates three approaches for handling tabular data in RAG systems: converting table rows to text, embedding tables in prompts, and using a text-to-SQL approach. See [Chapter 11](#) for more complete implementations of the third option.

Option 1: Converting table rows to text

This approach transforms each table row into a natural language description. Column headers become context labels, and values are integrated into readable text. [Figure 3-8](#) illustrates the concept.

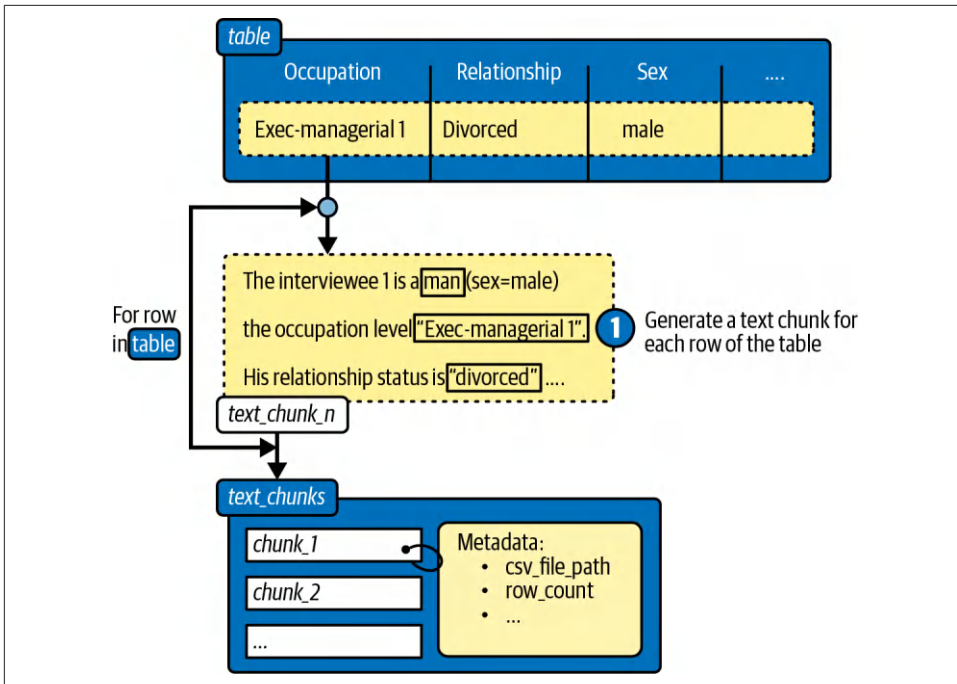


Figure 3-8. Loading tabular data by transforming table rows into text

This example uses the **Census Income dataset**, which is commonly used for binary classification tasks.

Download the dataset and save it as an Excel file containing 15 columns and 48,000 rows. You'll use the `openpyxl` library to open and load the Excel file, and `pandas` to process the data; install both as follows:

```
pip install openpyxl pandas
```

Next, define a new function `create_text_description` and apply it to each row of the `DataFrame` in order to create a new column, `text_description`, containing a text snippet explaining the values in the other columns:

```
import os
import pandas as pd

file_path = "../datasets/csv_files/census-income.xlsx"
df_excel = pd.read_excel(io=file_path)

def create_text_description_of_row(row):
    row["text_description"] = (
        f""The candidate {row['age']} years old is working in the
        {row['workclass']} sector. The candidate was born in
```

```

        {row['native-country']], is {row['marital-status']]
        and has a {row['relationship']] relationship.
        The candidate has a {row['education']] degree and is
        working as a {row['occupation']]. The income of the
        candidate is {row['income']}."""
    )

    return row

```

```

# Apply the function create_text_description_of_row
# to each row of the DataFrame
df_extended = df_excel.apply(create_text_description_of_row, axis=1)

```

Figure 3-9 shows the content of the newly created `text_description` column. Each row is a text snippet that describes the values in all the other columns.

```

[11]
... 0   The candidate 39 years old is working in the\n...
    1   The candidate 50 years old is working in the\n...
    2   The candidate 38 years old is working in the\n...
    3   The candidate 53 years old is working in the\n...
    4   The candidate 28 years old is working in the\n...
    Name: text_description, dtype: object

```

Python

Figure 3-9. Text snippets generated from tabular data rows

These text snippets can be treated like any other text by generating embeddings for them and storing them in a vector database. This works well for lookup queries, but remember that aggregate queries (percentages, averages, counts) require option 3 (text-to-SQL) instead.

Option 2: Embedding tables in prompts

As an alternative to converting rows into individual text snippets, you can embed the entire table directly into the LLM prompt. This works best for small tables whose full context fits into the model's context window.



LLMs are not designed for numerical computation or complex data analysis. However, modern LLMs can often recognize simple patterns, compare values across rows and columns, and summarize trends at a useful level. Because the model sees the complete table, it can identify relationships that would be lost if only a few rows were retrieved.

Figure 3-10 shows an example prompt template. The table is inserted into the prompt by using Markdown syntax. A common approach is to include the entire table in Markdown format.

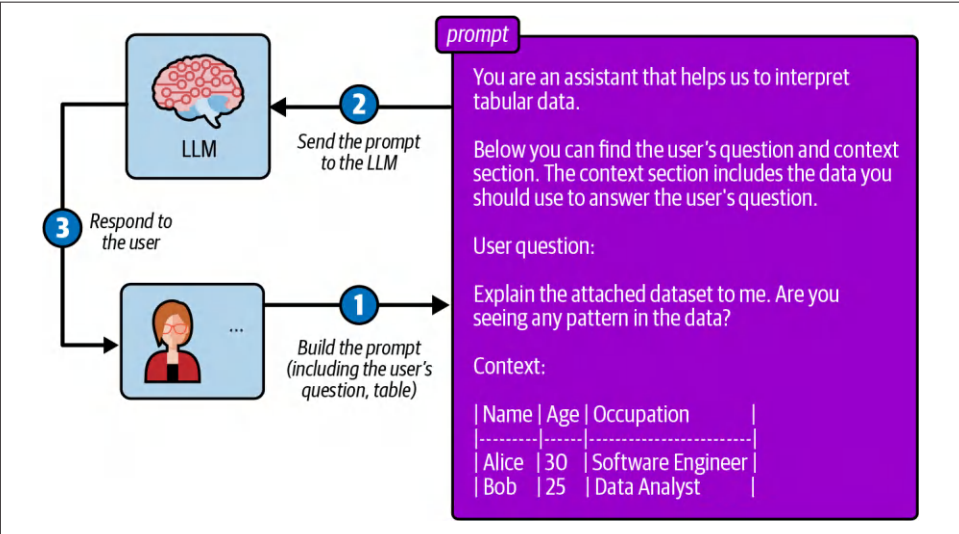


Figure 3-10. Loading tabular data by embedding the whole table into the prompt

Keep in mind that this approach is limited by the model's context window and can become expensive for tables with more than a few hundred rows. For larger tables or complex analytical queries, option 3 provides a more scalable solution.

Option 3: Using text-to-SQL for complex queries

For complex analytical queries or large datasets, the text-to-SQL approach is the most powerful option. The tabular data is uploaded to a SQL database. When a user asks a question, an LLM analyzes the table schema and the user's question to generate an appropriate SQL query. The query results are then interpreted by the LLM to produce a natural language answer.

Figure 3-11 illustrates this process at runtime in your RAG app.

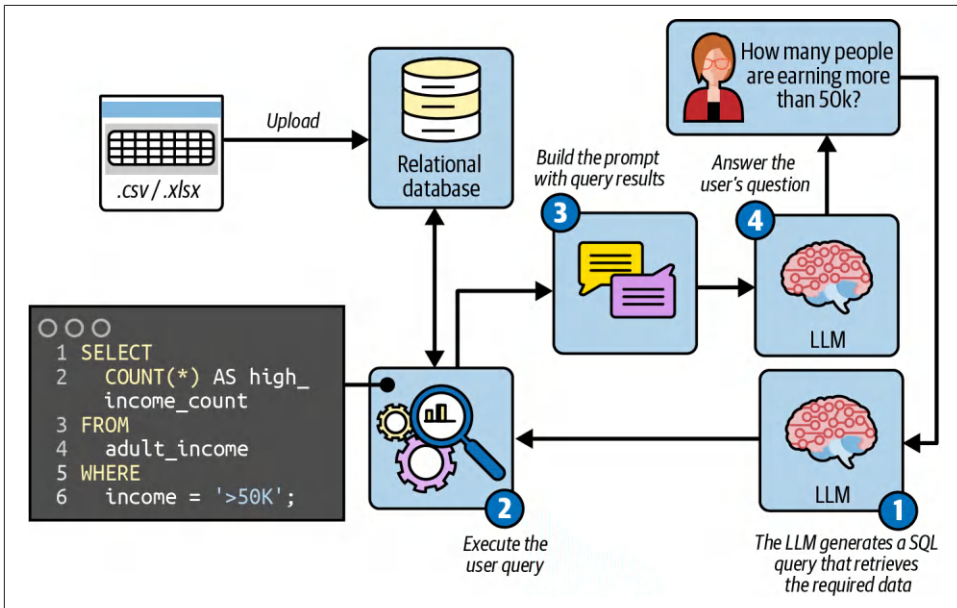


Figure 3-11. Loading tabular data by uploading it to a SQL database and using the text-to-SQL approach to answer user questions about the data

For a more comprehensive text-to-SQL implementation, see [Recipe 3.4](#) for database connectivity and [Chapter 11](#) for a full end-to-end text-to-SQL RAG application.

Discussion

Three fundamentally different approaches exist for handling tabular data in RAG. Each suits different query types and dataset sizes. [Table 3-1](#) summarizes the trade-offs.

Table 3-1. Choosing the right approach for tabular data in RAG

Approach	Best for	Limitations
Option 1: Row-to-text	Simple lookup queries on medium datasets (hundreds to thousands of rows). Example: "What is Sarah's job title?"	Cannot handle aggregate queries, percentage calculations, or comparisons across rows.
Option 2: Embedded table in prompt	Small tables (< 100 rows) where you need full context for analysis. Example: "Which employee has the highest salary in this table?"	Limited by the model context window. Expensive for large tables. Poor performance with complex queries on large data.
Option 3: Text-to-SQL	Large datasets requiring aggregations, filtering, or complex analysis. Example: "What percentage of engineers earn over \$100k?"	Requires database setup and SQL query generation. More complex implementation.

The key trade-off is retrieval scope versus computational cost. Row-to-text retrieves specific records through semantic search but cannot answer aggregate questions since

each row is indexed separately. Prompt embedding provides full table context for analysis but becomes prohibitively expensive beyond a few hundred rows. Text-to-SQL handles arbitrarily large datasets and complex analytics but requires SQL infrastructure and query generation logic.

Production systems often combine approaches based on table size and query patterns. Small reference tables (product catalogs, employee rosters) embed directly in prompts. Medium lookup datasets (customer profiles, transaction logs) use row-to-text. Large analytical datasets (sales history, sensor readings) require text-to-SQL.



The practical example in this recipe converted each row of the dataset into a text snippet. You can use the same approach for classification tasks. For more details, have a look at the paper “[TabLLM: Few-Shot Classification of Tabular Data with Large Language Models](#)” by Stefan Hegselmann et al.

See Also

- The [LlamaIndex Excel loader](#) offers built-in parsing for Excel and CSV files.
- The [LangChain Excel loader](#) provides document loader integration for tabular data.

3.4 Loading Structured Data from a PostgreSQL Database

Problem

You want to connect your Python workflows to a PostgreSQL database.

Solution

Install PostgreSQL and [pgAdmin](#). Download installers from [postgresql.org/download/](https://www.postgresql.org/download/).



Local installations suit quick experiments. For production, use managed PostgreSQL services (e.g., Amazon Aurora) for backups, patching, monitoring, and high availability. For smaller projects, [Amazon Lightsail](#) offers fixed monthly pricing.

Use pgAdmin to inspect schemas, browse tables, and run queries. For multiple database engines, consider multidatabase clients like **DBeaver** or **DataGrip**, which support PostgreSQL, MySQL, SQLite, Oracle, IBM Db2, and Microsoft SQL Server.

Connect to PostgreSQL from Python via **SQLAlchemy**, which uses the `psycopg2`-binary adapter. Install the required libraries:

```
pip install SQLAlchemy psycopg2-binary pandas python-dotenv
```

Launch pgAdmin, create a local server, and populate it with sample data from *w3schools.com*.

Next, create an engine for executing SQL commands in PostgreSQL:

```
import os
import pandas as pd
from sqlalchemy import create_engine

connection_string = (
    f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}"
)
engine = create_engine(connection_string)

with engine.connect() as connection:
    query = """SELECT * FROM categories ORDER BY category_id ASC """
    result = pd.read_sql(query, connection)
    print(result)
```

Once populated, `results_df` returns a DataFrame with all `categories` table records sorted by `category_id`.

Discussion

PostgreSQL serves dual roles in RAG applications. It stores structured application data (user sessions, query logs, feedback) while also functioning as a vector store through the `pgvector` extension. This consolidation reduces infrastructure complexity compared to maintaining separate databases for relational and vector data.

Use PostgreSQL when your queries require joining structured filters with vector searches. Examples include filtering customer embeddings by account tier, joining product descriptions with inventory status, or excluding documents based on user permissions stored in relational tables. PostgreSQL excels when your RAG app already uses it for transactional operations.

Specialized vector databases like Pinecone, Qdrant, or Weaviate offer better performance for pure similarity search workloads without relational joins. They provide simpler APIs and optimizations specifically for vector operations. Choose them when you don't need to combine vector search with complex SQL queries or when maximum vector search throughput is critical.

See Also

- The [SQLAlchemy Engine Configuration guide](#) covers connection strings, pooling, and dialect-specific options.
- The [SQLAlchemy tutorial with PostgreSQL examples](#) provides practical code for queries and database operations.

3.5 Loading Audio Files via Speech-to-Text Models

Problem

You want to load and preprocess audio files so you can add them to your RAG system's vector store.

Solution

Use OpenAI Whisper to transform audio into text. Whisper is available as both open source software and a hosted API.

In this example, you use OpenAI's SDK to send an audio file to the Whisper API endpoint:

```
import os
from openai import OpenAI

audio_file_path = "../datasets/audio_files/harvard.wav"

# Initialize the OpenAI client with your API key
client = OpenAI()

with open(audio_file_path, "rb") as audio_file:
    transcription = client.audio.transcriptions.create(
        model="whisper-1", file=audio_file
    )
```

Discussion

Whisper is available as both open source software (run locally) and a hosted API (via OpenAI). Choose based on data sensitivity and infrastructure:

Whisper API

Quick setup, supports 80+ languages, no infrastructure management. Use when data privacy permits external API calls and you want to minimize operational overhead.

Whisper locally

Full data control, no external dependencies. Use for sensitive audio that cannot leave your environment or when API costs exceed local compute costs at scale.

GPT-4o transcribe API

Higher accuracy than Whisper API, better punctuation and formatting, improved handling of technical terms. Use when transcription quality justifies higher per-minute costs.

Whisper handles batch transcription well but lacks features needed for specialized applications. Consider dedicated speech-to-text providers when you require these features:

- Real-time streaming for live captioning, voice assistants, or conversational AI when subsecond latency matters
- Speaker diarization to identify who spoke when in multispeaker recordings, critical for meeting summaries and interview analysis
- Custom vocabularies to improve accuracy on domain-specific terminology (medical procedures, legal terms, company names, technical jargon)

Table 3-2 compares major services.

Table 3-2. Popular speech-to-text alternatives

Service	Key features	Link
Google Cloud Speech-to-Text	Advanced speech models with automatic punctuation, speaker separation, custom vocabularies, and multichannel audio support	https://docs.cloud.google.com/speech-to-text/docs/overview
Amazon Transcribe	Fully managed speech-to-text service with automatic punctuation, custom vocabularies, language detection, speaker separation, and redaction of sensitive information	https://aws.amazon.com/transcribe/
Microsoft Azure Speech	Speech-to-text, text-to-speech, and translation with support for custom models and optional on-device deployment	https://learn.microsoft.com/en-us/azure/ai-services/speech-service/speech-to-text
Deepgram Speech-to-Text	Voice AI platform for real-time and batch speech-to-text with high accuracy, speaker separation, and punctuation	https://deepgram.com/product/speech-to-text
AssemblyAI Speech-to-Text	Speech-to-text for recorded and live audio with features such as speaker identification, automatic formatting, and language detection	https://www.assemblyai.com

See Also

- The **Hugging Face Whisper-Large-V3 model** provides open source transcription with code examples.

- The [OpenAI speech-to-text API documentation](#) covers API usage, model options, and parameter tuning.

3.6 Extracting Text from Images and PDFs via Tesseract OCR

Problem

You want to extract text from images or image-like documents by using open source OCR engines you can host on your own infrastructure.

Solution

This recipe extracts text by using *optical character recognition* (OCR) engines, which are traditional machine learning models trained for image-to-text conversion.

The workflow applies to any kind of images with written text. [Figure 3-12](#) shows the end-to-end process:

1. Convert documents to PDF format.
2. Render each PDF page as an image.
3. Run OCR on each page image and store the extracted text with metadata (page number, image path, timestamps, PDF reference).

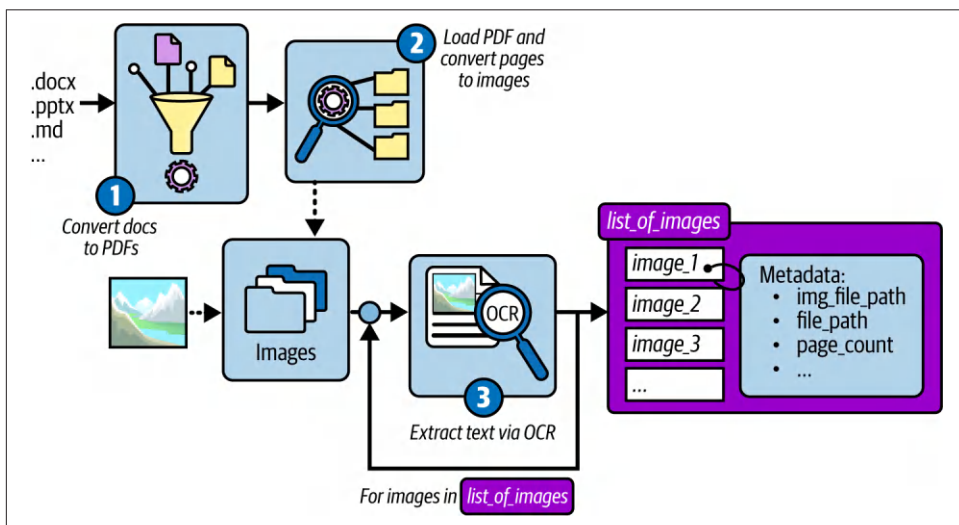


Figure 3-12. OCR engines can extract text from images (e.g., from photographs, technical drawings, or scanned contracts)

This recipe uses the most popular open source OCR engine, **Tesseract**. You can install it locally or use a managed OCR service.

You can find installation guides for various operating systems in the **Tesseract documentation**. If you're on Windows, you can use the **Windows installer** provided by the University of Mannheim.

On top of that, you need to install the Python packages `pdf2image`, `Python-tesseract`, and `Pillow`. Note that the `pdf2image` library depends on `Poppler` for PDF processing. The **pdf2image docs** provide installation instructions for `Poppler` for various operating systems. Use the following command to install the packages:

```
pip install pdf2image pytesseract pillow
```

Next, load a sample PowerPoint slide with finance data and extract the written text:

```
import os
from pdf2image import convert_from_path
from PIL import Image
import pytesseract

image = Image.open(
    fp="../datasets/images/example_finance_reporting_slide.png"
)

text = pytesseract.image_to_string(image)
```

If you have a scanned PDF, you can load it and convert each page into an image, and then use Tesseract to extract the text from each page:

```
import os
from pdf2image import convert_from_path
from PIL import Image
import pytesseract

file_path = (
    "../datasets/pdf_files/"
    "2023_Jan_7_Feature_Engineering_Techniques.pdf"
)

images = convert_from_path(pdf_path=file_path)

text = []
for i, image in enumerate(images):
    page_text = pytesseract.image_to_string(image)
    text.append(page_text)
```

The variable `text` contains a list of strings, each representing the extracted text from a page of the PDF.



For reliable source attribution, store traceable metadata alongside every extracted text chunk. At a minimum, capture the original PDF path, the derived image path, the page number, and any extraction timestamps or IDs. This lets you link retrieval results back to the exact document and page later (for citations, debugging, and reprocessing).

Discussion

Traditional OCR engines like Tesseract run locally, process documents quickly, and incur no per-page costs. They work well for clean text scans with standard fonts and simple layouts (contracts, forms, typed letters). Accuracy degrades on complex documents with tables, diagrams, mixed fonts, handwriting, or low-quality scans.

Multimodal models handle these complex cases by understanding layout, interpreting tables, and processing diverse visual elements simultaneously. The trade-off is cost. Multimodal API calls typically cost 100 times more per page than running Tesseract locally. At scale, this difference becomes substantial.

Production systems processing large document volumes often use a hybrid approach. Route simple text-heavy documents through OCR, and complex documents (those with tables, diagrams, or degraded quality) to multimodal models. A document classifier or layout analyzer can automate this routing, maximizing quality while controlling costs.

This list outlines when to use which approach:

Tesseract or similar OCR

Clean scans, typed text, high volume (thousands to millions of pages), cost-sensitive applications, or environments where data cannot leave your infrastructure

Newer OCR engines (EasyOCR, PaddleOCR)

Improved accuracy on handwriting and complex layouts while maintaining local deployment benefits

Multimodal models

Mixed content with tables and diagrams, handwritten notes, low-quality scans, or critical documents where extraction errors have high downstream costs (financial reports, legal contracts)

Table 3-3 shows open source OCR alternatives to Tesseract with improved capabilities.

Table 3-3. Open source alternatives to Tesseract

Tool	Key features	Official website
EasyOCR	Supports 80+ languages, GPU acceleration, Python-based implementation	https://oreil.ly/Bu4Fo
PaddleOCR	High accuracy with layout analysis, handwriting support, production-ready	PaddleOCR official documentation
docTR	Deep learning based, modular architecture, supports both PyTorch and TensorFlow	https://oreil.ly/28BDV
TrOCR	Transformer-based architecture, excels at handwritten text recognition	https://oreil.ly/Y2RF2

To help you choose the right approach for your use case, [Table 3-4](#) explains the decision criteria based on document characteristics, volume, and budget constraints.

Table 3-4. Decision guide: OCR versus multimodal models

Document type	Volume	Recommended approach
Simple text-only PDFs: plain documents, contracts, books, articles	< 1,000 docs/month	OCR (Tesseract): fast, free, runs locally
Simple text-only PDFs: plain documents, contracts, books, articles	> 10,000 docs/month	OCR (Tesseract or EasyOCR): cost-effective at scale
Mixed content: text + tables + images	< 500 docs/month	Multimodal models (GPT-5 mini, Claude Haiku, Gemini Flash): single-pass processing, robust results
Mixed content: text + tables + images	> 5,000 docs/month	Hybrid approach: classify documents first, use OCR for simple pages, multimodal for complex pages
Complex layouts: technical diagrams, handwritten notes, mixed fonts	Any volume	Multimodal models (GPT-5.2, Claude Sonnet, Gemini Pro): superior accuracy on complex documents
Sensitive data: cannot leave infrastructure	Any volume	OCR (open source: Tesseract, PaddleOCR, EasyOCR): full control, local deployment



For regulated or sensitive data, avoid external APIs. Run ingestion on your own infrastructure and lock down access to raw files and extracted text (least privilege, auditing, encryption).

Start with local OCR (Tesseract, EasyOCR, PaddleOCR). If you need layout-aware extraction (tables/forms/reading order), use a locally hosted vision or document model like Llama 3 Vision or Qwen-VL.

See Also

- The [Tesseract OCR documentation](#) covers installation, language support, and parameter tuning for accuracy.
- The [Python-tesseract package documentation](#) provides Python integration examples and an API reference.
- The [Google Cloud Vision OCR guide](#) covers complex layouts and handwriting with managed infrastructure.

3.7 Extracting Text from Images via Multimodal Models

Problem

You want to extract text, tables, and code listings from images without losing the original formatting.

Solution

Attach the image to a prompt and instruct the model to extract the content. You can instruct the model to return the text in Markdown format that reflects the original structure, including headings, bullet lists, tables, or code blocks.



Save important metadata like image path, timestamps, and any IDs along with the source document.

Figure 3-13 shows how to use multimodal models to extract text from images.

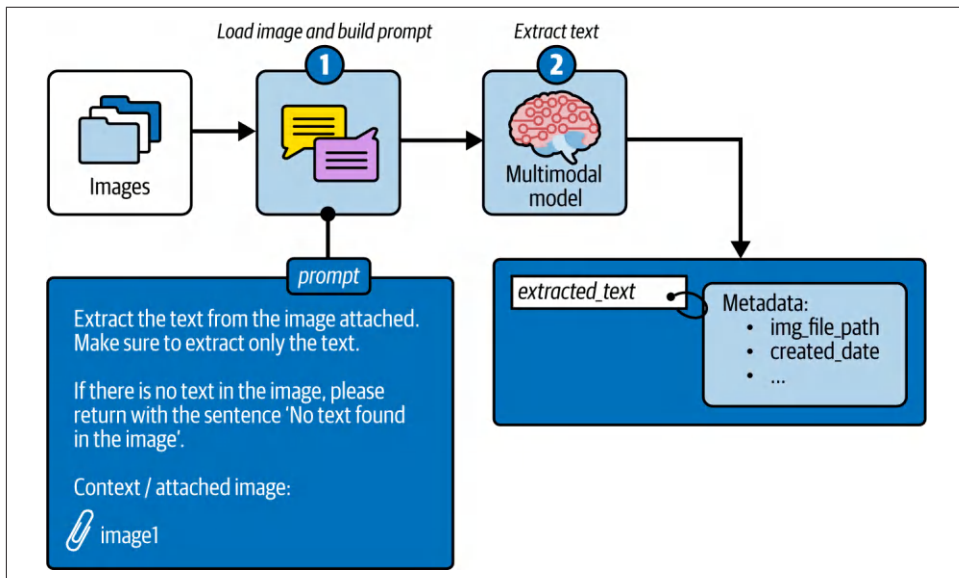


Figure 3-13. Extracting text from images via multimodal models

To start, define a sample prompt and send it to one of OpenAI's multimodal models along with an image of a financial reporting slide:

```
import os
import base64
from openai import OpenAI

png_file_path = "../datasets/images/example_finance_reporting_slide.png"

# Initialize the OpenAI client
client = OpenAI()

with open(png_file_path, "rb") as image_file:
    base64_image = base64.b64encode(image_file.read()).decode("utf-8")

    prompt = (
        "Extract the text from the image attached. Make sure to only "
        "extract only the text. If there is no text in the image, "
        "please return with the sentence 'No text found in the image.'"
    )

    response = client.chat.completions.create(
        model="gpt-5.2", # Define the model to use
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt},
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": (
                                f"data:image/jpeg;base64,"
                                f"{base64_image}"
                            ),
                        },
                    },
                ],
            },
        ],
        max_tokens=500,
    )

    content = response.choices[0].message.content
```

The content variable contains the extracted text from the image, preserving the original structure as closely as possible.

Discussion

Multimodal models extract text while preserving document structure. They can output Markdown that reflects the original formatting including headings, bulleted lists, tables, and code blocks. This structural preservation improves downstream processing compared to raw text extraction.

Cost scales linearly with document volume. Efficient multimodal models (GPT-5 mini, Claude Haiku 4.5, Gemini 2.5 Flash) balance cost and quality for most extraction tasks. Flagship models (GPT-5.2, Claude Sonnet 4.5, Gemini 3 Pro) provide better accuracy on complex documents but at higher per-page costs.

The following list details the use cases for each tier:

Efficient models

Straightforward documents with clear text and simple layouts, high-volume processing where cost matters

Flagship models

Dense tables, technical diagrams, mixed content types, or critical documents where extraction errors have significant consequences (financial statements, legal contracts, medical records)

Classical OCR

Plain-text documents at very high volume where cost sensitivity outweighs the benefits of structural understanding

Multimodal models may hallucinate numbers or skip content in low-quality scans. Improve accuracy by providing specific extraction instructions (“extract product name, price, and weight from each row”) or using multiple models for cross-validation on critical documents.

See Also

- The [OpenAI vision documentation](#) covers sending images to GPT models and includes code examples.
- The [Anthropic vision documentation](#) explains Claude’s image analysis capabilities and parameters.
- The [Gemini vision API documentation](#) provides image examples and model selection guidance.

3.8 Generating Text Description for Images via Multimodal Models

Problem

You want to extract insights from images by generating detailed textual descriptions.

Solution

Figure 3-14 shows the process and a sample prompt you can use. The prompt instructs the model to generate a detailed description of an image.

You can further enrich the resulting metadata with the following:

- The filepath of the original document
- The page where the image was found
- A link to the text sections surrounding the image

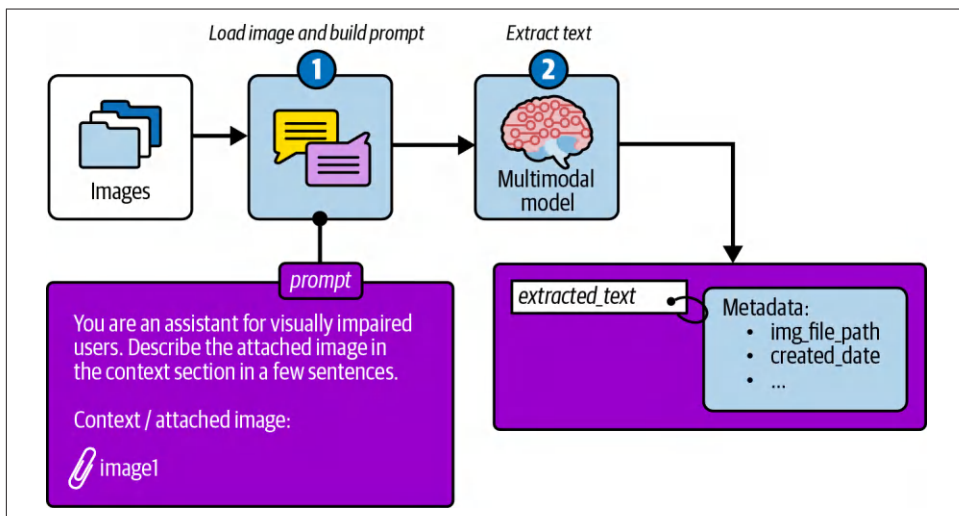


Figure 3-14. Using multimodal models to generate text summaries for images

This recipe uses the multimodal model GPT-5 mini to analyze images. Alternatively, you can use Anthropic’s Claude Haiku 4.5, Google’s Gemini 2.5 Flash, or other multimodal models from xAI, Mistral, or DeepSeek.

Load an example image of Vietnam’s capital city (Hanoi) and instruct the model accordingly:

```

import os
import base64
from openai import OpenAI

image_path = "../datasets/images/vietnam.png"

# Initialize the OpenAI client
client = OpenAI()

with open(image_path, "rb") as image_file:
    base64_image = base64.b64encode(image_file.read()).decode("utf-8")

    prompt = (
        "You are an assistant for visually impaired users. "
        "Describe the image in detail."
    )

    response = client.chat.completions.create(
        model="gpt-5-mini",
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt},
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": (
                                f"data:image/jpeg;base64,"
                                f"{base64_image}"
                            ),
                        },
                    },
                ],
            },
        ],
        max_tokens=150,
    )

    content = response.choices[0].message.content

```

The output of the model is a detailed description of the image, which can be stored as metadata alongside the image for future reference.

Discussion

Multimodal models adapt their output to image content. They describe emotions in portraits, explain structural relationships in diagrams, summarize data trends in charts, and extract text from documents. [Figure 3-15](#) illustrates this content-aware flexibility when processing mixed visual content without being able to predict image types in advance.

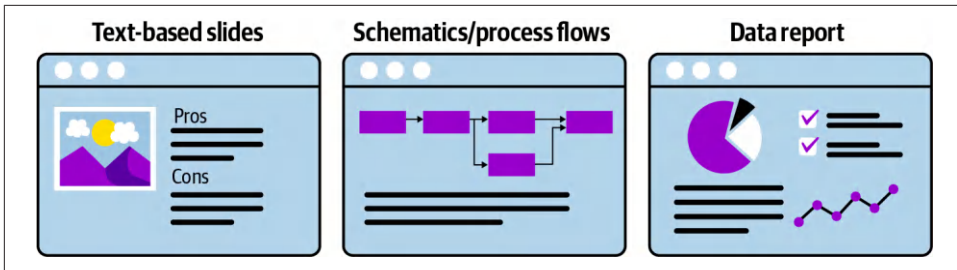


Figure 3-15. Extracting text from various image types

This recipe converts images to text descriptions first, then generates embeddings by using text embedding models. An alternative approach uses multimodal embedding models like OpenAI's CLIP that process images and text directly in the same vector space. CLIP enables direct image-to-image search and cross-modal retrieval (a text query matching images, or vice versa) without intermediate text conversion. See [Recipe 5.5](#) for implementation details.

See Also

- The [OpenAI CLIP repository](#) has the official implementation of Contrastive Language–Image Pre-training for multimodal embeddings.
- The [LangChain multimodal RAG tutorial](#) demonstrates combining text and image embeddings with code examples.
- The [LlamaIndex multimodal retrieval guide](#) provides patterns for indexing and retrieving multimodal content.

3.9 Generating Text Summaries for Embedded Tables via Multimodal Models

Problem

You want to load documents with embedded tables and preprocess them for your RAG system by converting the tables into text summaries.

Solution

[Figure 3-16](#) shows the steps to prepare the tables:

1. Identify and extract tables from the document.
2. Build a proper prompt by combining the table with instructions.

3. Send the prompt to the LLM to generate a text summary.

The summary is then stored together with important metadata. If the table comes from a PDF, the metadata might include the page number where the table appears and the filepath to the original PDF.

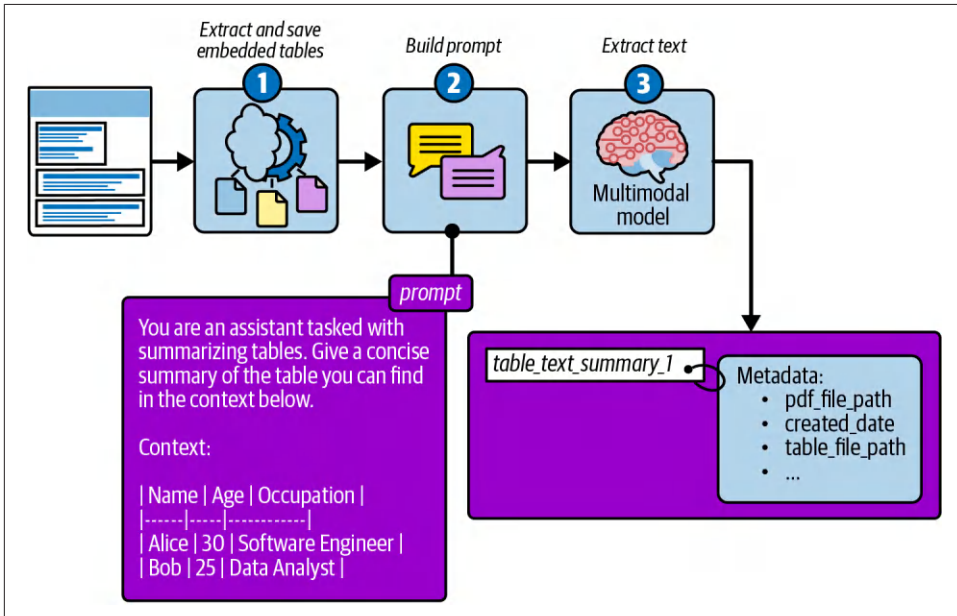


Figure 3-16. Using multimodal models to generate text summaries for tables

This example uses the Unstructured library to break a PDF into text, tables, and images. For interpreting tables, this recipe again uses OpenAI models. Install these dependencies:

```
pip install unstructured openai pandas
```

Load the PDF, partition it, and store all tables in a new list called tables:

```
import os
from unstructured.partition.pdf import partition_pdf

pdf_file_path = "../datasets/pdf_files/adult_data_article.pdf"

tables = []
texts = []

# Partition the PDF file into its elements
raw_pdf_elements = partition_pdf(
    filename=pdf_file_path,
    strategy="hi_res",
)
```

```

for element in raw_pdf_elements:
    if "unstructured.documents.elements.Table" in str(type(element)):
        tables.append(str(element))

```

Next, summarize the table content with a suitable LLM:

```

from openai import OpenAI
import pandas as pd

def summarize_tables(row):
    summary_prompt = (
        f"You are an assistant tasked with summarizing tables. "
        f"Give a concise summary of the table. "
        f"Table chunk: {row.table}"
    )

    # Initialize the OpenAI API client and generate table summary
    client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
    response = client.chat.completions.create(
        model="gpt-5-mini",
        messages=[{"role": "user", "content": summary_prompt}],
        temperature=0.7,
        max_tokens=150,
    )

    row["table_summary"] = response.choices[0].message.content

    return row

# Create a pandas dataframe from the tables
tables_df = pd.DataFrame(tables, columns=["table"])

# Add a column to the dataframe to store the summaries
tables_df = tables_df.apply(summarize_tables, axis=1)

```

The result is a DataFrame containing the summaries and all metadata fields.

Discussion

Converting tables to text summaries captures semantic meaning that raw table strings miss. Consider a quarterly revenue table. Embedding the raw string “Q1: 100, Q2: 120, Q3: 140” captures only numbers without the insight that these figures represent 20% quarter-over-quarter growth.

A text summary like “Revenue increased by 20% each quarter, rising from \$100M in Q1 to \$140M in Q3, indicating strong growth momentum” captures the pattern, trend, and business implication. When a user asks, “Which companies show

consistent revenue growth?,” the summary’s embedding matches semantically, while the raw table string does not.

This approach works well in these circumstances:

- Tables contain numeric data where patterns and trends matter (financial results, performance metrics, time series).
- Users ask analytical questions rather than looking up specific cells.
- The table insight can be expressed in two or three sentences.

Avoid this approach in these circumstances:

- Tables are primarily lookup data (employee directories, product specifications) where row-to-text conversion suffices.
- Tables are large reference datasets better suited for text-to-SQL.
- Users need exact cell values rather than summarized insights.

The trade-off is accuracy versus interpretability. Raw table text preserves exact values but lacks context. Summaries provide context but may miss details or introduce interpretation errors.

See Also

- The [OpenAI PDF parsing guide](#) shows using GPT-5.2 to identify and extract tables and diagrams from PDFs.
- A [Pinecone multimodal RAG example](#) demonstrates letting the model classify content types and extract information.

3.10 Parsing PDFs with Multimodal Content

Problem

You want to load and preprocess a PDF that contains text, listings, tables, and images so you can ingest it into your RAG system.

Solution

Figure 3-17 shows the end-to-end process for loading a PDF with multimodal content. It requires a multistep pipeline that partitions the PDF into its element types and processes each type differently:

1. Load the raw document and partition it into text, images, and tables.

2. Use a multimodal model (for example, OpenAI GPT-5.2, Anthropic Claude Sonnet 4.5, or Google Gemini 2.5 Flash) to generate text summaries for images and tables.
3. Generate text embeddings for text chunks, image summaries, and table summaries.

Then you add the embeddings to the vector store and store metadata such as the original filepath and page numbers.

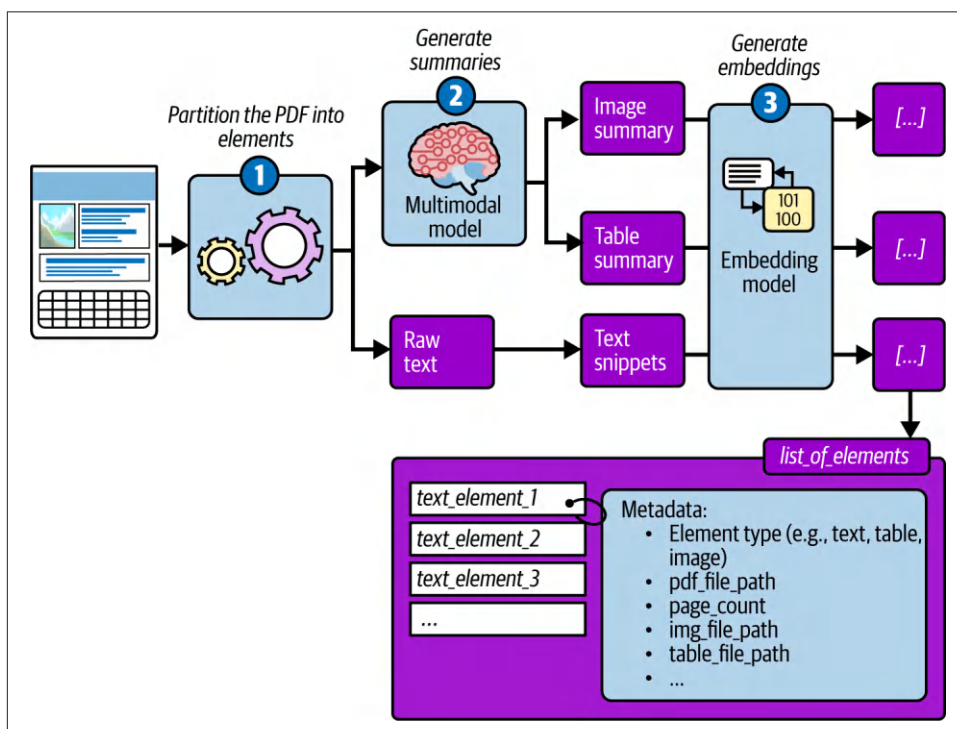


Figure 3-17. Loading and processing multimodal PDFs

This example uses the Unstructured library to split the PDF into text, images, and tables. This library relies on the Poppler utilities for PDF processing, especially for tasks such as extracting images and reading page-level information.

The way you install Poppler depends on your operating system:

Windows

Download the latest-release ZIP from the Poppler for Windows, extract it to a directory on your machine, and add that directory to your PATH environment variable. You'll find some useful links in the [pdf2image docs](#).

Ubuntu/Debian

Install Poppler as follows:

```
sudo apt-get install poppler-utils
```

To handle image-based documents such as scanned PDFs, Unstructured uses an OCR engine. This example uses Tesseract, so make sure you have Tesseract installed as well.

Then partition the PDF into its elements. Each element has a category such as narrative text, titles, images, or tables:

```
from unstructured.partition.pdf import partition_pdf
import os

# Set the OCR agent to tesseract
os.environ["OCR_AGENT"] = "tesseract"

pdf_file_path = "../datasets/pdf_files/adult_data_article.pdf"
image_output_dir = "../datasets/extracted_content_from_pdfs/images"

# Create output directory if it doesn't exist
os.makedirs(image_output_dir, exist_ok=True)

# Get elements by using the function extract_pdf_elements
raw_pdf_elements = partition_pdf(
    filename=pdf_file_path,
    extract_images_in_pdf=True,
    extract_image_block_types=["Image", "Table"],
    extract_image_block_to_payload=False,
    extract_image_block_output_dir=image_output_dir,
)

# Categorize elements by type
tables = []
texts = []
titles = []

# Fill the just created lists with the elements
for element in raw_pdf_elements:
    element_type = str(type(element))
    if "unstructured.documents.elements.Table" in element_type:
        tables.append(str(element))
    elif "unstructured.documents.elements.NarrativeText" in element_type:
        texts.append(str(element))
    elif "unstructured.documents.elements.Title" in element_type:
        titles.append(str(element))
```

The result is three lists, one per element type. Each element is then prepared further. [Table 3-5](#) shows some popular processing steps for each element type.

Table 3-5. Processing steps for PDF element types

Element type	Recommended processing steps
Text and titles	Split into smaller text chunks suitable for embedding models. Use techniques like recursive, semantic, and agentic chunking.
Images	Extract text via OCR engines or multimodal models. Alternatively, generate text summaries for images via multimodal models.
Tables	Load the plain text from the table or, preferably, extract key insights via multimodal models.



This recipe shows how to partition PDFs into elements. For the complete processing implementation (generating summaries, creating embeddings, and storing in vector stores), see [Recipe 3.7](#) for image processing and [Recipe 3.9](#) for table processing.

Discussion

PDFs frequently contain mixed content: narrative text, embedded images, data tables, and diagrams. Text embedding models process only text, making visual elements invisible to semantic search unless converted first.

The solution is modal conversion. Multimodal models analyze images and tables, generating text descriptions that capture their content and meaning. These descriptions are then embedded as text, making visual content discoverable through semantic search.

This enables comprehensive retrieval. A user query about “quarterly sales trends” can match both narrative paragraphs discussing growth and embedded tables showing quarterly figures, because both exist as searchable text embeddings.

Use this approach for these scenarios:

- Documents contain critical information in visual formats (charts, diagrams, tables).
- Users ask questions that span text and visual content.
- You need to preserve document fidelity rather than just extracting plain text.

Skip this complexity in these circumstances:

- Documents are text-only, or visual elements are decorative rather than informative.
- Visual content is already described in surrounding text (the table summary appears in the paragraph above the table).
- You can use specialized parsing that preserves table structure for text-to-SQL rather than converting to text summaries.

See Also

- The [OpenAI PDF parsing guide](#) demonstrates extracting text, tables, and images from complex PDFs.
- The [LangChain multimodal RAG tutorial](#) shows end-to-end implementation with code examples.

3.11 Loading Videos via Speech-to-Text and Multimodal Models

Problem

You want to load videos into your RAG system.

Solution

Split a video into its modalities and components, such as the audio track and individual frames, and process each component separately.

Figure 3-18 illustrates the steps needed:

1. Partition the video into sections. You can use fixed-length segments (for example, 10-second snippets) or detect logical breakpoints by analyzing the visuals and spoken content.
2. For each breakpoint, assign a label, save the corresponding frame as an image, and create a text summary of that image.
3. Extract the audio between two breakpoints, save it as an MP3 file, and transcribe it.
4. Use text embedding models to create embeddings for both the transcribed audio sections and the image summaries.
5. Store the embeddings in your vector store.

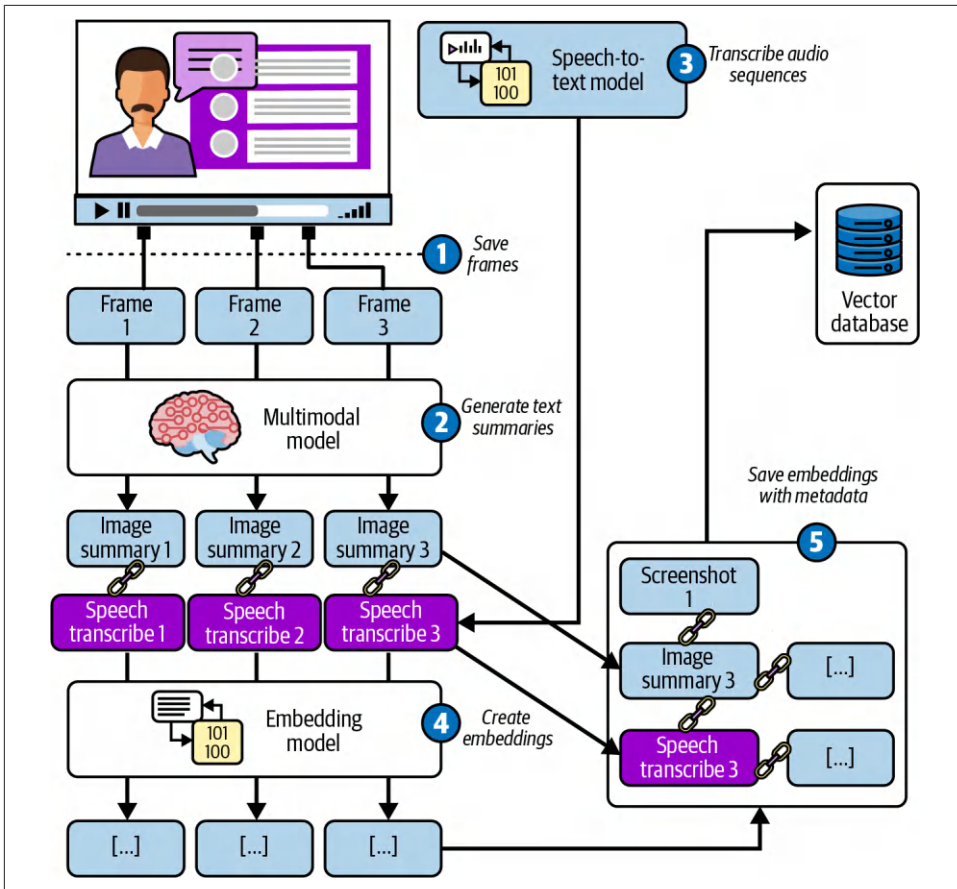


Figure 3-18. Loading videos into a RAG system by breaking them into their elements and processing them using speech-to-text and multimodal models

The code in this recipe uses the following:

- The MoviePy library to load the video, extract frames, and save audio clips between breakpoints
- Multimodal models to process the extracted images
- Speech-to-text models to transcribe audio sections
- One of OpenAI's multimodal models to process the extracted images
- One of OpenAI's speech-to-text models to transcribe the audio section

Use the following command to install the required dependencies:

```
pip install moviepy pandas
```


Start by partitioning the video and saving timestamps for each segment. You can pick timestamps at fixed intervals or manually choose the best split points.

The sample video used in this recipe is a beginner data science tutorial. The speaker presents a slide deck and typically spends about 30 seconds per slide before moving on.

Figure 3-19 shows how to extract frames from the loaded video and save them as images.

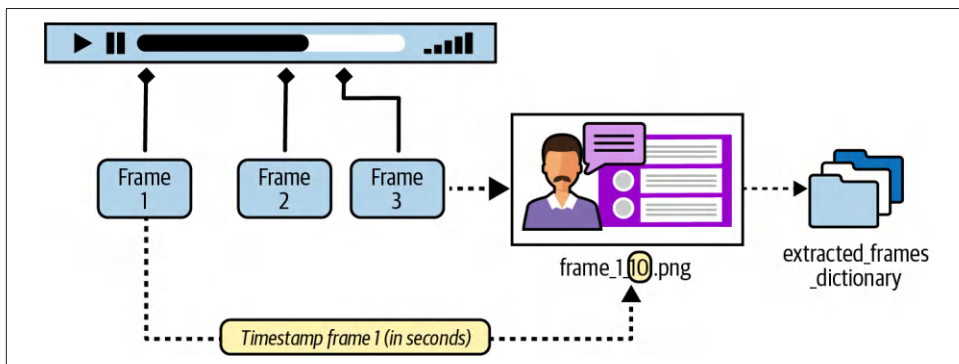


Figure 3-19. Extracting frames from the loaded video and saving them as images

Partition the video into 10-second segments and save the extracted frames as images:

```
import os
import pandas as pd

from moviepy import VideoFileClip, TextClip, CompositeVideoClip

video_file_path = "../datasets/videos/learn-data-science-tutorial.mp4"
image_output_folder = "../datasets/videos/video_extracted_images"

# Create output folder if it doesn't exist
os.makedirs(image_output_folder, exist_ok=True)

clip = VideoFileClip(video_file_path)

# Create a list of timestamps from which to extract a frame
time_step = 10 # Time in seconds
timestamps = list(range(0, int(clip.duration) - time_step, time_step))

# For each timestamp extract a frame
for timestamp in timestamps:
    frame_image_path = os.path.join(
        image_output_folder, f"frame_{timestamp}.png"
    )
    clip.save_frame(frame_image_path, t=timestamp)
```

The result is a list of images, each representing a frame from the video (see [Figure 3-20](#)).

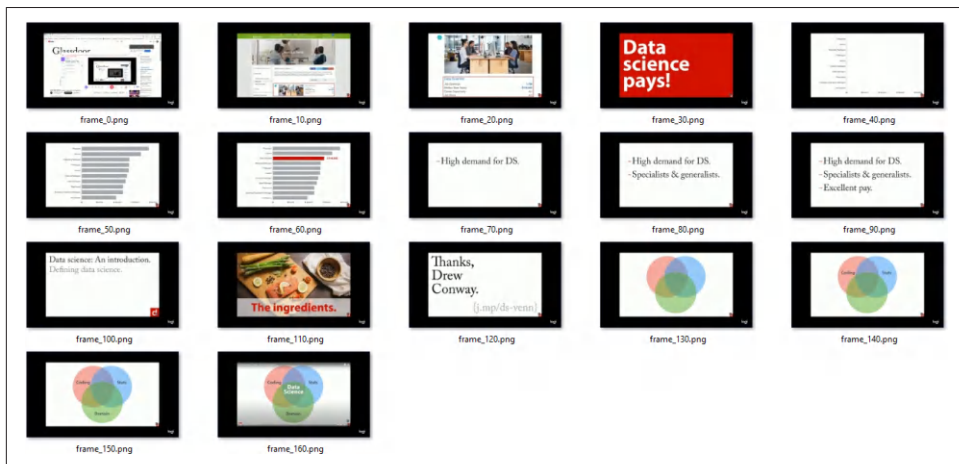


Figure 3-20. Frames extracted from the video

Next, decide how to process the saved images. The most common approach is to use multimodal models to interpret the content in each image.

The next step extracts the audio between each pair of timestamps and saves it as an MP3 file. [Figure 3-21](#) shows how to save audio sequences from the video to MP3 files.

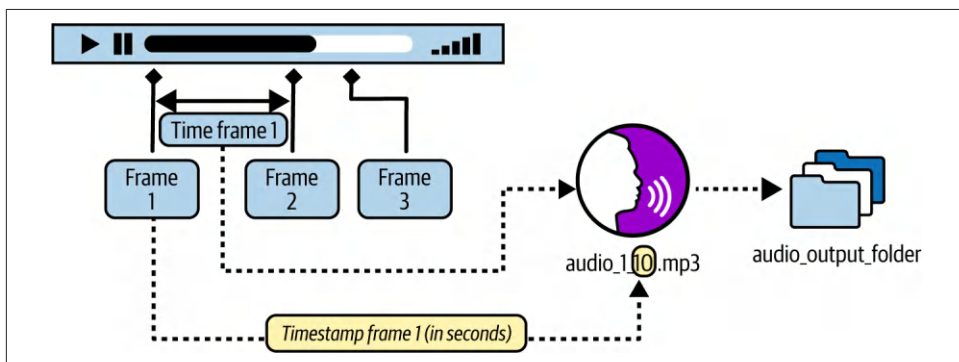


Figure 3-21. Saving audio sequences from the video to MP3 files

Iterate over the saved timestamps and write the audio between two timestamps to an MP3 file (for example, using a clip/subclip function from a video library):

```
# For each timestamp extract the audio sequence and save it to a .mp3 file
audio_output_folder = "../datasets/videos/video_extracted_audio"

# Create output folder if it doesn't exist
```

```

os.makedirs(audio_output_folder, exist_ok=True)

for timestamp in timestamps:
    audio_clip = clip.subclip(timestamp, timestamp + time_step).audio
    output_audio_path = os.path.join(
        audio_output_folder, f"audio_{timestamp}.mp3"
    )
    audio_clip.write_audiofile(output_audio_path)

```

Next, transcribe each audio segment using a speech-to-text model. See [Recipe 3.5](#) for details.

After that, each video segment yields at least two text snippets, one from the audio transcription and one derived from the first frame of the segment (for example, via image understanding).

Discussion

Video segmentation strategy depends on content structure and retrieval needs. Choose a strategy based on your video type:

Fixed intervals (10–30 seconds)

Lectures, presentations, tutorials with consistent pacing. Simple to implement, works well when content changes gradually.

Scene detection (visual analysis)

Movies, documentaries, training videos with clear visual transitions. Automatically detects cuts, fades, and major visual changes.

Transcript-based segmentation

Meetings, interviews, podcasts where topic shifts occur mid-scene. Use text segmentation techniques to analyze transcripts for topic boundaries.

Chapter markers (when available)

Educational content, webinars with pre-defined sections. Using these markers for segmentation is the most accurate approach, but in practice you rarely deal with content that has such markers available.

The dual-modality approach (frames + audio) captures complementary information. Slide presentations benefit more from frame analysis since slides contain key information. Podcasts and interviews depend primarily on audio transcription. Action-heavy videos need dense frame sampling to capture visual events.

Segment metadata enables precise source attribution. Users get not just answers but exact video timestamps. A question like “How do I configure authentication?” returns both the answer and a deep link to the relevant video moment, significantly improving user experience over text-only responses.

See Also

- The [MoviePy documentation](#) details the library's support for video editing, frame extraction, and audio processing.
- The [OpenAI Whisper API guide](#) provides official documentation for audio transcription and speech recognition.
- The [LlamaIndex video processing guide](#) demonstrates frame extraction, audio transcription, and CLIP embeddings for multimodal search.

Data Preparation

When feeding longer texts into RAG systems, preprocessing breaks the document into smaller chunks. Each chunk is converted into an embedding vector that captures its semantic meaning. During retrieval, the system measures similarity by calculating distances between these vectors to find relevant text.

This works best when each chunk contains exactly one piece of information and can be understood independently. The key challenge is preparing chunks that stand alone without relying on surrounding context. A robust processing pipeline cleans raw text and splits it at the right points to produce meaningful chunks.

Figure 4-1 illustrates common preprocessing techniques:

Text preparation

Replace abbreviations and clean the text.

Metadata collection

Store page numbers, source, and author.

Text splitting

Apply character, recursive, semantic, or agentic chunking.

The goal is producing clear, unambiguous chunks that don't require surrounding context.

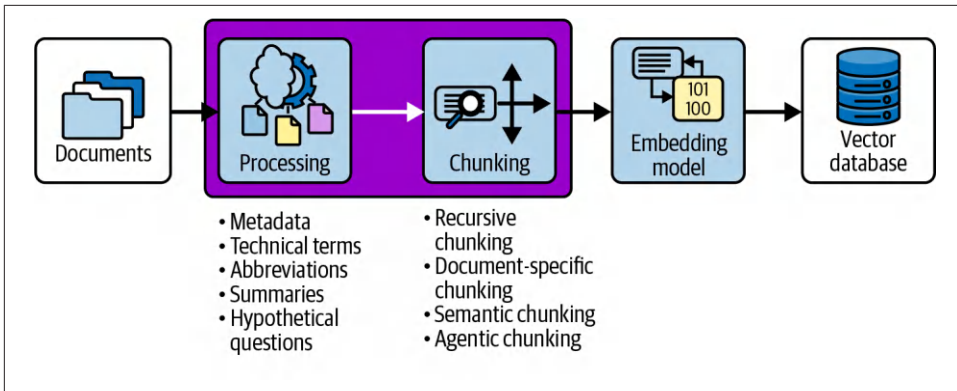


Figure 4-1. A simplified RAG indexing pipeline including potential data processing techniques

Store useful metadata with each chunk to enable filtering during retrieval, making the process both faster and more accurate.

You can find all the code examples for this chapter in the [book's GitHub repository](#).

4.1 Adding Metadata to Enable Metadata Filtering

Problem

You want to store metadata alongside the text chunks in your vector store so that retrieval can apply metadata filtering.

Solution

Extract existing metadata from documents and compute additional fields. For specialized use cases, optionally use LLM-generated metadata derived from the document content to enrich the metadata stored with the documents. Follow these steps:

1. Extract metadata already stored in the document (author, title, creation date).
2. Add computed metadata fields (file location, size, page count, text length).
3. (Optional) Generate content-based metadata by analyzing document text with an LLM.
4. Save the metadata along with the text chunk.

Figure 4-2 illustrates this workflow.

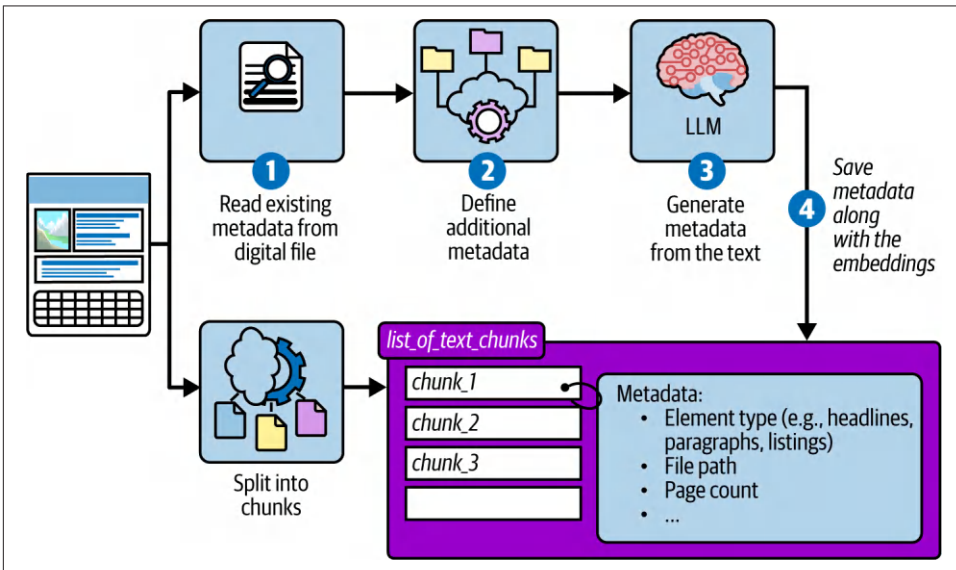


Figure 4-2. Extracting and generating metadata fields

To follow along, install the required libraries:

```
pip install PyPDF2 openai pydantic
```

Next, you import a PDF and store its metadata fields in a dictionary. The completeness of these fields depends heavily on the software that created the PDF. Many PDFs include author, title, subject, creator, creation date, and modification date. Here's the code:

```
import PyPDF2
import os

file_path = "../datasets/pdf_files/attention_is_all_you_need_paper.pdf"

with open(file_path, "rb") as file:
    reader = PyPDF2.PdfReader(file)
    metadata = reader.metadata

    text = ""
    for page in reader.pages:
        text += page.extract_text()
```

Figure 4-3 shows the existing metadata fields extracted from the sample PDF document.

```
[7]
✓ Os
  metadata
  {
    '/Author': '',
    '/CreationDate': 'D:20240410211143Z',
    '/Creator': 'LaTeX with hyperref',
    '/Keywords': '',
    '/ModDate': 'D:20240410211143Z',
    '/PTEX.Fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023)
    kpathsea version 6.3.5',
    '/Producer': 'pdfTeX-1.40.25',
    '/Subject': '',
    '/Title': '',
    '/Trapped': '/False'}
```

Figure 4-3. Existing metadata fields extracted from a PDF

Next, add additional data points to your metadata dictionary—file size, filename, file-path, page count, and total text length:

```
metadata_ext = dict(metadata)
metadata_ext["page_count"] = len(reader.pages)
metadata_ext["file_size"] = os.path.getsize(file_path)
metadata_ext["file_name"] = os.path.basename(file_path)
metadata_ext["file_path"] = file_path
metadata_ext["text_length"] = len(text)
```

For specialized scenarios, you can use LLMs to enrich metadata with content-based fields. In the sample PDF loaded in this recipe, the existing Author field is empty. Since the PDF is a scientific paper, the author name should appear somewhere in the content.

To ensure that the extracted information is returned as clean values in the correct format, follow the process in Figure 4-4. First you define a Pydantic model describing the expected data, then provide a system instruction that asks the model to populate that schema from the document text. Finally, you send both to the LLM.

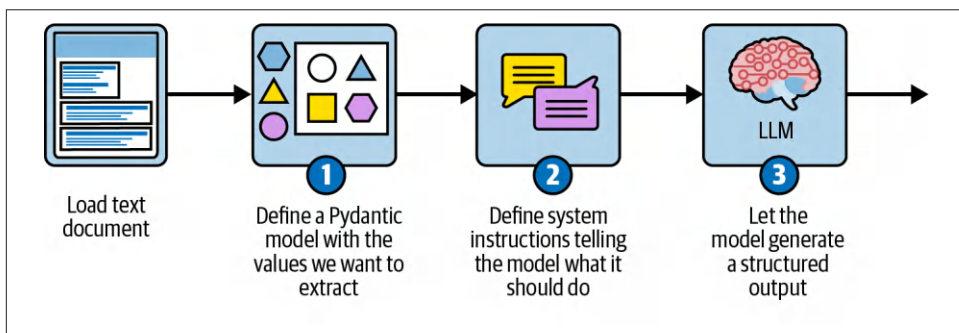


Figure 4-4. Generating metadata from the document text by using LLMs

Let's now implement these steps. Define a Pydantic model for the author contact information (author name, email address, and company name) and use OpenAI Structured Outputs to extract those fields:

```
from pydantic import BaseModel
from openai import OpenAI

client = OpenAI()

class AuthorContact(BaseModel):
    name: str
    company: str
    email: list[str]

class Contacts(BaseModel):
    entries: list[AuthorContact]

system_message = """Extract the contact information of all authors."""

response = client.beta.chat.completions.parse(
    model="gpt-5-mini",
    messages=[
        {
            "role": "system",
            "content": system_message,
        },
        {
            "role": "user",
            "content": text,
        },
    ],
    response_format=Contacts,
)

author_contacts = response.choices[0].message.parsed

metadata_ext_llm = dict(metadata_ext)
metadata_ext_llm["author_contacts"] = author_contacts
```

Figure 4-5 shows the full metadata dictionary after enrichment. With native PDF fields, computed metadata, and optional LLM-generated fields, this provides a strong foundation for the retriever to run targeted searches, such as filtering by creation date or by a specific author.

```
[14]
✓ Os
  metadata_ext_llm
  {
    '/Author': '',
    '/CreationDate': 'D:20240410211143Z',
    '/Creator': 'LaTeX with hyperref',
    '/Keywords': '',
    '/ModDate': 'D:20240410211143Z',
    '/PTeX.Fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023)
kpathsea version 6.3.5',
    '/Producer': 'pdfTeX-1.40.25',
    '/Subject': '',
    '/Title': '',
    '/Trapped': '/False',
    'page_count': 15,
    'file_size': 2215244,
    'file_name': 'attention_is_all_you_need_paper.pdf',
    'file_path': '../datasets/pdf_files/attention_is_all_you_need_paper.pdf',
    'text_length': 39472,
    'author_contacts': Contacts(entries=[AuthorContact(name='Ashish Vaswani',
company='Google Brain', email=['avaswani@google.com']), AuthorContact(name='Noam
Shazeer', company='Google Brain', email=['noam@google.com']),
AuthorContact(name='Niki Parmar', company='Google Research', email=
['nikip@google.com']), AuthorContact(name='Jakob Uszkoreit', company='Google
Research', email=['usz@google.com']), AuthorContact(name='Llion Jones',
company='Google Research', email=['llion@google.com']), AuthorContact(name='Aidan N.
Gomez', company='University of Toronto', email=['aidan@cs.toronto.edu']),
AuthorContact(name='Łukasz Kaiser', company='Google Brain', email=
['lukaszkaiser@google.com']), AuthorContact(name='Illia Polosukhin', company='',
email=['illia.polosukhin@gmail.com'])]))}
```

Figure 4-5. The extended metadata dictionary, including the generated metadata fields

Discussion

Metadata filtering works by narrowing the search space before semantic similarity calculations. Vector databases first apply metadata constraints (e.g., `author == "Smith"` or `date >= 2023`), then perform semantic search only on the filtered subset. This reduces false positives from semantically similar but contextually irrelevant content.

Use metadata filtering when your documents come from clearly different buckets and users usually care about one bucket at a time. Here are some examples:

- You store HR, engineering, and finance documents in the same vector index—for example, filter to `department = "engineering"` before searching.
- You index content from different years or quarters—for example, search only `year = 2025` when the user asks about last year.
- You mix manuals, tickets, and emails—for example, filter to `source = "manual"` to avoid pulling support chatter.

- Many documents use the same words for different things—for example, “release” appears in both product docs and press releases.

Figure 4-6 illustrates why metadata matters: content from related fields can be semantically similar but contextually irrelevant. Information about antioxidants applies to both medicine and environmental engineering, for example. Metadata filtering separates these contexts before semantic search, preventing false positives from overlapping terminology.

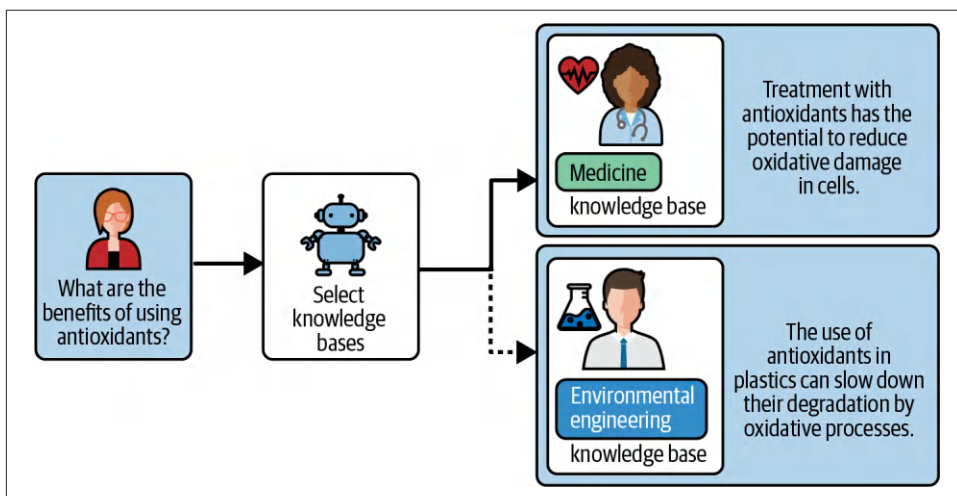


Figure 4-6. Two semantically similar information sets can be contradictory



LLM-generated metadata adds preprocessing cost and latency. For a 1,000-document corpus, expect several minutes and \$1–\$5 in API costs. Balance this against improved retrieval precision for your use case.

See Also

- [The OpenAI structured outputs guide](#) describes how to extract metadata fields from text via LLMs.
- [The Pinecone metadata filtering documentation](#) explains how to apply metadata filtering during retrieval in a Pinecone vector database.
- [The Chroma metadata filtering documentation](#) provides examples on how to use metadata filters in Chroma.

4.2 Enhancing Data Quality by Replacing Abbreviations and Technical Terms

Problem

You want every text chunk in your vector store to be self-explanatory so it can be understood as a standalone piece of information.

Solution

Use regexes and LLMs to expand abbreviations and clarify technical terms before generating embeddings. This makes text chunks self-explanatory when stored as standalone pieces.

First, install the OpenAI SDK:

```
pip install openai
```

Next, load a text file and replace abbreviations with their long form followed by the abbreviation in parentheses:

```
import re

abbreviations_dict = {
    "NLP": "Natural Language Processing",
    "RNN": "Recurrent Neural Network",
    "LSTM": "Long Short-Term Memory",
    "GRU": "Gated Recurrent Unit",
    "TF": "Transformer",
    "MHA": "Multi-Head Attention",
    "FFN": "Feed-Forward Network",
}

file_path = "../datasets/text_files/blog_post_transformers.txt"
with open(file_path, "r") as file:
    text = file.read()

# Replace abbreviations in the text
for abbr, full_form in abbreviations_dict.items():
    text = re.sub(rf"\b{abbr}\b", f"{full_form} ({abbr})", text)
```

Finally, use an LLM to expand abbreviations and clarify technical terms from extracted text:

```
import os
from openai import OpenAI

file_path = "../datasets/text_files/EMEA_drives_revenue.txt"

with open(file_path, "r") as file:
```

```

text = file.read()

prompt = f"""
The text below contains a financial report including a lot of
abbreviations and technical terms from the finance domain.
Please replace the abbreviations with their full forms and
provide a brief explanation of the technical terms, so the
whole text gets easier to read and understandable for everyone.

Make sure it's easy enough, so that a 10-year-old school kid could
understand it.

Often used abbreviations are:
- EMEA: Europe, Middle East, and Africa
- BD: Business Development
- YoY: Year-over-Year
- APAC: Asia-Pacific

Text:
{text}
""".strip()

client = OpenAI()
chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": prompt,
        }
    ],
    model="gpt-5.2",
)

enhanced_text = chat_completion.choices[0].message.content

```

Figure 4-7 shows the outcome: on the left, the raw extracted text from the slide; on the right, the preprocessed version, which includes a clear textual description of all information pieces, whether they come from the diagram or from the slide text. The LLM analyzes the full context and informational content of the slide and converts it into a textual form.

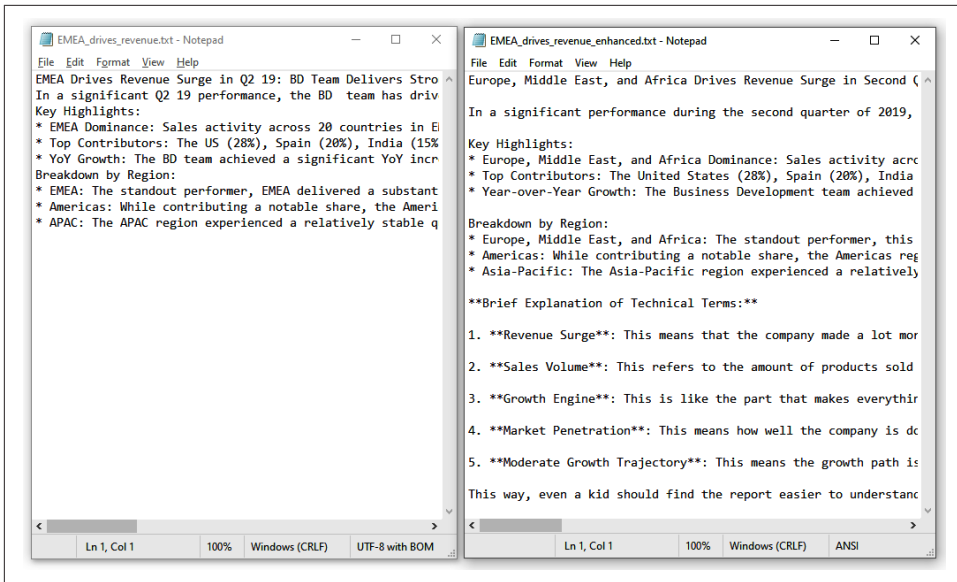


Figure 4-7. Enhanced text chunks after replacing abbreviations and technical terms

Discussion

When you expand abbreviations and clarify technical terms, each text chunk becomes understandable on its own. This matters because RAG systems retrieve chunks in isolation. The embedding model only sees the words inside the chunk. If those words are vague or abbreviated, the model has less semantic signal to work with.

For example, “NLP” is much less informative than “natural language processing.” Users usually search with short or mixed language queries, and the embedding model benefits from having both the abbreviation and the expanded form available in the text.

This approach works for these reasons:

- Embeddings built from expanded terms contain richer semantic signals than acronyms alone.
- Queries like “NLP models,” “language models,” or “text understanding” still match chunks that contain the expanded phrase.
- Retrieved chunks are easier for the LLM to interpret without guessing what an acronym means.

Use this approach when you have the following:

- Technical documents full of acronyms
- Financial or legal reports with shorthand
- Medical or clinical text
- Any document that will be read by people outside the original expert audience

Don't use this approach in the following circumstances:

- Abbreviations are the standard form, like “HTTP” in web development.
- Preprocessing very large collections would be too expensive.
- You must ingest documents in real time with very low latency.

Figure 4-8 illustrates three common problems this solution addresses:

- Abbreviations like “PO” (purchase order, post office, or product owner) become ambiguous without context.
- References such as “MJ” lose meaning when isolated from their original context.
- Image descriptions remain incomplete without the visual information.

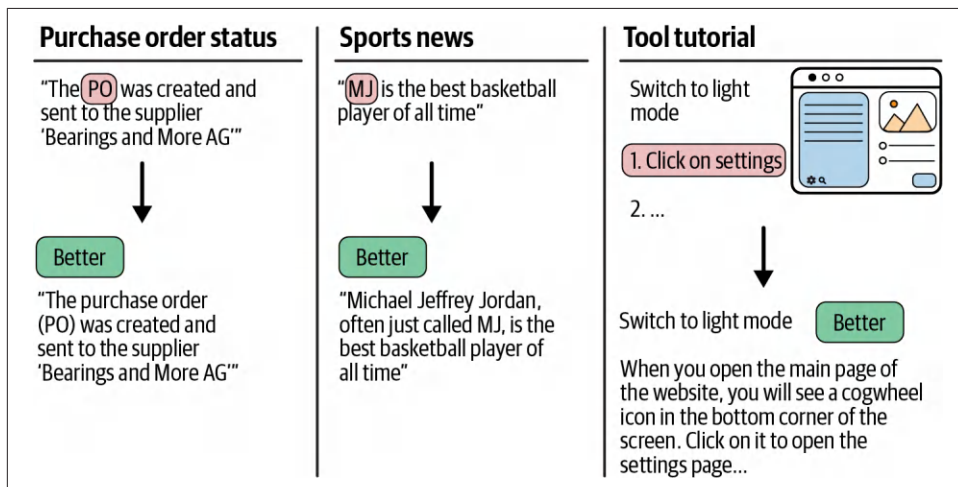


Figure 4-8. The way that documents are structured influences the performance of our RAG system

Table 4-1 shows additional techniques that improve chunk clarity.

Table 4-1. Techniques to enhance data quality

Technique	Instead of	Use
Adding contextual sentences	It improved efficiency by 40%.	The new indexing algorithm improved search efficiency by 40%.
Avoiding pronouns and implicit references	It was implemented last year.	The customer feedback system was implemented last year.
Preserving full names of entities	Google launched it in 2019.	Google launched the BERT language model in 2019.
Avoiding ambiguous words	The system works better now.	The system's response time improved from 500 ms to 200 ms after optimization.

See Also

- [The OpenAI prompt engineering guide](#) covers using LLMs to enhance text quality

4.3 Improving Search Accuracy by Creating Hypothetical Questions for Text Chunks

Problem

You want to improve retriever quality by comparing the user's question to hypothetical questions derived from text chunks, instead of comparing it directly to the text chunks.

Solution

Figure 4-9 shows the full process: the preprocessing and indexing steps, as well as retrieval at runtime in the RAG app.

Before indexing the data, you need to do the following:

1. Load the data and split it into chunks.
2. Iterate over the chunks and generate at least one hypothetical question per text chunk.
3. Generate embeddings for each hypothetical question.
4. Store the hypothetical questions and their embeddings in the database.

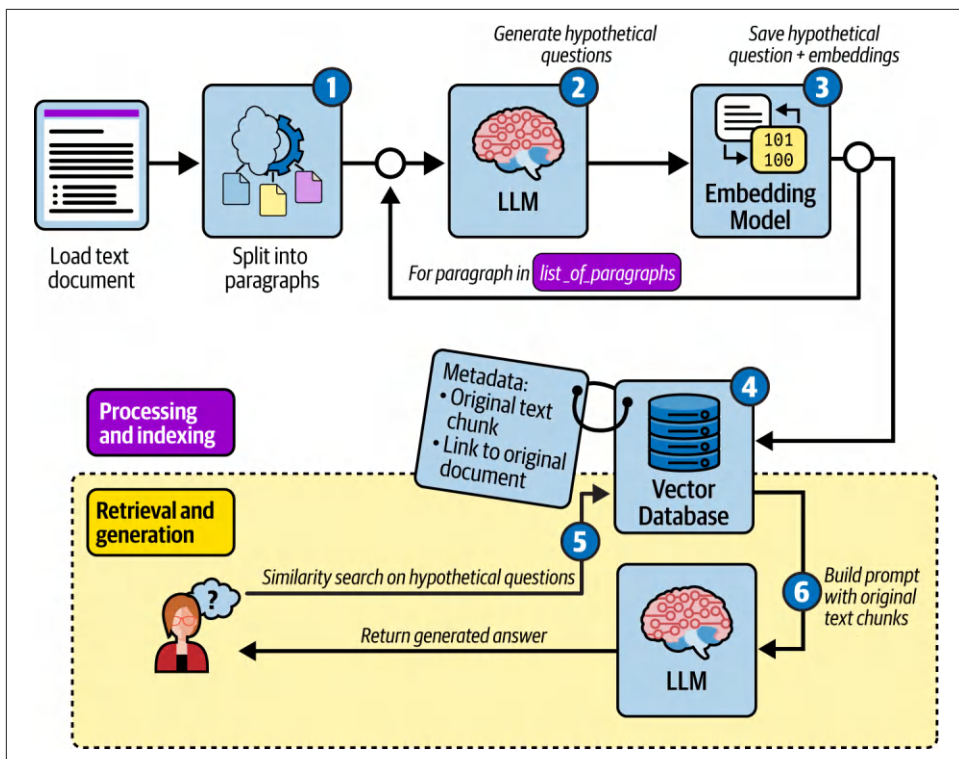


Figure 4-9. Using LLMs to generate hypothetical questions

Each hypothetical question links back to its original text chunk. During runtime of the RAG app, the retriever searches the hypothetical questions, but you typically pass the underlying text chunks to the LLM, so the model receives the full context rather than only the derived hypothetical questions.

To execute the code, first install the OpenAI SDK and Pydantic:

```
pip install openai pydantic
```

Next, you load a chat history between two students discussing AI in manufacturing. The LLM analyzes the text and generates five hypothetical questions. Each question can be answered using the information contained in the text. Here's the code:

```
import textwrap
from openai import OpenAI
from pydantic import BaseModel

file_path = "../datasets/text_files/AI_in_factories_chat.txt"

with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()
```

```

client = OpenAI()

prompt = textwrap.dedent(
    f"""
        Below you can find a chat history between two students.

        Please generate 5 hypothetical questions that could be
        answered using the information from the discussion.
        The questions should focus on key details, definitions, and
        information present in the text.

        Chat History:
        {text}
        """
)

class HypotheticalQuestions(BaseModel):
    questions: list[str]

result = client.beta.chat.completions.parse(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": prompt}],
    response_format=HypotheticalQuestions,
)

hypothetical_questions = result.choices[0].message.parsed.questions
hypothetical_questions

```

Figure 4-10 shows the generated output.

```

hypothetical_questions
[69] ✓ 0.0s Python
... ['How can AI, specifically through robotic process automation (RPA), contribute to reducing operational costs in manufacturing?'
      'What measures could be taken to minimize workplace injuries in manufacturing using AI technology?'
      'How does machine vision in AI contribute to improving quality control in manufacturing processes?'
      'What potential challenges have companies faced due to system reliability issues with AI implementation?'
      'How might AI adoption potentially exacerbate inequality between large corporations and smaller businesses?']

```

Figure 4-10. Hypothetical questions generated by LLMs

Discussion

Hypothetical questions reduce the gap between the way users phrase queries and the way documents are written. Instead of embedding raw chunks like code, logs, or tables, you embed questions that describe what those chunks are about. This makes retrieval behave more like the way users actually search.

This approach works for these reasons:

- Query embeddings are closer to question-style text than to raw technical content.
- Generated questions express the intent behind a chunk in natural language.
- This pulls hard-to-embed content like code and tables into the same semantic space as user queries.

Use hypothetical questions in the following scenarios:

- Documents contain code, configuration files, or database schemas.
- Content includes logs, transcripts, or chat histories.
- Data is stored in tables or other structured formats
- Users typically ask natural language questions like “How do I...” or “What is...”

Don't use hypothetical questions in these circumstances:

- Documents are already written as questions and answers, such as FAQs.
- You can't afford the extra LLM calls during ingestion.
- The text is already clear, narrative prose.
- Ingestion must happen in real time, and latency matters.

Figure 4-11 illustrates the semantic alignment problem: code blocks, chat logs, and tables often sit far from embeddings of natural language user queries, even when the underlying content is relevant. Hypothetical questions reposition this content closer to typical user question patterns in the embedding space.

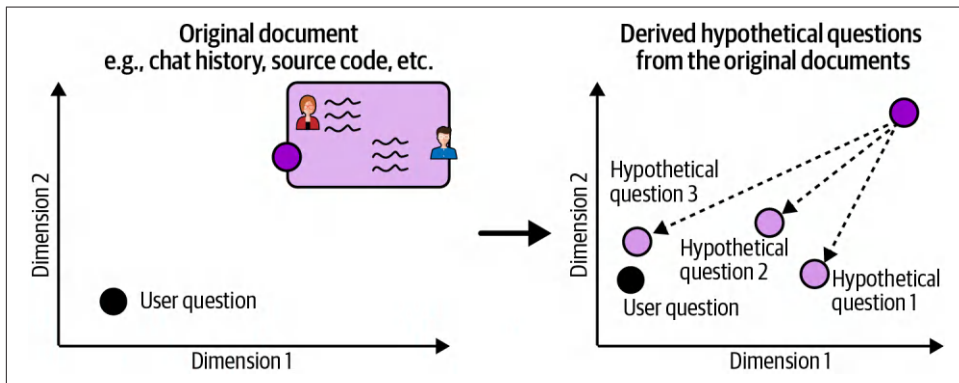


Figure 4-11. Hypothetical questions are closer to the location of potential user questions in the embedding space

At retrieval time, the system compares the user’s query to stored hypothetical questions (not raw chunks). When a hypothetical question matches, the retriever fetches the original text chunk linked to that question. This provides the LLM with full context rather than only the derived questions.



This recipe addresses the semantic alignment problem during indexing by generating hypothetical questions from your text chunks. For an alternative approach that addresses alignment at query time, see the hypothetical document embeddings (HyDE) recipe in [Chapter 7](#). HyDE generates hypothetical documents from the user’s query instead of preprocessing your data. You can use one or both techniques depending on your retrieval quality needs and performance requirements.

See Also

The following resources cover HyDE, which has become the more popular approach for addressing semantic alignment in retrieval. While HyDE generates hypothetical documents from queries at runtime, this recipe takes the inverse approach—generating hypothetical questions from your text chunks during indexing. HyDE implementations are more widely adopted because they don’t require preprocessing all documents, though both techniques can be effective, depending on your use case. See these resources for more details:

- [The Haystack HyDE implementation](#) shows how to implement HyDE via the Haystack framework.
- [The LangChain HyDE retriever API reference](#) explains how to use HyDE-style retrieval in LangChain.
- [The LlamaIndex query transformations](#) documentation covers query-transformation approaches, including HyDE-style techniques.

4.4 Splitting Documents via Character Splitting

Problem

You need to break lengthy documents into smaller sections that are small enough to create embeddings, using existing embedding models.

Solution

Character-based splitting is the simplest way to split text. It uses a fixed-size window that slides over the text and cuts out snippets. [Figure 4-12](#) illustrates this with an example. In this case, chunk 2 and chunk 3 overlap.

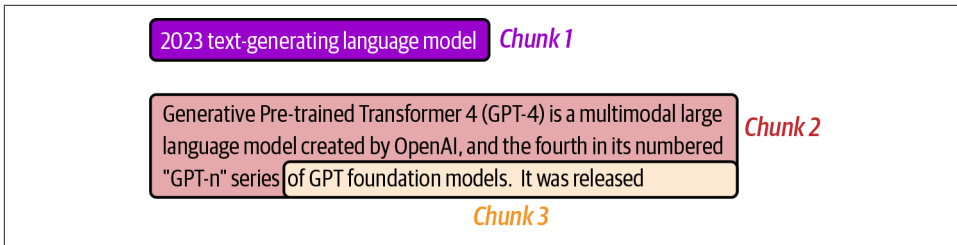


Figure 4-12. Splitting text by using a fixed character or token size

In this example, you use a chunk length of 100 characters:

```
file_path = "../datasets/text_files/blog_post_transformers.txt"

with open(file_path, "r") as file:
    text = file.read()

def split_by_characters(text, chunk_size, overlap):
    chunks = []
    step = max(1, chunk_size - overlap)

    for start in range(0, len(text), step):
        end = start + chunk_size
        chunk = text[start:end]
        if chunk:
            chunks.append(chunk)

    return chunks

chunks = split_by_characters(text, chunk_size=100, overlap=20)
```

Discussion

Character splitting works by moving a fixed-size window across text and extracting substrings. This technique ignores all document structure, treating text as a continuous stream of characters. This is the simplest chunking approach but also the least sophisticated. It counts characters and splits, often tearing sentences and paragraphs apart. Figure 4-12 shows this clearly: chunk boundaries fall arbitrarily, potentially splitting words or breaking sentences mid-thought.

Use character splitting when you have the following:

- Raw audio or video transcriptions with no natural structure
- Concatenated logfiles or data dumps without delimiters
- Quick prototyping before implementing sophisticated chunking

Don't use character splitting for the following:

- Any document with paragraph breaks, headings, or other structure markers
- Content where mid-sentence splits damage meaning (most documents)



Overlap considerations: Setting an overlap (e.g., 20 characters) means consecutive chunks share text at their boundaries. This helps preserve context that spans chunk edges but increases storage requirements and redundancy.

Table 4-2 summarizes when to use character splitting versus alternative approaches.

Table 4-2. When to use character splitting versus alternatives

Scenario	Recommended approach
Unstructured text like raw audio transcriptions with no natural boundaries	Use character splitting.
Structured documents (blog posts, reports)	Use recursive chunking (Recipe 4.5) or document-aware splitting (Recipe 4.6) to preserve paragraphs and sections.
Markup documents (Markdown, LaTeX)	Use document-aware splitting to leverage syntax as splitting cues.
Complex unstructured text	Use semantic chunking (Recipe 4.7) or agentic chunking (Recipe 4.8) to find meaning-based breakpoints.

See Also

- [The LangChain character text splitter documentation](#)

4.5 Splitting Documents with Recursive Text Splitters

Problem

You want to split documents that have a clear structure, including headings, chapters, and paragraphs, and by doing so, preserve this structure.

Solution

A *recursive text splitter* identifies characters that humans typically use to structure a document, such as newlines, spaces, and punctuation marks. It splits text at natural boundaries, favoring larger structural breaks like newlines first, then falling back to punctuation marks when chunk size limits are reached.

Figure 4-13 shows how an example text would be chunked. The first split is naturally at a newline; the second split (between chunk 2 and chunk 3) happens at a punctuation mark once a certain length is reached.

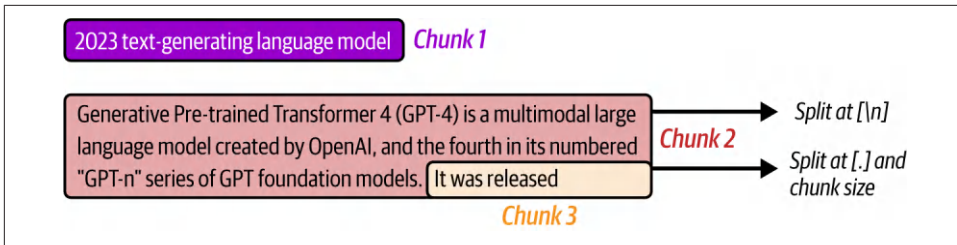


Figure 4-13. Using recursive chunking to split text

This recipe uses the LangChain text splitters for demonstration purposes.



Alternatively, you can define your own recursive text splitter by using regexes to identify separators such as newlines and punctuation marks. This way, you avoid adding dependencies on orchestration frameworks to your project, and your implementation is more lightweight.

Install the required packages:

```
pip install PyPDF2 langchain-text-splitters
```

Then you load and chunk a newsletter page. The splitter counts characters (or tokens), and once it reaches the defined maximum size, it looks for the next separator to split the text:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
import PyPDF2

file_path = "../datasets/pdf_files/daily_insights.pdf"

with open(file_path, "rb") as file:
    reader = PyPDF2.PdfReader(file)

    text = ""
    for page in reader.pages:
        text += page.extract_text()

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=0,
    length_function=len,
    is_separator_regex=False,
)

chunks = text_splitter.split_text(text)
```

Figure 4-14 shows the resulting list of chunks and the character length of each chunk. Each chunk has a maximum length of 200 characters.


```
print(text_chunks)
print(f"Number of chunks: {len(text_chunks)}")

for chunk in text_chunks:
    print(f"Chunk length: {len(chunk)}")
```

[53] ✓ 0.0s Python

```
... ['The\nDaily\nInsight\nPolitics:\nNew\nClimate\nAccord\nSets\nAmbitious\nGoals\nWorld\nleaders\nNumber of chunks: 8
Chunk length: 200
Chunk length: 191
Chunk length: 189
Chunk length: 199
Chunk length: 197
Chunk length: 193
Chunk length: 197
Chunk length: 60
```

Figure 4-14. Recursive text splitting



The default list of separators works well for most texts and languages. However, for languages like Chinese, Japanese, and Thai, which lack word boundaries, these separators might split words incorrectly. To avoid this, you can customize the list of separators. Best practices can be found in the [LangChain documentation](#).

Discussion

Recursive chunking identifies natural text boundaries in priority order: paragraph breaks first, then single newlines, then punctuation, and finally spaces. When a chunk reaches its size limit, the splitter uses the highest-priority separator available.

This approach works for these reasons:

- Preserves semantic coherence by respecting how humans structure text
- Keeps related sentences within the same chunk
- Reduces mid-sentence splits that damage meaning

Use recursive chunking for these scenarios:

- Documents with clear paragraph structure (articles, reports, books)
- General-purpose RAG systems processing mixed document types
- As a default choice when document format is unknown

Don't use recursive chunking for the following:

- Markup documents (Markdown, LaTeX, HTML) where document-aware splitting ([Recipe 4.6](#)) preserves heading structure better
- Domain-specific content where semantic boundaries matter more than structural ones, requiring semantic chunking ([Recipe 4.7](#))

[Figure 4-15](#) illustrates the risk with fixed-size chunking: when you split text every fixed number of tokens (for example, every 1,000 tokens), you can inadvertently combine content from different sections of a document. In a newspaper, for instance, politics, sports, and healthcare articles appearing on the same page might end up in the same chunk, mixing unrelated topics. Recursive chunking reduces this risk by respecting paragraph and section boundaries, keeping related information together.

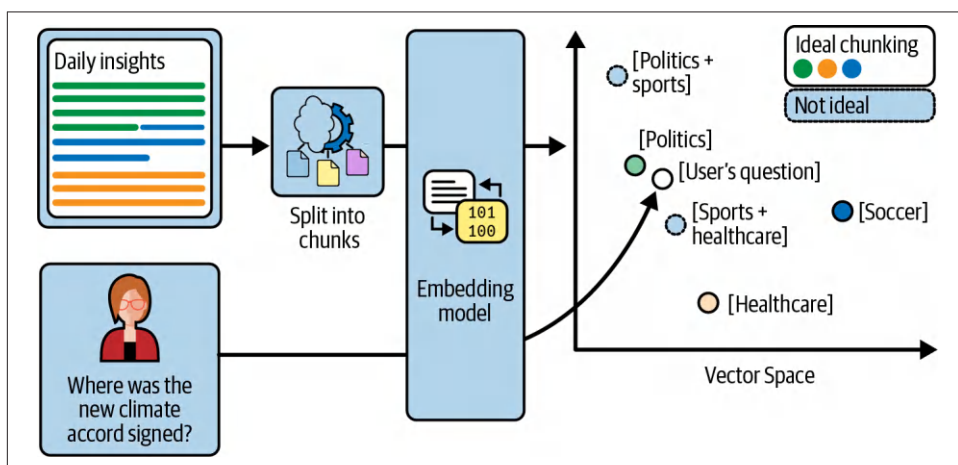


Figure 4-15. Each text chunk should represent one particular piece of information as clearly as possible

You can add overlaps so that the last sentence of one chunk also appears at the beginning of the next chunk. This helps the embedding model capture context that spans multiple chunks. However, overlaps increase storage costs and can cause redundant retrievals.



Recursive character chunking is straightforward to implement from scratch, using regexes to search for separators such as new-lines and punctuation marks. For production use, consider implementing a simple version yourself to avoid adding orchestration framework dependencies to your project.

See Also

- [The LangChain recursive text splitter documentation](#) explains how recursive splitting works and how to configure separators, chunk size, and overlap.

4.6 Chunking Documents with Document-Aware Splitting

Problem

You need to load markup documents and want to use the document-specific syntax to split them into chunks.

Solution

Markup formats such as Markdown use a specific syntax. For example, headings start with a hash sign (#), bullet lists often use an asterisk (*), and bold text is delimited by two asterisks (**). *Document-aware splitters* understand this syntax and use it to chunk the text.

Figure 4-16 shows example code snippets for various file formats.



Figure 4-16. Sample code for HTML, Python, Markdown, and LaTeX

This recipe uses LangChain Text Splitters to avoid defining syntax and splitting rules for each file format. Install the package:

```
pip install langchain-text-splitters
```

This example uses three text splitters: one for Python code, one for LaTeX code, and one for Markdown files. The code checks the file extension and selects the appropriate splitter. In this example, it loads a Markdown file with the *.md* extension:

```
import os
from langchain_text_splitters import import (
```

```

        PythonCodeTextSplitter,
        LatexTextSplitter,
        MarkdownHeaderTextSplitter,
    )

    file_path = "../datasets/markdown_files/random_md_code.md"
    file_extension = os.path.splitext(file_path)[1]

    with open(file_path, "r") as file:
        file_text = file.read()

    if file_extension == ".py":
        splitter = PythonCodeTextSplitter(chunk_size=500, chunk_overlap=50)
    elif file_extension == ".tex":
        splitter = LatexTextSplitter(chunk_size=500, chunk_overlap=50)
    elif file_extension == ".md":
        headers_to_split_on = [
            ("#", "Header 1"),
            ("##", "Header 2"),
            ("###", "Header 3"),
        ]

        splitter = MarkdownHeaderTextSplitter(headers_to_split_on)

    chunks = splitter.split_text(file_text)

```

The output is a list of Python objects. Each object represents a text chunk and includes metadata such as `header_1`, `header_2`, and `header_3`, along with the page content. In other words, each chunk corresponds to a document section while preserving structural metadata such as the heading levels 1, 2, and 3.

Figure 4-17 shows the resulting text chunks for the example document.

```

[44]: text_chunks
✓ 0.0s Python
... [Document(metadata={'Header 1': 'Random Markdown Page'}, page_content='---'),
Document(metadata={'Header 1': 'Random Markdown Page', 'Header 2': 'Table of Contents'}, page_content='1. [Introducti
Document(metadata={'Header 1': 'Random Markdown Page', 'Header 2': 'Introduction'}, page_content='Welcome to a randor
Document(metadata={'Header 1': 'Random Markdown Page', 'Header 2': 'Introduction', 'Header 3': 'Highlights'}, page_cc
Document(metadata={'Header 1': 'Random Markdown Page', 'Header 2': 'Code Snippets'}, page_content='```python\n# Pyth
Document(metadata={'Header 1': 'Random Markdown Page', 'Header 2': 'Lists', 'Header 3': 'Unordered List'}, page_conti
Document(metadata={'Header 1': 'Random Markdown Page', 'Header 2': 'Lists', 'Header 3': 'Ordered List'}, page_content
Document(metadata={'Header 1': 'Random Markdown Page', 'Header 2': 'Lists', 'Header 3': 'Task List'}, page_content='

```

Figure 4-17. Document-aware splitting for a Markdown file

Discussion

Document-aware splitting leverages format-specific syntax to identify logical boundaries. Markdown splitters recognize heading markers (`#`, `##`, `###`), Python splitters identify function and class definitions, HTML splitters respect tag structures, and LaTeX splitters understand section commands.

Document-aware splitting works for these reasons:

- Preserves the logical structure authors intended when writing
- Keeps code functions, sections, or paragraphs intact
- Captures hierarchical information (heading levels) as metadata
- Is more accurate than generic text splitting for structured formats

Use document-aware splitting for the following:

- Source code repositories (Python, JavaScript, Java, etc.)
- Markdown documentation and READMEs
- HTML pages or web-scraped content
- LaTeX papers and academic documents
- Any content written in a markup language with formal syntax

Don't use document-aware splitting for the following:

- Plain text without markup (use recursive chunking instead)
- PDF or Word documents extracted to plain text (no syntax to leverage)
- Mixed-format documents that need multiple splitters
- Unstructured formats like chat logs or transcripts

Document-aware splitting differs from recursive chunking:

- Recursive chunking looks for generic separators (newlines, punctuation) that work across all text.
- Document-aware splitting understands format-specific structure (HTML tags, Python function definitions, Markdown headers).
- Document-aware splitting preserves hierarchical metadata (e.g., which heading a chunk belongs to).

Using a splitter that leverages the existing document structure is one of the most popular and efficient ways to chunk documents. This approach is fast, and by using headings, paragraphs, and punctuation, you keep content that belongs together in the same chunk.

You can use regexes to search the text and build your own splitting rules, or you can use an existing text-splitting library such as LangChain Text Splitters, which already includes rules for formats like Markdown, Python, HTML, and LaTeX.

To use the syntax of specific file types, your splitter must understand the format's syntax elements. In this recipe, LangChain Text Splitters is used because it saves you from encoding each syntax element as a dedicated regex. For production solutions, consider avoiding LangChain if you want to minimize dependencies, and implement your own document-aware splitter using regexes.

The key trade-offs are that document-aware splitting requires format detection and format-specific parsers. This adds complexity but provides superior chunk quality for structured content compared to generic approaches.

See Also

- [The LangChain Text Splitters API reference](#) provides an overview of available splitters and their configuration options.
- [The LlamaIndex node parsers documentation](#) explains how LlamaIndex parses and structures documents into nodes for retrieval.

4.7 Splitting Text with Semantic-Aware Chunkers

Problem

You want to chunk unstructured documents, such as audio transcriptions, chat logs, or concatenated text, using semantic topic boundaries rather than arbitrary character counts.

Solution

Figure 4-18 shows the semantic chunking pipeline:

1. Break the text into small pieces, such as individual sentences.
2. Generate embeddings for each small text piece by using an embedding model.
3. Measure the semantic similarity between consecutive pieces by calculating the distance between their embedding vectors.
4. If two consecutive pieces are very similar, merge them into a larger chunk. Continue adding sentences until the similarity to the next piece drops below a certain threshold.

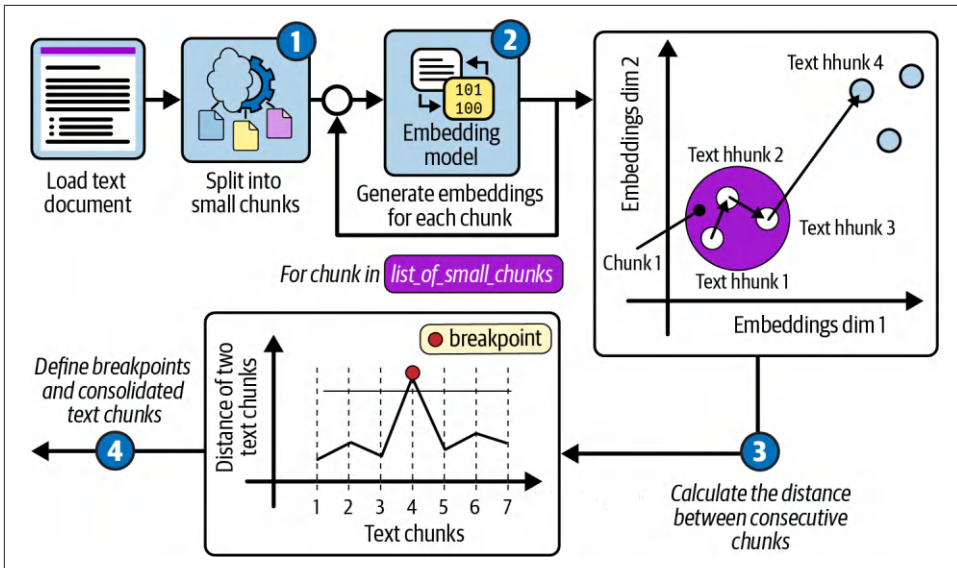


Figure 4-18. Using semantic similarity to define the breakpoint

This recipe uses OpenAI text embedding models. To keep the recipe short, instead of implementing semantic chunking from scratch, you'll use the `SemanticChunker` from the LangChain [Experimental package](#). Install it as follows:

```
pip install langchain-experimental langchain-openai
```

Next, load a text sample that lacks any structure (no headings, no paragraphs, no bullet points). In this example, the splitting threshold is set to the 90th percentile:

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai.embeddings import OpenAIEmbeddings

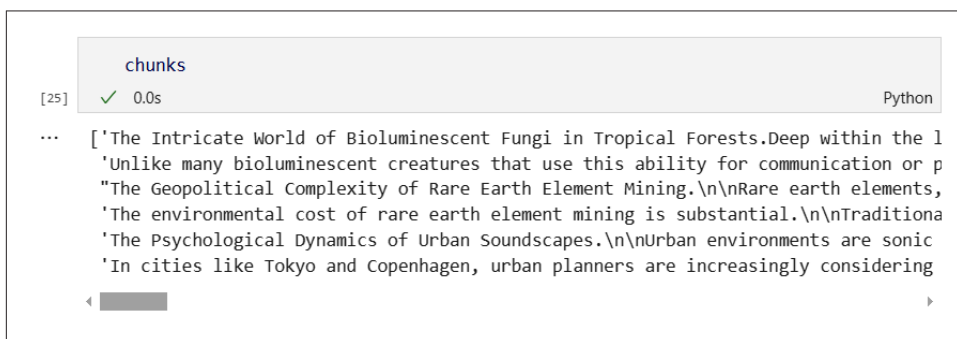
file_path = (
    "../datasets/text_files/"
    "random-text-about-5-different-stories.txt"
)

with open(file_path, "r") as file:
    text = file.read()

text_splitter = SemanticChunker(
    OpenAIEmbeddings(),
    breakpoint_threshold_type="percentile",
    breakpoint_threshold_amount=90,
)

chunks = text_splitter.split_text(text)
```

Figure 4-19 shows the chunks split from the sample document.



```
chunks
[25] ✓ 0.0s Python

... ['The Intricate World of Bioluminescent Fungi in Tropical Forests. Deep within the l
      'Unlike many bioluminescent creatures that use this ability for communication or p
      "The Geopolitical Complexity of Rare Earth Element Mining.\n\nRare earth elements,
      'The environmental cost of rare earth element mining is substantial.\n\nTraditiona
      'The Psychological Dynamics of Urban Soundscapes.\n\nUrban environments are sonic
      'In cities like Tokyo and Copenhagen, urban planners are increasingly considering
```

Figure 4-19. The split text chunks after using the semantic chunker

Discussion

Semantic chunking analyzes content meaning to identify natural breakpoints rather than relying on document structure or fixed character counts. This approach generates embedding vectors for small text pieces (typically sentences), then measures similarity between consecutive pieces. When similarity drops below a certain threshold, the system creates a new chunk.

This recipe uses the percentile method to set breakpoints. The system calculates distances between all consecutive sentence pairs, then sets the threshold at the 90th percentile of those distances. The system creates a new chunk only when the similarity drop is larger than 90% of all observed drops in the document.

This approach works for these reasons:

- Identifies topic shifts that don't align with structural boundaries
- Groups semantically related content even when formatting is inconsistent
- Creates chunks based on meaning rather than arbitrary rules

Use semantic chunking for the following:

- Unstructured text with no clear formatting (transcripts, concatenated emails, chat logs)
- Documents with topic boundaries that don't match paragraph breaks
- Use cases when retrieval quality justifies higher preprocessing costs

Don't use semantic chunking for the following:

- Documents with clear structure where document-aware or recursive splitting work well

- Large corpora that would result in embedding costs that are too high
- Real-time ingestion pipelines where embedding latency creates bottlenecks



Semantic chunking requires embedding each piece of text twice: once during the chunking process to identify boundaries, and again when ingesting chunks into the vector store. At \$0.02 per 1M tokens (typical embedding model pricing), this adds measurable cost. Balance this cost against retrieval quality improvements for your use case.



For production systems, implement your own version by using the LangChain source code as a reference rather than depending on the Experimental package. This gives you better control over the implementation and avoids potential breaking changes in experimental APIs.

See Also

- [“5 Levels of Text Splitting”](#) tutorial by Greg Kamradt provides practical examples of progressively more advanced splitting strategies.
- [LlamaIndex semantic chunker examples](#) show how to implement semantic chunking workflows with the LlamaIndex SemanticChunker.
- [LangChain SemanticChunker source code](#) shows the implementation details of the LangChain SemanticChunker.

4.8 Splitting Text with Agentic Chunkers

Problem

You’re dealing with long, complex documents and want to bundle related information into meaningful text chunks.

Solution

Agentic chunking creates standalone statements called *propositions*, where each proposition describes one clear piece of information. Agentic chunking uses an LLM to analyze text and break it down into these propositions.

Figure 4-20 shows the agentic chunking workflow:

1. Split text into chunks.
2. Iterate over each chunk, send it to the LLM, and generate propositions.
3. Iterate over all propositions and compare them with one another. If they contain redundant information, merge them.

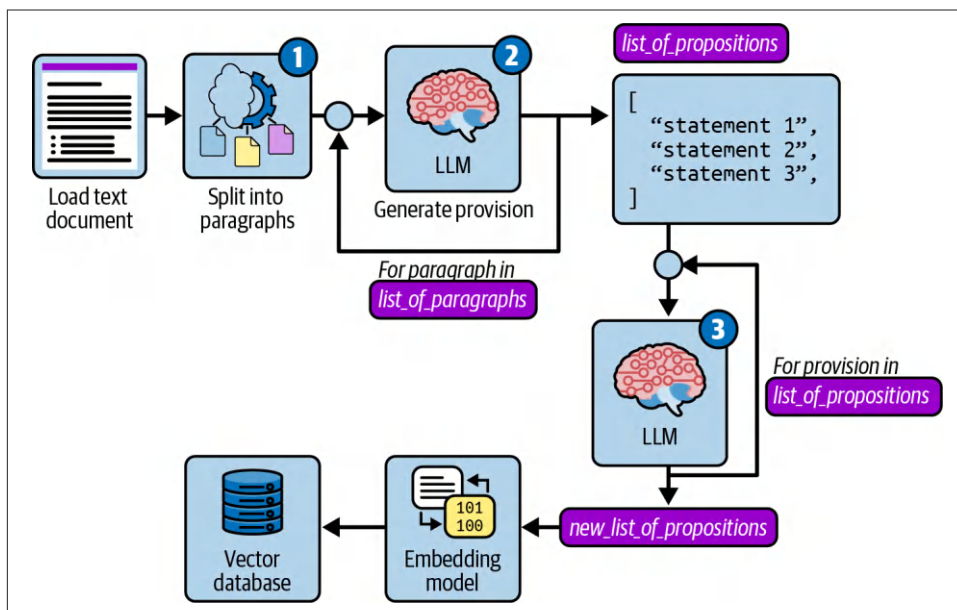


Figure 4-20. How agentic chunkers work

The prompt template used in this recipe comes from one of the first [papers](#) that described the concept of agentic chunking. You can find the full prompt in the [Lang-Chain Hub](#). Here's a shortened version to give you an idea of what it looks like:

Decompose the "Content" into clear and simple propositions, ensuring they are interpretable out of context.

1. Split compound sentence into simple sentences.
Maintain the original phrasing from the input whenever possible.
2. For any named entity that is accompanied by additional descriptive information, separate this information into its own distinct proposition.
3. Decontextualize the proposition by adding necessary modifier to nouns or entire sentences and replacing pronouns (e.g., "it", "he" "she", "they", "this", "that") with the full name of the entities they refer to.
4. Present the results as a list of strings, formatted in JSON.

Example:

Input: Title: Eostre. Section: Theories and interpretations,
Connection to Easter Hares. Content:

The earliest evidence for the Easter Hare (Osterhase) [...] America where it evolved into the Easter Bunny.

Output: ["The earliest evidence for the Easter Hare was recorded in south-west Germany in 1678 by Georg Franck von Franckenau.", "Georg Franck von Franckenau ..", "..."]

You send the prompt to the LLM along with the text chunk to be processed. The only packages needed are the OpenAI SDK and Pydantic, which you install as follows:

```
pip install langchain langchain-openai pydantic
```

Then use this prompt and create propositions:

```
from langchain import hub
from langchain_openai import ChatOpenAI
from pydantic import BaseModel
from typing import List

# Pull the prompt template from the langchain hub
obj = hub.pull("wfh/proposal-indexing")

llm = ChatOpenAI(model="gpt-5.2")

class Sentences(BaseModel):
    sentences: List[str]

extraction_llm = llm.with_structured_output(Sentences)

# Create the sentence extraction chain
extraction_chain = obj | extraction_llm

propositions = extraction_chain.invoke(
    """
    On July 20, 1969, astronaut Neil Armstrong walked on the moon .
    He was leading the NASA's Apollo 11 mission.
    Armstrong famously said, "That's one small step for man, one
    giant leap for mankind" as he stepped onto the lunar surface.
    """
)
```

Figure 4-21 shows the propositions that the LLM derives from the text snippet in the preceding code. From the short text sample about the Apollo 11 mission, the model generates four standalone statements.

```
for sentence in provisions.sentences:
    print(sentence)
```

[38] ✓ 0.0s Python

... On July 20, 1969, Neil Armstrong walked on the moon.
Neil Armstrong was an astronaut.
Neil Armstrong was leading NASA's Apollo 11 mission.
Neil Armstrong famously said, "That's one small step for man, one giant leap for mankind" as he s

Figure 4-21. The generated propositions

You can now either use these statements directly or merge related propositions first.

Discussion

Agentic chunking uses LLMs to decompose text into standalone propositions—atomic statements that each convey one clear piece of information without relying on surrounding context. This produces the highest-quality chunks for retrieval but requires the most preprocessing work.

This works for the following reasons:

- Eliminates pronouns and implicit references that create ambiguity
- Creates truly self-contained statements that can be understood in isolation
- Resolves cross-references within documents during preprocessing
- Merges redundant information into single, comprehensive statements

Use agentic chunking for the following:

- Legal contracts with extensive cross-references between clauses
- Technical specifications that reference other sections or external standards
- Documents with pronouns and implicit references that make chunks ambiguous
- Use cases where retrieval accuracy justifies high preprocessing costs

Don't use agentic chunking for these use cases:

- Well-written documentation with clear, self-contained paragraphs
- Large document corpora that would result in LLM costs that are too high
- Real-time ingestion pipelines (LLM latency makes this impractical)

Agentic chunking differs from alternatives:

- All other chunking methods split existing text at boundaries.

- Agentic chunking transforms text into new standalone statements.
- Semantic chunking groups by topic; agentic chunking decontextualizes and atomizes.
- This is the only approach that actively resolves references and pronouns.

The following example illustrates the transformation. The original text chunk is “Sarah bought a new book. She enjoys reading fantasy novels.”

From these two sentences, the LLM derives standalone propositions:

- Sarah bought a new book.
- Sarah enjoys reading fantasy novels.

Note that “She” becomes “Sarah” in the second proposition, making it self-contained.

To merge similar propositions, follow these steps:

1. Generate embeddings for all propositions.
2. Calculate semantic similarity between proposition pairs.
3. Merge highly similar propositions into a single statement.
4. Keep distinct propositions separate.

For implementation, use a simple similarity threshold (e.g., cosine similarity > 0.9) to identify candidates for merging, then send them to an LLM to generate the final merged statement.

Contracts are a typical use case. A 100-page contract with clauses referencing other sections or external documents creates challenges for basic RAG systems. Converting this content into standalone statements that don’t rely on surrounding context significantly improves retrieval accuracy.



Agentic chunking requires two or three LLM calls per chunk (for generation and optional merging). For a 500-chunk document, expect 30–60 minutes and \$15–\$40 in API costs when using GPT-4 class models. This is 10 to 20 times more expensive than semantic chunking. Use only when retrieval quality critically matters.

See Also

- [“5 Levels of Text Splitting”](#) tutorial by Greg Kamradt provides practical examples of progressively more advanced splitting strategies.
- [The agentic chunking implementation guide](#) by Thuwarakesh Murallie explains how to implement agentic chunking in practice.

Embeddings

Embedding models convert text, images, and other content into vectors that capture semantic meaning. In a RAG system, these vectors let the retriever search large unstructured collections for content that is relevant to a user's question. The retriever embeds the query, compares it to vectors stored in a vector database, and ranks candidates by distance. Smaller distances indicate higher semantic similarity, which determines what fits into the LLM's limited context window.

A typical embedding-based retrieval flow looks like this:

1. Split documents into chunks and embed them when building the vector database.
2. Embed each incoming user query with the same model.
3. Compute distances between the query vector and stored vectors.
4. Retrieve the closest chunks and pass them to the LLM as context.

This chapter shows how to work with embedding models from providers such as OpenAI, Google, and open source projects. You will generate embeddings, visualize semantic relationships, measure vector distances, and use them in practical RAG pipelines. The recipes also cover model selection, multimodal embeddings, and hybrid retrieval that combines vectors with keyword or metadata filters.

You can find all the code examples for this chapter in the [book's GitHub repository](#).

5.1 Mapping the Linguistic Meaning of Text Chunks to a Numerical Representation

Problem

You want to map the semantic meaning of words and sentences into a numerical representation.

Solution

Use an embedding model to convert text into numerical vectors. The following code examples demonstrate this with OpenAI and open source models. **Figure 5-1** shows how the resulting vectors position semantically similar text closer together in the embedding space.

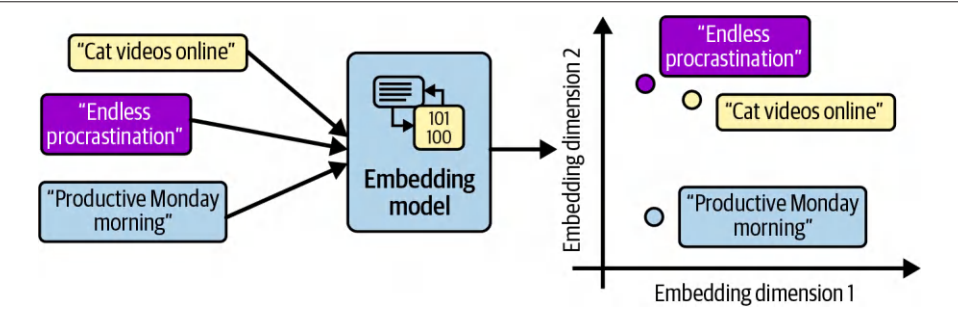


Figure 5-1. Semantic similarity between text chunks in vector space

The maximum size of a text chunk cannot exceed the maximum token length supported by the embedding model. **Table 5-1** shows the maximum token window and the number of dimensions for popular embedding models.

Table 5-1. Modern embedding models

Model	Company	Platform	Max tokens	Dimensions
text-embedding-3-small	OpenAI	OpenAI API	8,191	1,536
voyage-large-2 instruct	Voyage AI	Voyage API	16,000	1,536
text-embedding-005	Google	Gemini API	2,048	768
all-MiniLM-L6-v2	Open source	Hugging Face	512	384

This recipe contains an example for two sample models:

- One of OpenAI’s powerful embedding models

- The popular open source model all-MiniLM-L6-v2, which offers lower accuracy but runs locally

To use them, you need to install the OpenAI SDK and the Sentence Transformers library:

```
pip install openai sentence-transformers
```

Then use the OpenAI SDK and one of OpenAI's text embedding models to generate embedding vectors for two sample text strings:

```
from openai import OpenAI

text_chunks = ["The sky is blue.", "The grass is green."]

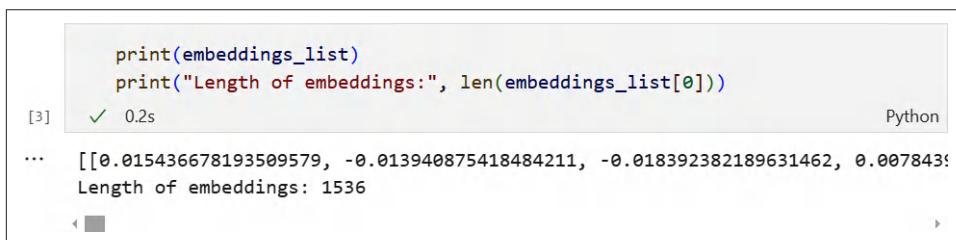
client = OpenAI() # Uses the environment variable OPENAI_API_KEY

embeddings_list = []

for text_chunk in text_chunks:
    response = client.embeddings.create(
        input=[text_chunk], model="text-embedding-3-small"
    )
    embedding = response.data[0].embedding
    embeddings_list.append(embedding)
```

The response object returned by the OpenAI API contains a `data` attribute, which is a list of embedding objects. Each embedding object has an `embedding` attribute that holds the actual vector, a Python list of floating-point numbers. For the `text-embedding-3-small` model, this list contains 1,536 values. The embeddings are normalized, with individual values typically ranging from -1 to 1 , though most fall within a narrower range. The preceding code extracts the first embedding from the response via `response.data[0].embedding`.

Figure 5-2 shows what the list of embedding vectors looks like. Here, the generated embeddings have 1,536 dimensions per vector.



```
print(embeddings_list)
print("Length of embeddings:", len(embeddings_list[0]))
```

[3] ✓ 0.2s Python

```
... [[0.015436678193509579, -0.013940875418484211, -0.018392382189631462, 0.007843...
Length of embeddings: 1536
```

Figure 5-2. Sample output of 1,536-dimensional embedding vectors

As an alternative to OpenAI, you can use smaller open source models. The following example uses one of the most popular open source models, all-MiniLM-L6-v2. While the accuracy and performance is not comparable to modern transformer-based foundation embedding models from Google or OpenAI, it remains a popular choice because it's fast, cost-efficient, and can run on your own infrastructure, which allows you to deploy it even on smaller edge devices or your local laptop.

In this example, you're using the Sentence Transformers library, which builds on Hugging Face Transformers and is specifically designed for generating text embeddings:

```
from sentence_transformers import SentenceTransformer

# Load the model
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
text_chunks = ["The sky is blue.", "The grass is green."]

embeddings = model.encode(text_chunks)
```

The encode method returns a NumPy array with each row representing the embedding vector for the corresponding text chunk. The resulting embeddings array has shape (2, 384), meaning two text chunks with 384-dimensional vectors.

Discussion

Embedding models transform text into numerical vectors that capture semantic meaning. This is the core mechanism enabling RAG systems to find relevant content by measuring similarity in vector space rather than relying on exact keyword matches.

Modern text embedding models learn semantic relationships by training on large text datasets. Understanding how they differ from simpler encoding methods like one-hot encoding clarifies their power. One-hot encoding treats categories as independent labels with no semantic relationship, while embedding models learn meaningful representations when similar concepts are positioned close together in vector space.

Figure 5-3 shows one-hot encoding, which represents “Bachelor’s” as [0, 1, 0], “Master’s” as [1, 0, 0], and “PhD” as [0, 0, 1]. These randomly assigned vectors lose all semantic information. They don’t capture that a PhD student typically has a bachelor’s degree first, or that these degrees represent different levels of education.

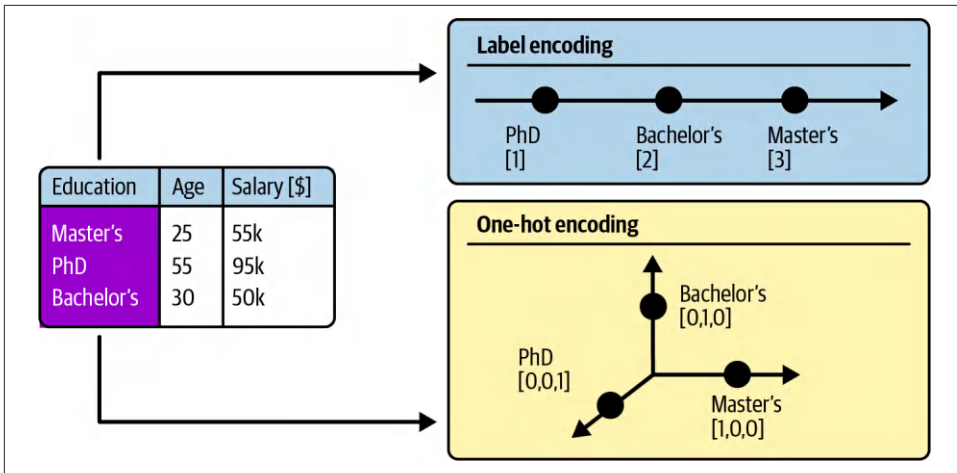


Figure 5-3. Label encoding versus one-hot encoding for categorical data



A *vector* is a list of numbers representing something in a structured way (like coordinates on a map where [5, 3] means “five blocks north and three blocks east”). Vectors let us represent abstract concepts (like word meanings) as numbers that computers can work with and compare mathematically.

RAG systems depend on embedding models that capture semantic meaning because they enable retrieval based on conceptual relationships rather than just keyword matches. Instead of random numbers, embedding models create vectors and give each dimension a clear meaning, mapping characteristics into vector space.

In the example in [Figure 5-4](#), the model maps the words queen, king, princess, and prince by using three meaningful dimensions: age (kings are typically older than princesses), gender (queens and princesses are female), and royal status (hierarchy levels in the royal family). Each number in the vector indicates one of the three characteristics.

So a king gets the vector [0.8, 0, 1], which stands for an older male with high status. A princess is represented as [0.3, 1, 0.5] (younger, female, lower status).

You can even perform arithmetic with these vectors. When you add or subtract vectors, you add or subtract each corresponding number:

$$\begin{aligned}
 &\text{king} - \text{man} + \text{woman} = \text{queen} \\
 &[0.8, 0, 1] - [0.7, 0, 0.6] + [0.6, 1, 0.5] \\
 &= [0.8 - 0.7 + 0.6, 0 - 0 + 1, 1 - 0.6 + 0.5] \\
 &= [0.7, 1, 0.9]
 \end{aligned}$$

This works because subtracting *man* and adding *woman* changes the gender dimension while preserving age and status, resulting in a vector that describes a queen.

Figure 5-4 also shows a second example using capitals and countries: France - Germany + Berlin = Paris.

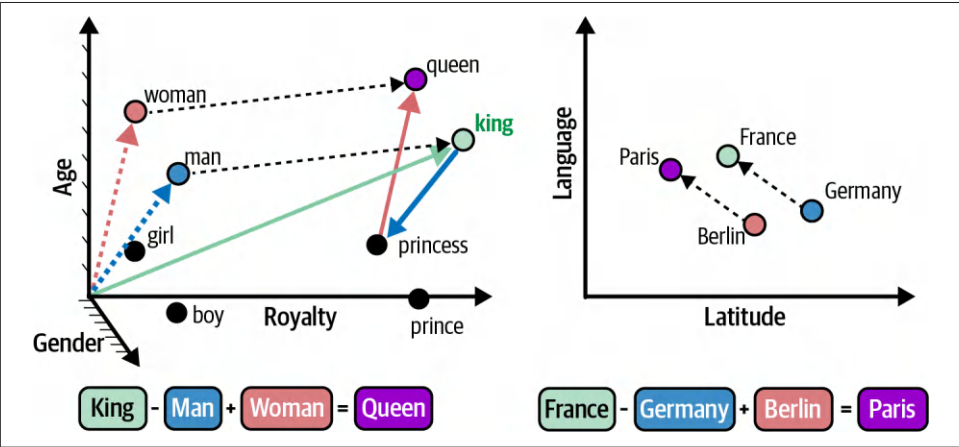


Figure 5-4. Vector arithmetic with meaningful embedding dimensions

Embedding models are neural networks trained on large text datasets. Each unique word in the dataset is assigned to a neuron in the input layer. Figure 5-5 shows a simple example with a hidden layer containing only two neurons. During training, the neural network predicts the next word for each input word. For example, when the input neuron for Google is activated, the network is trained to predict the word is. Once the model is fully trained, the weights of the hidden-layer neurons become the embedding model. These learned weights map every word or sentence into a space and determine each word's position.

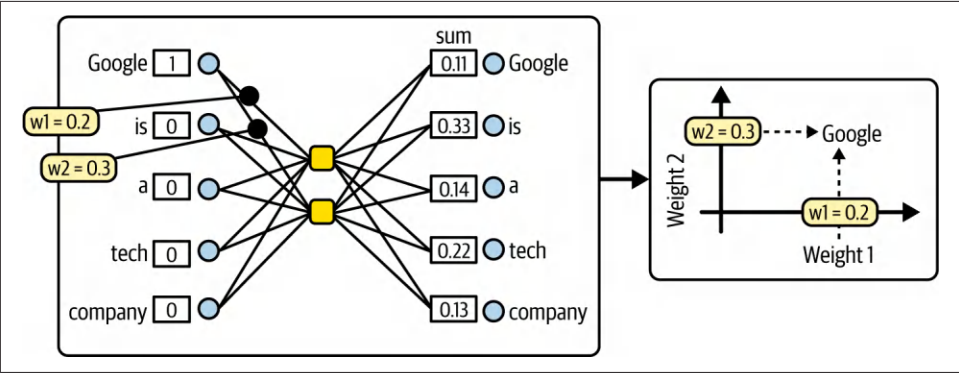


Figure 5-5. Neural network architecture for generating text embeddings

Use embedding models for any RAG system that needs to find relevant content based on semantic meaning rather than exact keyword matches. This includes question answering, document retrieval, and content recommendation systems.

Avoid embeddings when you need exact string matching (use full-text search), when dealing with highly structured data like database queries (use SQL), or when interpretability is critical (embeddings are opaque boxes). The main trade-off is cost versus accuracy: cloud-based models like OpenAI's text-embedding-3-small offer better accuracy but incur API costs, while open source models like all-MiniLM-L6-v2 run locally but with lower retrieval quality.

While their evolution has not been as dramatic as LLMs, embedding models remain fundamental to RAG systems. Most current models, even those from a few years ago, deliver strong performance, making the choice more about infrastructure constraints (cloud versus local) than accuracy differences.

See Also

- The [Word Embedding and Word2Vec playlist](#) by Joshua Starmer provides visual explanations of how embedding models work.
- The [Hugging Face embeddings guide](#) covers implementation details and model architectures.

5.2 Visualizing Semantic Relationships Between Text Chunks via Dimensionality Reduction Techniques

Problem

You want to visualize high-dimensional embedding vectors in two dimensions to verify that semantically similar text appears clustered together.

Solution

Figure 5-6 shows the dimensionality reduction process. Use principal component analysis (PCA) to reduce embedding dimensions from 1,536 to 2 and plot them with Matplotlib. This reveals semantic patterns by showing which text chunks cluster together based on meaning.

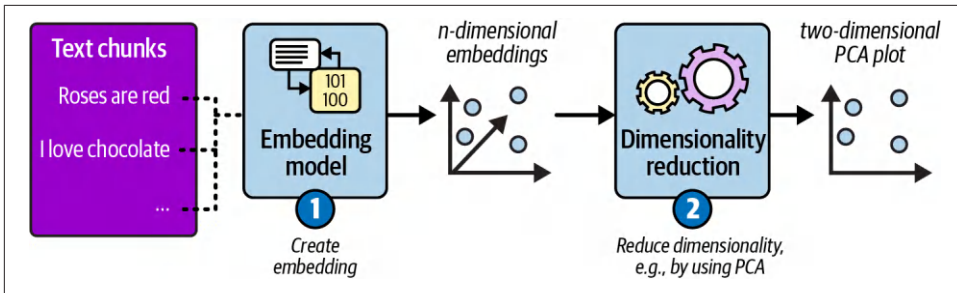


Figure 5-6. Dimensionality reduction process using PCA

The code uses PCA from scikit-learn, a popular machine learning library that provides tools for dimensionality reduction and data analysis. Matplotlib is used to create the visualization, and pandas helps manage the embedding data in tabular form.

To run the code, install the following dependencies:

```
pip install openai scikit-learn matplotlib pandas
```

Then generate embeddings for six example sentences and store them together with the text chunks in a DataFrame:

```
import os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from openai import OpenAI

# Define text chunks
text_chunks = [
    "The sky is blue.",
    "The sun is shining.",
    "I love chocolate.",
    "Ice cream is delicious.",
    "Roses are red.",
    "Violets are blue.",
]

# Initialize OpenAI client
# Assumes the OPENAI_API_KEY environment variable is set
client = OpenAI()
embeddings = []

# Generate embeddings for text chunks
for text_chunk in text_chunks:
    response = client.embeddings.create(
        input=[text_chunk], model="text-embedding-3-small"
    )
    embeddings.append(response.data[0].embedding)
```

```

# Convert embeddings to a DataFrame
# Each row represents one text chunk, each column one embedding dimension
embeddings_df = pd.DataFrame(
    embeddings, columns=[f"dim_{i}" for i in range(len(embeddings[0]))]
)

```

Next, the PCA function from scikit-learn transforms the embedding vectors into a new coordinate system. Each principal component is a new axis that captures as much variation in the data as possible. The first components therefore contain most of the information. By visualizing only the first two components, you can often identify semantic patterns:

```

# Perform PCA with 2 components
pca = PCA(n_components=2)
df_reduced = pca.fit_transform(embeddings_df)

# Create a new DataFrame with reduced dimensions
df_reduced = pd.DataFrame(df_reduced, columns=["PC1", "PC2"])
df_reduced["text"] = text_chunks # Add original text chunks for labeling

# Create a scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(df_reduced["PC1"], df_reduced["PC2"])

# Add labels to each point
for i, label in enumerate(df_reduced["text"]):
    plt.text(df_reduced["PC1"][i], df_reduced["PC2"][i], label, fontsize=9)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Scatter Plot")

# Save the plot
plt.savefig("../principal_component_plot.svg", format="svg")

```

The plot in [Figure 5-7](#) shows the results. In this example, sentences about food and drinks appear grouped on the right, weather-related sentences in the lower left, and sentences about flowers in the upper left.

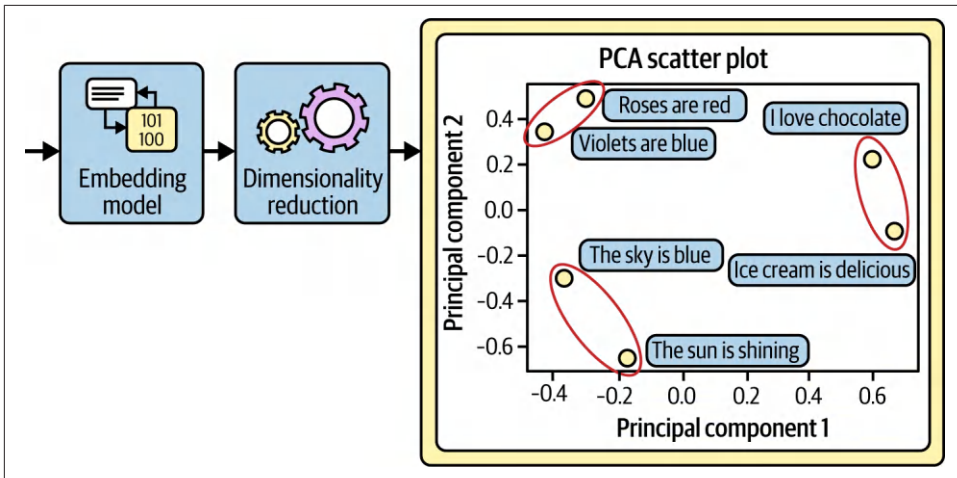


Figure 5-7. PCA visualization showing semantic clusters in two dimensions

The visualization confirms that semantically related text chunks cluster together in the reduced space. This validates that the embedding model captures meaningful semantic relationships, with similar topics grouping naturally despite the dramatic dimensionality reduction from 1,536 to 2 dimensions.

Discussion

Dimensionality reduction works by projecting high-dimensional vectors onto a lower-dimensional space—typically, two-dimensional (2D) or three-dimensional (3D)—while preserving relative distances between points. PCA finds the directions of maximum variance in the data and uses them as the new axes. This means semantically similar text chunks that are close in 1,536-dimensional space often remain close in 2D.

You use visualization during development to verify that your embedding model clusters semantically similar content together. This validates that “legal documents” group separately from “marketing materials” before deploying the system. Visualization also builds stakeholder trust in regulated industries by making retrieval behavior tangible during demos.

Don’t use 2D plots in production for diagnostics. They simplify 1,536 dimensions down to 2, which means they can’t show all relationships. Use them during development to verify clustering behavior, but in production track retrieval metrics like precision@k instead.

The main trade-off is simplicity versus accuracy. Uniform Manifold Approximation and Projection (UMAP) and t-SNE (t-Distributed Stochastic Neighbor Embedding) preserve local structure better than PCA but take longer to compute and can produce

misleading patterns if parameters are poorly tuned. PCA is faster and deterministic but may not capture nonlinear relationships as well.

See Also

- The [PCA documentation](#) in scikit-learn covers implementation and parameters for dimensionality reduction.
- The [manifold learning guide](#) in scikit-learn explains t-SNE, UMAP, and other dimensionality reduction techniques.

5.3 Calculating the Distance Between Embeddings

Problem

You want to measure the similarity of two pieces of text by calculating the distance between their embedding vectors.

Solution

Calculate cosine similarity between embedding vectors with NumPy or the Sentence Transformers library.

This recipe uses OpenAI text embedding models to generate embeddings and NumPy to calculate cosine similarity. Install them as follows:

```
pip install openai numpy sentence-transformers pandas
```

Then generate embeddings for three historical statements and one user question. The goal is to find the historical fact that best matches the user's question, which asks for information about diseases in history:

```
import numpy as np
from numpy.linalg import norm
import pandas as pd
import os
from openai import OpenAI

text_chunks = [
    "The Great Fire of London in 1666 destroyed over 13,000 houses.",
    "Julius Caesar was assassinated on the Ides of March (March 15) in 44 BCE.",
    "The Black Death is estimated to have killed nearly one-third of the "
    "European population.",
]

users_question = "Tell me something interesting about diseases in history"

embeddings_df = pd.DataFrame(text_chunks, columns=["text_chunk"])
```

```

client = OpenAI()
embeddings = []

def create_embeddings(text_chunk, client):
    return client.embeddings.create(
        input=[text_chunk],
        model="text-embedding-3-small"
    ).data[0].embedding

# Apply function create_embeddings to the correct column
embeddings_df["embedding"] = embeddings_df["text_chunk"].apply(
    create_embeddings, client=client
)

users_question_embedding = create_embeddings(
    text_chunk=users_question, client=client
)

```

Next, calculate cosine similarity by using NumPy:

```

# Create a list to store the calculated cosine similarity
cos_sim = []

def calculate_cosine_similarity(text_chunk_embedding, users_question_embedding):
    A = text_chunk_embedding
    B = users_question_embedding

    # Calculate the cosine similarity
    cosine = np.dot(A, B) / (norm(A) * norm(B))
    return cosine

# Apply function calculate_cosine_similarity to the correct column
embeddings_df["similarity"] = embeddings_df["embedding"].apply(
    calculate_cosine_similarity, users_question_embedding=users_question_embedding
)

```

Figure 5-8 shows the output. The DataFrame with the embeddings is now enriched with a similarity column, which shows the cosine similarity of each sentence to the user's question. In this basic example, the sentence about the Black Death shows the highest similarity.

embeddings_df			
✓	0.0s	Python	
	text_chunk	embedding	similarity
0	The Great Fire of London in 1666 destroyed ove...	[0.04129223898053169, -0.021087143570184708, 0...	0.203967
1	Julius Caesar was assassinated on the Ides of ...	[-0.025326920673251152, -0.004735556431114674, ...	0.108710
2	The Black Death, is estimated to have killed n...	[0.04071861132979393, 0.019235173240303993, 0...	0.319896

Figure 5-8. Cosine similarity scores for historical text queries

Figure 5-9 shows the cosine similarity formula. Using PCA from Recipe 5.2, the embedding vectors are also plotted as a 2D visualization on the right side of the figure.

Cosine similarity is the inverse of cosine distance, so the Black Death with a cosine similarity of 0.31 is the closest match. Keep in mind that cosine similarity measures the angle between vectors in high-dimensional space, not absolute magnitude. The 2D plot illustrates the concept but does not reflect the true distance in the original high-dimensional space.

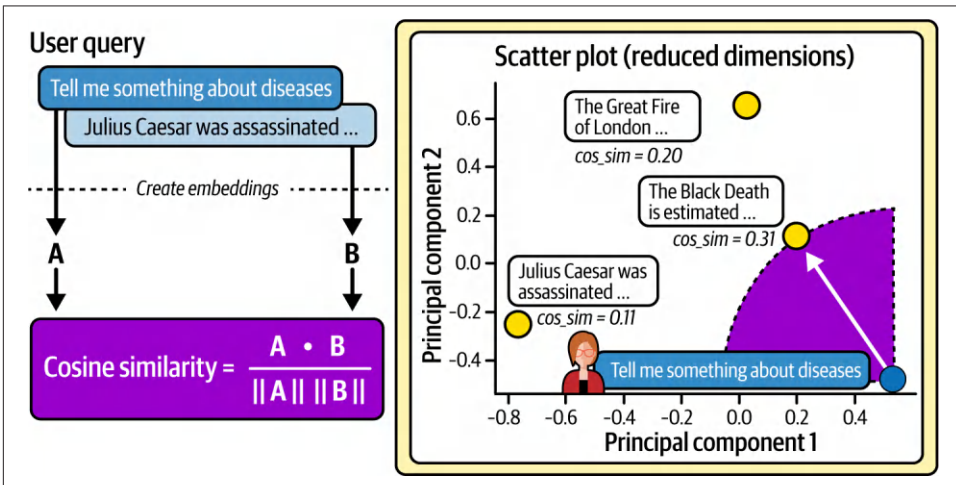


Figure 5-9. Cosine similarity formula and geometric representation

Discussion

Cosine similarity measures the angle between two vectors rather than their absolute distance. For RAG systems, this is the preferred distance metric because it focuses on

semantic direction rather than magnitude. A short query like “reset password” and a long paragraph explaining password reset procedures point in the same semantic direction and receive a high similarity score, even though their vector magnitudes differ significantly.

Figure 5-10 shows three common distance metrics used with embeddings: cosine similarity, Euclidean distance, and Manhattan distance.

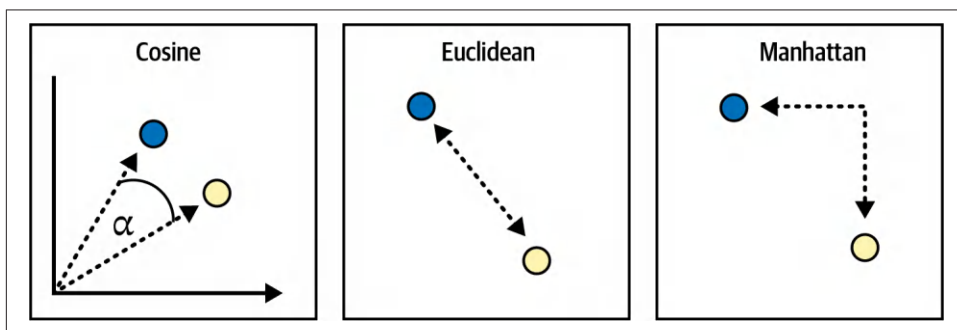


Figure 5-10. Common distance metrics for measuring vector similarity

This robustness to length differences matters because user queries are typically much shorter than retrieved documents. Without normalization, longer documents would dominate rankings simply because of magnitude, not relevance.

Use cosine similarity for standard RAG retrieval with modern transformer-based embeddings. It works well across different text lengths and effectively captures semantic meaning. Use Euclidean distance when vector magnitude carries semantic meaning, which is rare in modern embeddings but can occur in specialized domains where embedding norms encode additional information. Use dot product for speed in production systems where you can normalize vectors once during indexing and reuse them, avoiding repeated normalization overhead.

Avoid Manhattan distance unless working with high-dimensional sparse vectors where it offers computational advantages. In practice, cosine similarity remains the default for over 95% of RAG systems using transformer-based embeddings.

See Also

- The [NumPy linear algebra documentation](#) covers norm, dot, and other operations for vector calculations.
- The [Sentence Transformers util module](#) provides functions for cosine similarity, dot product, and pairwise distances.
- The [scipy.spatial.distance documentation](#) covers implementation of cosine, Euclidean, and Manhattan distance metrics.

5.4 Choosing the Right Embedding Model

Problem

You need to pick the right embedding model for your project.

Solution

Follow these steps to select the right embedding model for your RAG system:

1. Check benchmarks: review the Massive Text Embedding Benchmark (MTEB) leaderboard at [Hugging Face](#) to see model performance across tasks. For multilingual systems, check MIRACL (Multilingual Information Retrieval Across a Continuum of Languages) benchmark scores.
2. Identify deployment constraints: determine whether you can use commercial cloud APIs (OpenAI, Google, Cohere) or need local deployment because of data sensitivity or offline requirements.
3. Establish baselines: start with proven models like OpenAI's text-embedding-3-small when using commercial providers or all-MiniLM-L6-v2 for local deployment.
4. Test with real data: run end-to-end tests using 50–100 representative queries from your actual use case. Measure retrieval accuracy by checking whether the correct documents appear in the top results.
5. Compare alternatives: if baseline accuracy is insufficient, test a larger model (e.g., text-embedding-3-large) with the same query set to quantify the improvement.
6. Measure performance: use your vector database's query endpoint to measure average retrieval latency. Ensure that it meets your application's requirements (< 100 ms for interactive apps).
7. Calculate costs: for cloud APIs, multiply expected monthly query volume by per-token pricing. Compare the result against the baseline to validate budget fit.

The main decision is whether you can use commercial cloud models or need open source models for local deployment. For RAG applications, commercial models like text-embedding-3-small or Google's text-embedding-005 work well for most use cases.

Discussion

[Table 5-2](#) provides guidance on selecting models based on your use case.

Table 5-2. Embedding model selection criteria

Use case	Recommended approach
Large datasets (> 1M documents) or high query volume (> 1,000/day)	Prioritize speed and cost with smaller models.
Critical accuracy needs (legal, medical, financial) with < 100K documents	Use larger models despite higher cost.
Real-time user-facing applications	Favor models with < 50 ms latency.
Batch processing or offline indexing	Accuracy can outweigh speed.

Use cloud-based models (such as OpenAI’s text-embedding-3-small or Google’s text-embedding-005) when you can tolerate API latency and costs in exchange for state-of-the-art accuracy. These models excel at understanding nuanced queries and work well across multiple languages without retraining.

Use open source models (such as all-MiniLM-L6-v2 or BGE-small-en-v1.5) when data cannot leave your infrastructure because of compliance requirements, API costs exceed budget constraints at your query volume, or you need offline operation. The accuracy gap has narrowed considerably—modern open source models approach commercial performance for many retrieval tasks.

Avoid oversizing your embedding model. If your baseline tests with text-embedding-3-small show 85% retrieval accuracy and upgrading to text-embedding-3-large achieves 87%, the 2% improvement rarely justifies doubling costs and latency for most applications. Focus optimization efforts on chunking strategy and retrieval configuration instead.

The main risk is committing to a model without validating on your actual data. Benchmarks like MTEB measure general performance across diverse tasks, but your specific domain (legal contracts, medical records, code documentation) may behave differently. Always test with 50–100 real queries from your use case before production deployment.



Embedding models have not advanced as rapidly as LLMs in recent years. Many RAG systems built years ago still use OpenAI’s text-embedding-ada-002 because the accuracy remains sufficient for retrieval tasks. This stability means model selection is a one-time decision that rarely needs revisiting.

See Also

- The [MTEB leaderboard](#) on Hugging Face provides model benchmarks and details including context window sizes.
- The [OpenAI embeddings guide](#) covers API usage, model selection, and best practices.

5.5 Generating Embeddings for Images and Text with CLIP

Problem

You want to classify images or perform cross-modal matching between images and text by using embeddings.

Solution

Use Contrastive Language–Image Pre-training (CLIP) to generate embeddings for both images and text, enabling cross-modal matching and classification.

Figure 5-11 illustrates how to use CLIP as part of your retrieval system:

1. Generate embeddings for all images and text you want to add to the vector store.
2. When a user asks a question, use CLIP to generate an embedding for the query and search the vector store.
3. Compare the newly generated embedding vector to the embedding vectors stored in the vector store.
4. Send the retrieved images and text to a multimodal model that can interpret both at once and answer the user's question.

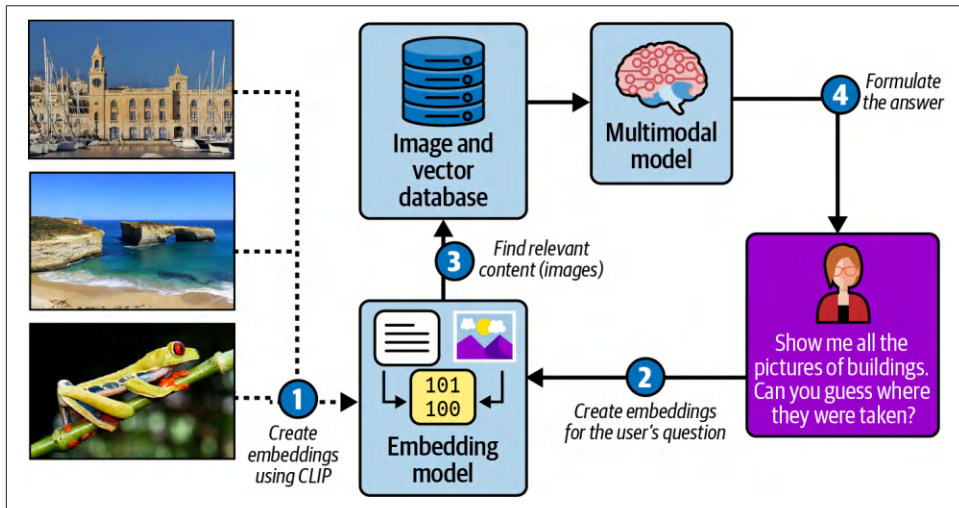


Figure 5-11. CLIP-based multimodal retrieval architecture

This approach allows you to build a large image database and query it by asking questions, because a CLIP-based retriever can connect text and images in both directions.

This recipe demonstrates classification with CLIP. While the earlier diagrams showed how CLIP enables multimodal retrieval in RAG systems, now we'll focus on classification because it clearly demonstrates CLIP's core capability: mapping images and text into a shared embedding space. An image classifier works similarly to a RAG retriever—both compare embeddings to find the closest match.

With a few lines of code, you can build a classifier for images of cats and dogs. **Figure 5-12** shows how this works with CLIP. You generate embeddings for images and embeddings for descriptive text like “a photo of a cat” and “a photo of a dog.” When you classify a new image, you compare its embedding to the text embeddings. The closest match determines the predicted label.

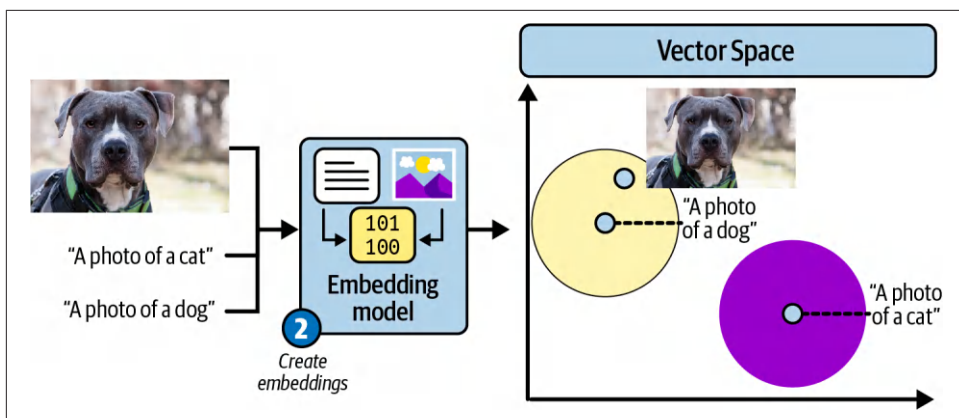


Figure 5-12. Cross-modal classification with clip embeddings

CLIP is open source, which means you can load it and run it on your local machine by using PyTorch. To preprocess images, you can use the Pillow library. Install both as follows:

```
pip install transformers torch pillow
```

Then load the CLIP model to create embeddings for images and text chunks. The CLIP processor tokenizes the text and preprocesses the images. Next, CLIP generates the embeddings, and you compute similarity scores for each image-text pair. The output contains similarity scores between each image embedding and each text embedding. The probabilities are derived from these similarity scores by using the softmax function, ensuring that they sum to 1:

```
import torch
from PIL import Image
from transformers import CLIPProcessor, CLIPModel

# Load the model and processor
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
```



```

# Text and image inputs
descriptions = ["A photo of a cat", "A photo of a dog"]
images = [
    Image.open("../datasets/images/cat.jpg"),
    Image.open("../datasets/images/dog.jpg"),
]

# Disable gradient tracking since this code only runs inference
with torch.no_grad():
    inputs = processor(
        text=descriptions,
        images=images,
        return_tensors="pt",
        padding=True,
        truncation=True,
    )
    outputs = model(**inputs)

dot_products_per_text = outputs.logits_per_text

# Calculate probabilities
probabilities = dot_products_per_text.softmax(dim=1)

```

The final output is a probability matrix. The first column shows the likelihood that each image contains a cat, and the second column shows the likelihood that each image contains a dog. In other words, the classification approach will still assign the similarity of each image to the text descriptions.

Figure 5-13 shows the results. In this example, the probability that the first image shows a cat is 0.9999, which means the probability that it shows a dog is extremely low. Remember that the probability always sums to 1 across all classes, so if the image is of an elephant, but elephant is not one of the classes, the classification approach will still assign a high probability to either cat or dog.

```

print(probabilities)

print("Probability that the first image is a cat:", probabilities[0][0].item())
print("Probability that the second image is a dog:", probabilities[1][1].item())

```

✓ 0.0s Python

```

tensor([[9.9995e-01, 5.3789e-05],
        [1.2071e-01, 8.7929e-01]])
Probability that the first image is a cat: 0.9999462366104126
Probability that the second image is a dog: 0.8792922496795654

```

Figure 5-13. CLIP classification probabilities for cat and dog images

Discussion

CLIP works by training on millions of image-text pairs from the internet, learning to map both modalities into a shared vector space. The key insight is that semantically similar content ends up close together in this space regardless of modality. An image of a dog produces a vector near the text “a photo of a dog” because the model learned this association during training.

The model requires no task-specific training—you simply provide candidate text descriptions, and CLIP ranks them by similarity to the image.

Use CLIP for image classification when you have many possible categories and want to avoid training a traditional classifier. Adding new categories requires only writing new text descriptions, not retraining the model. This makes CLIP powerful for rapidly evolving taxonomies or when labeled training data is scarce.

Use CLIP for cross-modal retrieval in domains where images and text are tightly coupled, such as product catalogs where users might search by description or image, or scientific papers where figures relate closely to textual explanations.

However, for most RAG applications, a simpler approach works better: use multimodal models (GPT-5.2, Claude Sonnet 4.5) to generate text descriptions of images, then embed those descriptions with standard text embedding models. Store the text descriptions in your vector database and pass only the original image to the LLM when that chunk is retrieved. This approach leverages text embedding models that are optimized for semantic text matching, while CLIP’s direct image-text mapping adds complexity.



CLIP’s main limitation is that it can’t reliably handle text within images. A photo of a No Parking street sign may not embed near the text “no parking” if the model treats the input as a generic sign image. Extracting and embedding the text separately avoids this problem.

See Also

- The [CLIP research page](#) from OpenAI explains the model architecture and training approach.
- The [CLIP model documentation](#) on Hugging Face covers implementation details and usage examples.
- The [LlamaIndex multimodal retrieval example](#) demonstrates building a multimodal RAG system with CLIP.

5.6 Performing Text Classification with Embeddings

Problem

You want to route user questions to specific data subsets (e.g., by topic, department, or document type) by training a classifier on text embeddings.

Solution

Train a classifier on text embeddings to route user questions to specific data subsets in your vector store. This prefiltering step narrows the search space before semantic retrieval runs.

Figure 5-14 shows the two-step process: prepare your dataset by chunking documents and generating embeddings, then train a classifier by using these embedding vectors as features to predict document categories.

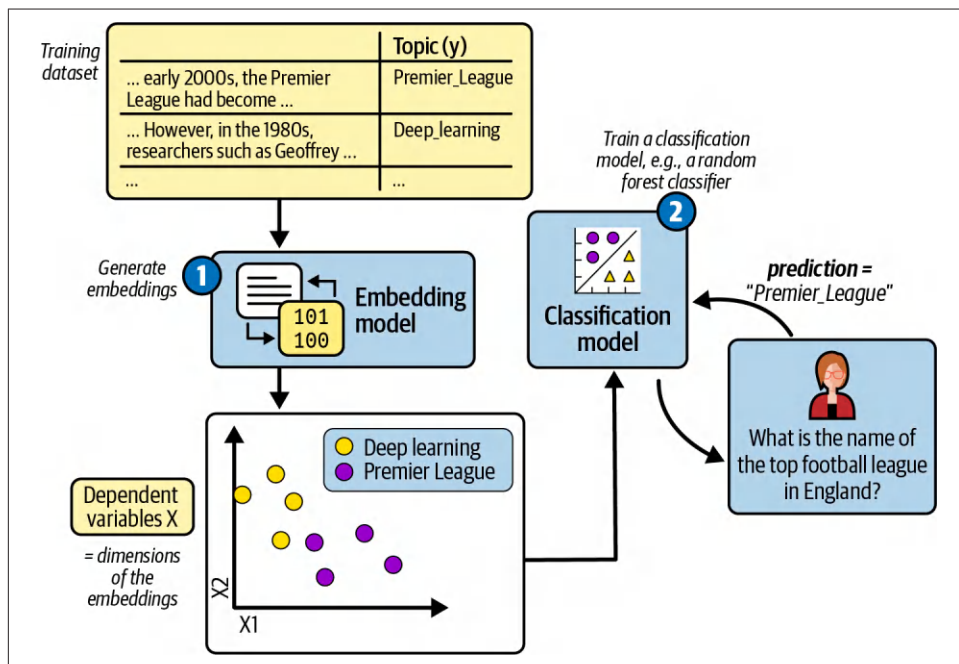


Figure 5-14. Training a random forest classifier with text embeddings

This recipe demonstrates the process with two PDF files. First, install the required dependencies:

```
pip install openai scikit-learn pandas PyPDF2 langchain-text-splitters
```



The following code loads example PDF files from the [book's GitHub repository](#) at `datasets/pdf_files/`. Download these files or adjust the paths to your own documents.

Next, load two example text files, one about the history of the Premier League and one about the history of deep learning, and then apply recursive chunking to create text chunks:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
import PyPDF2
import pandas as pd

pdf_files = [
    {
        "file_path": "../datasets/pdf_files/history_of_deep_learning.pdf",
        "label": "Deep_Learning",
    },
    {
        "file_path": "../datasets/pdf_files/premier_league_history.pdf",
        "label": "Premier_League",
    },
]

chunks_dict_list = []

# Split both documents into chunks and append to the same list of dicts
for pdf_file in pdf_files:
    with open(pdf_file["file_path"], "rb") as file:
        reader = PyPDF2.PdfReader(file)

        text = ""
        for page in reader.pages:
            text += page.extract_text()

        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=200,
            chunk_overlap=0,
            length_function=len,
            is_separator_regex=False,
        )

        chunks = text_splitter.split_text(text)

        for chunk in chunks:
            chunks_dict_list.append({"text": chunk, "label": pdf_file["label"]})

chunks_df = pd.DataFrame(chunks_dict_list)
```

Now iterate over the just-created dataset and generate an embedding vector for each text chunk:

```

client = OpenAI()

def create_embeddings(text_chunk, client):
    return client.embeddings.create(
        input=[text_chunk],
        model="text-embedding-3-small"
    ).data[0].embedding

chunks_df["embedding"] = chunks_df["text"].apply(
    create_embeddings,
    client=client
)

```

The resulting DataFrame is the foundation for training the classifier.

You set the labels as the target vector y . The feature matrix X is defined by the embeddings. Using a lambda function, each dimension in the embedding vector is mapped to one column in the input matrix X . So the resulting input matrix has 1,536 columns, one for each dimension of the embedding vector.

Finally, train the random forest classifier by using scikit-learn:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

y = chunks_df["label"]
X = chunks_df["embedding"].apply(
    lambda x: pd.Series(eval(x)) if isinstance(x, str) else pd.Series(x)
)

# Train a random forest classifier
clf = RandomForestClassifier()
clf.fit(X, y)

```

Next, you test the model with a question that is clearly about football (soccer). The random forest predicts the probability for each class. In this example, the model is 94% confident that the question is related to the Premier League content:

```

test_embedding = create_embeddings(
    text_chunk="What is the name of the top football league in England?",
    client=client,
)

X_test = [test_embedding]

# Predict the most likely class
predicted_classes = clf.predict(X_test) # e.g., ['Premier_League']
probabilities = clf.predict_proba(X_test) # e.g., array([[0.06, 0.94]])

```

The trained model can now be used as part of your RAG workflow.

Discussion

Classification routing works by training a model to map embedding vectors to predefined categories. The random forest classifier learns patterns in the 1,536-dimensional embedding space that distinguish topics. When a new query arrives, its embedding is fed to the classifier, which predicts the most likely category with a probability score.

This prefiltering narrows the search space before semantic retrieval runs. Instead of searching all 100,000 documents in your vector store, the classifier routes sports queries to the 20,000 sports documents, reducing retrieval latency and improving precision.

Use classification routing when your vector store combines drastically different content types where semantic similarity can be misleading. A question about “market performance” might retrieve sports articles about team performance instead of financial data because embedding models see them as semantically similar, for example. Classification prevents this by routing the query to the correct domain first.

Use classification routing when you have clear, stable categories (sports, politics, finance) with sufficient training examples per category (at least 100–200 labeled chunks). The classifier needs enough examples to learn discriminative patterns in the embedding space.

Avoid classification routing when categories overlap significantly or when domain boundaries are fuzzy. Questions about “sports betting regulations” span both sports and legal domains, and forcing a single category loses relevant information. For overlapping domains, metadata filtering works better: assign category metadata fields during ingestion, then filter by `category IN ['sports', 'legal']` before semantic search. This preserves documents from multiple relevant categories without requiring a classifier.

Avoid classification routing when your document collection is small (< 10,000 documents) or homogeneous. The overhead of training and maintaining a classifier outweighs benefits when semantic search alone performs well. Start with semantic search. Add classification only when you observe cross-domain confusion in retrieval logs.

The main trade-off is precision versus maintenance burden. Classification improves routing precision for distinct domains but requires labeled training data, periodic retraining as content evolves, and additional inference latency (typically 10–20 ms). The classifier can also make mistakes—misrouting queries loses relevant documents entirely, unlike semantic search that might return suboptimal but still-relevant results.

See Also

- The [random forest documentation](#) in scikit-learn covers classifier implementation and parameters.
- The [model evaluation guide](#) in scikit-learn explains metrics for assessing classification performance.
- The [scikit-llm library documentation](#) shows how to integrate transformer models with scikit-learn workflows.

5.7 Improving Search Results with a Hybrid Search Approach

Problem

You want to improve retrieval quality by combining multiple search techniques instead of relying on a single one.

Solution

Combine BM25-based keyword search with vector similarity search, then merge their rankings by using reciprocal rank fusion (RRF).

[Figure 5-15](#) shows the process step by step:

1. A keyword search uses BM25 to rank documents based on exact term matches.
2. An embedding model generates an embedding for the user's query.
3. A semantic search ranks documents based on vector similarity.
4. The rankings from both searches are merged into a single ranking.

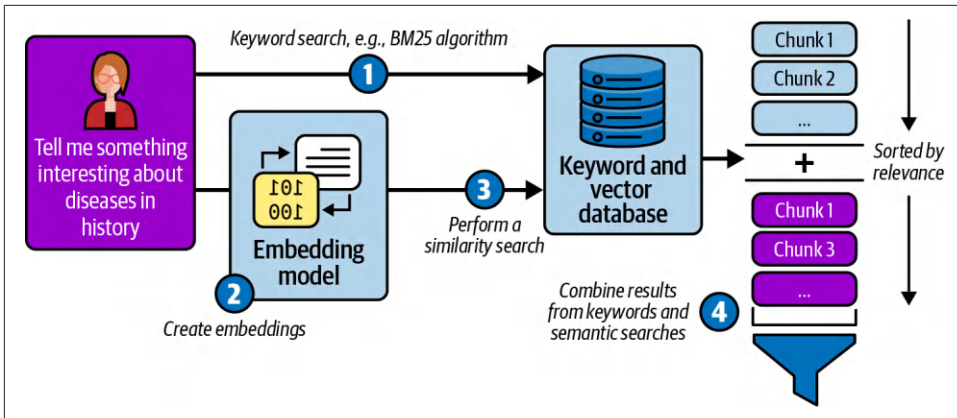


Figure 5-15. Combining keyword and semantic search with reciprocal rank fusion

To implement hybrid search, you need three components:

- BM25 for keyword-based ranking, which is especially useful for product names, IDs, codes, and domain-specific terms
- A text embedding model for semantic similarity search
- A rank fusion method to merge both rankings into a single result list

Install these components as follows:

```
pip install openai rank-bm25 sentence-transformers pandas
```

In this example, you use three short text chunks. First, run a BM25 keyword search:

```
from rank_bm25 import BM25Okapi
import pandas as pd

text_chunks = [
    "The Great Fire of London in 1666 destroyed over 13,000 houses.",
    "Julius Caesar was assassinated on the Ides of March (March 15) in 44 BCE.",
    "The Black Death is estimated to have killed nearly one-third of the "
    "European population.",
]

# Tokenize text into words
tokenized_chunks = [chunk.split(" ") for chunk in text_chunks]

bm25 = BM25Okapi(tokenized_chunks)

user_query = "Tell me something interesting about diseases in history"
tokenized_query = user_query.split(" ")

# BM25 scores for each document
bm25_scores = bm25.get_scores(tokenized_query)
```



```

# Document IDs ordered by keyword relevance (best first)
keyword_ranking_doc_ids = (
    pd.DataFrame(bm25_scores, columns=["score"])
    .sort_values(by="score", ascending=False)
    .index
    .to_list()
)

```

Next, run a semantic search using vector embeddings. Semantic search is strong when the query expresses intent and context, but it may miss exact identifiers or rare keywords. Here's the code:

```

from sentence_transformers.util import cos_sim
from openai import OpenAI

client = OpenAI()

def create_embedding(text):
    return (
        client.embeddings.create(
            input=[text], model="text-embedding-3-small"
        )
        .data[0]
        .embedding
    )

embeddings_df = pd.DataFrame(text_chunks, columns=["text_chunk"])
embeddings_df["embedding"] = embeddings_df["text_chunk"].apply(create_embedding)

query_embedding = create_embedding(user_query)

# Compute cosine similarity
embeddings_df["similarity"] = cos_sim(
    embeddings_df["embedding"], query_embedding
)

# Document IDs ordered by semantic similarity (best first)
semantic_ranking_doc_ids = (
    embeddings_df
    .sort_values(by="similarity", ascending=False)
    .index
    .to_list()
)

```

At this point, you have two ranked lists of document IDs. The final step is to merge these rankings by using RRF. RRF assigns higher scores to documents that appear near the top of both rankings:

```

# Reciprocal Rank Fusion parameter
# Higher k reduces the influence of very top ranks
k = 60

```

```

# Build a table with one row per document
rrf_df = pd.DataFrame({"doc_id": range(len(text_chunks))})

# Map document IDs to their rank positions
keyword_rank_map = {
    doc_id: rank for rank, doc_id in enumerate(keyword_ranking_doc_ids, start=1)
}
semantic_rank_map = {
    doc_id: rank for rank, doc_id in enumerate(semantic_ranking_doc_ids, start=1)
}

rrf_df["keyword_rank"] = rrf_df["doc_id"].map(keyword_rank_map)
rrf_df["semantic_rank"] = rrf_df["doc_id"].map(semantic_rank_map)

# Reciprocal Rank Fusion score
rrf_df["rrf_score"] = (
    1 / (k + rrf_df["keyword_rank"]) +
    1 / (k + rrf_df["semantic_rank"])
)

# Final hybrid ranking
rrf_df = rrf_df.sort_values("rrf_score", ascending=False)

final_ranking_doc_ids = rrf_df["doc_id"].to_list()

```

Each row in the table represents one document. Documents that rank highly in both keyword search and semantic search receive the highest combined scores.

Discussion

Hybrid search combines two independent ranking signals that capture different aspects of relevance. Vector similarity search measures semantic closeness in embedding space. RRF merges both rankings by giving higher scores to documents that appear near the top of multiple lists.

The mechanism works because keyword and semantic signals are complementary. Keyword matching excels at exact terms (product IDs, medical codes, legal citations) where character-level precision matters. Semantic matching captures intent and handles synonyms, paraphrasing, and conceptual similarity that a keyword search misses.

Use hybrid search when your corpus contains identifiers or specialized terminology that embeddings can't capture reliably. Examples include product SKUs ("SKU-4792" must match exactly), legal statute numbers where "article 230" differs from "section 230," medical diagnosis codes (ICD-10 codes), and technical identifiers (API end-point names, database table names). Keyword matching provides a critical ranking signal for these cases.

Use hybrid search when users mix natural language with precise terms in queries. A question like “How do I configure SSL for endpoint `/api/v2/users?`” benefits from semantic understanding of “configure SSL” and exact matching of the endpoint path.

Avoid hybrid search for pure knowledge retrieval where users phrase questions naturally without technical identifiers. A question like “How do I reset my password?” does not need keyword matching—semantic search captures the intent perfectly. Adding BM25 would introduce complexity without improving results.

Avoid hybrid search when semantic search alone meets your quality requirements. Hybrid search adds computational overhead (you run two retrievers plus rank fusion) and increases system complexity. Start with semantic search. Add hybrid search only when you observe exact-match failures in your retrieval logs or user feedback.

The main trade-off is recall versus complexity. Hybrid search improves recall for identifier-heavy queries but requires maintaining two retrieval systems, tuning the rank fusion parameter k , and handling edge cases where the two rankers disagree. The parameter k controls how much to favor top-ranked results—a higher k (60–100) gives more weight to lower ranks, while a lower k (10–20) strongly favors items that rank highly in both retrievers.

See Also

- The [hybrid search introduction](#) from Pinecone explains combining dense and sparse retrieval methods.
- The [LlamaIndex BM25 retriever example](#) demonstrates implementing keyword-based search alongside semantic search.

Vector Databases and Similarity Searches

Vector databases have grown rapidly in popularity, especially since the rise of LLMs. **Figure 6-1** shows some of the most popular vector databases and when they were released.

FAISS was released in 2017, followed by Milvus and Weaviate in 2019, Vald in 2020, Pinecone in 2021, and Chroma in 2023. At the same time, traditional databases such as PostgreSQL and Elasticsearch have added vector search features to their existing platforms. This means you don't necessarily need to add a dedicated vector database to your tech stack if your existing SQL or NoSQL database already supports vector search.

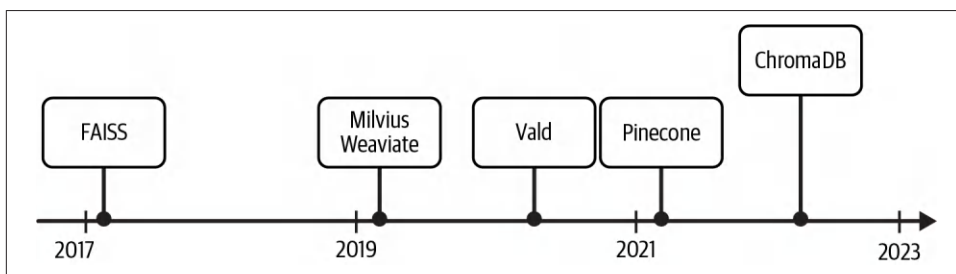


Figure 6-1. The evolution of vector stores

This chapter introduces popular libraries and databases that support vector operations, including FAISS, Chroma, and PostgreSQL. You'll learn how to choose the right vector store for your needs, implement similarity searches, and optimize performance by using indexing techniques.



Throughout this chapter, you'll encounter both “similarity search” and “semantic search.” *Similarity search* refers to the technical operation of finding vectors that are close together in vector space. *Semantic search* describes the user-facing capability of searching by meaning rather than exact keywords. Semantic search is implemented using similarity search as its underlying mechanism.

You can find all the code examples for this chapter in the [book's GitHub repository](#).

6.1 Choosing the Right Vector Database

Problem

You need to pick a vector database that fits your RAG workload, your team, and your infrastructure, without overengineering or locking yourself into the wrong tech stack.

Solution

Use the following decision path to narrow down your options to one or two realistic choices. Follow these steps to evaluate and choose a vector database for your RAG application:

Step 1: Define your requirements

Is vector search only used during preprocessing, experiments, or offline pipelines? Or is it part of a live application serving users?

If vector search is not part of a live system, use a vector library or an in-file vector store like one of the following:

- FAISS or Annoy for pipelines and experiments
- Chroma for local RAG prototypes

If vector search is part of a live application, continue to step 2.

Step 2: Consider operational requirements

Do embeddings need to live next to your business data, users, documents, or permissions?

If yes, use a database you already operate:

- PostgreSQL with pgvector
- MongoDB with vector search

- Redis or Cassandra with vector support

If no, continue to step 3.

Step 3: Decide what scale and operations you need

Will the system grow to millions or tens of millions of vectors? Do you need filtering, access control, and predictable latency?

If yes, use a purpose-built or managed vector database:

- Weaviate, Qdrant, Milvus for self-hosted systems
- Pinecone for fully managed systems

If no, an in-file store like Chroma may still be sufficient.

Step 4: Validate against practical constraints

Ask yourself the questions shown in [Table 6-1](#).

Table 6-1. Practical questions for vector database selection

Question	If yes, lean toward
Are you allowed to send your data to a third-party managed cloud service?	Pinecone or cloud-managed PostgreSQL
Do you already operate PostgreSQL or MongoDB?	pgvector or MongoDB vector search
Do you need keyword search combined with vectors?	Elasticsearch, OpenSearch, or Weaviate
Do you need minimal setup for a small team?	Chroma or managed Pinecone

Once you reach this point, you should have one or two realistic candidates. [Table 6-2](#) provides a comparison to make the final call.

Table 6-2. Pros and cons of common vector stores

Database	Type	Pros	Cons
FAISS	Vector library	Very fast similarity search GPU acceleration Great for pipelines	No persistence No filtering or access control Means you must build everything else
Chroma	Embedded vector store	Instant setup Works well for notebooks and prototypes Python-first API	Not production grade Limited scaling and durability
PostgreSQL (pgvector)	Relational database with vectors	Vectors live next to business data Uses existing backups and security Easy joins and filters	Slower than specialized engines Needs tuning for large scale
Weaviate	Vector database	Built for large-scale RAG Supports hybrid search Rich filtering	New operational stack Higher complexity

Database	Type	Pros	Cons
Pinecone	Managed vector database	No infrastructure to manage Strong performance guarantees Easy to scale	Cloud only Vendor lock-in Cost grows with usage

Discussion

Vector databases exist because traditional databases are not optimized for nearest neighbor search over high-dimensional vectors. Similarity search works by computing the distance between vectors in high-dimensional space. Modern embedding models produce vectors with hundreds or thousands of dimensions, which makes brute-force search too slow. Vectors that are close together represent semantically similar content, while distant vectors represent different concepts.

The main requirement for a vector database in a RAG application is fast similarity search over thousands or millions of vectors. The store must also support high-dimensional vectors, since modern embedding models typically produce vectors with thousands of dimensions.

The mistake many teams make is choosing based on benchmarks instead of workflow. A RAG prototype with a few thousand chunks does not need a distributed vector database. A production assistant serving many users does.

Figure 6-2 shows how the ecosystem clusters into libraries, embedded stores, databases with vector extensions, and purpose-built vector engines.

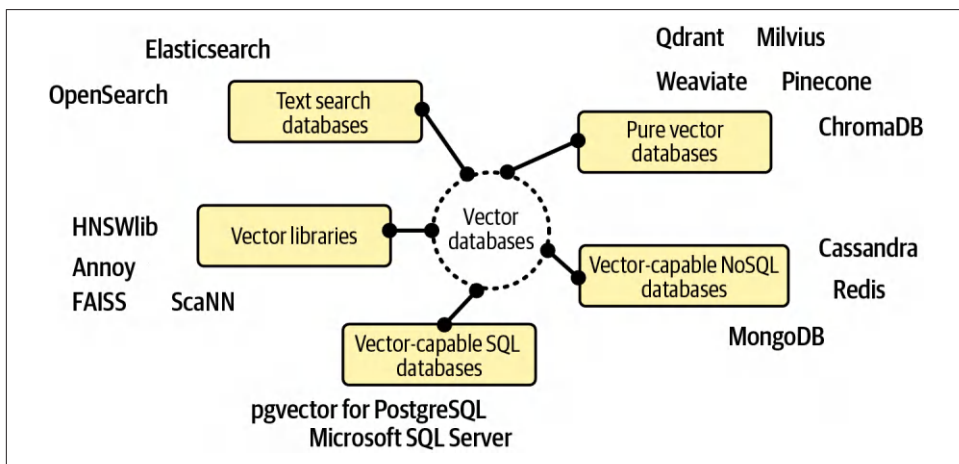


Figure 6-2. Vector stores organized by type and use case

Purpose-built vector databases such as Pinecone, Weaviate, and Qdrant are optimized for vector search and also offer core database capabilities such as persistence and user management.

Text-search-oriented databases such as Elasticsearch and OpenSearch are popular in RAG applications. A retriever that searches semantic text chunks behaves like a specialized search engine, which makes these systems a natural fit when keyword search already plays an important role.

SQL and NoSQL systems including PostgreSQL, MongoDB, Redis, and Cassandra support vector search through extensions or native features. This allows you to keep application data, metadata, and embeddings in one place and combine filtering with semantic search while using familiar query languages.

SQL databases with vector support, or dedicated vector databases like Pinecone and Milvus, are best suited for production systems. They support hybrid search, metadata filtering, and workflows common in larger RAG and agentic applications. For smaller projects and prototypes, vector libraries or lightweight vector stores like Chroma require minimal setup and provide excellent performance for standalone vector operations.

The main trade-off is simplicity versus capability. Vector libraries are simple to set up but lack persistence and access control. Purpose-built vector databases provide comprehensive features but require infrastructure management or vendor costs. SQL databases with vector extensions offer the best integration when you already use them, but may not match the raw performance of specialized solutions at very large scale.

If you start with a simple vector library and later need a full database, the migration is usually straightforward. You must rebuild the index but don't need to regenerate embeddings, which is the costly part.

See Also

- The [ANN Benchmarks repository](#) compares performance of vector search engines.
- The [Chroma documentation](#) shows how to run an in-file vector store.
- The [FAISS repository](#) explains how vector libraries implement similarity search.

6.2 Storing and Searching Embeddings with FAISS

Problem

You need a lightweight solution to store embeddings and perform similarity searches.

Solution

FAISS is well suited for RAG workflows which don't need persistence or metadata filtering. **Figure 6-3** illustrates how FAISS fits into a RAG workflow for temporary analysis tasks.

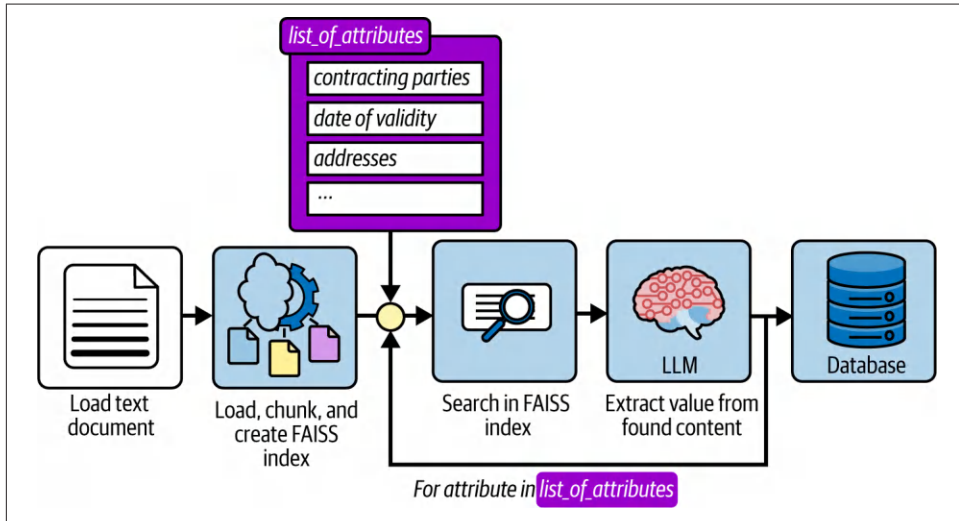


Figure 6-3. Using FAISS for a RAG workflow

This recipe uses the CPU version of FAISS, so it can run on almost any laptop. Install the FAISS library for CPU-based execution and the Python libraries NumPy and OpenAI needed to generate and handle embeddings:

```
pip install faiss-cpu openai numpy
```

Next, you'll generate a FAISS index and populate it with sample text embeddings. An index is the data structure where FAISS stores vectors and performs searches.

You'll also use OpenAI embedding models to generate vectors and then add them to the FAISS index. The example uses IndexFlatL2, which measures similarity by using L2 distance. L2 distance and cosine distance are the most commonly used similarity metrics. Here's the code:

```
import faiss
import numpy as np
from openai import OpenAI

# Example list of sample strings
text_chunks = [
    "The sky is blue.",
    "The sun is shining.",
    "I love chocolate.",
    "Ice cream is delicious.",
```

```

    "Roses are red.",
    "Violets are blue.",
]

# Initialize the OpenAI embeddings model
client = OpenAI()
model = "text-embedding-3-small"

# Generate embeddings for the sample strings
def get_embedding(text):
    response = client.embeddings.create(input=text, model=model)
    return response.data[0].embedding

document_embeddings = np.array(
    [get_embedding(text) for text in text_chunks]
)

# Convert embeddings to float32 (FAISS requires float32 type)
document_embeddings = document_embeddings.astype("float32")

# Create a FAISS index (using L2 distance)
index = faiss.IndexFlatL2(document_embeddings.shape[1])

# Add embeddings to the index
index.add(document_embeddings)

```

The next step is to use the index you just created. When a user asks a question, you generate an embedding for the query and then search the index by calculating the L2 distance.

The following code demonstrates this with a sample query asking what color violets are:

```

# Generate a query embedding for the user query
query = "What color are violets?"
query_embedding = np.array(get_embedding(query)).astype("float32")

# Perform the search: k = number of closest documents you want to retrieve
k = 5
distances, indices = index.search(query_embedding.reshape(1, -1), k)

# Retrieve the documents corresponding to the indices
retrieved_documents = [text_chunks[i] for i in indices[0]]

```

Figure 6-4 shows the search results, including the distance, chunk location, and retrieved text chunks. The sentence stating that violets are blue has the smallest distance, which means it has the highest semantic similarity.

```
print("Distances: ", distances)
print("Indices: ", indices)
print("Retrieved document: ", retrieved_documents)
```

[15] ✓ 0.0s Python

```
... Distances: [[0.47384575 1.0964983 1.4761887 1.6999562 1.7092482 ]]
Indices: [[5 4 0 2 1]]
Retrieved document: ['Violets are blue.', 'Roses are red.', 'The sky is blue.', 'I lov
```

Figure 6-4. The FAISS query result

Discussion

FAISS is a library for fast similarity search over dense vectors. It was released by Meta in 2017 and remains widely used because it's lightweight, supports both CPU and GPU, and provides excellent performance for in-memory vector operations.

FAISS works by organizing vectors into efficient index structures that enable fast nearest neighbor search. The L2 (Euclidean distance) and cosine similarity metrics measure how close vectors are in high-dimensional space. L2 measures the straight-line distance between two points, while cosine similarity measures the angle between vectors. Both effectively identify semantically similar content.

Use FAISS when your workflow doesn't require persistence between runs, metadata filtering, or user-specific access controls. FAISS excels at temporary, single-use analysis tasks where you build an index, query it, and discard it. For example, analyzing a contract to extract parties and dates needs vector search only during that processing job. Similarly, research scripts that process a corpus once and save results work well with FAISS.

Don't use FAISS when you need data to survive application restarts, when multiple users need different access permissions, or when you must filter by metadata fields before searching. FAISS stores everything in memory or in a simple file format with no built-in query language or access control.

The main trade-off with FAISS is simplicity versus features. FAISS offers the fastest pure vector search performance with minimal code but provides no database capabilities. You gain speed and simplicity but must handle persistence, access control, and metadata filtering in your application layer.

FAISS is an in-memory vector index that can be serialized to disk, but it doesn't provide database-level persistence, metadata, or concurrency control. Chroma adds lightweight persistence and basic metadata filtering. PostgreSQL with pgvector adds full SQL capabilities, transactions, and production-grade access control, but requires

database management. Choose FAISS for batch jobs and offline indexing, Chroma for prototypes that need simple persistence, and PostgreSQL for production systems.

For production systems requiring persistence, metadata filtering, or multiuser access, use PostgreSQL with pgvector (when you already use PostgreSQL) or dedicated vector databases like Weaviate or Pinecone. These solutions support role-based access control and are available as managed services on cloud platforms.

See Also

- The [FAISS repository](#) contains the official documentation and usage examples.
- The [FAISS wiki](#) provides detailed guidance on index types and performance tuning.
- The [original FAISS paper](#) explains the algorithms and design principles behind the library.

6.3 Storing and Working with Embeddings in a Chroma Vector Database

Problem

You need a lightweight vector database for a smaller project that you can set up in minutes.

Solution

Chroma is a lightweight vector database designed for quick prototyping and small projects. It provides persistence and metadata handling while remaining simple to set up and use.

Chroma can store your data in two ways:

Ephemeral client

Stores data in memory, which means the data is lost when the application stops

Persistent client

Saves data to disk so you can reload it when restarting your app

Chroma has built-in functions that can automatically handle text processing and embedding generation. For prototypes, these built-in features speed up development. For production systems you plan to migrate later, generate embeddings in your own code and use Chroma only to store and search them. This makes switching to PostgreSQL or Pinecone straightforward.

In **Figure 6-5**, you can see the two main steps:

1. Load the text, chunk it, and generate embeddings by using an embedding model.
2. Insert those embeddings into Chroma. If persistence is enabled, Chroma also stores them on disk.

When a user asks a new question, you generate an embedding for the question by using the same embedding model and then execute a vector search in Chroma.

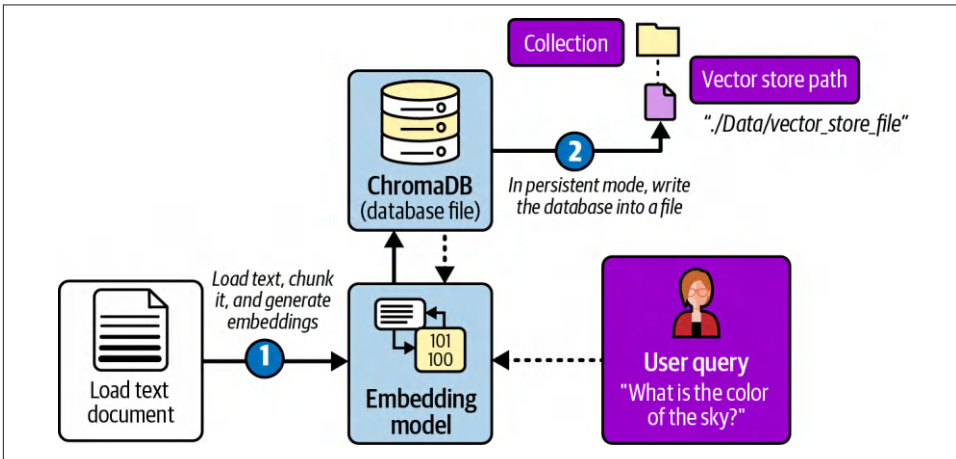


Figure 6-5. Storing and retrieving vector embeddings from Chroma

To use Chroma, you only need the Chroma Python package. Because this example uses an OpenAI embedding model, you also need the OpenAI SDK. Use this command to install both:

```
pip install chromadb openai
```

To get started with Chroma, you create a Chroma client and a new collection. Because this example uses the persistent client, you must provide a directory path where Chroma can store its files:

```
import chromadb

# Vector store settings
VECTOR_STORE_PATH = r"./02_Data/00_Vector_Store"
COLLECTION_NAME = "my_collection"

# Get/create a Chroma client and collection
chroma_client = chromadb.PersistentClient(path=VECTOR_STORE_PATH)
collection = chroma_client.get_or_create_collection(name=COLLECTION_NAME)
```

In the next step, you add text chunks to the collection. Chroma can handle tokenization, embedding, and indexing for you and uses the Sentence Transformer model

all-MiniLM-L6-v2 by default to create embeddings. However, Chroma also allows you to supply embeddings generated elsewhere.

In this example, embeddings are generated in your application code with an OpenAI text embedding model, and only stored and searched in Chroma. Using the OpenAI embedding model, create a list of embeddings for the text chunks you want to insert into Chroma, and then pass that list to the vector store by using the add function:

```
from openai import OpenAI

text_chunks = [
    "The sky is blue.",
    "The sun is shining.",
    "I love chocolate.",
    "Ice cream is delicious.",
    "Roses are red.",
    "Violets are blue.",
]

# Generate embeddings
client = OpenAI()
model = "text-embedding-3-small"
def get_embedding(text):
    response = client.embeddings.create(input=text, model=model)
    return response.data[0].embedding

embeddings_model = client # For compatibility with later code
embeddings_list = [get_embedding(text) for text in text_chunks]

# Add data frame to collection
collection.add(
    embeddings=embeddings_list,
    documents=text_chunks,
    ids=[
        str(i) for i in range(len(text_chunks))
    ], # Create a list of strings as index
)
```

Now you can test the setup by querying the collection. In this example, you'll ask for the color of the sky. The question is embedded and used to query the collection:

```
# Query collection
query = "What is the color of the sky?"
query_embedding = get_embedding(query)
results = collection.query(query_embeddings=[query_embedding], n_results=3)
```

The query returns the three closest matching text chunks along with their cosine distance. [Figure 6-6](#) shows the resulting dictionary. It includes not only the text chunks and IDs of the best matches but also any metadata stored in Chroma.

```
results
[21] ✓ 0.0s Python
... {'ids': [['0', '5', '1']],
     'embeddings': None,
     'documents': [['The sky is blue.',
                    'Violets are blue.',
                    'The sun is shining.']],
     'uris': None,
     'included': ['metadatas', 'documents', 'distances'],
     'data': None,
     'metadatas': [[None, None, None]],
     'distances': [[0.7066932916641235, 1.2916922569274902, 1.4443373680114746]]}
```

Figure 6-6. Working with embeddings in Chroma

Discussion

Chroma works by storing vectors with their metadata in a simple embedded database. It persists data to disk automatically, so you can restart your application without rebuilding the index. This makes Chroma suitable for prototypes and small projects needing more than FAISS but less than a full production database.

Use Chroma when you need persistence and basic metadata filtering but want minimal setup. Chroma is ideal for prototypes, proof-of-concept projects, and small production systems with datasets under a few hundred thousand vectors. It works well when you want to quickly test a RAG application without configuring PostgreSQL or signing up for a managed service.

Don't use Chroma when your dataset grows beyond a few hundred thousand vectors, when you need role-based access control, when you require SQL joins with application data, or when query latency must be under 10 ms at scale. Chroma lacks advanced security features, sophisticated query optimization, and horizontal scaling capabilities.

The main trade-off with Chroma is ease of use versus scalability. You can start coding immediately with minimal dependencies, but performance degrades as your dataset grows. Chroma provides a middle ground between FAISS (which can be saved to disk but lacks database features) and PostgreSQL (which has full database features).

Chroma differs from FAISS by adding persistence and metadata filtering, which means you can restart your app without rebuilding the index. Chroma differs from PostgreSQL with pgvector by being simpler to set up but lacking SQL capabilities, transaction support, and production-grade access control. When your dataset grows beyond a few hundred thousand vectors or you need role-based access and SQL joins, migrate to PostgreSQL with pgvector or a managed vector database like Pinecone.

See Also

- The [Chroma documentation](#) provides comprehensive setup and usage guidance.
- The [Chroma repository](#) contains code examples and implementation details.
- The [Chroma cookbook](#) offers practical examples for common use cases.

6.4 Storing Embeddings in PostgreSQL with the pgvector Extension

Problem

You want to store your application data and vector data in the same database by using PostgreSQL with the pgvector extension.

Solution

PostgreSQL with the pgvector extension lets you store application data and high-dimensional vectors in the same database. Vector search becomes just another SQL operator, so you can extend existing tables and queries with similarity search capabilities.

If you haven't installed PostgreSQL yet, you can either use Docker to spin up a container with PostgreSQL and the pgvector extension installed, or you can install PostgreSQL directly on your machine.



Docker is a platform that packages applications and their dependencies into containers—lightweight, portable units that run consistently across different environments. A Docker container includes everything needed to run an application, making it easy to share and deploy without worrying about system-specific configurations.

To install PostgreSQL directly, go to the [PostgreSQL website](#) and download the latest installer for your operating system. In addition, you need to install the pgvector extension. The installation steps depend on your operating system:

Windows

Clone the [pgvector repository](#). The README contains instructions for installing it correctly.

macOS

Use Homebrew to install pgvector with a single command.

Docker provides a simpler and more reproducible approach. A RAG application that uses PostgreSQL as a vector store can run in a container with a fixed configuration, making it straightforward to deploy the same setup to AWS, Google Cloud, or Azure.

To run Docker containers locally, download and install Docker Desktop from the Docker website. Once Docker is installed, create a Docker Compose file that defines the configuration of the container you want to run, and let Docker start the container.

Let's walk through that process now. In your project directory, create a new *docker-compose.yml* file, add the content shown here, and save it:

```
version: '3.8'
services:
  db:
    image: ankane/pgvector
    container_name: postgres_with_pgvector
    restart: always
    environment:
      POSTGRES_USER: rag_cookbook_user
      POSTGRES_PASSWORD: rag_cookbook_user_pw
      POSTGRES_DB: rag_cookbook
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

To start the container, navigate to the directory where the *docker-compose.yml* file is located and run the following:

```
docker-compose up -d
```

This command searches the current directory for a Docker Compose file and downloads the container image *ankane/pgvector*, which contains PostgreSQL and pgvector. The Docker Compose file sets the PostgreSQL user, password, and database name, and exposes the port used to reach the PostgreSQL database.

To verify that your container is running correctly, open the Docker Dashboard and check the status of the container. [Figure 6-7](#) shows the Docker Dashboard when the PostgreSQL container is running.

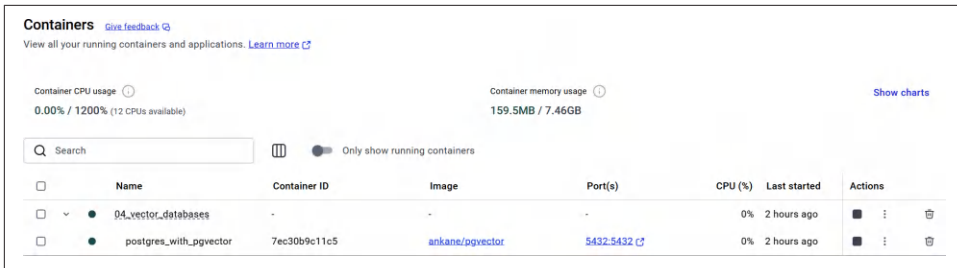


Figure 6-7. Checking that the Docker container is running

To connect to the newly created PostgreSQL database from Python, this recipe uses the Psycopg 2 package, which you install as follows:

```
pip install psycopg2-binary openai
```

In this example, you'll connect to the database by using the defined username, password, and database name, and then create a new table that contains the following:

- An ID for the text chunk
- The text chunk
- The embedding vector

To enable vector search and store embeddings, the embedding column uses the vector data type with the correct number of dimensions. In this case, the vector has 1,536 dimensions because this recipe uses OpenAI's text-embedding-3-small model, which produces embeddings with that dimensionality. Remember that the vector data type is available only if you have installed the pgvector extension. Run the following code:

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    port=5432,
    dbname="rag_cookbook",
    user="rag_cookbook_user",
    password="rag_cookbook_user_pw",
)

cur = conn.cursor()

cur.execute("CREATE EXTENSION IF NOT EXISTS vector;")

cur.execute("""
    CREATE TABLE IF NOT EXISTS embeddings (
        id SERIAL PRIMARY KEY,
        chunk TEXT NOT NULL,
```

```

        embedding vector(1536) NOT NULL
    );
"""

conn.commit()
cur.close()
conn.close()

```

The next step is to populate the table with sample data. You loop through a list of sample text chunks, generate embeddings via the OpenAI embedding model, and then insert the text chunks and their embeddings into the table:

```

from openai import OpenAI

# Define text chunks
text_chunks = [
    "The sky is blue.",
    "The sun is shining.",
    "I love chocolate.",
    "Ice cream is delicious.",
    "Roses are red.",
    "Violets are blue.",
]

client = OpenAI()
model = "text-embedding-3-small"

def get_embedding(text):
    response = client.embeddings.create(input=text, model=model)
    return response.data[0].embedding

index = 0
cur = conn.cursor()

# Insert the embeddings into the table
for text_chunk in text_chunks:
    embedding = get_embedding(text_chunk)
    cur.execute(
        """INSERT INTO embeddings
        (id, chunk, embedding)
        VALUES (%s, %s, %s)""",
        (index, text_chunk, embedding),
    )
    index += 1

```

The function fills the PostgreSQL table with text chunks and their corresponding embeddings. **Figure 6-8** shows how the table is populated: six text chunks along with their embeddings.

```
cur = conn.cursor()
cur.execute("SELECT * FROM embeddings_table")
rows = cur.fetchall()

for row in rows:
    print(row)
```

[57] ✓ 0.0s Python

```
... (0, 'The sky is blue.', '[0.015439852,-0.013955711,-0.01844404,0.007833631,0.01365649,-0.0
(1, 'The sun is shining.', '[-0.020074036,0.008307903,0.036631625,-0.008651397,0.047786456
(2, 'I love chocolate.', '[0.017230779,-0.040228788,-0.0276307,-0.014264437,0.016391695,-0
(3, 'Ice cream is delicious.', '[0.019605028,-0.05108732,-0.061606564,-0.024875429,0.01894
(4, 'Roses are red.', '[-0.016459428,-0.020555496,-0.0058184206,0.013002196,0.028710052,-0
(5, 'Violets are blue.', '[0.0094178775,-0.017430486,-0.026738778,0.004644477,0.0076516224
```

Figure 6-8. The embeddings table in PostgreSQL

You can now use this table like any other SQL table. You can perform SELECT, CREATE, and UPDATE operations. Since the table contains a vector column, you can also perform vector searches using cosine similarity, Euclidean distance, or inner product as the distance metric.

Discussion

PostgreSQL with pgvector works by storing embeddings as native vector columns alongside your application data. The pgvector extension adds vector data types and similarity search operators directly to SQL, so you can query embeddings by using familiar PostgreSQL syntax. This enables you to join user tables, document metadata, and embeddings in a single query without managing separate systems.

Use PostgreSQL with pgvector when your application already uses PostgreSQL for structured data, when you need to filter embeddings by metadata (user permissions, dates, categories) before similarity search, or when you want to avoid managing a separate vector database. PostgreSQL works well for production RAG systems with up to tens of millions of vectors, especially when you need ACID (atomicity, consistency, isolation, durability) transactions and role-based access control.

Don't use PostgreSQL with pgvector when you need to search hundreds of millions or billions of vectors with single-digit millisecond latency. At that scale, purpose-built vector databases like Pinecone or Weaviate provide better performance. Also avoid PostgreSQL if your workload is purely vector search with no metadata filtering, as FAISS or specialized vector databases will be faster.

The main trade-off with PostgreSQL is integration versus raw performance. You gain unified data management, SQL joins, and transactional guarantees, but specialized vector databases can be faster for pure vector-only workloads at very large scale. For

most RAG applications, the integration benefits outweigh the performance difference.

PostgreSQL with pgvector differs from FAISS by providing persistence, ACID transactions, and SQL capabilities. It differs from Chroma by offering production-grade access control, query optimization, and the ability to join embeddings with structured data. It differs from specialized vector databases by integrating with your existing PostgreSQL infrastructure but potentially offering lower performance at extreme scale (billions of vectors).

See Also

- The [pgvector repository](#) contains installation instructions and configuration options.
- The [PostgreSQL documentation](#) provides comprehensive database setup and management guidance.
- The [pgvector installation guide](#) offers platform-specific setup instructions.

6.5 Performing Similarity Search in PostgreSQL

Problem

You want to use PostgreSQL to find the text chunks most similar to a user's query.

Solution

This recipe assumes you have a PostgreSQL table with a vector column. The example uses a table called `embeddings` with columns `id` (integer), `chunk` (text), and `embedding` (vector).

If you're following this recipe standalone, first establish a database connection and ensure that the table exists. The following code creates the connection and table structure:

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    port=5432,
    dbname="rag_cookbook",
    user="rag_cookbook_user",
    password="rag_cookbook_user_pw",
)

cur = conn.cursor()
cur.execute("CREATE EXTENSION IF NOT EXISTS vector;")
```

```

cur.execute("""
    CREATE TABLE IF NOT EXISTS embeddings (
        id SERIAL PRIMARY KEY,
        chunk TEXT NOT NULL,
        embedding vector(1536) NOT NULL
    );
""")
conn.commit()
cur.close()

```

For full setup details including Docker configuration, see [Recipe 6.4](#).

The pgvector extension supports the most common similarity metrics, such as cosine similarity, Euclidean distance, and inner product.

This example uses the Psycopg 2 library to connect to PostgreSQL and execute SQL queries. Embeddings are created with OpenAI embedding models. Install these dependencies:

```
pip install psycopg2-binary openai
```

In this recipe, you'll perform a cosine similarity search by using the `<=>` operator.

Alternatively, you can use the following:

- Euclidean distance with the `<->` operator
- Inner product with the `<#>` operator
- Taxicab (Manhattan) distance with the `<+>` operator

To start, search the newly created table, which contains random text chunks about weather, flowers, and food. The sample query is “sweet,” so the hypothesis is that the text chunks related to food will rank higher:

```

from openai import OpenAI

# Initialize the OpenAI embeddings model
client = OpenAI()
model = "text-embedding-3-small"

# Example text chunk
text_chunk = "Sweets are delicious."

# Get the embedding
response = client.embeddings.create(input=text_chunk, model=model)
embedded_query = response.data[0].embedding

cur = conn.cursor()
cur.execute(
    f"""
        SELECT 1 - (embedding <=> '{embedded_query}') AS cosine_similarity, *
        FROM embeddings
    """
)

```

```

ORDER BY 1 - (embedding <=> '{embedded_query}') DESC
LIMIT 20;
"""
)

results = cur.fetchall()

```

Figure 6-9 shows the ranked results, including the similarity score, the text chunk, and the embedding vector itself. The two highest-ranked text chunks are about ice cream and chocolate.

```

results
[65] ✓ 0.0s Python
... [(0.571485459804539,
      3,
      'Ice cream is delicious.',
      '[0.019605028,-0.05108732,-0.061606564,-0.024875429,0.018947575,-0.00661764,-0.032075077,0.0377,0.47613173865486835,
      2,
      'I love chocolate.',
      '[0.017230779,-0.040228788,-0.0276307,-0.014264437,0.016391695,-0.0068308576,-0.006913584,0.039,0.25766646862030207,
      4,
      'Roses are red.',
      '[-0.016459428,-0.020555496,-0.0058184206,0.013002196,0.028710052,-0.004449933,-0.012444781,-0.0.22350402334308528,
      5

```

Figure 6-9. Calculating cosine similarity in PostgreSQL

Discussion

Similarity search in PostgreSQL works by using SQL operators to compute distance metrics (cosine similarity, Euclidean distance, inner product) between the query embedding and stored embeddings. The `<=>` operator calculates cosine distance, `<->` calculates Euclidean distance, and `<#>` calculates inner product. PostgreSQL executes these calculations either by scanning all rows or by using vector indexes such as Hierarchical Navigable Small Worlds (HNSW) or Inverted File Indexing (IVF) to narrow the search space.

The power of PostgreSQL similarity search comes from combining it with SQL filtering. You can use `WHERE` clauses to filter by metadata before computing similarity, reducing the search space and improving relevance. For example, filter to documents accessible by the current user, published after a certain date, or tagged with specific categories, and then find the most similar vectors within that filtered set.

Use SQL filtering with similarity search when your RAG application needs user-specific results, when document permissions matter, when you want to restrict search to recent content, or when categories and tags improve relevance. For example, a legal

RAG system should search only contracts the user has permission to view. A news RAG system might search only articles from the last 30 days.

Don't use SQL filtering when all documents are equally accessible to all users and no metadata filtering is needed. In that case, pure vector search without filtering is simpler and faster. Also avoid complex multitable joins in the same query as similarity search, as this can slow down performance. Filter first in a subquery or common table expression (CTE), then run a similarity search.

The main trade-off is flexibility versus complexity. SQL filtering enables sophisticated query logic but requires careful query construction to maintain performance. Improperly indexed filter columns or overly complex joins can make queries slow. Test query performance with realistic data volumes before deploying to production.

This approach differs from pure vector search (FAISS, standalone Pinecone) by enabling metadata filtering within the database. It differs from hybrid search (used in the next recipe) by focusing on metadata filtering rather than keyword matching. Use simple SQL filtering when you need to restrict the search space by metadata. Use hybrid search when exact keyword matches also matter.

See Also

- The [pgvector repository](#) documents similarity metrics and query operators.
- The [PostgreSQL indexing documentation](#) explains index types and optimization strategies.
- The [pgvector query guide](#) provides examples for different distance functions.

6.6 Accelerating Vector Searches in PostgreSQL with Indexing Techniques

Problem

You need to accelerate vector searches when working with datasets that contain millions of vectors.

Solution

Use indexing techniques to organize vectors for faster similarity searches. PostgreSQL with the `pgvector` extension supports two indexing methods: IVFFlat and HNSW. This recipe uses a dataset with around 200,000 text chunks to demonstrate both methods.



This recipe uses a large dataset for meaningful performance comparison. You can use the sample dataset from [Kaggle](#) or substitute your own data with a similar volume. The key requirement is having enough rows (100,000+) to see indexing benefits.

The practical code uses Psycopg 2 to connect to PostgreSQL, OpenAI to generate the embeddings, and pandas for data preprocessing. Install these three as follows:

```
pip install psycopg2 openai pandas
```

First, create a table to hold your job descriptions and their embeddings. The following code creates the necessary table structure and establishes a database connection:

```
import psycopg2
from psycopg2 import Error

conn = psycopg2.connect(
    dbname="rag_cookbook",
    user="rag_cookbook_user",
    password="rag_cookbook_user_pw",
    host="localhost",
    port="5432",
)

cur = conn.cursor()
cur.execute("""CREATE EXTENSION IF NOT EXISTS vector""")

cur.execute(
    """
    CREATE TABLE IF NOT EXISTS job_description_table(
        id integer PRIMARY KEY,
        text_chunk TEXT,
        embedding vector(1536)
    )
    """
)
conn.commit()
```

This recipe compares full-search performance with indexed vector search by using the HNSW index and the IVFFlat index.

The following example starts with a full search using cosine similarity. This method sequentially scans all entries without using an index, which makes it the slowest option for large datasets. For small datasets, a full search can sometimes be faster. Run the following:

```
query = "I am looking for a job as a data scientist in Berlin."
client = OpenAI()
model = "text-embedding-3-small"
response = client.embeddings.create(input=query, model=model)
query_embedding = response.data[0].embedding
```

```

full_search_sql = f"""
    EXPLAIN ANALYZE SELECT 1 - (embedding <=> '{str(query_embedding)}')
    AS cosine_similarity,
    * FROM job_description_table
    ORDER BY 1 - (embedding <=> '{str(query_embedding)}') DESC
    LIMIT 20;
    """

cur.execute(full_search_sql)

results_full_search = cur.fetchall()
# [..., ('Planning Time: 13.669 ms',), ('Execution Time: 85.582 ms',)]

```

This baseline will be used to compare performance against the indexed searches.



To ensure that the index is actually used, you may need to force PostgreSQL to disable sequential scans. PostgreSQL sometimes falls back to a sequential scan when it isn't clear that an index search is the better option.

Next, you'll create an IVFFlat index and perform a similarity search on it. First, duplicate the existing job description table and create an index on the new table. To analyze the query plan and execution time, run the query in EXPLAIN ANALYZE mode:

```

import psycopg2
from psycopg2 import Error

ivfflat_sql = f"""
    DROP TABLE IF EXISTS test_embedding_table;
    CREATE TABLE test_embedding_table AS
    SELECT * FROM job_description_table;
    CREATE INDEX ON test_embedding_table
    USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 30);
    -- Reduce the number of probes for faster search
    SET ivfflat.probes = 3;
    EXPLAIN ANALYZE SELECT 1 - (embedding <=> '{str(query_embedding)}')
    AS cosine_similarity, *
    FROM test_embedding_table
    ORDER BY 1 - (embedding <=> '{str(query_embedding)}') DESC
    LIMIT 20;
    """

cur.execute(ivfflat_sql)
ivfflat_search = cur.fetchall()

```

In this test with a dataset of 200,000 job descriptions, the index search is roughly twice as fast as a full search. This difference becomes even more significant for very large datasets with millions or billions of data points.

Next, you'll use the HNSW indexing method. This code uses the `vector_cosine_ops` option, which creates an index optimized for cosine similarity search. Alternatively, you can use `vector_ip_ops`, which is optimized for inner product similarity search, or `vector_l2_ops`, which is optimized for Euclidean (L2) distance vector search.

When creating the index, you must set these hyperparameters:

- `m` sets the maximum number of connections per layer in the HNSW graph. It defines how many connections each data point has to its neighbors. More connections make the graph denser, which typically speeds up search queries but increases index build time and memory usage. Typical values range from 16 to 64.
- `ef_construction` controls the size of the dynamic candidate list used during index construction. Higher values usually improve index quality.
- `ef_search` controls the number of candidates to consider during search, letting you trade off search accuracy versus speed. Higher values return more accurate results, but produce slower queries.

Here's an example query:

```
import psycopg2
from psycopg2 import Error

hnsw_test_sql = f"""
DROP TABLE IF EXISTS test_embedding_table;
CREATE TABLE test_embedding_table AS
    SELECT * FROM job_description_table;
CREATE INDEX ON test_embedding_table
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 200);
SET hnsw.ef_search = 50;
EXPLAIN ANALYZE SELECT 1 - (embedding <=> '{str(query_embedding)}')
AS cosine_similarity, *
FROM test_embedding_table
ORDER BY 1 - (embedding <=> '{str(query_embedding)}') DESC
LIMIT 20;
"""

cur.execute(hnsw_test_sql)
hnsw_search = cur.fetchall()

# [..., ('Planning Time: 1.109 ms',), ('Execution Time: 13.927 ms',)]
```

Discussion

For tables under 100,000 rows, full scans are often fast enough. Skip indexing in this case. Above one million rows, indexing becomes essential.

Choose HNSW when query speed is critical and you have sufficient memory (HNSW stores more data in RAM). Choose IVF when memory is constrained or you need faster index building at the cost of slightly slower queries. For most production RAG systems with millions of vectors, HNSW provides the best query performance.

When creating indexes, you can choose different operators based on your similarity metric. The `vector_cosine_ops` option creates an index optimized for cosine similarity search. Alternatively, use `vector_ip_ops` for inner product similarity search, or `vector_l2_ops` for Euclidean (L2) distance vector search.

HNSW indexes require tuning several hyperparameters. The `m` parameter sets the maximum number of connections per layer in the HNSW graph. More connections make the graph denser, which typically speeds up search queries, but increases index build time and memory usage. Typical values range from 16 to 64. The `ef_construction` parameter controls the size of the dynamic candidate list used during index construction. Higher values usually improve index quality. The `ef_search` parameter controls the number of candidates to consider during search, letting you trade off search accuracy with speed. Higher values return more accurate results but produce slower queries.

Understanding how these indexing methods work helps you tune parameters for your dataset. *IVF* stands for *inverted file*. It connects content in the index to its position in a table or document. When building an IVF index, the system divides the vector space and all data points in it into partitions. Each partition has a centroid, and each item belongs to only one partition at a time. When you start a new search, the index first looks for the right partition by comparing the centroids' locations to the user's query, as illustrated in [Figure 6-10](#).

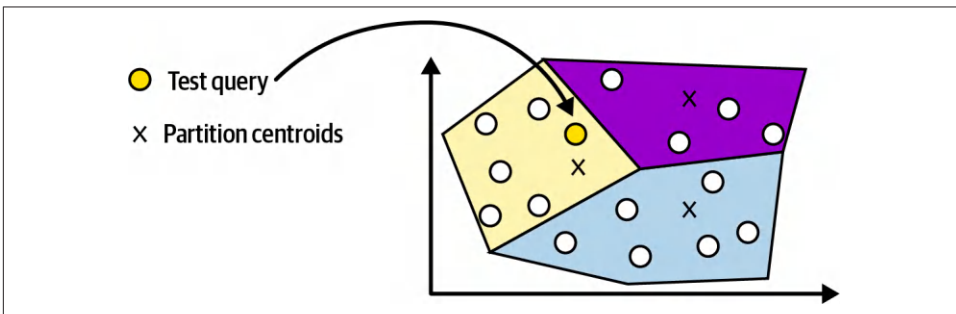


Figure 6-10. Querying an IVF index by finding the closest centroid

Once the closest partition is identified, the search continues within that partition to find the nearest neighbor, as shown in [Figure 6-11](#). The query vector (shown as the blue dot) is compared to the data points in the selected partition to identify the closest match. Note that while the partition's centroid is the closest among all centroids, the actual nearest neighbor could be in an adjacent partition if it's near the partition boundary.

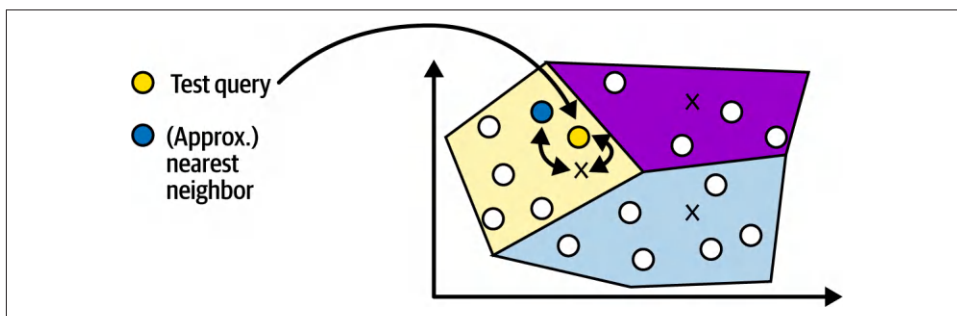


Figure 6-11. Querying an IVF index by identifying the closest data point within the partition

With all indexing methods, you typically trade some accuracy for speed. With an IVF index, the challenge is that when you search within a particular partition, you might miss relevant data points that are close to the edge of a neighboring partition. In practice, that's often acceptable because the accuracy loss is relatively small, while the performance gain is significant.

Another alternative in PostgreSQL is the HNSW index, which builds a multilayer graph. Higher layers contain fewer connections but span longer distances across the data space, acting as shortcuts for efficient search. Lower layers contain denser connections that provide fine-grained navigation.

Think of HNSW as a multilayer map. The highest layer is like a national highway map with very few roads, each covering long distances. From that highest layer, you move down to lower layers with more roads and more detail. The bottom layer shows every small local street. [Figure 6-12](#) illustrates this structure.

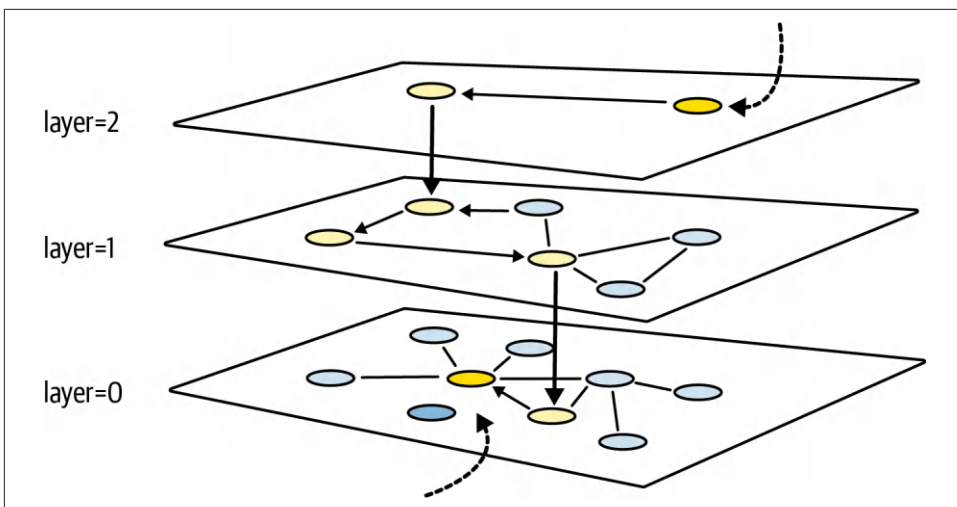


Figure 6-12. The HNSW layer approach

Different vector databases use different indexing methods. FAISS supports product quantization (PQ), IVF, and HNSW. Annoy uses a tree-based approach, while HNSWlib focuses solely on HNSW. Milvus supports HNSW, IVF, and PQ.

See Also

- The [pgvector indexing documentation](#) describes HNSW and IVFFlat configuration options and performance characteristics.
- The [HNSWlib repository](#) provides the reference implementation and technical details of HNSW indexing.
- The [PostgreSQL CREATE INDEX documentation](#) explains the syntax and options for creating indexes.

6.7 Combining Keyword and Similarity Search to Improve Retrieval Accuracy with PostgreSQL

Problem

You want to increase the robustness of your retrieval step by combining keyword search and semantic search.

Solution

Hybrid search combines keyword search and semantic search into a single result set. This recipe implements hybrid search directly in PostgreSQL by executing both search types in one SQL query and merging the results with weighted scoring.

This recipe assumes you have a PostgreSQL table with embeddings already created. The example reuses the `job_description_table` from [Recipe 6.6](#). If you're following this recipe standalone, you need (1) a PostgreSQL database with `pgvector` extension enabled, (2) a table with columns `id`, `text_chunk`, and `embedding vector(1536)`, and (3) the table populated with at least a few dozen text chunks and their embeddings. See [Recipe 6.4](#) or [Recipe 6.6](#) for setup instructions.

This recipe uses the `Psycopg2` library to connect to PostgreSQL and execute SQL queries. Here's the installation command:

```
pip install psycopg2-binary openai
```

Next, copy the job description table and add a new column with the `tsvector` data type (`tsvector` is designed for full-text search and is used to perform keyword search in a subsequent step):

```
import psycopg2
from psycopg2 import Error

conn = psycopg2.connect(
    dbname="rag_cookbook",
    user="rag_cookbook_user",
    password="rag_cookbook_user_pw",
    host="localhost",
    port="5432",
)

cur = conn.cursor()

hybrid_search_table = f"""
    DROP TABLE IF EXISTS test_embedding_table;
    CREATE TABLE test_embedding_table AS
    SELECT *, to_tsvector(text_chunk) AS tsv
    FROM job_description_table;
"""

cur.execute(hybrid_search_table)
```

To run a hybrid search query, you can use a standard SQL `SELECT` statement. In addition to returning rows, the query computes the following columns:

- `text_score`, which stores the full-text search ranking score
- `vector_score`, which stores the cosine similarity score

- `hybrid_score`, which combines both rankings into a single score

In this example, the combined score is computed by weighting both rankings equally:

- Keyword ranking weight: 0.5
- Vector similarity ranking weight: 0.5

You can adjust these weights. For example, to prioritize semantic search, you could use 0.7 for semantic similarity and 0.3 for keyword ranking:

```
import psycopg2
from psycopg2 import Error
from openai import OpenAI

query = "I am looking for a job as a data scientist in Berlin."
client = OpenAI()
model = "text-embedding-3-small"
response = client.embeddings.create(input=query, model=model)
query_embedding = response.data[0].embedding

hybrid_search_query = f"""
    WITH ranked_docs AS (
        SELECT
            id,
            text_chunk AS text,
            ts_rank(tsv, plainto_tsquery('PostgreSQL')) AS text_score,
            1 - (embedding <=> '{str(query_embedding)}') AS vector_score,
            ts_rank(tsv, plainto_tsquery('PostgreSQL'))
            * 0.5 + (1 - (embedding <=> '{str(query_embedding)}')) * 0.5
            AS hybrid_score
        FROM test_embedding_table
        -- Filter for relevant text first
        WHERE tsv @@ plainto_tsquery('PostgreSQL')
        ORDER BY hybrid_score DESC
        LIMIT 10
    )
    SELECT * FROM ranked_docs;
"""

cur.execute(hybrid_search_query)
results = cur.fetchall()
```

Discussion

Keyword search (full-text search) uses PostgreSQL's `tsvector` type and `ts_rank` function to identify documents containing query terms. Semantic search uses cosine similarity to find vectors close to the query embedding. By computing both scores and combining them with weights, you retrieve documents that either contain the exact terms or express similar concepts.

Use hybrid search when exact keyword matches matter for your domain. This includes legal documents (searching for “force majeure” or “indemnification”), technical documentation (searching for specific error codes or API names), medical records (searching for drug names or diagnosis codes), or any scenario where users expect results containing their exact query terms. Hybrid search also helps when queries contain proper nouns, acronyms, or domain-specific terminology that may not be well represented in embedding spaces.

Don’t use hybrid search when semantic meaning is sufficient and exact terms are not critical. For example, searching conversational customer support tickets or general knowledge articles often works better with pure semantic search. Also avoid hybrid search when your keyword search index is not well tuned, as poor keyword results will drag down the combined score.

The main trade-off with hybrid search is complexity versus robustness. You must maintain both a text search index and a vector index, tune two sets of parameters, and determine appropriate weights. In return, you get more robust retrieval that handles both semantic queries and exact term matching.

Start with equal weights (0.5 text, 0.5 vector). If users complain that relevant results are missing because they lack the exact term, increase the vector weight to 0.7. If results drift too far from the literal query, increase the text weight to 0.7. Different domains may need different default weights: legal and technical applications often need higher text weights, while conversational applications often need higher vector weights.

Hybrid search differs from pure semantic search by also considering exact term matches. It differs from pure keyword search by also finding semantically similar content that may not contain query terms. It differs from SQL filtering (previous recipe) by combining two search methods rather than filtering and then searching. Use hybrid search when both literal terms and meaning matter. Use SQL filtering when you need to restrict by metadata. Use pure semantic search when meaning alone is sufficient.

See Also

- The [PostgreSQL full-text search documentation](#) explains keyword search capabilities and configuration.
- The [pgvector query documentation](#) provides examples for combining vector and keyword searches.
- The [PostgreSQL UNION documentation](#) describes techniques for merging query results.

Retrieval

Improving the retriever step is the most effective way to increase the accuracy and relevance of a RAG system. You need a solid toolkit to build customized, highly efficient systems. This chapter introduces advanced retrieval techniques that go beyond a basic vector search. **Figure 7-1** provides an overview of techniques for optimizing the pre-retrieval and retrieval steps.

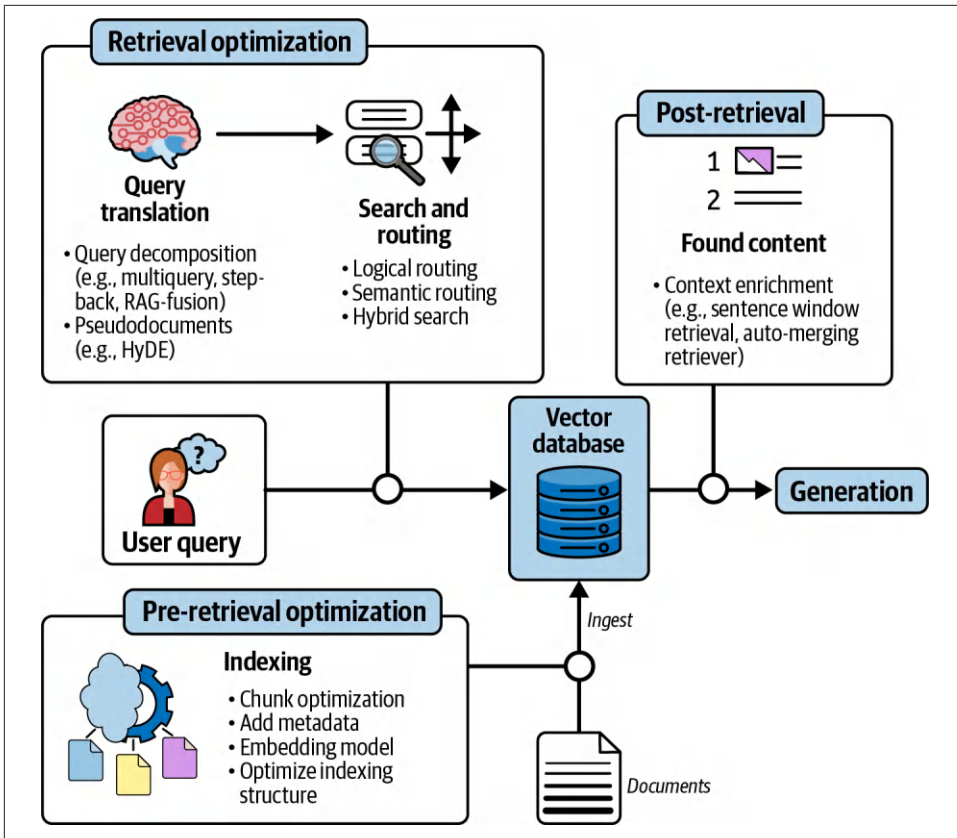


Figure 7-1. Techniques to enhance the retrieval process

Real-world RAG workflows typically combine several of these techniques, depending on the use case. **Figure 7-2** shows an example workflow that combines multiple steps and advanced retrieval techniques. The workflow illustrates three key stages: (1) breaking a complex query into focused subqueries, (2) routing each subquery to the appropriate tool or data source to retrieve relevant information, and (3) reasoning across the answers to synthesize a comprehensive response.

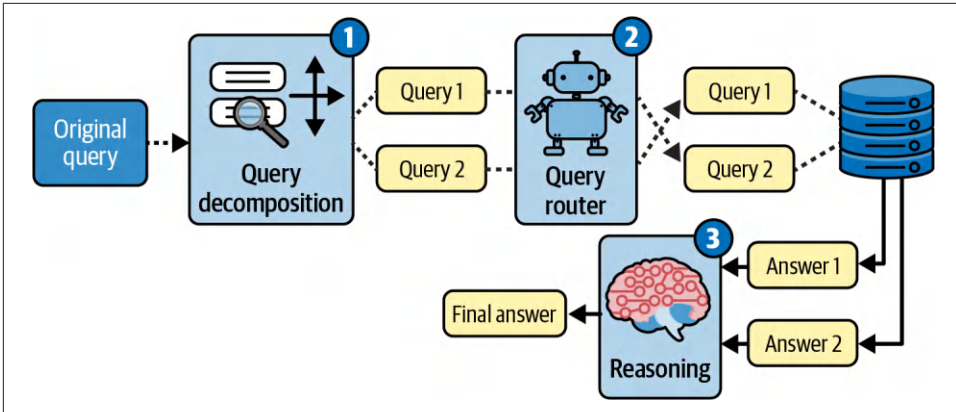


Figure 7-2. A multistep retrieval process

Think of the recipes in this chapter as building blocks that you can combine to improve your workflow and achieve better search results. You need a toolkit of advanced retrieval techniques that you can apply in the way that best fits your use case.

Table 7-1 summarizes seven techniques discussed in this chapter. You should understand each one and decide which makes sense for your specific dataset and task.

Table 7-1. Retrieval enhancement techniques

Technique	Description	Recipe reference
Metadata filtering	Using metadata to filter search results based on what you know about the user	Recipe 7.1
Multiquery retrieval	Creating multiple versions of the same prompt to find more relevant documents	Recipe 7.3
Query routing system	Using a query routing system to identify the best data sources or tools to use to answer the question	Recipe 7.4
Auto-merging retriever	Retrieving larger, more meaningful text chunks by grouping related data	Recipe 7.5
Sentence window retrieval	Including nearby sentences to add context to the retrieved text	Recipe 7.6
Hypothetical document embeddings (HyDE)	Generating hypothetical documents to improve search results	Recipe 7.2
Query decomposition	Breaking complex queries into simpler subqueries	Recipe 7.8
Reranking	Reviewing and evaluating the relevance of the retrieved documents via LLMs	Recipe 7.7

You can find all the code examples for this chapter in the [book's GitHub repository](#).

7.1 Optimizing Query Results via Metadata Filtering in PostgreSQL

Problem

You want to make search results more robust and accurate by applying metadata filters before running semantic search.

Solution

Storing metadata alongside embeddings lets you apply filters before semantic search. This narrows the search space to a relevant subset of your data. Imagine you're building a chatbot for a sports website covering football, tennis, and basketball. A user query such as "Who's the best player of all time?" is too vague. Without knowing the sport, answering properly is impossible.

If you have context about the user, you can filter the vector dataset down to the relevant sport. If the user is a football enthusiast, you filter to football content only and search within that subset.

In **Figure 7-3**, Jim is a football fan from Liverpool. When he asks for the latest match results of his favorite club, user preferences likely indicate that he's looking for last night's match results for FC Liverpool.

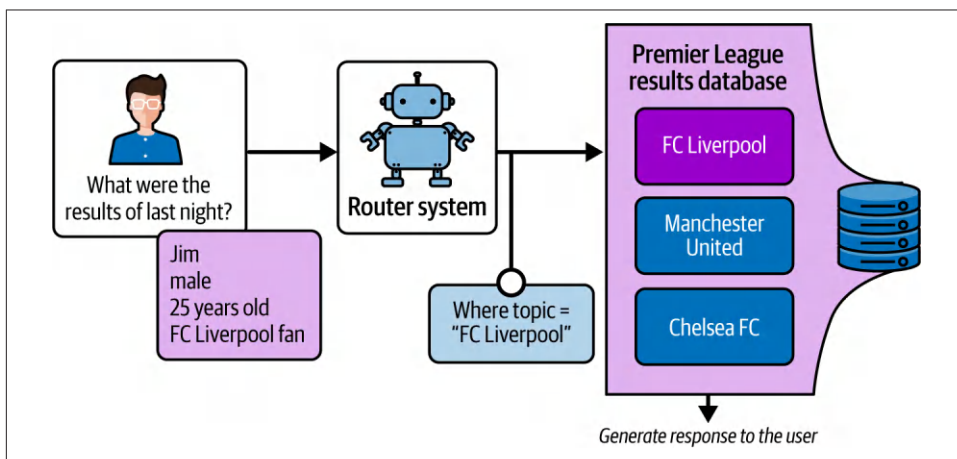


Figure 7-3. Using metadata filtering as part of a router retriever. The router prefilters the search space based on metadata before performing the search

Let's implement this approach with PostgreSQL as the vector database. You need only the Psycopg 2 library to connect to PostgreSQL and the OpenAI client to generate vector embeddings:

```
pip install psycpg2 openai
```

For this example, imagine a dataset with general knowledge about football and tennis. This recipe shows how to answer questions about the greatest player by filtering based on user preferences.

Start by setting up your database schema. To apply filters later, store the relevant metadata up front. Create a table with embedding vectors and a metadata column that holds a JSON object:

```
# Create the vector extension if it does not exist
cur.execute("""CREATE EXTENSION IF NOT EXISTS vector""")

# Create table with metadata column
cur.execute(
    """
    CREATE TABLE IF NOT EXISTS embeddings_table_with_metadata (
        id SERIAL PRIMARY KEY,
        text_chunk TEXT,
        embedding VECTOR(1536),
        metadata JSONB
    )
    """
)
conn.commit()
```

With the table structure in place, populate it with sample data. You'll insert four text chunks—two about tennis and two about football. Each row contains the text, its embedding vector, and metadata as a JSON object. For this basic example, the only metadata field is the topic of the text chunk, set to either tennis or football:

```
# Define text chunks and metadata
text_chunks = [
    {
        "text": "Roger Federer has won 20 Grand Slam titles in tennis.",
        "topic": "tennis",
    },
    {
        "text": "The FIFA World Cup is the most prestigious "
        "football tournament.",
        "topic": "football",
    },
    {
        "text": "Serena Williams is one of the greatest tennis "
        "players of all time.",
        "topic": "tennis",
    },
    {
        "text": "Lionel Messi has won multiple Ballon d'Or "
        "awards in football.",
        "topic": "football",
    },
]
```

```

]
# Initialize OpenAI client
client = OpenAI()

# Insert text chunks with embeddings and metadata
for chunk in text_chunks:
    # Get embedding using OpenAI API
    response = client.embeddings.create(
        input=chunk["text"],
        model="text-embedding-3-small"
    )
    embedding = response.data[0].embedding

    metadata = {"topic": chunk["topic"]}
    cur.execute(
        "INSERT INTO embeddings_table_with_metadata "
        "(text_chunk, embedding, metadata) VALUES (%s, %s, %s)",
        (chunk["text"], embedding, json.dumps(metadata)),
    )

conn.commit()

```

Figure 7-4 shows what the table looks like in pgAdmin.

	id [PK] integer	text_chunk text	embedding vector	metadata jsonb
1	1	Roger Federer has won 20 Grand Slam titles in tennis.	[-0.03446877,0.014871765,0.0043380065,...]	({'topic': 'tennis'})
2	2	The FIFA World Cup is the most prestigious football tou...	[-0.00076958816,-0.0010350262,0.009684...	({'topic': 'football'})
3	3	Serena Williams is one of the greatest tennis players of ...	[0.03279612,0.017161418,-0.00071505905...	({'topic': 'tennis'})
4	4	Lionel Messi has won multiple Ballon d'Or awards in foo...	[-0.019959558,-0.026812702,0.002324525,...]	({'topic': 'football'})

Figure 7-4. The populated metadata filtering table as viewed in pgAdmin

Now that the data is loaded with metadata, use this metadata column to perform filtering before semantic search. Imagine a user query asking for the best player of all time—an ambiguous question unless you know which sport the user has in mind.

If you know the user's preferences—say they're primarily a football fan—apply a metadata filter for entries with the topic of football, then perform a cosine-similarity search on the filtered subset:

```

# Query the table with metadata filtering
query = "Who is the best player?"
response = client.embeddings.create(
    input=query,

```



```

    model="text-embedding-3-small"
)
query_embedding = response.data[0].embedding
topic_filter = "football"

cur.execute(
    f"""
        SELECT text_chunk, 1 - (embedding <=> %s::vector) AS similarity
        FROM embeddings_table_with_metadata
        WHERE metadata->>'topic' = %s
        ORDER BY similarity DESC
        LIMIT 5
    """,
    (query_embedding, topic_filter),
)
results = cur.fetchall()

```

Figure 7-5 shows the results. The snippet about Lionel Messi and his multiple Ballon d'Or awards is ranked highest and is therefore the most relevant match.

```

results
[33] ✓ 0.0s Python
... [("Lionel Messi has won multiple Ballon d'Or awards in football.",
      0.9999992847442627),
      ('The FIFA World Cup is the most prestigious football tournament.',
      0.3422972747925217)]

```

Figure 7-5. Query results showing metadata filtering in action

The query successfully retrieves football-related content because the metadata filter was set to `football` based on prior knowledge about the user. An alternative approach is to have an LLM analyze the question and the available content in the vector database, and then let the model propose the best metadata filters.

To avoid increasing latency, you can use a fast and efficient model for this step, because in many cases it is obvious what the user is looking for, assuming it's clear what data is available in the knowledge source. The more difficult part is analyzing complex snippets and reaching a good conclusion. For that reasoning step, using a bigger, more capable model makes sense.

Discussion

Metadata filtering works by narrowing the search space before semantic search runs. This matters when your vector database contains diverse content and a global search returns irrelevant matches. The mechanism is straightforward: filter by metadata first, then run cosine similarity only within that subset.

Use metadata filtering when you have distinct user segments (logged-in users with preferences), content naturally partitions into categories (products by department, articles by topic), or queries become ambiguous without context (“best player” means different things in different sports).

Skip this approach when your entire dataset is topically homogeneous, you lack metadata to filter on, or the overhead of maintaining metadata tags exceeds the accuracy benefit.

The main trade-off is up-front metadata collection effort versus ongoing accuracy gains. Testability improves significantly: when you segment retrieval by metadata (e.g., football versus tennis), you can build evaluation datasets per topic, making it easier to measure performance and identify weak spots.

Start with one category, validate retrieval quality, then expand incrementally. Often the hardest part is collecting the right metadata. [Recipe 4.1](#) shows how to extract existing metadata from documents and generate new metadata by having an LLM analyze content.

See Also

- [The Chroma metadata filtering documentation](#) shows practical examples with operators like `$eq`, `$gt`, and `$lt`.
- [The Pinecone metadata filtering guide](#) demonstrates combining vector search with metadata filters.
- [The PostgreSQL JSONB documentation](#) covers JSON querying and indexing for metadata storage.

7.2 Enhancing Retrieval Accuracy with HyDE

Problem

You want to improve search results by creating pseudodocuments that match the style of real content in your database.

Solution

Hypothetical document embeddings (HyDE) generate hypothetical documents that match the user’s query. These hypothetical documents are then used for semantic search.

The hypothetical documents are not intended to be factually correct. They simply resemble plausible passages from articles or books that could contain an answer,

written in a style similar to real documents in your database. This style matching can improve retrieval results.

The process is straightforward:

1. The user asks a question.
2. The LLM generates a hypothetical document that could answer the user's question.
3. You embed the hypothetical document(s) and search the vector database.
4. You use the retrieved real documents to generate the final answer.

Figure 7-6 visualizes these steps by using two examples from the **paper** that first introduced the HyDE concept. The figure shows a medical query generating a passage about wisdom tooth removal time, and a scientific query generating a passage with statistics about COVID's mental health impact. Both demonstrate how HyDE creates text that stylistically matches the target corpus.

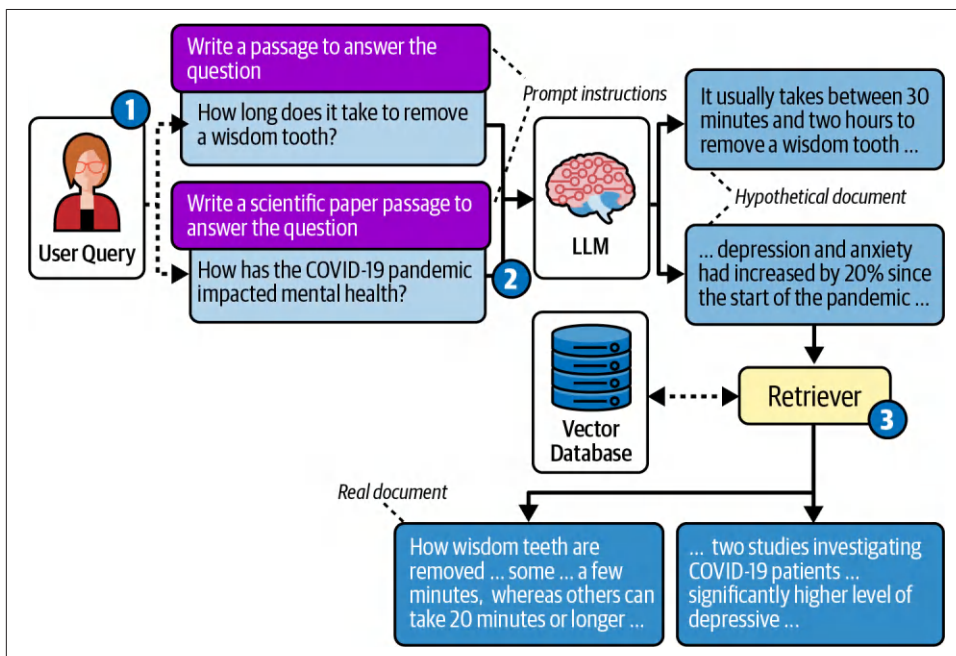


Figure 7-6. The LLM generates a hypothetical paragraph to assist in answering the question

In this recipe, you use an OpenAI model to generate hypothetical documents.

A smaller, efficient model is often sufficient because it needs to produce only plausible text snippets that match the query. The model should return only the hypothetical documents and nothing else.

To ensure that the model returns a list of hypothetical documents in a predictable format, use Pydantic to define the expected response schema. Install it as follows:

```
pip install openai pydantic
```

Then generate hypothetical documents for a query asking for the revenue of Company X in 2024:

```
from pydantic import BaseModel
from openai import OpenAI

user_query = "What is the revenue of Company X in 2024?"

client = OpenAI()

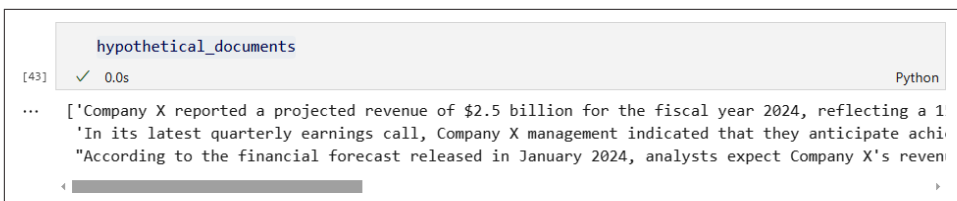
class HypotheticalDocuments(BaseModel):
    documents: list[str]

prompt = f"""
You are an AI assistant. Based on the user query below, generate
three hypothetical text chunks that contain relevant information to
answer the query.
"""

completion = client.beta.chat.completions.parse(
    messages=[
        {"role": "system", "content": prompt},
        {"role": "user", "content": user_query},
    ],
    model="gpt-5-mini",
    response_format=HypotheticalDocuments,
)

hypothetical_documents = completion.choices[0].message.parsed.documents
```

Figure 7-7 shows the generated hypothetical documents. The LLM creates paragraphs that resemble financial reports, which can help retrieve better results from the vector database.



```
hypothetical_documents
[43] ✓ 0.0s Python
... ['Company X reported a projected revenue of $2.5 billion for the fiscal year 2024, reflecting a 1
      'In its latest quarterly earnings call, Company X management indicated that they anticipate achi
      "According to the financial forecast released in January 2024, analysts expect Company X's reven
```

Figure 7-7. Hypothetical documents generated to answer “What is the revenue of Company X in 2024?”

The next step follows the basic RAG approach: use the hypothetical documents to generate embeddings, search the vector database, and then build a prompt around the retrieved content.

The hypothetical documents are not included in the final prompt. They are replaced with the real documents retrieved from the vector database.

Discussion

HyDE works by transforming queries into text that stylistically matches your corpus. If your documents are scientific papers with formal language, HyDE generates formal hypothetical passages. If your documents are casual blog posts, HyDE generates casual text. The embedding similarity becomes style + topic rather than just topic.

Use HyDE when you have a vocabulary or style mismatch between user queries and stored documents. For example, users ask, “How do I fix my car?” but your documents say, “automotive repair procedures.” HyDE bridges this gap by generating text in your corpus’s style.

Skip this approach when query and document styles already match, your baseline retrieval works well (validate with eval metrics first), or the added latency (one extra LLM call per query) and cost make the trade-off unappealing.

The main risk is generating plausible but incorrect hypothetical documents that lead to wrong retrievals. This happens more with specialized domains where the model lacks knowledge. Test thoroughly before deploying to production.



HyDE differs from query expansion: query expansion generates alternative phrasings of the same question, while HyDE generates hypothetical answer passages. HyDE is most effective with cross-domain retrieval where linguistic style matters more than exact keyword matching.

See Also

- [The LlamaIndex HyDE implementation](#) provides a complete working example you can adapt.
- [The LangChain RAG tutorial](#) includes HyDE integration in a production pipeline.
- [The original HyDE paper](#) describes the technique and evaluation results across multiple domains.

7.3 Improving Search Results with Multiquery Retrieval

Problem

You want to send multiple queries to your vector database that cover numerous facets of the original user query to ensure that you retrieve all relevant content.

Solution

Multiquery retrieval creates several variations of the original user's question, performs a semantic search with each one, and then combines the results to produce a more comprehensive answer.

The process is straightforward, as shown in [Figure 7-8](#):

1. Take the original query and generate two to three alternative phrasings.
2. Search the vector database with each query.
3. Merge the documents retrieved from each search run and send them to the LLM.

Each query variant can surface different relevant documents, giving you broader coverage of the content in your knowledge base.

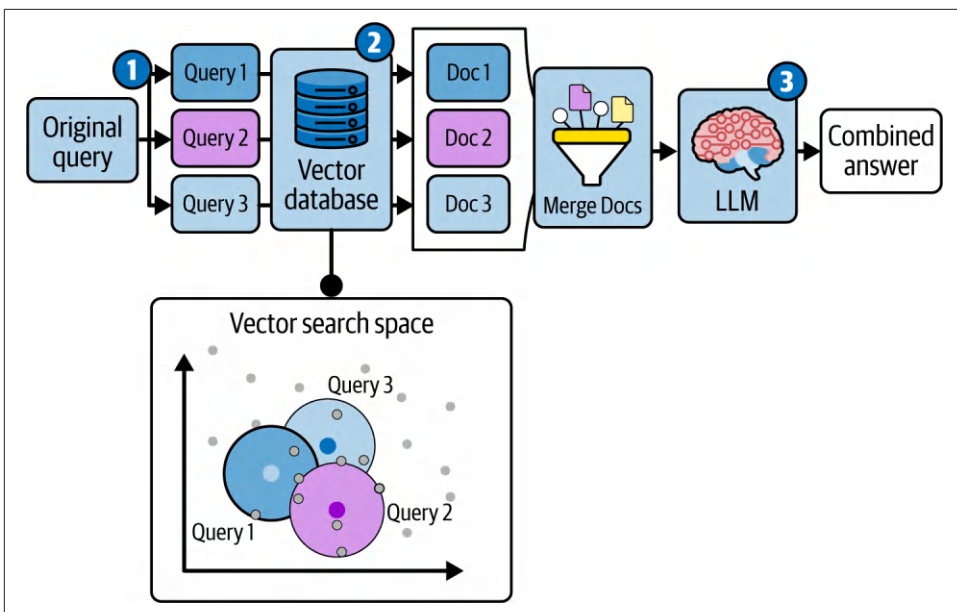


Figure 7-8. Multiquery retrieval

A sample user query might be “What are the benefits of renewable energy?” Multiquery retrieval breaks this straightforward-sounding question into multiple facets that matter for answering it well.

In this case, here are three sample queries derived from the original query:

- What are the environmental benefits of renewable energy?
- How does renewable energy contribute to economic growth?
- What are the drawbacks of renewable energy?

Each of these targets a different aspect that may be relevant to answering the original question. Based on what the LLM knows about the world, the environmental impact, economic effects, and potential challenges are all likely to be important for generating a thorough answer.

For the practical example in this recipe, you use an OpenAI LLM to generate multiple variations of the original query. To ensure that the result is strictly a list of strings, the code uses Pydantic to define a retrieval template. To run the code, install the OpenAI SDK and Pydantic:

```
pip install openai pydantic
```

You use a prompt template to instruct the LLM to rephrase the original query into several subqueries. This prompt is intentionally simple: it asks for diverse variations

of the same query and clarifies the purpose of retrieving documents from a vector database and mitigating limitations of distance-based similarity search:

```
from openai import OpenAI
from pydantic import BaseModel
import os

client = OpenAI()

question = "What are the benefits of renewable energy?"

query_prompt = f"""You are an AI language model assistant. Your task is
to create three alternative versions of the provided user query to
enhance the retrieval of relevant documents from a vector database.
By offering diverse variations of the query, your goal is to help
mitigate the limitations of distance-based similarity search. Provide
these alternative queries, each on a new line.

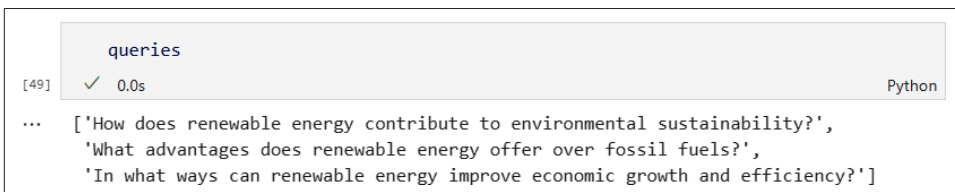
Original query: {question}
"""

# Send the query prompt to OpenAI
class QueryVariations(BaseModel):
    queries: list[str]

completion = client.beta.chat.completions.parse(
    model="gpt-5-mini",
    messages=[
        {
            "role": "user",
            "content": query_prompt,
        },
    ],
    response_format=QueryVariations,
)

queries = completion.choices[0].message.parsed.queries
```

Figure 7-9 shows the output. In this example, the model generates three alternative versions of the original question (“What are the benefits of renewable energy?”).



```
[49] ✓ 0.0s Python
... ['How does renewable energy contribute to environmental sustainability?',
     'What advantages does renewable energy offer over fossil fuels?',
     'In what ways can renewable energy improve economic growth and efficiency?']
```

Figure 7-9. The derived versions of the query

Next, the retrieval step performs a semantic search for all three generated queries plus the original query. That means you have four vector searches in total, returning four sets of results. You can then use a merger to combine these results, and, if needed, apply a reranking method to score the retrieved snippets and keep only the best-matching ones. Reranking is covered in [Recipe 7.7](#).

Discussion

Multiquery retrieval works by expanding one query into N variants, running N vector searches, and merging results. The mechanism exploits the fact that different phrasings surface different documents, improving recall when the original query misses relevant content because of a vocabulary mismatch.



Recall measures the number of relevant documents the retriever finds out of all relevant documents that exist in your database. For RAG systems, high recall means the retriever successfully finds most of the useful information available, even if it also retrieves some irrelevant content that gets filtered later.

Use multiquery retrieval when user questions are broad or ambiguous (“renewable energy benefits” can mean environmental, economic, or social), single queries consistently miss relevant documents that your eval set shows exist, or improving recall justifies the higher cost of multiple searches.

Skip this approach when users ask narrow, specific questions that single queries handle well, your retrieval latency budget can’t absorb N parallel searches plus one LLM call, or eval metrics show that single-query retrieval already achieves target recall.

The main trade-off is cost and latency versus recall. Running three searches plus one LLM call for query generation roughly triples retrieval time and compute cost. This makes sense for complex research queries but not for simple FAQ lookups.

Multiquery differs from query decomposition: multiquery generates semantically similar variants of the same question, while decomposition breaks complex questions into distinct subquestions that need separate answers. Combine both when you have complex multipart questions that also benefit from multiple phrasings per part.

See Also

- [The LlamaIndex multiquery retriever tutorial](#) shows how to combine retrieval results from multiple queries and indexes.
- [The LangChain MultiQueryRetriever](#) provides an alternative implementation with result merging.

7.4 Addressing Complex Requests by Designing a Query Routing System

Problem

You want to handle complex user questions by directing them to the right data source.

Solution

A query routing system selects the most suitable tool or data source to solve a user's query. For example, a user asking for the result of multiplying 87 by 99 is requesting a basic mathematical operation that is best handled by a calculator tool. By contrast, a question about Google's revenue in a particular year is better handled via vector search, because the answer is likely written in one of Google's financial reports.

If you have a RAG system connected to multiple data sources and tools that can perform a certain action, you need an orchestrator that routes each question to the right component.

Figure 7-10 shows a basic example of such a system. It has access to financial reports for multiple companies—for example, a knowledge store containing Microsoft revenue data and Google financial figures. In addition, the query routing system is connected to a SQL database that contains time-series data such as each company's share price.

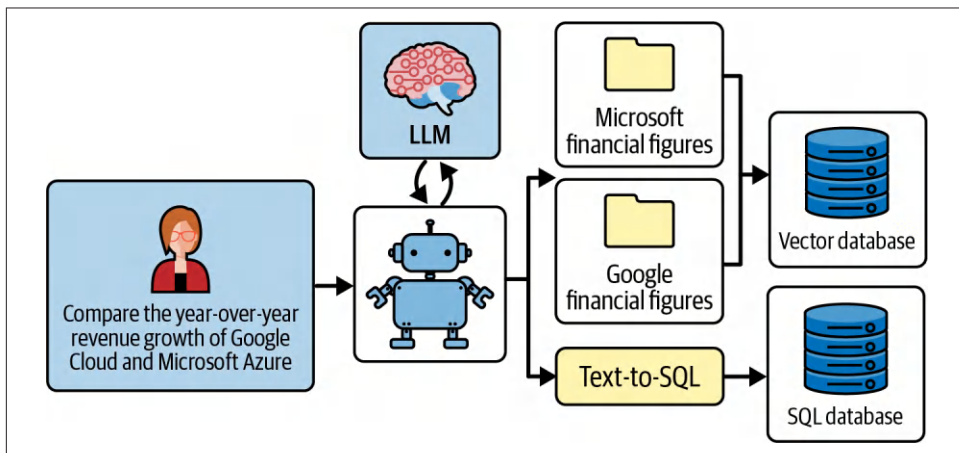


Figure 7-10. Query routing with function calling models. Here, the agent can query either the vector database or the SQL database. The agent should decide which route is most appropriate.

The router selects the data source that best fits the user's question. For complex questions, the router can also break the task into multiple subquestions that can be

answered using different data sources and tools, then merge the results of the different queries into a final answer.

This recipe uses an LLM-based router. An efficient model is prompted with the available functions, tools, and data sources, and it decides where to retrieve the information from.

Typically, every tool or database that can deliver a particular kind of information is exposed as a function. The router selects which function to call, and the Python code executes the function. The router in this recipe uses an OpenAI LLM to make the routing decision, while Pydantic ensures that the output contains only the name of the function to call. Install the required dependencies with the following command:

```
pip install openai pydantic
```

You'll define a prompt that instructs the model to choose from these three tools:

- A tool connected to a vector database with general football knowledge, for questions such as “What is the offside rule in football?”
- A tool connected to a vector database about tennis.
- A tool that simulates a connection to a SQL database holding the latest football match results.

You implement this router by defining a Pydantic schema that constrains the LLM to select one of these three tools. Based on the user's question, the router step calls the LLM and lets it decide which tool to use. The LLM returns only a string label identifying the selected tool. Here's the code:

```
from pydantic import BaseModel
from openai import OpenAI

client = OpenAI()

user_queries = [
    {
        "query": "Who is the all-time top scorer in the FIFA World Cup?",
        "selected_data_source": None,
    },
    {
        "query": "What are the four Grand Slam tennis tournaments?",
        "selected_data_source": None,
    },
    {
        "query": "Did Manchester United win their last game?",
        "selected_data_source": None,
    },
]

prompt = f"""
```

```

You are an expert at routing a user question to the appropriate
data source. Given a user question choose which of the data sources
in list_of_data_sources is the best to answer the question.
"""

```

```

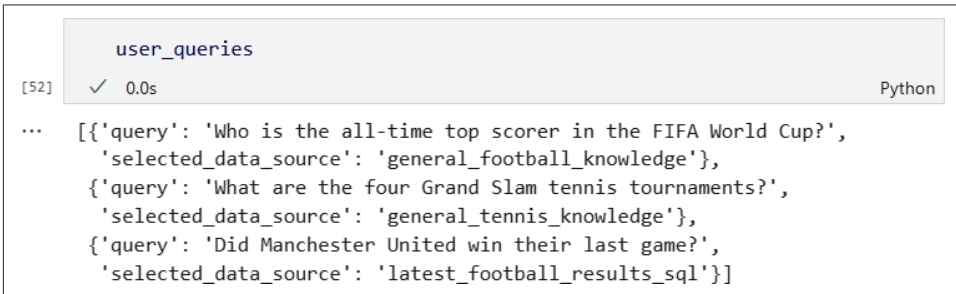
from typing import Literal
from pydantic import Field

class RouterDecision(BaseModel):
    data_source: Literal[
        "general_football_knowledge",
        "general_tennis_knowledge",
        "latest_football_results_sql",
    ] = Field(
        ...,
        description="The best data source to answer the question."
    )

for user_query in user_queries:
    completion = client.beta.chat.completions.parse(
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": user_query["query"]},
        ],
        model="gpt-5-mini",
        response_format=RouterDecision,
    )
    user_query["selected_data_source"] = completion.choices[
        0
    ].message.parsed.data_source

```

Figure 7-11 shows the results of the query routing system.



```

user_queries
[52] ✓ 0.0s Python
... [{"query": "Who is the all-time top scorer in the FIFA World Cup?",
      "selected_data_source": "general_football_knowledge"},
     {"query": "What are the four Grand Slam tennis tournaments?",
      "selected_data_source": "general_tennis_knowledge"},
     {"query": "Did Manchester United win their last game?",
      "selected_data_source": "latest_football_results_sql"}]

```

Figure 7-11. Query routing system results

The first query about the FIFA World Cup top scorer routes to general football knowledge. The second query about Grand Slam tournaments routes to tennis knowledge. The last query about Manchester United's recent match routes to the SQL database with the latest results.

For more complex questions, you can also instruct the model to break the overall query into multiple subqueries. Each subquery can be routed to a different tool, and a final LLM reasoning step consolidates the results.

Discussion

Query routing works by delegating retrieval decisions to a classifier that selects the appropriate data source or tool. The mechanism is simple: analyze the query, pick the best source, and execute the retrieval. The preceding code demonstrates the routing decision. In a complete implementation, you would execute the appropriate retrieval function (e.g., `search_vector_db(query)` for knowledge bases or `query_sql_db(query)` for structured data) based on the router's selection.

Use routing when you have multiple distinct data sources (vector stores, SQL databases, APIs) and queries clearly map to one source, query types are predictable enough to classify reliably, or the cost of searching wrong sources (irrelevant results, wasted compute) exceeds routing overhead.

Skip routing when you have a single data source, it's faster to search everything and filter than to route first, or your queries are too ambiguous for reliable classification.

LLM-based routing adds latency of 500 ms to 2 seconds per request. While 2 seconds isn't necessarily a deal-breaker, it adds noticeable delay to user-facing chat applications where users expect quick responses. Routing works well for asynchronous workflows like batch processing or email automation. Two faster alternatives exist:

Majority vote

Search all collections and route to the one with the most results. Latency matches a single search but multiplies compute cost.

Embedding classifier

Train a lightweight model to predict the best collection from query embeddings. This adds latency of 10 to 50 ms with minimal compute overhead.

Start with LLM routing to validate your routing logic. If latency becomes a problem and collections are topically distinct, switch to an embedding classifier. If collections overlap heavily, LLM routing may be your only reliable option despite the latency cost.

Routing differs from metadata filtering: routing selects from separate data sources before search, while filtering narrows results within a single source. Combine both when queries need source selection (routing) and then category filtering (metadata) within that source.

Figure 7-12 shows alternative approaches to build a query routing system.

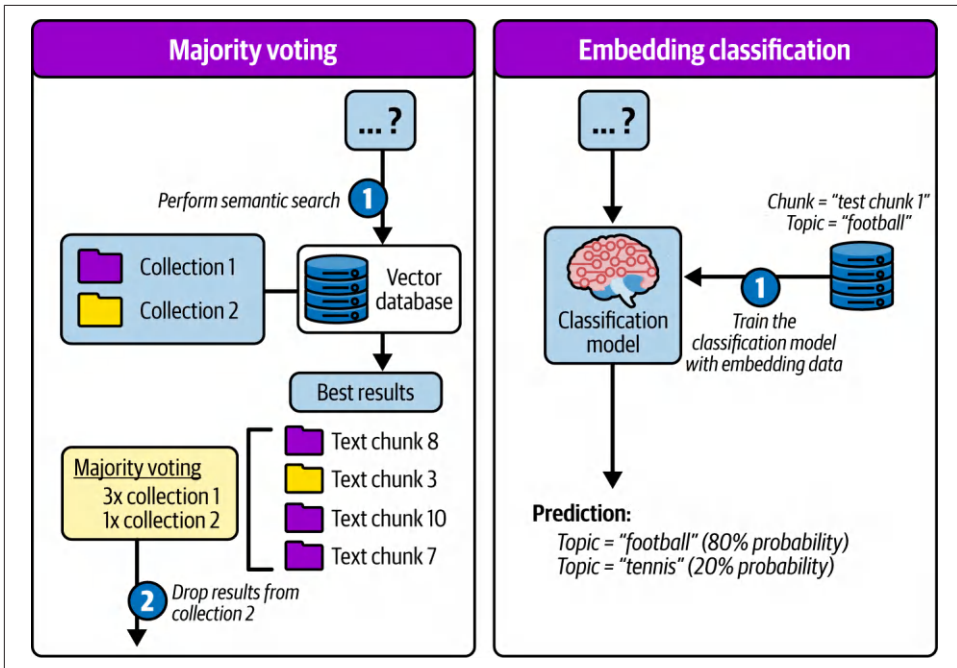


Figure 7-12. Alternative approaches to building a query routing system



Frameworks like LangChain, LlamaIndex, Amazon Bedrock Agents, and Azure AI Agent Service offer prebuilt routing solutions. However, since a router is often just a well-crafted prompt, consider whether you need the additional complexity before adding framework dependencies.

See Also

- [The LlamaIndex router query engine](#) provides a complete routing implementation for multisource systems.
- [The LangChain multiagent router](#) offers routing components for agents and workflows.
- [The AWS Bedrock Agents documentation](#) demonstrates production routing patterns for enterprise systems.

7.5 Enhancing Retrieved Documents by Designing an Auto-Merging Retriever

Problem

You want to optimize retrieval and generation by using small text chunks for semantic search while providing larger parent chunks to the LLM for the generation step.

Solution

An *auto-merging retriever* organizes text into smaller child chunks linked to larger parent chunks. Child chunks are used for vector search because they're focused enough to capture distinct information units. After retrieval, the system counts the number of child chunks that belong to the same parent. If a certain threshold number of child chunks from a given parent are identified as relevant, the system provides the entire parent chunk to the LLM, giving it broader context for generation.

Figure 7-13 shows a three-level tree structure with parent nodes at the top, child nodes in the middle, and leaf nodes at the bottom. The leaf nodes are used for semantic search. If a threshold number of leaf nodes from a specific child or parent node are retrieved, the system merges them and provides the whole child or parent chunk to the prompt. For example, if leaf nodes 1, 2, and 5 are relevant, and two of them belong to child node 1, the system retrieves the complete text from child node 1, but from child node 2 it retrieves only leaf node 5.

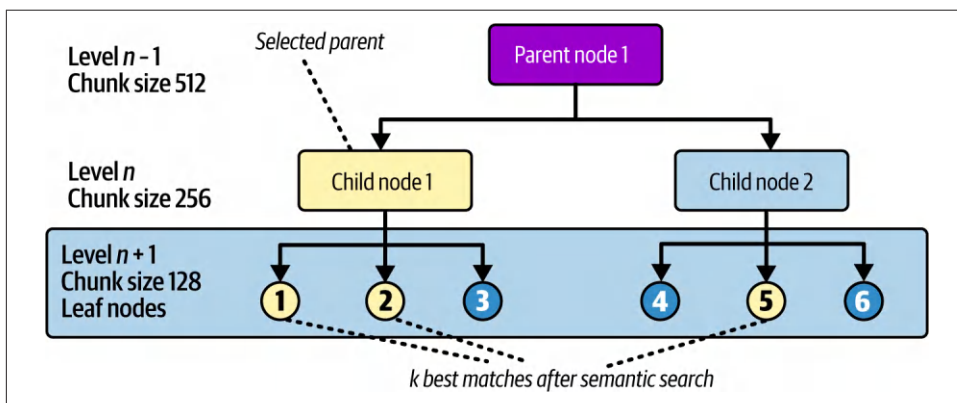


Figure 7-13. The auto-merging retriever working with parent and child nodes

This recipe uses PostgreSQL as a vector database and OpenAI embedding models, which you install as follows:

```
pip install psycopg2 openai
```

Next, you create a simple two-level structure. Each child chunk contains roughly 250 characters, and four child chunks are merged into one parent chunk, resulting in roughly 1,000 characters per parent chunk. Each leaf node clearly belongs to exactly one parent node.

To use these relationships later, each parent node and each leaf node has an ID. Additionally, each child chunk stores both the child node ID and the parent node ID in separate columns:

```
file_path = (
    "../datasets/text_files/"
    "random-text-about-5-different-stories-with-paragraphs.txt"
)

with open(file_path, "r", encoding="utf-8") as f:
    text = f.read()

leaf_node_size = 250 # Size of the leaf nodes in characters
parent_merge = 4 # Number of leaf nodes to merge into a parent node
parent_node_size = (
    parent_merge * leaf_node_size
) # Size of the parent nodes in characters
text_chunks = [] # List to store the text chunk dictionaries

for leaf_node_id in range(0, len(text) // leaf_node_size, 1):
    parent_node_id = leaf_node_id // parent_merge
    leaf_chunk_start = leaf_node_id * leaf_node_size
    parent_chunk_start = parent_node_id * parent_node_size

    text_chunk = {
        "leaf_node_id": leaf_node_id,
        "leaf_chunk": text[
            leaf_chunk_start : leaf_chunk_start + leaf_node_size
        ],
        "parent_node_id": parent_node_id,
        "parent_chunk": text[
            parent_chunk_start : parent_chunk_start + parent_node_size
        ],
    }
    text_chunks.append(text_chunk)
```

Create the table that stores the leaf node, leaf chunk, leaf node ID, parent node, parent node ID, and the embeddings for the leaf chunk:

```
# Create a new table for storing chunks and embeddings
cur.execute("""CREATE EXTENSION IF NOT EXISTS vector""")

cur.execute(
    """
    CREATE TABLE auto_merging_retriever_text_chunks (
        leaf_node_id SERIAL PRIMARY KEY,
        leaf_chunk TEXT,
```



```

        parent_node_id INTEGER,
        parent_chunk TEXT,
        leaf_chunk_embedding VECTOR(1536)
    )
"""
)
conn.commit()

```

You then populate the table by using the previously created dictionary:

```

from openai import OpenAI

client = OpenAI() # Initialize OpenAI client

# Insert chunks and their embeddings into the table
for text_chunk in text_chunks:
    leaf_embedding = (
        client.embeddings.create(
            input=[text_chunk["leaf_chunk"]],
            model="text-embedding-3-small"
        )
        .data[0]
        .embedding
    )

    cur.execute(
        """
        INSERT INTO auto_merging_retriever_text_chunks
        (leaf_node_id, leaf_chunk, parent_node_id, parent_chunk,
        leaf_chunk_embedding)
        VALUES (%s, %s, %s, %s, %s)
        """,
        (
            text_chunk["leaf_node_id"],
            text_chunk["leaf_chunk"],
            text_chunk["parent_node_id"],
            text_chunk["parent_chunk"],
            leaf_embedding,
        ),
    )
conn.commit()

```

When a user now asks a question, perform a vector search over the leaf chunks. After the vector search, count whether any parent node has more than two leaf nodes marked as relevant. For those parent nodes, retrieve the parent text chunk via the parent node ID and provide the parent chunk to the generation model.

Discussion

Auto-merging searches at the child level for precision, then expands to parent chunks when multiple children from the same parent are relevant.

Use auto-merging when your documents have strong hierarchical structure (books with chapters and sections, research papers with sections), retrieval quality suffers from information spanning multiple chunks, or evaluation shows that larger context chunks improve generation quality without excessive prompt bloat.

Skip this approach when simple flat chunking works well, your documents lack clear hierarchy (FAQs, product descriptions), larger prompts increase costs without quality gains, or the added complexity of maintaining parent-child relationships isn't justified by evaluation metrics.

The main trade-off is storage complexity versus context quality. You store both parent and child chunks, maintain ID relationships, and implement merge logic. In exchange, you retrieve more coherent context when queries match multiple related fragments.

See Also

- [The LlamaIndex auto-merging retriever](#) works with a hierarchical node parser for multilevel chunk hierarchies.
- [The Haystack auto-merging implementation](#) uses a hierarchical document splitter for parent-leaf structures.
- [The pgvector documentation](#) covers efficient storage and retrieval of hierarchical embeddings in PostgreSQL.

7.6 Retrieving More Complete Text Chunks with a Sentence Window Retriever

Problem

You want to use small text chunks for semantic search, but still provide enough surrounding context to the LLM so it doesn't miss important information around the retrieved sentences.

Solution

A *sentence window retriever* adds a few sentences before and after the most relevant chunk to provide additional context. [Figure 7-14](#) shows an example with a question about FC Bayern Munich. A semantic search might identify a paragraph about recent successes and titles, but cut the historical context explaining what the club is, where the name comes from, and where it is located. Including the surrounding sentences ensures that this relevant context reaches the prompt.

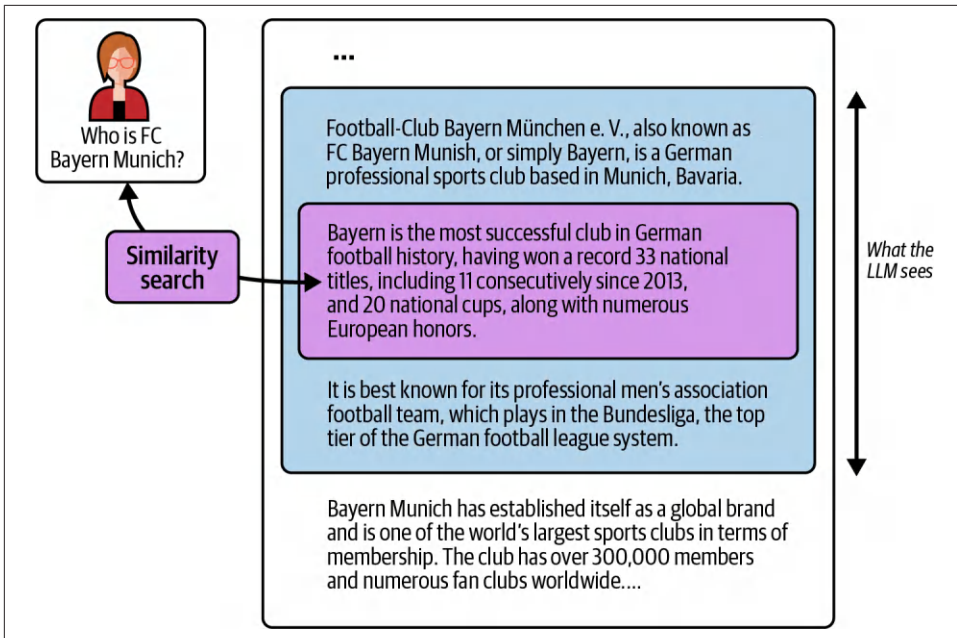


Figure 7-14. Sentence window retrieval

In [Figure 7-15](#), the sentence window is defined by three text chunks. The original document is split into small text elements for vector search. If element 4 is identified as relevant, the full window including elements 3, 4, and 5 is sent to the LLM.

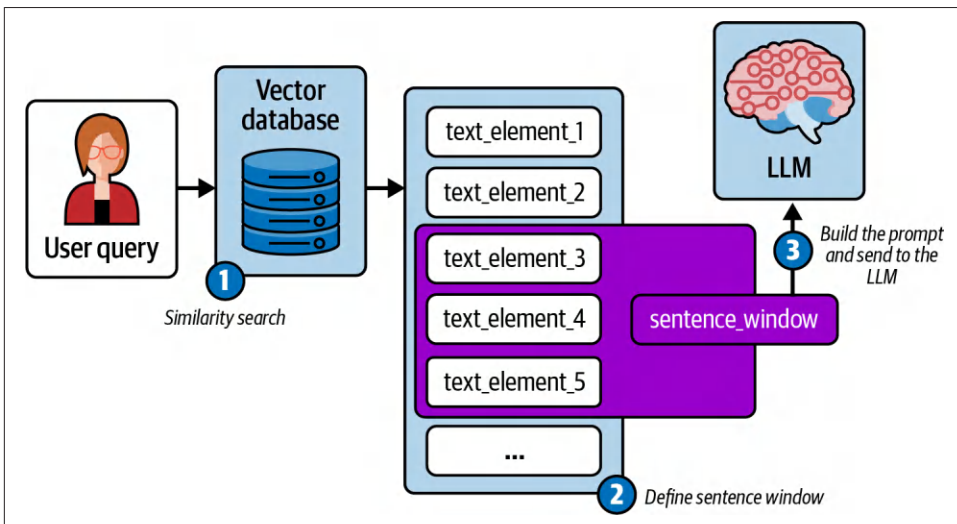


Figure 7-15. The sentence window retriever in practice

You need a vector store that can store the text chunks, their embeddings, and the original order of the chunks in the source document. This recipe uses PostgreSQL as the vector database and the Psycopg 2 package to connect from Python. Install Psycopg 2:

```
pip install psycopg2
```

Next, you'll define the table you need to create. It stores the chunk ID, the chunk text, the chunk embedding, and a document ID.

When loading documents into the vector database, ensure that the chunk ID reflects the original order. The first text snippet should have chunk ID 1, the second chunk ID 2, and so on. This later allows you to retrieve the chunks immediately before and after the chunk returned by semantic search. Define the table with this code:

```
cur.execute(
    """
    CREATE TABLE sentence_window_retriever_text_chunks (
        chunk_id SERIAL PRIMARY KEY,
        chunk TEXT,
        chunk_embedding VECTOR(1536)
    )
)
conn.commit()
```

To fill the table, you split the text, generate embeddings, and insert them as in previous recipes. The important part is that chunk IDs are sequential, showing the order of text in the document. Split the document into small chunks of roughly 250 tokens. When a chunk is identified as relevant during semantic search, select the chunk before and after it, then provide the combined window of roughly 750 tokens to the LLM.

This helps ensure that important information isn't separated across chunk boundaries. It also increases the likelihood that relevant surrounding context is included.

Discussion

Sentence window retrieval works by expanding retrieved chunks with adjacent context from the original document. The mechanism is simple: identify the most relevant chunk via semantic search, then include N chunks before and after based on their sequential position in the source document.

Use sentence window retrieval when your chunks are small (200–500 tokens) for retrieval precision, adjacent context frequently matters for understanding (legal docs, technical manuals, narrative text), and expanding by one or two chunks doesn't excessively bloat your prompt relative to your model's context limit.

Skip this approach when your chunks already contain 500+ tokens with plenty of context, documents are FAQ-style with each chunk self-contained, or evaluation metrics show that the added complexity provides no quality improvement.

The main trade-off is prompt size versus completeness. Expanding from 1,000 to 3,000 tokens per retrieved chunk triples context length, increasing cost and latency. The benefit is reducing boundary effects where relevant information gets split across chunk edges.

Sentence window retrieval differs from auto-merging: the former uses fixed positional expansion (always adding N adjacent chunks) regardless of content, while auto-merging uses relevance-based expansion (adding the parent chunk to the retrieved context only when multiple child chunks match). Sentence window retrieval is simpler to implement but less adaptive. Use it for straightforward positional context needs; use auto-merging when document structure and relevance patterns should drive expansion decisions.

See Also

- [The LlamaIndex sentence window retriever](#) shows a complete implementation with metadata replacement for context expansion.
- [LangChain ParentDocumentRetriever](#) provides a similar approach for retrieving parent documents with child chunks.

7.7 Improving Retrieval Relevancy with Reranking Methods

Problem

You have a system that retrieves documents from multiple databases and tools, and you want to filter out the most relevant documents by reranking the retrieved results.

Solution

Earlier recipes in this chapter described techniques for executing multiple queries, each returning potentially different results. The challenge is merging these results while identifying which documents truly matter for the final answer.

When you use HyDE, you generate multiple hypothetical documents and run vector search multiple times. Multiquery retrieval executes several reformulated queries in parallel. Agent systems may query multiple tools such as SQL databases, vector stores, APIs, or Python calculations. All these approaches produce a combined result set that needs filtering.

Reranking addresses this by scoring merged results for relevance, keeping only the top items. This reduces noise and improves the quality of the final prompt. [Figure 7-16](#) visualizes the process.

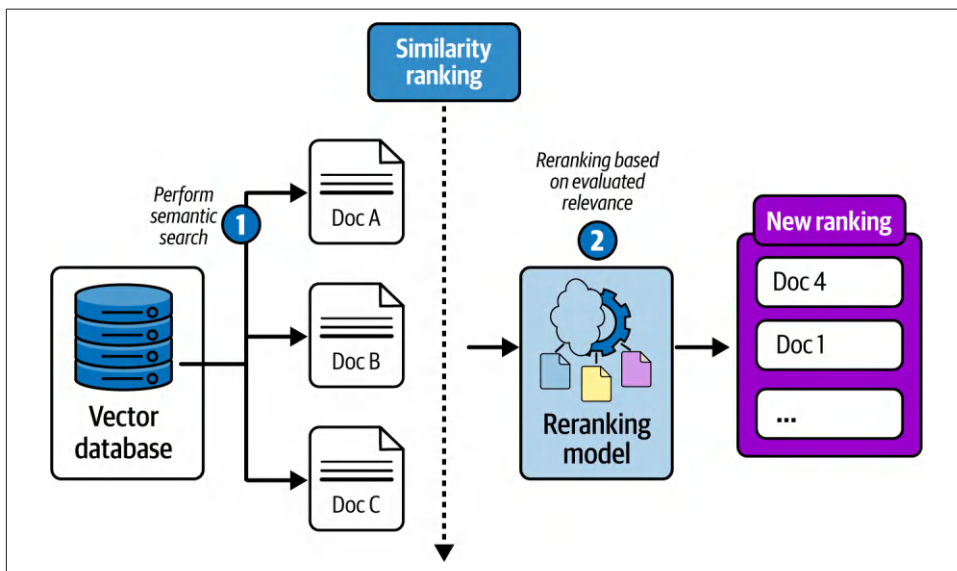


Figure 7-16. Applying reranking to the retrieved documents

The diagram shows three semantic searches, but in practice you could use any kind of retrieval step: SQL databases, vector stores, external APIs, or Python calculations. You merge all results into a single list, then filter the combined set before the LLM step.

This recipe uses an LLM-based reranker. An efficient LLM reviews the retrieved documents and assesses their relevance. This recipe also uses an OpenAI model to rerank the results and returns them by using a response schema defined as a Pydantic model. Install these dependencies:

```
pip install openai pydantic
```

This recipe provides the reranker with all retrieved documents. The original user query is about whether Tesla will remain the market leader in electric vehicles. Upstream, you might use a multiquery approach that derives five subqueries, performs vector search for each of them, and then selects the best results per query. You then merge those results into a single set.

Here, you define a prompt template that instructs the LLM to rerank the retrieved text chunks according to their relevance to the user's question:

```
import textwrap
from openai import OpenAI
from pydantic import BaseModel
```

```

text_chunks = {
    1: "Tesla's Supercharger network and tech lead face rising "
        "competition from BYD and established automakers.",
    2: "Tesla's production grows, but price competition threatens "
        "its market share.",
    3: "The automotive industry is shifting to EVs due to climate "
        "change and regulations.",
    4: "Semiconductor shortages are disrupting automotive supply "
        "chains.",
    5: "Consumer demand for autonomous driving and advanced tech "
        "impacts EV competition.",
}

prompt = textwrap.dedent(
    f"""
    Query: Will Tesla remain the market leader in electric vehicles?

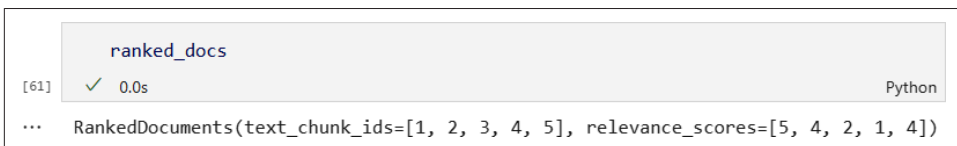
    Documents:
        1. {text_chunks[1]}
        2. {text_chunks[2]}
        3. {text_chunks[3]}
        4. {text_chunks[4]}
        5. {text_chunks[5]}

    Instructions:
    Please assess the relevance of each document to the query and
    provide a relevance score from 1 to 5, where 5 is the most relevant.

    Relevance Scores:
    """
)

```

Figure 7-17 shows the reranking output: a new ordering of the documents. According to the LLM, the fifth text chunk appears to be the most relevant to the user's question, followed by chunks 4, 2, and 1.



```

ranked_docs
[61] ✓ 0.0s Python
... RankedDocuments(text_chunk_ids=[1, 2, 3, 4, 5], relevance_scores=[5, 4, 2, 1, 4])

```

Figure 7-17. The reranking results

You still have the same five retrieved documents, but now you know which ones are most relevant. To improve response time and avoid adding too much noise to the final generation model, you can select only the top three text chunks and combine them into the final prompt.

Discussion

Reranking scores and filters merged retrieval results before generation, keeping only the top- K most relevant items.

Use reranking when you're combining 20+ candidates from multiple sources, initial retrieval is noisy with many marginally relevant results, or filtering to the top five after reranking improves quality versus taking the top five from each source separately.

Skip reranking when you have a single retrieval source with good native ranking, your retrieval already returns highly relevant results, or the added LLM call or cross-encoder cost isn't justified by quality gains measured through precision@ k metrics.

The main trade-off is latency and cost versus relevance. LLM-based reranking adds one inference call to process all candidates. Cross-encoder models are faster but still add overhead. In exchange, you get more relevant final results and can confidently filter from 50+ candidates down to 5–10 high-quality items.

Reranking differs from initial retrieval scoring: initial scoring (cosine similarity, BM25) is fast but treats the query and document independently, while reranking uses cross-attention between query and candidates for deeper relevance assessment. The two-stage approach balances speed (initial retrieval) and accuracy (reranking). Use reranking as a second pass over merged results, not as a replacement for fast initial retrieval.

See Also

- [LangChain RankLLM Reranker](#) provides LLM-based reranking for document relevance scoring.
- [LlamaIndex ColbertRerank](#) uses ColBERT models for fast and accurate reranking.
- [Haystack SentenceTransformersSimilarityRanker](#) uses Hugging Face cross-encoder models for document ranking.

7.8 Decomposing Complex Queries into Multiple Subqueries

Problem

You want to answer complex questions by breaking them into simpler, manageable parts.

Solution

Figure 7-18 shows the end-to-end process:

1. The complex query is broken into simpler subqueries.
2. Each subquery triggers a retrieval step.
3. The retrieved context generates an answer for each subquery.
4. A final reasoning step combines everything to produce one answer.

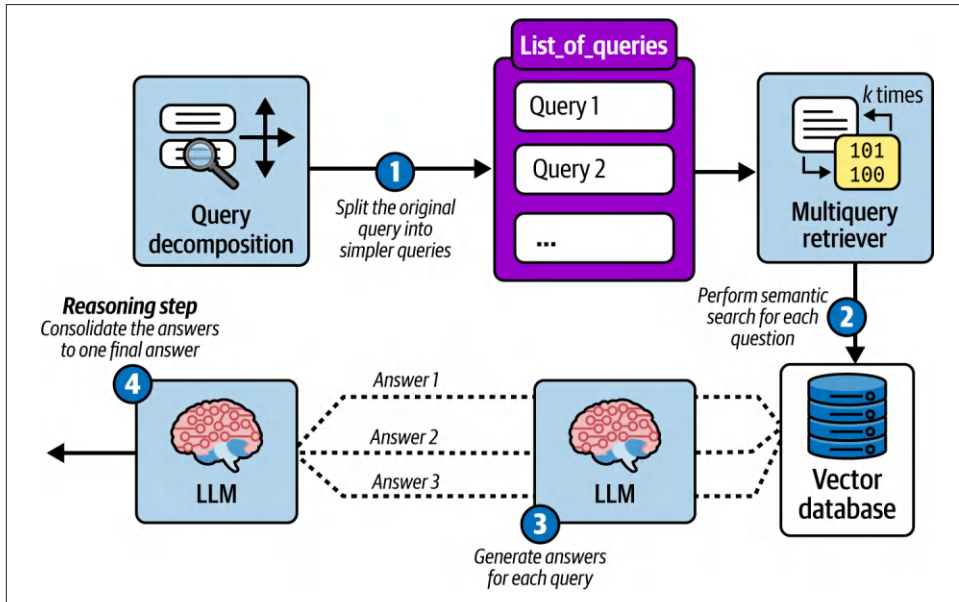


Figure 7-18. Decomposing complex queries

This recipe uses an OpenAI LLM to break a complex user query into simpler subqueries. You also use the OpenAI SDK and define the response template as a Pydantic model. Install the dependencies:

```
pip install openai pydantic
```

You define a Pydantic model that captures question-answer pairs. The final response is a list of questions, and each question is actually a Python object with the question and answer.

Here's the code to define the prompt instructions and the response template as a Pydantic model:

```
from pydantic import BaseModel
from typing import Optional
from openai import OpenAI
```

```

class Question(BaseModel):
    question: str
    answer: Optional[str] = None

class Questions(BaseModel):
    questions: list[Question]

splitter_prompt = """
You are a helpful assistant for a RAG chatbot.

Your job is to break down complex questions into simpler ones that
are easy to answer. When the answers to these simpler questions are
combined, they should fully answer the original question. If the
question is already simple, leave it as it is. Handle one question
at a time.

Example:
    1. Query: Did Microsoft or Google make more money last year?

Decomposed Questions:
    1. How much profit did Microsoft make last year?
    2. How much profit did Google make last year?
"""

```

Next, you apply these templates to a sample query about the benefits of renewable energy compared to fossil fuels. The question is not highly complex, but it is unclear from which angle the user is asking.

Because the query is vague, a human would typically explore it from multiple perspectives to provide a complete answer. This RAG process does the same by splitting the original query into subqueries that approach the topic from different angles:

```

client = OpenAI()

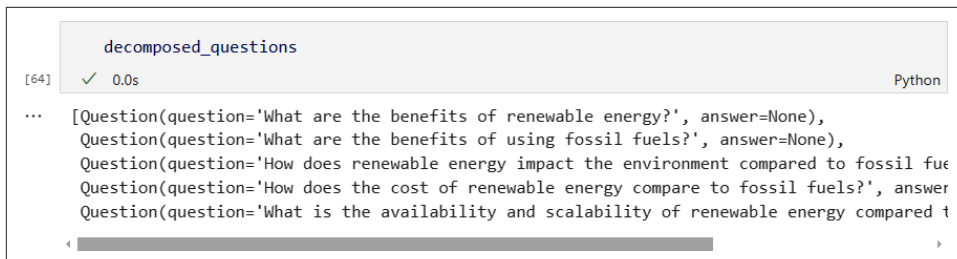
query = (
    "What are the benefits of renewable energy compared to "
    "fossil fuels?"
)

completion = client.beta.chat.completions.parse(
    model="gpt-5-mini",
    messages=[
        {"role": "system", "content": splitter_prompt},
        {"role": "user", "content": query},
    ],
    response_format=Questions,
)

decomposed_questions = completion.choices[0].message.parsed.questions

```

Figure 7-19 shows the result of the decomposition step. The LLM divides the original question about the benefits of renewable energy into five simpler subqueries. For example, the first subquery asks about the benefits of renewable energy, and the second asks about the benefits of using fossil fuels. Additional subqueries cover key aspects such as environmental impact, cost, and availability.



```
[64]: ✓ 0.0s Python
... [Question(question='What are the benefits of renewable energy?', answer=None),
      Question(question='What are the benefits of using fossil fuels?', answer=None),
      Question(question='How does renewable energy impact the environment compared to fossil fuels?', answer=None),
      Question(question='How does the cost of renewable energy compare to fossil fuels?', answer=None),
      Question(question='What is the availability and scalability of renewable energy compared to fossil fuels?', answer=None)]
```

Figure 7-19. The decomposed subqueries

The next step in the RAG system is to perform a semantic search for each subquery and generate separate answers. Once all subqueries are answered, the subqueries and their answers are fed into a final reasoning prompt. That prompt instructs the model to synthesize the insights into a single consolidated answer.

Discussion

Query decomposition breaks complex questions into simpler subquestions, retrieves context for each independently, and synthesizes sub-answers into a final response.

Use decomposition when questions naturally require aggregating information from multiple sources (“Compare X and Y” needs X data and Y data separately), breaking down the query makes each subsearch more precise, you can run subqueries in parallel to limit latency impact, or intermediate answers help you debug why final answers fail.

Skip query decomposition when questions are simple and direct (“What is X?”), a single retrieval call returns sufficient context, the added LLM calls (decomposition + final reasoning) make response time unacceptable, or your evaluation shows that single-step retrieval performs equally well.

The main trade-off is complexity and latency versus completeness. Decomposition adds two LLM calls (one for breaking down, one for synthesis) plus multiple retrieval steps. In exchange, you can answer questions that no single vector search handles well, like “Which company was more profitable: Microsoft or Google?”

Query decomposition differs from multiquery retrieval: decomposition creates distinct subquestions requiring separate answers that get combined (“How much did Microsoft earn?” + “How much did Google earn?”), while multiquery creates

semantically similar variants of one question to improve recall. Combine both when complex questions also benefit from multiple phrasings per subquestion.

See Also

- [The LlamaIndex Query Transform Cookbook](#) provides subquestion decomposition examples and implementation patterns.
- [The Haystack query decomposition guide](#) offers a practical walk-through with code examples.
- [The Demonstrate-Search-Predict paper](#) introduced multihop question decomposition for retrieval systems.

Agentic RAG

Agents are systems in which an LLM acts as a decision-maker. The model observes the current situation, plans a sequence of actions, and selects tools to execute those actions. Like a human who adjusts travel plans when a train is canceled, an agent adapts its strategy as new information becomes available (Figure 8-1).

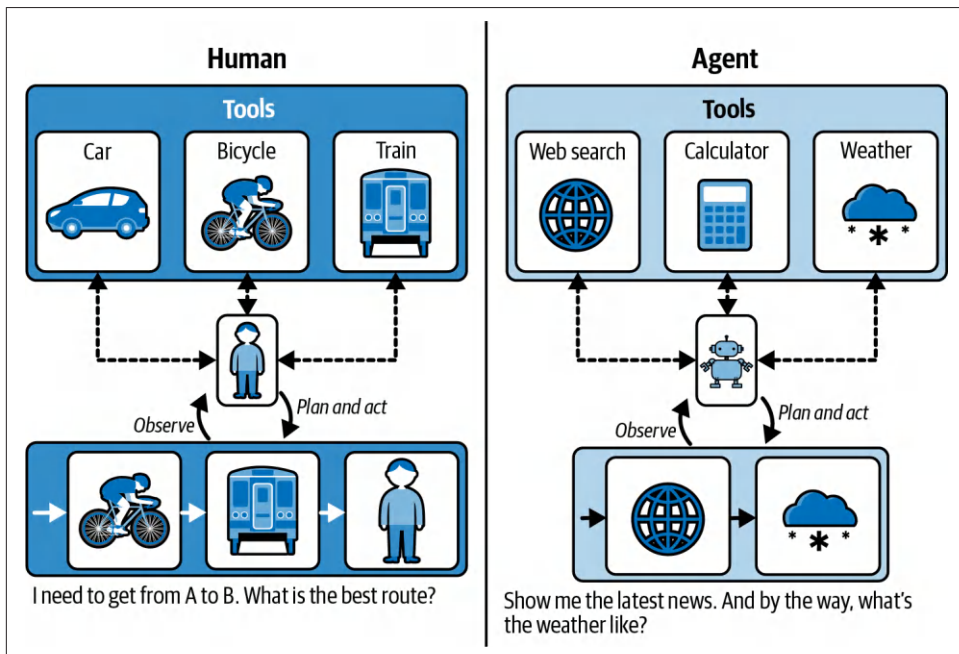


Figure 8-1. Complex problem-solving with agents mimics human behavior

This way of thinking about problem-solving did not emerge overnight. Early LLM applications were built as isolated features such as summarization or translation. Over time, these grew into multistep workflows, and finally into fully agentic systems that can decide for themselves which tools to use.

Figure 8-2 shows the progression from single LLM features to orchestrated workflows to autonomous agents that freely select and combine tools.

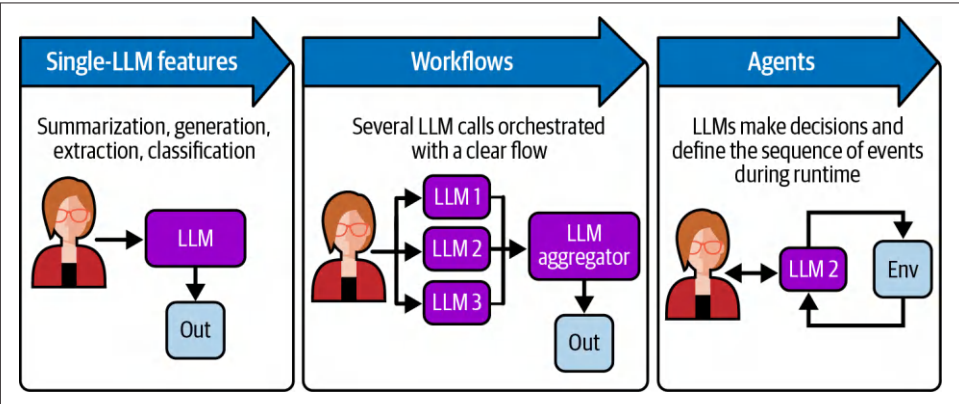


Figure 8-2. From single-LLM features to autonomous agentic systems

Once you view an agent as an autonomous problem solver, its internal structure becomes easier to reason about. At the core, an agent runs in a continuous loop. It performs an action, observes the result, and decides what to do next.

Figure 8-3 shows the main components that make this possible. The agent uses an LLM to reason and plan, relies on tools to perform actions, and stores information in short- and long-term memory so that past results can influence future decisions.

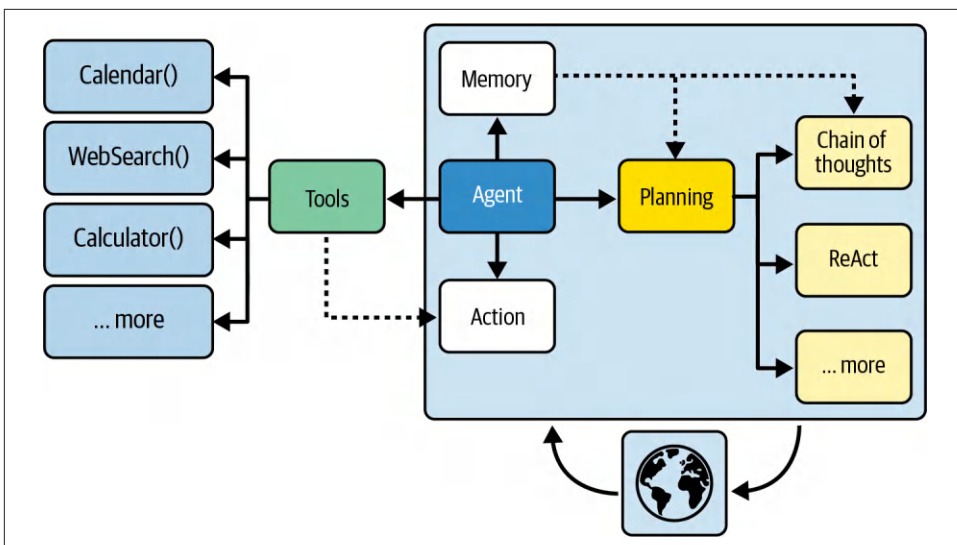


Figure 8-3. A typical agent architecture consists of an LLM for decision making, tools for actions, and memory for context

There is no single way to implement this loop. Several planning patterns exist, but the most widely used is ReAct. In this pattern, the agent alternates between reasoning about what to do next and acting by calling tools, using the results to refine its plan. The loop continues until the agent reaches a goal or hits a predefined limit, such as a maximum number of tool calls. ReAct was introduced in the 2023 paper “[ReAct: Synergizing Reasoning and Acting in Language Models](#)” by Shunyu Yao et al. ReAct combines explicit reasoning with tool use so that an agent can adapt its problem-solving strategy instead of following a fixed workflow.

To see how this plays out in practice, [Figure 8-4](#) shows an agent working through a concrete problem. A complex question is decomposed into smaller, searchable sub-questions, information is retrieved from the web, and the results are combined to produce a final answer.

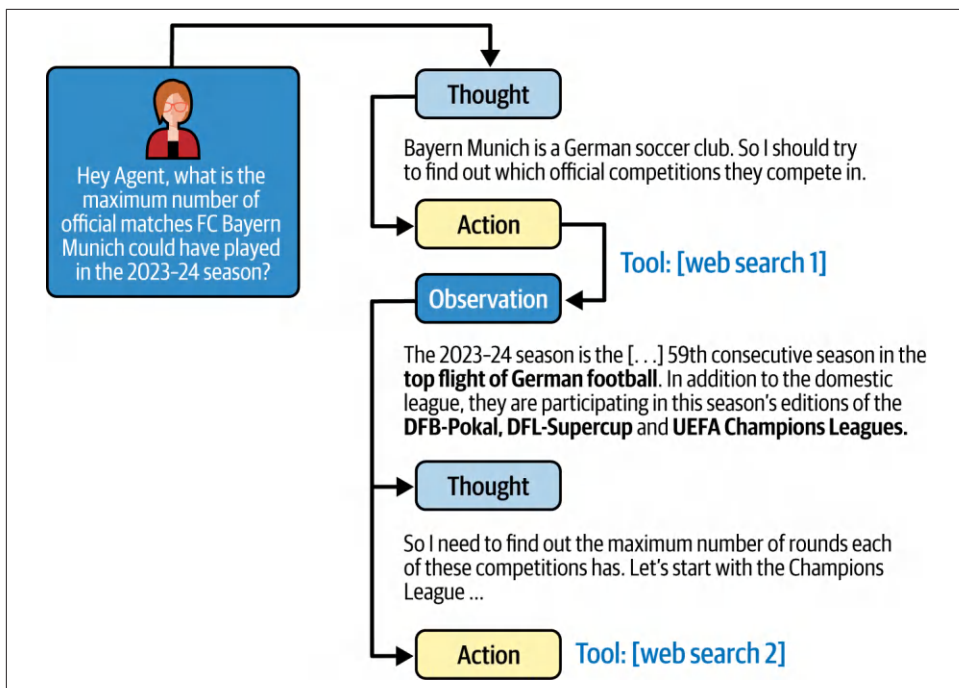


Figure 8-4. An agentic system divides complex problems into manageable steps, gathers information, and performs calculations to reach a conclusion

This chapter builds on these ideas by introducing the tools and frameworks used to implement agentic RAG systems, from low-level function calling to higher-level runtimes that manage state, memory, and tool orchestration.

You'll find recipes for building agents using either direct function calling or orchestration frameworks like the OpenAI Agents SDK or LangGraph, each representing a different point on the spectrum from lightweight orchestration to full agent runtimes. Beyond these, frameworks such as LlamaIndex, CrewAI, and AutoGen are also worth exploring when comparing approaches to building agentic systems.

Before exploring these frameworks, you need to understand the building blocks they orchestrate. Every agentic system relies on tools, which are discrete functions that perform specific actions. The first recipe shows how to design a custom tool from scratch.

You can find all the code examples for this chapter in the [book's GitHub repository](#).

8.1 Designing a Custom Tool in Python

Problem

You need to define your own custom tool for your agent toolkit.

Solution

Create a Python function with a detailed docstring that describes its purpose, inputs, and outputs. The docstring enables the LLM to understand when and how to use this tool:

```
import requests

def get_weather(latitude, longitude):
    """
    This function calls the open-meteo API to get the weather data for a
    given latitude and longitude.

    Args:
        latitude (float): The latitude of the location to get weather
            data for.
        longitude (float): The longitude of the location to get
            weather data for.

    Returns:
        dict: A dictionary containing the weather data for the given
            latitude and longitude.
    """
    response = requests.get(
        "https://api.open-meteo.com/v1/forecast"
        f"?latitude={latitude}"
        f"&longitude={longitude}"
        "&current=temperature_2m,wind_speed_10m"
        "&hourly=temperature_2m,relative_humidity_2m,wind_speed_10m"
    )
    data = response.json()
    return data['current']['temperature_2m']
```

In your agentic system, the LLM decides when to call this tool based on the user's request. For example, if the user asks, "What is the weather like in New York tomorrow?", the LLM recognizes that this tool can provide the answer, fills in the parameters, and calls the function. The function retrieves the weather data and returns it to the LLM for further processing.

Discussion

Tools are implemented by using code that performs an action. In Python agents, tools are functions, API calls, database queries, or mathematical operations.

Well-designed tools define their purpose, inputs, and outputs through docstrings. The LLM reads these docstrings to determine which tool fits the task and how to call it. This enables agents to chain tools in multistep workflows without hardcoded logic.

Use custom tools for domain-specific functionality—internal databases, proprietary APIs, or business calculations. Keep each tool focused on a single responsibility. Avoid creating tools for tasks that prebuilt solutions already handle, like web search or code execution.

Note that each tool requires testing, debugging, and updates as APIs change. Tools that call external services need error handling for timeouts, rate limits, and failures.

See Also

- The [OpenAI Agents SDK tool creation guide](#) provides a tutorial on converting Python functions into agent tools.
- The [OpenAI Function Calling guide](#) contains official documentation on designing and implementing tools for LLM function calling.
- The [Anthropic tool use documentation](#) offers comprehensive patterns for tool definition and implementation with Claude models.

8.2 Using Workflow Patterns in Multiagent Systems

Problem

You want to optimize the performance and latency of your agentic system by using the right workflow patterns for your tools and sub-agents.

Solution

This recipe describes five common workflow patterns defined by Anthropic. Use it as a reference to choose which pattern fits your use case. For concrete implementations, see the recipes later in this chapter, particularly [Recipe 8.5](#) for parallel tasks and [Recipe 8.8](#) for orchestrator-workers patterns.

The key difference among these patterns is that some rely heavily on concurrency (tasks run in parallel), while others focus on sequential steps, with each step grounded in the output of the previous step. Which approach you need depends on your specific use case: sometimes you need a sequential workflow, and other times you can run parts in parallel.

Workflow pattern 1: Prompt chaining

In *prompt chaining*, the workflow divides a complex task into sequential steps, and each LLM call processes the previous step's output.

Figure 8-5 shows a workflow that extracts text from a PDF and converts it into simple English. The workflow splits subtasks (extraction, translation, reformulation) into separate LLM calls. Because each call depends on the previous output, this workflow must be sequential.

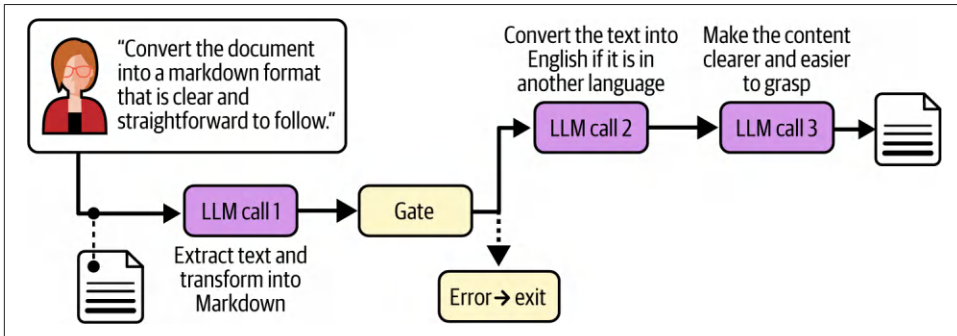


Figure 8-5. The prompt chaining workflow links prompts sequentially to build context

Workflow pattern 2: Routing

In a *routing* workflow, an initial LLM call acts as a router that analyzes the request and selects the most appropriate tool or subworkflow.

Figure 8-6 shows a router that receives an email and generates a response. If the email includes attachments, the router triggers processing steps to analyze, preprocess, and save the content. For a simple email, the router answers directly by using information from your calendar or vector store. With roughly 5 to 10 scenarios, you can cover most actions required to handle email.

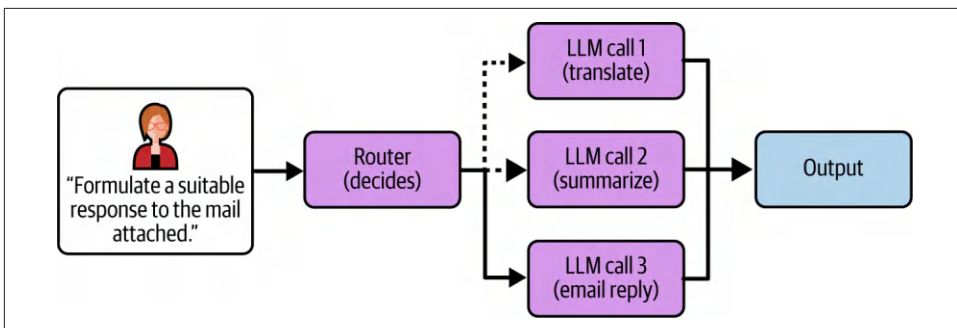


Figure 8-6. In a routing workflow, the LLM decides which path to take based on the input

Workflow pattern 3: Parallel tasks

In the *parallel tasks* workflow, you run multiple LLM calls or tool calls *in parallel*, each handling a different part of the task independently. At the end, an aggregator collects and combines the results.

Figure 8-7 shows an agent analyzing a long scanned contract. The system must extract text and analyze it page by page. To speed up processing, send one page at a time to a multimodal model to extract specific values like expiration date and cancellation terms.

Sequential processing takes 10 to 20 seconds per page even with an efficient model. Running 10 or 20 LLM calls in parallel reduces processing time from hours to minutes.

The main constraint is the token-per-minute limit of your API endpoint. Running too many tasks concurrently may hit the token limit and cause errors.

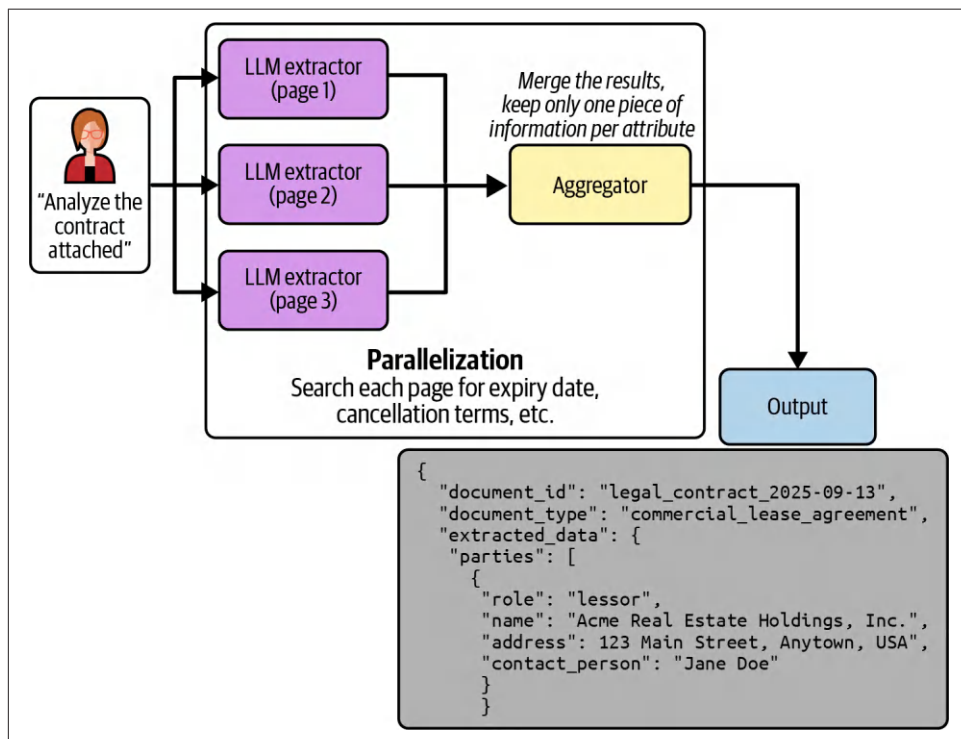


Figure 8-7. In the parallel task workflow, multiple concurrent LLM calls analyze multiple pages of the contract in parallel



Recipe 8.5 uses this exact concept to parallelize reading and analyzing documents. Each LLM call reads and analyzes one page, and an aggregator merges the results.

Workflow pattern 4: Orchestrator-workers

The *orchestrator-workers* workflow uses an agent that acts as a coordinator. The orchestrator has access to multiple worker agents, each specialized in a particular task or domain.

Figure 8-8 shows an orchestrator with access to three worker agents:

- A worker that queries a SQL database by translating the user's request into an appropriate SQL query and executing it
- A worker that searches wiki data (for example, via a vector database and semantic search)
- A worker that performs a direct LLM call by using a specialized prompt

Depending on the user's question, the orchestrator can invoke one or more workers. A synthesizer then considers all gathered content and generates a single final response for the user.

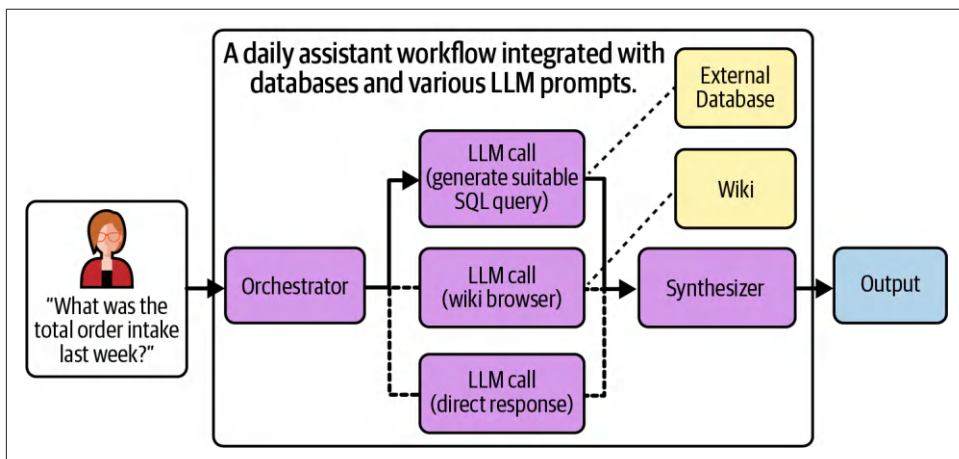


Figure 8-8. In an orchestrator workflow pattern, the orchestrator decides which specialized workers to call

Workflow pattern 5: Evaluator-optimizer

In the *evaluator-optimizer* pattern, one LLM creates an initial draft, and another LLM reviews it. The system iterates between drafting and evaluation until the evaluator is satisfied or reaches a maximum number of iterations.

Figure 8-9 shows an agent system for writing a technical blog post. One LLM produces draft paragraphs, and the evaluator provides feedback. The cycle continues until the evaluator approves the result.

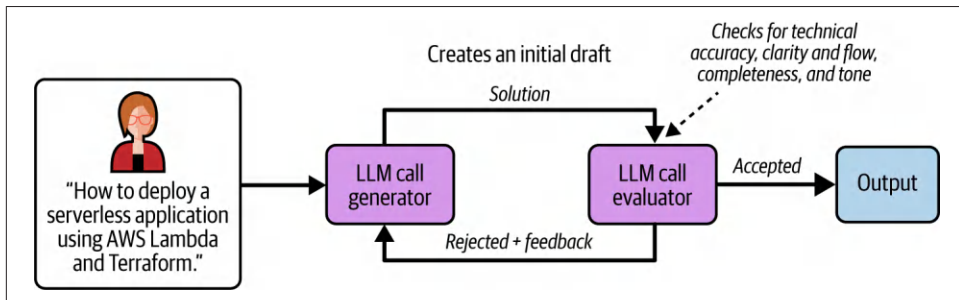


Figure 8-9. In the evaluator-optimizer workflow, output is iteratively refined through evaluation and optimization cycles

Discussion

Sequential patterns work when tasks happen in order—each step processes the previous output. Use this approach when data flows linearly, like analyzing and then summarizing a document.

Parallel patterns run independent tasks concurrently. Use this technique when sub-tasks don't depend on one another, like querying multiple databases at the same time.

Routing patterns classify input and choose execution paths accordingly. Use routing patterns when you have distinct task types, like routing customer queries to sales, support, or billing.

Orchestrator patterns coordinate specialized sub-agents. Use this strategy when tasks require diverse expertise. One orchestrator delegates to domain-specific agents.

Choose workflow patterns when reliability matters more than flexibility. Alternatively, you can choose autonomous agents when tasks require adaptive decision making. In the second case, the agent decides which tools to use at runtime rather than following a fixed workflow.

Avoid overengineering. Parallel patterns add orchestration overhead that pays off only for long-running subtasks. Routing patterns require clear categories—avoid them when classification is ambiguous. Orchestrator patterns increase latency through multiple LLM calls.

See Also

- [Anthropic’s guide to building effective agents](#) provides an overview of workflow patterns and agent design principles.
- The [Claude Cookbook on agent workflow patterns](#) includes code examples for basic multi-LLM workflows.

8.3 Choosing an Agentic Framework

Problem

You’re building your first agentic AI applications and looking for the optimal setup for your team and use cases.

Solution

Figure 8-10 shows a simple way to group frameworks and platforms to build agents by abstraction level.

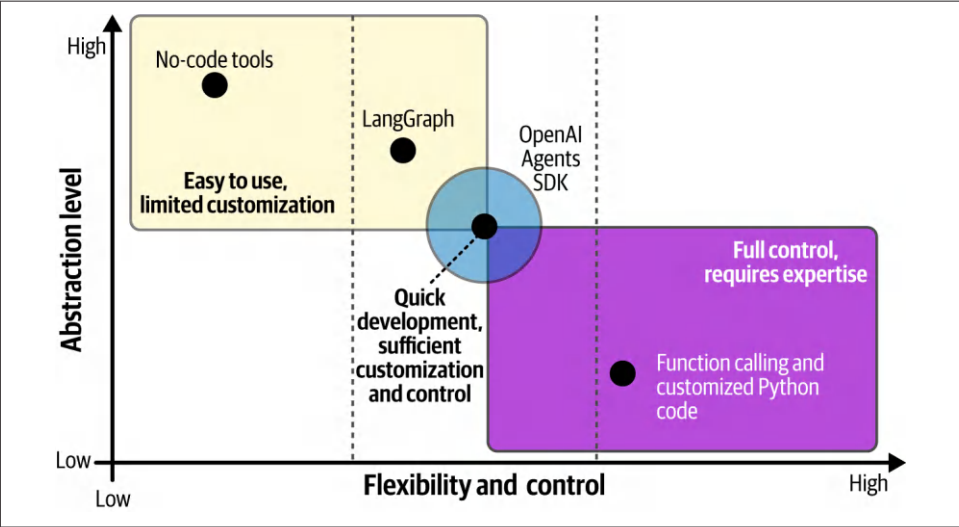


Figure 8-10. Framework abstraction levels: control versus convenience

Table 8-1 summarizes the three abstraction levels.

Table 8-1. Abstraction levels for agentic tooling

Level	Example tools/ frameworks	What defines the level	Notes (when to use)
1. No-code (extremely high abstraction)	Microsoft Copilot Studio, n8n, Zapier, Make	Drag-and-drop workflow builders with prebuilt connectors and minimal coding.	Good for internal workflows with two to five steps and for teams without engineering resources. Helps you start quickly. If your company already uses Microsoft, Copilot Studio can fit well into that ecosystem. Pricing is typically subscription based rather than per token.
2. Agentic frameworks (code-first)	OpenAI Agents SDK, LangGraph	Libraries that provide agent primitives (tools, memory/ persistence, and tracing) and patterns for building agentic applications.	Use when you want faster development than from- scratch builds while still keeping code-level control. Lightweight options (for example, the OpenAI Agents SDK) minimize lock-in, making it easier to switch frameworks later or move to a from-scratch approach.
3. From scratch (maximum control)	Direct model APIs, your own orchestration	You directly call LLM provider endpoints and implement your own orchestration, tools, state, memory, and observability.	Use when you need maximum flexibility or robustness and have the engineering expertise to own the full stack (including logging, tracing, and guardrails).

This chapter contains recipes that describe how to build agents with plain Python code from scratch, as well as how to use agentic frameworks like LangGraph and the OpenAI Agents SDK. No-code tools are not covered in this book. Therefore, [Table 8-2](#) ignores no-code tools and summarizes scenarios and recommendations for choosing between lightweight frameworks, high-abstraction frameworks, or building from scratch in Python. Use it as a reference.

Table 8-2. Choosing an agentic framework: scenarios and recommendations

Scenario	From scratch	Lightweight framework	High- abstraction framework	Notes
You need to set up an agentic RAG app in weeks and bring it to production.			✓	LangGraph has many built-in features that help you quickly build and ship a solution. Building from scratch can take longer but may be more robust.
You have many developers maintaining your app with low-to-mid experience.		✓	✓	Build from scratch only if the team is highly experienced and it's clear what the agent should look like. Otherwise, a framework provides a standardized approach and makes it easier for junior developers to follow established patterns than to write high-quality infrastructure from scratch.
You're building a solution that serves thousands to millions of users, and you need robustness and scalability.	✓	✓		From-scratch implementations or lightweight frameworks reduce dependencies. Fewer dependencies typically mean less maintenance effort and fewer components that can break.

Scenario	From scratch	Lightweight framework	High-abstraction framework	Notes
You need a highly flexible and customizable agent tailored to one particular need.	✓	✓		Building from scratch gives you full control over how tools are defined and how and when they interact. You can also freely design the application flow and state management for your specific use case.
You need real-time debugging, observability, transparency, and audit/compliance logs.	✓	✓	✓	The OpenAI Agents SDK includes a tracing view on the OpenAI website that lets you follow each step of the agent. LangGraph provides similar capabilities via LangSmith. If you build from scratch, you typically need to implement logging and tracing yourself.

This recipe does not recommend specific frameworks. Instead, it provides criteria to evaluate options based on your team and use case.

Discussion

The framework's abstraction level determines control versus complexity. At the lowest level, you call LLM APIs directly and implement tools as Python functions. This maximizes transparency and debugging. Use this level when you need auditable behavior and your team has strong Python expertise.

Mid-level frameworks like the OpenAI Agents SDK provide structure without heavy abstraction. They enforce tool patterns and handle orchestration while remaining inspectable. Use these frameworks when you want established patterns without extensive dependencies.

High-level frameworks like LangGraph manage state through graph structures. They bundle observability, type safety, and coordination primitives. Use these frameworks for complex multiagent systems needing built-in debugging tools.

No-code tools like Microsoft Copilot Studio use visual interfaces. Use these when time-to-market trumps customization and your team lacks Python expertise. You trade control for speed.

Avoid heavy frameworks early on. Start with direct API calls or lightweight frameworks to understand requirements. Each abstraction layer obscures behavior and makes debugging harder. Each dependency expands your security attack surface.

Framework lock-in compounds over time. LangGraph's state management enforces a structure that's hard to migrate from. The OpenAI Agents SDK minimizes lock-in through simpler abstractions.

See Also

- [Anthropic’s guide to building effective agents](#) provides recommendations for implementing agents with direct LLM API calls.
- The [OpenAI Agents SDK documentation](#) offers a reference for the lightweight agent framework with minimal dependencies.

8.4 Building an Agentic System via Function Calling

Problem

You want to build an agentic system without using any framework.

Solution

Use the function-calling approach. Define each tool the agent can use as a Python function, and provide the LLM with a dictionary that contains the key information about those functions so it can decide what makes sense for the task at hand.

In this setup, the LLM acts as the planner and plans a sequence of tool calls based on the user’s query. The Python runtime then executes those tool calls step by step. [Figure 8-11](#) shows how it works: the LLM only decides; the code executes. This recipe implements an agent that uses Python functions to solve math problems. The agent breaks a math task like $15 + 23 * 4$ into a series of smaller steps and decides which mathematical function to apply at each point.

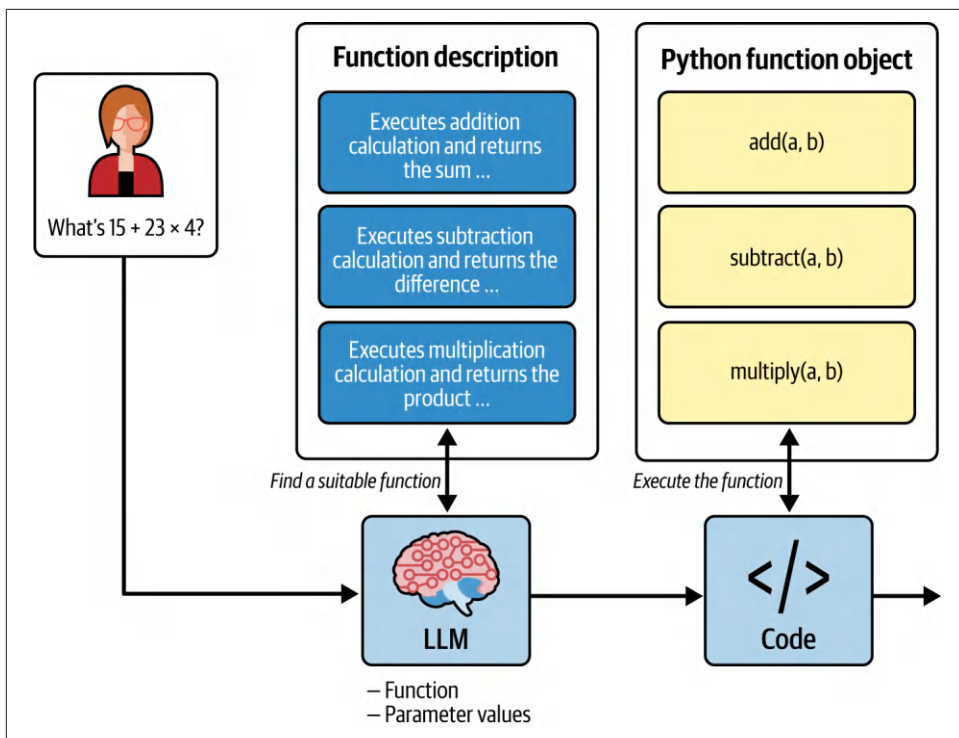


Figure 8-11. The LLM decides which tool to call, and Python executes it

This recipe uses one of OpenAI’s LLMs to decide which functions to call and in what sequence. Later, the system will also have access to a weather API so it can handle not only mathematical operations but also queries like “What is the current weather?”

In the first step, you define the mathematical functions needed. In this example, you define `add_numbers`, which allows the agent to add two numbers.

The implementation is only a single line (`a + b`), but the docstring is the important part: it describes the function’s behavior, the input arguments, return value, and data types, and includes an example. This docstring becomes the foundation the LLM uses to decide which tool to use later:

```
def add_numbers(a, b):
    """
    Tool for adding two numbers together.

    This tool takes two numbers as input and returns their sum. It can be
    used for:
    - Basic addition operations
    - Combining numeric values
    - First step in more complex calculations
    """
```

```

Args:
    a (float): The first number to add
    b (float): The second number to add

Returns:
    float: The sum of the two input numbers (a + b)

Example:
    add_numbers(5.0, 3.0) -> 8.0
    add_numbers(-1.0, 1.0) -> 0.0
    """
    return a + b

```

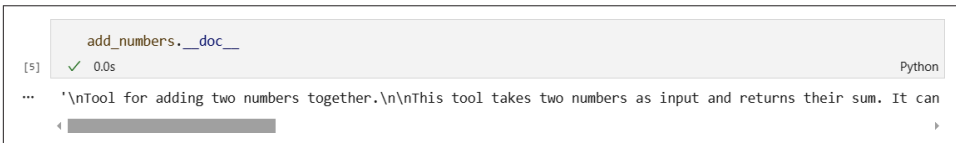
In addition, the recipe defines math functions for subtraction, multiplication, and division. They follow the same pattern and are omitted here for brevity. You can find all functions in the [Jupyter notebook](#) for this recipe.

With the mathematical functions and the weather tool (shown earlier in [Recipe 8.1](#)), the system has a set of tools to solve requests.

To make sure the LLM understands which functions it can use, you must package the function information into a dictionary. This dictionary contains the following:

- The function name
- A detailed description
- The input parameters (and their schema)

A convenient way to populate the description field is to reuse the function's docstring. [Figure 8-12](#) shows how to access the docstring from a function by reading its `__doc__` attribute.



```

add_numbers.__doc__

```

```

'\nTool for adding two numbers together.\n\nThis tool takes two numbers as input and returns their sum. It can

```

Figure 8-12. Getting the docstring of a function

Next, define the tools dictionary required for function calling. This example shows only the first entry for `add_numbers`:

```

tools = [
    {
        "type": "function",
        "function": {
            "name": "add_numbers",
            "description": add_numbers.__doc__,
            "parameters": {
                "type": "object",

```

```

        "properties": {
            "a": {
                "type": "number",
                "description": "The first number to add"
            },
            "b": {
                "type": "number",
                "description": "The second number to add"
            }
        },
        "required": ["a", "b"]
    }
},
...
]

```

You use the tools dictionary together with the LLM to plan which tools should be used to answer the user's question.

To do that, you use the following code to call OpenAI's endpoint, include the user's query in the messages, and pass the tools dictionary via the `tools` parameter. This gives the LLM the information it needs to select the best tool to call. The response contains a list of tool calls.

The final part of the code iterates over the tool calls, finds the corresponding function, loads the arguments specified by the LLM, and then calls the tool by resolving the real Python object:

```

import openai
import json

user_query = "What is the result of (5.0 + 3.0) * 2.0?"
messages = [{"role": "user", "content": user_query}]

# Call the LLM with the available tools
response = openai.chat.completions.create(
    model="gpt-5-mini", messages=messages, tools=tools
)

# --- Extraction and Execution Logic ---
message = response.choices[0].message
tool_calls = message.tool_calls
finish_reason = response.choices[0].finish_reason

for tool_call in tool_calls:
    tool_name = tool_call.function.name
    # 1. Extract and parse arguments
    arguments = json.loads(tool_call.function.arguments)

    # 2. Locate the function object
    tool = globals().get(tool_name)

```

```
# 3. Execute the function and capture the result
result = tool(**arguments) if tool else {}
```

When you run this snippet, you'll see that it executes only a single tool call, which is the first addition step inside the parentheses. The result of this first tool call doesn't answer the user question yet, but it is a correct first step. [Figure 8-13](#) shows the content of the LLM response with the tool that was selected and the values for the input parameters.



Figure 8-13. Function calling for math problems: only the first set of parentheses is resolved by the LLM

To solve the entire problem, the process must run in a loop. After executing the first function, the LLM must decide how the process should continue.

The following code encapsulates the execution of the selected function into a helper function that can be called repeatedly. That helper is then used in a while loop that continuously calls the LLM, receives tool calls if any, executes the tool, appends the tool result to the messages, and repeats until the model returns a final answer. Implement the code:

```
def process_function_invocations(invocations):
    """Processes LLM tool requests and executes the functions."""
    results = []
    for invocation in invocations:
        function_name = invocation.function.name
        function_args = json.loads(invocation.function.arguments)

        # Get and execute the function
        function = globals().get(function_name)
        # Execute the function with unpacked arguments
        function_result = function(**function_args) if function else {}

        # Format the result to be sent back to the LLM
        results.append({
            "role": "tool",
            "content": json.dumps(function_result), # Must be a string
            "tool_call_id": invocation.id,
        })
    return results

user_query = "What is the result of (6.7 + 3.3) * 2.0?"
messages = [{"role": "user", "content": user_query}]
finish_reason = "tool_calls" # Initialize to enter the loop
```

```

while finish_reason == "tool_calls":
    # Call the LLM with the conversation history and tools
    response = openai.chat.completions.create(
        model="gpt-5-mini",
        messages=messages,
        tools=tools
    )

    # Extract response details
    message = response.choices[0].message
    tool_calls = message.tool_calls
    finish_reason = response.choices[0].finish_reason

    # Add the LLM's response (tool request or final answer) to history
    messages.append(message)

    if finish_reason == "tool_calls":
        # Execute the tool and format the results for feedback
        tool_results = process_function_invocations(tool_calls)
        # Add the tool results to the history
        messages.extend(tool_results)

```

Figure 8-14 shows the message list after the loop completes. It contains three entries: the user's question, the first tool call, and the second tool call.

```

... [{"role": "user", "content": "What is the result of (5.0 + 3.0) * 2.0?}],
ChatCompletionMessage(content=None, role='assistant', function_call=None, tool_calls=[Chat
{'role': 'tool',
'tool_call_id': 'call_7yG7L4utDNhzY0irngkPPazp',
'content': "add_numbers({'a': 5.0, 'b': 3.0}) = 8.0"},
ChatCompletionMessage(content=None, role='assistant', function_call=None, tool_calls=[Chat
{'role': 'tool',
'tool_call_id': 'call_vzp6KyTYDuZzK0FvuK6lmdNA',
'content': "multiply_numbers({'a': 8.0, 'b': 2.0}) = 16.0"}]

```

Figure 8-14. An example of iterative function calls for a simple math problem

Now, let's test the same approach with a question that also requires weather information. This sample query tells the agent that the user will be in London in a few days, and asks the system to retrieve the weather and calculate the number of days until the return:

```

user_query = (
    "I'm planning a trip to London. "
    "I'll arrive in 5 days and stay for 6 days. "
    "Can you tell me what the current weather is in London, "
    "and also calculate in how many days I will return home from today?"
)

messages = [{"role": "user", "content": user_query}]

```

```

finish_reason = "tool_calls" # Initialize finish_reason

while finish_reason == "tool_calls":
    print("messages:", messages)
    response = openai.chat.completions.create(
        model="gpt-5-mini",
        messages=messages,
        tools=tools
    )

    finish_reason = response.choices[0].finish_reason

    if finish_reason == "tool_calls":
        message = response.choices[0].message
        tool_calls = message.tool_calls
        results = process_function_invocations(tool_calls)

        messages.append(message)
        messages.extend(results)

final_content = response.choices[0].message.content
print("Final content:", final_content)

```

The sample query is executed using the same flow. If you print the outcome, you'll see that the first tool call retrieves the current weather in London, and the next call calculates the number of days until the return.

Discussion

LLMs are unreliable at arithmetic, so tool calling delegates calculations to Python functions that perform them accurately. The docstring acts as the contract between model and tool, describing what the function does and how to call it.

Function calling enables step-by-step agent workflows. The model decides which tool to use, runs it, and reasons over the result. Because each call is explicit, it is easy to inspect, log, and debug.

Use function calling when a task requires external data or computation. For simple one-shot questions, a single LLM call is simpler and faster.

The main trade-off is latency: each tool call adds another model round trip. In performance-sensitive systems, reduce this with batching or asynchronous execution, as shown in [Recipe 8.5](#).

See Also

- The [OpenAI function calling guide](#) provides detailed documentation on implementing function calling.

- [Agentic AI code examples on GitHub](#) show sample implementations for building custom agents.

8.5 Accelerating Agents with asyncio

Problem

You want to speed up independent agent tasks that are limited by I/O wait time, not by sequential dependencies.

Solution

Use Python's asyncio module to run I/O-bound tasks concurrently. Add `async` to the function definition to turn it into a coroutine, then use `await asyncio.gather` to run multiple coroutines concurrently.

In this example, you define two coroutines that simulate API calls by sleeping for different durations:

```
import asyncio
import time

async def fetch_data():
    await asyncio.sleep(2) # Simulating an API call that takes 2 seconds
    return {"status": "success", "data": "Some data from API"}

async def process_data():
    # Simulating data processing that takes 1 second
    await asyncio.sleep(1)
    return {"status": "complete", "result": "Processed data"}

async def main():
    start_time = time.time()
    # Run both coroutines concurrently;
    # total time ≈ longest single task (2s)
    data, result = await asyncio.gather(fetch_data(), process_data())
    end_time = time.time()

    print(f"Total time: {end_time - start_time:.2f} seconds")

asyncio.run(main())
```

Figure 8-15 shows the timing results. With asyncio, total runtime is roughly the duration of the longest task. The overall time drops from about 3 seconds to about 2 seconds.

```
15 data, result = await asyncio.gather(fetch_data(), process_data())
16 end_time = time.time()
17
18 print(f"Total time: {end_time - start_time:.2f} seconds")
19
20 asyncio.run(main())
21
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [] [X] [] [X]

```
PS C:\rag-oreily-book> & C:\rag-oreily-book/.venv/Scripts/Activate.ps1
(.venv) PS C:\rag-oreily-book> & C:\rag-oreily-book/.venv/Scripts/python.exe c:\rag-oreily-book/
asyncio_example.py
Total time: 2.01 seconds
(.venv) PS C:\rag-oreily-book>
```

Figure 8-15. Using `asyncio` to run tasks concurrently

For a practical example, imagine an agentic chatbot that analyzes uploaded PDFs. The first step is to digitize the content and scan for specific values. A scanned PDF is essentially a long document of images, so each page can be converted to an image and processed by a multimodal model to extract text. This approach is powerful but time-consuming. The following implementation parallelizes processing across individual pages.

Figure 8-16 shows the process to extract content from all pages by running tasks concurrently:

1. The controller agent sends each page to a multimodal model to extract text.
2. Each multimodal-model call runs concurrently and extracts text from one page at a time.
3. The merger agent collects and combines the results from the individual pages into a cohesive output.

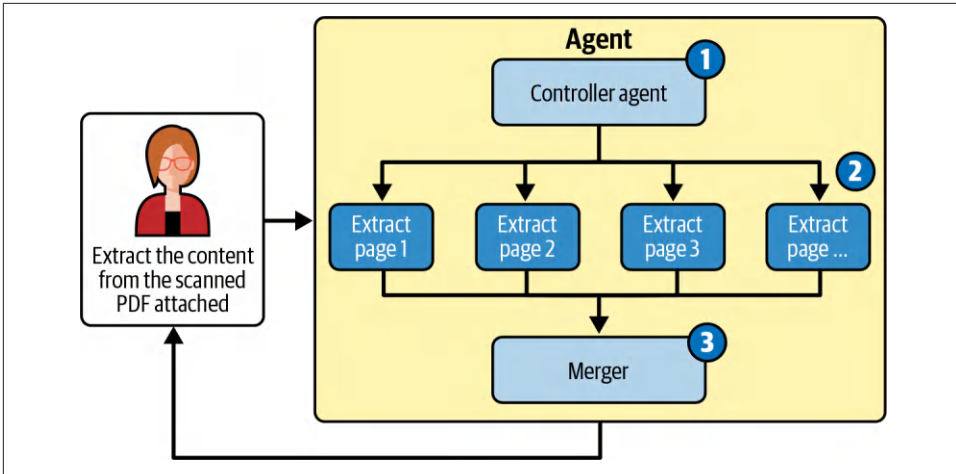


Figure 8-16. Entity extraction workflow with asyncio

You define the coroutine that does the actual heavy lifting by using a multimodal model to extract entities from a single image:

```

async def extract_entities_from_image(image, page_num):
    start_time = time.time()

    buffer = io.BytesIO()
    image.save(buffer, format="PNG")
    image_base64 = base64.b64encode(buffer.getvalue()).decode("utf-8")

    client = openai.AsyncOpenAI()
    response = await client.chat.completions.create(
        model="gpt-5.2",
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": "Extract entities as JSON."},
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"data:image/png;base64,{image_base64}"
                        }
                    },
                ],
            },
        ],
        max_tokens=1000,
    )

    # Write response to individual JSON file
    filename = f"../datasets/sample_data_asyncio/page_{page_num}_entities.json"
  
```

```

with open(filename, "w") as f:
    f.write(response.choices[0].message.content)

end_time = time.time()

return response.choices[0].message.content

```

The next step loads the PDF, converts each page to an image, and extracts text from the first three pages concurrently. You can choose the number of pages to process in parallel. The limiting factor is usually the API rate limit.

Model providers typically document how to estimate token usage for images at a given resolution. With that information and processing time per image, you can estimate the number of images to process concurrently without exceeding rate limits:

```

async def main():
    # Convert PDF to images
    images = convert_from_path(
        "../datasets/sample_data_asyncio/Laptop_Order_Invoice.pdf", dpi=200
    )
    # Process only first 3 pages for demo purposes
    end_page_to_process = 3
    results = await asyncio.gather(
        *[
            extract_entities_from_image(img, i + 1)
            for i, img in enumerate(images[:end_page_to_process])
        ]
    )

    # Merge the results to one JSON,
    # they are all from the same list of entities
    client = openai.AsyncOpenAI()
    response = await client.chat.completions.create(
        model="gpt-5.2",
        messages=[
            {
                "role": "user",
                "content": [
                    {
                        "type": "text",
                        "text": "Merge these JSON lists into one."
                    },
                    {
                        "type": "text",
                        "text": "\n".join(results),
                    },
                ],
            }
        ],
    )

```

```
merged_json = response.choices[0].message.content
return merged_json
```

Figure 8-17 shows the resulting dictionary. The code stores extracted values as a JSON file for each image and merges the results into a single dictionary.

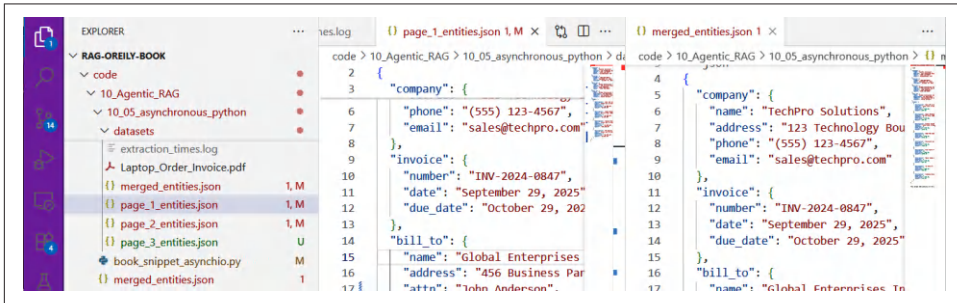


Figure 8-17. Extracted entities saved as JSON

To verify that asyncio improves performance, log execution times. In this example, page 1 took 19 seconds, page 2 took 34 seconds, and page 3 took 38 seconds. The entire script took 49 seconds total. Without asyncio, sequential processing would have taken at least 90 seconds.

Discussion

Agent systems make multiple API calls to LLMs, embedding models, and vision models. Each call takes seconds. Python's asyncio suspends tasks during I/O waits and switches to other ready tasks. Sequential execution takes the sum of all durations. With asyncio, total time is roughly the longest task duration.

Use asyncio when your agent queries multiple independent data sources, calls several tools in parallel, or generates embeddings for multiple chunks simultaneously. Fetching three data sources concurrently takes as long as the slowest call, not the sum of all three.

Avoid asyncio for sequential workflows where each step depends on the previous result. Also avoid it for simple scripts with few API calls where overhead adds complexity without meaningful gains.

The trade-off is code complexity. Asynchronous programming with asyncio requires understanding coroutines, event loops, and await semantics. Debugging is harder because errors can occur in multiple tasks running at the same time, making it difficult to trace which task caused the problem. Error handling must account for partial failures.

Use `asyncio` for user-facing agents where response time matters. Waiting for multiple API calls to finish sequentially frustrates users; `asyncio` reduces wait time by running independent tasks concurrently.

See Also

- The [Python `asyncio` documentation](#) is the official reference on asynchronous I/O, event loops, and coroutines.
- [Real Python's `asyncio` tutorial](#) provides a hands-on guide to Python's `asyncio` for concurrent programming.

8.6 Building a Sales Negotiation Agent with OpenAI's Agents SDK and Chroma

Problem

You want to build a sales assistant agent by using a lightweight agent framework, so you don't need to build everything from scratch but still have full flexibility.

Solution

The OpenAI Agents SDK is a lightweight framework for building agents. It covers the most important concepts, such as how to define guardrails, how agents can hand off tasks to each other, and how to trace an agent's behavior step by step.

This recipe builds a sales assistant agent that negotiates on your behalf. Later in the recipe, a customer agent and a moderator agent are added. The moderator alternately triggers the customer and salesman agents. The salesman agent has access to a vector store containing information about previous chats with the customer, pricing, and manufacturing costs.

Figure 8-18 shows the full setup with all three agents and how they interact with one another.

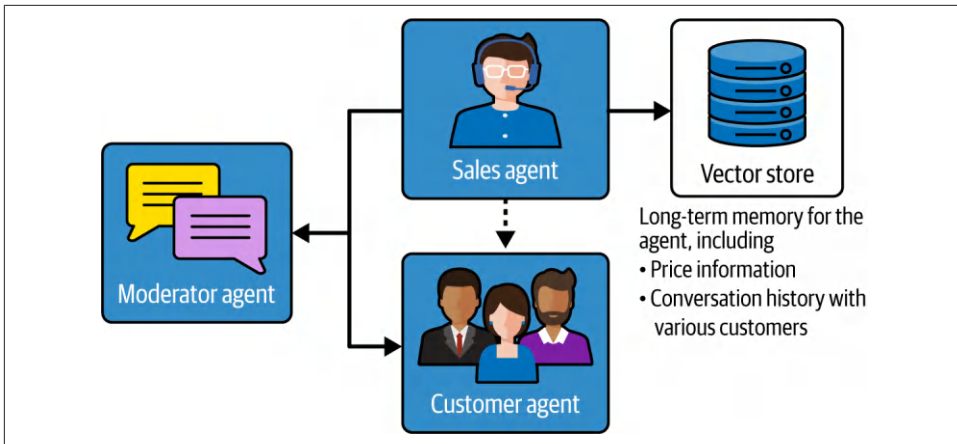


Figure 8-18. Agentic workflow chart for the agent to build

To run the code in this recipe, you need the OpenAI Agents SDK library. The agent uses a Chroma vector database to store pricing and historical sales information as long-term memory. Install the library and database:

```
pip install openai-agents chromadb
```

To run the pdf2image library, you need additional system utilities. On Linux, most distributions include the Poppler utilities. If not, install them with your package manager:

```
sudo apt-get install poppler-utils
```

On Windows, you can download the Poppler binaries from the official Poppler repository and add them to your system PATH so the pdf2image library can find them.

The first step is to set up the vector database that will serve as the sales agent's long-term memory. This recipe uses a Chroma vector database and populates it with sample emails from an email chain between a salesperson and a potential buyer. The email history also includes business information such as retail prices, common manufacturing costs, and product details. This gives the sales agent additional context and more room for negotiation.

To start, you create a Chroma vector database and a collection called `email_history`. Populate the collection by loading an email history text file, splitting it into chunks, embedding those chunks, and uploading them to the vector database:

```
import os
import shutil
import chromadb
from chromadb.config import Settings
```

```

def fill_chroma_db():
    chroma_dir = "./chroma_email_history_db"

    # If the directory exists, remove it to overwrite
    if os.path.exists(chroma_dir):
        shutil.rmtree(chroma_dir)

    client = chromadb.Client(
        Settings(persist_directory=chroma_dir)
    )
    collection = client.get_or_create_collection("email_history")

    with open(
        "../datasets/sample_data_negotiation_agents/sample_email_history.txt",
        "r",
        encoding="utf-8"
    ) as f:
        email_text = f.read()

    email_docs = [
        e.strip()
        for e in email_text.split("\n---\n")
        if e.strip()
    ]

    for idx, doc in enumerate(email_docs):
        collection.add(
            documents=[doc],
            ids=[f"email_{idx+1}"]
        )
        print(f"Added email_{idx+1}")

fill_chroma_db()

```

Next, make the vector database queryable by the agent by creating a tool that can search the `email_history` collection. In this example, you define a tool called `query_database`, which uses Chroma to query the collection. It connects to `email_history` and submits a query based on the input text.

To turn a standard Python function into a tool when using the OpenAI Agents SDK, add the `@tool` decorator above the function definition.

A *decorator* in Python is a special type of function that modifies the behavior of another function. When you apply a decorator to a function, it wraps the original function and enhances it with additional functionality. Here's the code:

```

import chromadb
from chromadb.config import Settings

@function_tool
def query_database(query_text, n_results=3):

```



```

"""
Query emails using semantic search from a local Chroma collection.
"""

# Set up Chroma client and collection
chroma_dir = "./chroma_email_history_db"
client = chromadb.Client(Settings(persist_directory=chroma_dir))
collection = client.get_or_create_collection("email_history")

# Perform the query
results = collection.query(
    query_texts=[query_text],
    n_results=n_results,
    include=["documents"],
)
return {
    "query": query_text,
    "documents": (
        results["documents"][0] if results["documents"] else []
    ),
}

```

Next, define the sales agent. This agent has access to the tool you just created for negotiating with customers.

The code imports the required modules and uses the Agent class to create a new agent. When creating an agent, provide a name, instructions, the tools it can access, and the LLM it uses:

```

from agents import Agent, Runner
from agents import trace

# Create a negotiation agent with email search capability
salesman_agent = Agent(
    name="Negotiation Agent",
    instructions="""You are a skilled negotiation agent
representing a salesperson.

Your role:
- Analyze the current email conversation history
- Search for relevant information using the query_database tool
- Generate informed responses with strategic counter-offers

Process:
1. Use query_database to find relevant context from email history
2. Analyze customer concerns and pricing constraints
3. Craft a professional response with a competitive counter-offer
4. Maintain a collaborative tone while protecting profit margins
""",
    tools=[query_database],
    model="gpt-5.2"
)

```

```

# Helper function to run the negotiation agent
async def negotiate_with_customer(email_history):
    with trace("Negotiation Response"):
        result = await Runner.run(
            salesman_agent,
            f"Email conversation history:\n{email_history}"
        )
    return result.final_output

```

Now test the agent by providing the existing email history. The email history contains an email chain between a customer and a salesperson:

1. The customer wants to buy 15 laptops for no more than €950 per laptop, though the retail price is €1,299.
2. The salesperson offers a 12% discount.
3. The customer responds that the discount is insufficient and claims they can get a similar laptop from another supplier for €1,000.

The discussion currently ends there.

Feed this email history to the agent so it can continue the negotiation:

```

# Example: Using the negotiation agent
import asyncio
from agents import trace

async def demonstrate_negotiation():
    with open(
        "../datasets/sample_data_negotiation_agents/sample_email_history.txt",
        "r",
        encoding="utf-8"
    ) as f:
        email_history = f.read()

    # Generate negotiation response using the agent
    with trace("Automated Customer Response"):
        response = await negotiate_with_customer(email_history)

    print(response)
    return response

await demonstrate_negotiation()

```

So far, this works in only one direction. To simulate a full negotiation, define a customer agent that replies on behalf of the customer.

Additionally, define a moderator agent. The moderator facilitates the negotiation by alternating turns between customer and salesperson.

To allow the moderator to trigger the other agents, the code wraps each agent as a tool by using the `AsTool` function and provides a tool description. This gives the moderator access to two tools:

- A customer-agent tool, which calls the customer agent
- A salesperson-agent tool, which calls the salesperson agent

Here is the complete code that defines the moderator and calls the salesperson and customer agents separately:

```
from agents import Agent, Runner

async def run_negotiation_process():
    """
    Orchestrates a negotiation between customer and salesperson agents
    using a moderator agent to facilitate the conversation.
    """
    # Load the email history from file
    with open(
        "../datasets/sample_data_negotiation_agents/sample_email_history.txt",
        "r",
        encoding="utf-8"
    ) as f:
        email_history = f.read()

    # Define the customer agent
    customer_agent = Agent(
        name="Customer",
        instructions="""Negotiate for the best laptop deal.
                        Be polite but persistent.
                        Respond to the email chain below.""",
        model="gpt-5-mini",
    )

    # Convert agents to tools for the moderator
    customer_agent_tool = customer_agent.as_tool(
        tool_name="customer",
        tool_description=(
            "Tool that represents the customer in negotiations."
        )
    )

    salesman_agent_tool = salesman_agent.as_tool(
        tool_name="salesman",
        tool_description=(
            "Tool that represents the salesperson in negotiations."
        )
    )

    # Define the moderator agent with clear instructions
    moderator_instructions = """
```

You are a moderator facilitating negotiation between customer and salesperson.

Process:

1. Receive email history with the last message from customer
2. Use `salesman_agent_tool` to generate salesperson response
3. Append response to email history
4. Use `customer_agent_tool` to generate customer response
5. Continue alternating until agreement or breakdown

Rules:

- Aim for mutually beneficial agreement
- """

```
moderator_agent = Agent(  
    name="Moderator",  
    instructions=moderator_instructions,  
    model="gpt-5-mini",  
    tools=[customer_agent_tool, salesman_agent_tool]  
)  
  
# Execute the negotiation with tracing  
with trace("Negotiation Process"):  
    responses = await Runner.run(  
        moderator_agent,  
        f"Begin negotiation with this email history: {email_history}"  
    )  
  
return responses
```

If you run the full code, the moderator alternately calls the customer-agent tool and the salesperson-agent tool. The conversation continues until they reach an agreement or hit a maximum number of iterations.

If you're using the OpenAI Agents SDK with the OpenAI API endpoint, you can view **traces** in your OpenAI account. The interface shows, for each agent call, the input, output, and time taken.

Figure 8-19 shows the trace view of the negotiation workflow in the OpenAI platform, where the salesperson, customer, and negotiation agents interact and call tools such as database queries. You can see the sequence of steps and how long each action takes. In this example, the customer reviews the updated invoice and approves it, confirming that the card can be charged, thereby ending the negotiation successfully.

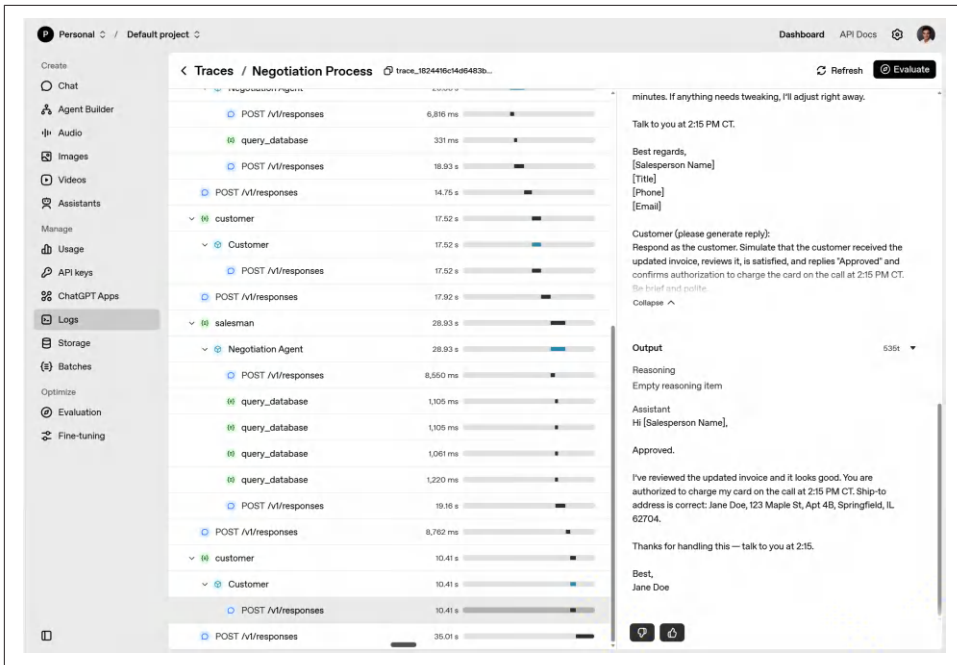
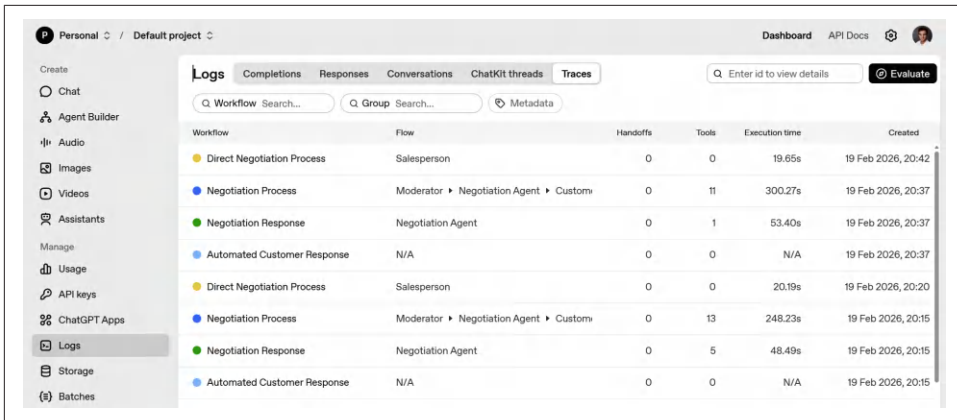


Figure 8-19. An OpenAI Agents SDK trace showing the negotiation process between customer and salesman agents

Discussion

The OpenAI Agents SDK is a lightweight framework for building agents. It covers guardrails, agent hand-offs, and step-by-step tracing. In Python, a decorator modifies function behavior. The `@function_tool` decorator converts a function into a tool that agents can invoke.

The SDK provides built-in tracing, guardrails, and controls to prevent endless loops. You can review traces at [OpenAI Platform Traces](#). Each run logs as a separate entry showing the complete execution steps. Figure 8-20 displays the agent trace dashboard where you can monitor individual agent runs and examine their full execution history.



Workflow	Flow	Handoffs	Tools	Execution time	Created
Direct Negotiation Process	Salesperson	0	0	19.65s	19 Feb 2026, 20:42
Negotiation Process	Moderator → Negotiation Agent → Custom	0	11	300.27s	19 Feb 2026, 20:37
Negotiation Response	Negotiation Agent	0	1	53.40s	19 Feb 2026, 20:37
Automated Customer Response	N/A	0	0	N/A	19 Feb 2026, 20:37
Direct Negotiation Process	Salesperson	0	0	20.19s	19 Feb 2026, 20:20
Negotiation Process	Moderator → Negotiation Agent → Custom	0	13	248.23s	19 Feb 2026, 20:15
Negotiation Response	Negotiation Agent	0	5	48.49s	19 Feb 2026, 20:15
Automated Customer Response	N/A	0	0	N/A	19 Feb 2026, 20:15

Figure 8-20. With an OpenAI agent trace, you can monitor and visualize agent actions, tool calls, and model usage

The SDK wraps agents as tools that other agents can invoke, enabling hierarchical delegation. The moderator pattern coordinates specialized sub-agents with distinct objectives but shared conversation context. Each agent exposes a simple interface (input text, output text), making them interchangeable and testable in isolation.

Use this pattern for multiparty interactions like negotiations, support routing, or collaborative editing when different roles need distinct objectives. The moderator passes conversation history to each sub-agent.

Avoid this pattern when latency is critical. Each agent hand-off requires a full LLM call, and multiturn conversations accumulate costs linearly. For simple task delegation without dialogue, direct function calls or routing patterns are more efficient.

The trade-off between lightweight and heavyweight frameworks becomes apparent at scale. The OpenAI Agents SDK provides minimal abstractions but requires you to implement coordination logic, error handling, and state management. LangGraph manages shared state automatically but requires understanding graph-based execution models. Ready-to-use OpenAI tools reduce development time but introduce vendor coupling.



You might have heard of the agent framework **Swarm**. Swarm was also from OpenAI and served as a lightweight multiagent orchestration framework. However, it has been completely replaced by the OpenAI Agents SDK. If you used Swarm for your agent setup, I recommend migrating to the OpenAI Agents SDK.

See Also

- The [OpenAI Agents SDK documentation](#) provides a comprehensive guide including agent hand-offs and advanced patterns.

8.7 Enriching Your Agent’s Capabilities with MCP Tools

Problem

You want to enhance and extend your agents’ toolkits by using tools developed by others and shared in the form of an MCP server.

Solution

This recipe builds a cooking-assistant agent that searches the web for recipes, analyzes them, and saves them. To browse the web, the code uses the Playwright MCP server to control a web browser.



Some MCP servers don’t run smoothly on Windows, particularly in Jupyter Notebook. If you encounter too many errors, you may need to switch to Linux. With Microsoft Windows Subsystem for Linux (WSL), you can install a Linux subsystem directly on a Windows machine. To set up WSL on Windows 10 or 11, open PowerShell and run `wsl --install`, which installs Ubuntu by default. For details, see the [installation guide](#) on the Microsoft website.

MCP servers exist in many popular languages, including Go, Java, Kotlin, C#, PHP, Node.js, and Python. This recipe uses the Node.js-based Playwright tooling, so you’ll need Node.js and the Node Package Manager (NPM). If you use a Python-based MCP server instead, you’ll need Python and pip.

You will also need a package-execution tool that can run scripts and packages without permanently installing them. For Python-based servers, this is commonly `uvx`; for JavaScript-based tools, it is `npm`. The MCP ecosystem offers a wide range of servers across numerous categories, including filesystem operations, web browsing, data access, and various integrations, as shown in [Figure 8-21](#).

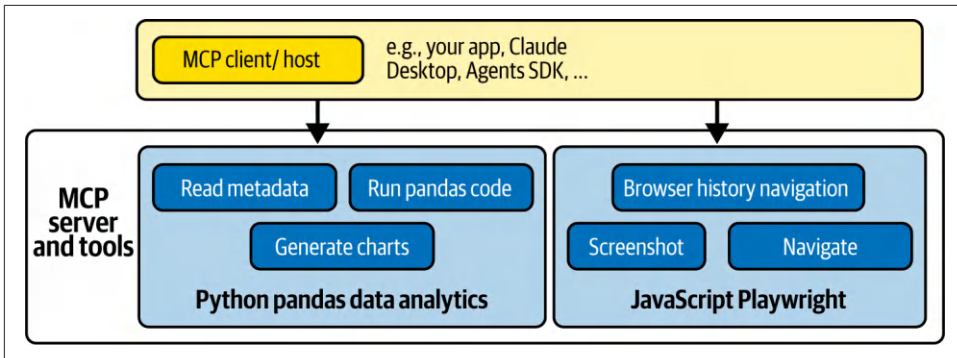


Figure 8-21. Example MCP servers and tools

In this recipe, the MCP client is an agent built with the OpenAI Agents SDK. To use it, install the OpenAI Agents package:

```
pip install openai-agents
```

The rest of the recipe shows how this agent can connect to multiple MCP servers. To see available MCP servers in the NPM registry, run the following command:

```
npm search @modelcontextprotocol
```

You will get a list of available servers. [Figure 8-22](#) shows one example: the Figma MCP server.

```

!npm search @modelcontextprotocol
[1] ✓ 1.9s Python
...
figma-mcp
ModelContextProtocol server for Figma
Version 0.1.4 published 2025-05-04 by mjd
Maintainers: mjd
https://npm.im/figma-mcp

```

Figure 8-22. List of available MCP servers

This recipe uses the Playwright MCP server, which is one of the most popular MCP tools. Install the MCP server (for additional instructions, see the [Playwright MCP GitHub repo](#)):

```

# Using npm
npm install -g @modelcontextprotocol/server-playwright

# Or using the community version
npm install -g @playwright/mcp@latest

```

Next, you configure the MCP client to use the MCP server you just installed. The client uses the MCP server wrapper class from the OpenAI Agents SDK to connect to

the Playwright MCP server. The parameters specify which server the agent is allowed to use. When you execute the snippet, it lists all tools available from that MCP server and stores them in the `tools` variable:

```
import asyncio
from agents.mcp import MCPServerStdio

async def connect_to_playwright():
    # Define connection parameters
    # "npx" runs Node.js packages,
    # "@playwright/mcp@latest" specifies the package
    params = {"command": "npx", "args": ["@playwright/mcp@latest"]}

    # Create server connection with extended timeout
    # Increased from default 5s
    async with MCPServerStdio(
        params=params, client_session_timeout_seconds=30
    ) as server:
        # List available tools from the server
        tools = await server.list_tools()
        print("Available tools:", tools)

# Run the async function
if __name__ == "__main__":
    asyncio.run(connect_to_playwright())
```

In [Figure 8-23](#), you can see the result. The `tools` object is a dictionary containing a list of tools. For example, one tool closes the browser, another resizes the browser, another uploads a file, and so on. In general, most browser actions are exposed as clearly described tools.

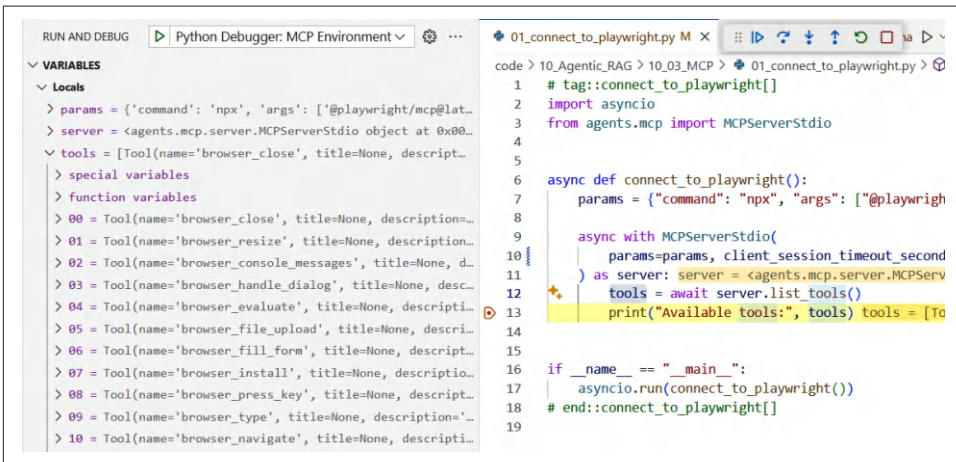


Figure 8-23. Playwright MCP server tools

The next step is to use the Playwright tools for something useful. You'll define a research agent that uses the MCP server, the MCP Playwright server, and the MCP filesystem server to save a file.

Instruct the agent that it can browse websites by using the tools from the Playwright MCP server and save information to a file. Ask the agent to find a chocolate chip cookie recipe and save the recipe to a new Markdown file:

```
import asyncio
from agents.mcp import MCPServerStdio
from agents import Agent, Runner

async def create_cookie_research_agent():
    # MCP server configurations
    files_params = {
        "command": "npx",
        "args": ["-y", "@modelcontextprotocol/server-filesystem", "."],
    }
    browser_params = {"command": "npx", "args": ["@playwright/mcp@latest"]}

    # Agent instructions
    instructions = "You can browse websites and save information to files."

    async with MCPServerStdio(
        params=files_params, client_session_timeout_seconds=60
    ) as files:
        async with MCPServerStdio(
            params=browser_params, client_session_timeout_seconds=60
        ) as browser:

            # Create agent
            agent = Agent(
                name="research_agent",
                instructions=instructions,
                model="gpt-5-mini",
                mcp_servers=[files, browser],
            )

            # Run task
            result = await Runner.run(
                agent,
                "Find a chocolate chip cookie recipe and "
                "save it to recipe.md"
            )

            print(result.final_output)

if __name__ == "__main__":
    asyncio.run(create_cookie_research_agent())
```

Once you run the script and it completes successfully, you should find a new Markdown file containing a chocolate chip cookie recipe saved in the same directory as your script.

Discussion

The Model Context Protocol (MCP) standardizes how LLMs and tools interact. Like HTTP for browsers, MCP lets any MCP-compatible agent use any MCP-compatible tool without custom integration code. This solves tool integration: without MCP, each framework requires its own tool format.

MCP has three components: client, server, and protocol. The client is your agent application. Servers provide tools and data sources, running locally or remotely. You can build custom MCP servers for your own tools. **Figure 8-24** illustrates this architecture, showing how MCP clients (such as custom agents or Claude) connect to both local servers (like Playwright, FileSystem, and custom tools) and remote servers (such as GitHub, Stripe, and Slack) through the standardized MCP.

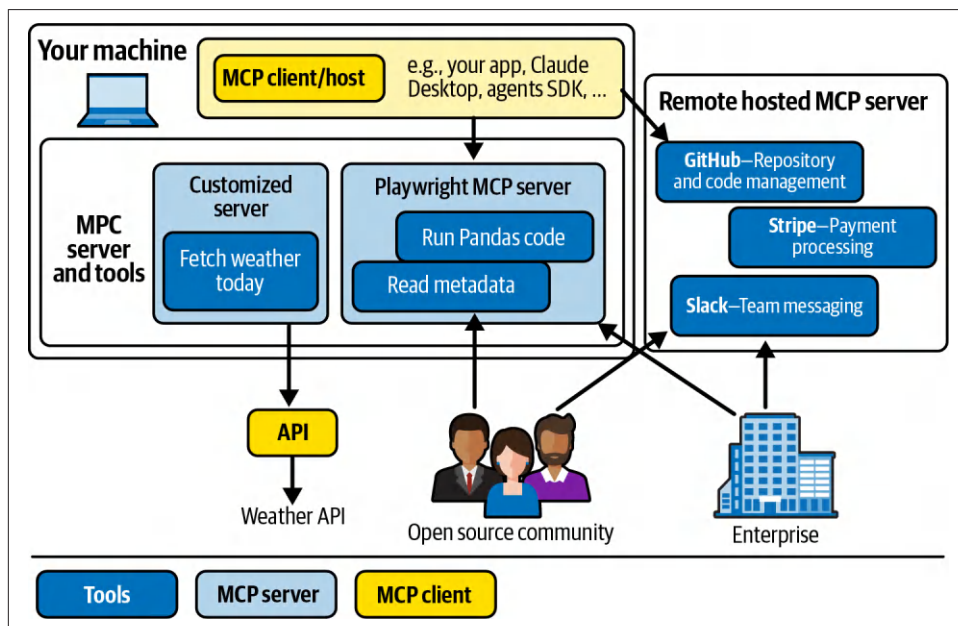


Figure 8-24. In the MCP architecture, clients connect to local or remote servers to access tools and services

Use MCP when building tools to share across projects or teams. A database query tool as an MCP server works with any agent framework supporting MCP. Update the MCP server once rather than maintaining multiple framework-specific versions.

Avoid MCP for simple single-purpose agents where tools are tightly coupled to agent logic. The protocol overhead pays off only when tool reusability or cross-framework compatibility matter.

The trade-off is additional infrastructure. MCP servers run as separate processes, requiring deployment, monitoring, and network-level error handling. Your agent must handle unreachability, timeouts, and malformed responses.

Security differs from local function calls. MCP servers accept network requests, requiring authentication, authorization, and input validation. Each MCP server expands your attack surface.

See Also

- The [Model Context Protocol documentation](#) provides an official guide to building MCP servers and clients.
- The [OpenAI Agents SDK MCP integration guide](#) offers instructions for using MCP with the OpenAI Agents SDK.
- [OpenAI Cookbook MCP tutorials](#) include practical walk-throughs on MCP implementation.
- The [MCP Servers directory](#) contains a collection of available MCP servers for integration.

8.8 Building an Agentic System with LangGraph

Problem

You want to build a complex agent that can plan, call tools, observe results, and loop until completion while you keep control over execution by defining nodes, edges, and stop conditions.

Solution

This recipe creates an assistant that can provide weather information and, based on the weather, suggest the best means of transportation to get from *A* to *B* and the kind of clothes to wear.

To define the flow of this agentic system in LangGraph, you will work with three core components: state, nodes, and edges. The state tracks the conversation, nodes perform actions, and edges control the flow.

To run the code, you need the LangGraph library. In addition, the weather API example uses GeoPy to resolve coordinates. The weather API is then called using the Requests library. Install these dependencies:

```
pip install langgraph geopy requests langchain_openai langchain-core pydantic
```

For this agent, the state stores the ongoing conversation history between you and the agent as a list of messages. After each node, the state is updated. LangGraph provides a helper called `add_messages` that updates the messages list in the state object by appending it. The function works like a reducer; it takes the current list and new messages and returns the combined list.

Start by defining the initial state for the agent, which contains an empty list of messages:

```
from typing import Annotated, Sequence, TypedDict

from langchain_core.messages import BaseMessage
# Helper function to add messages to the state
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    """The state of the agent."""
    messages: Annotated[Sequence[BaseMessage], add_messages]
    number_of_steps: int
```

Next, you'll use the LangChain tools library to turn functions into tools. Tool inputs are defined as a Pydantic model.

You'll define two tools:

- A tool that enables the agent to fetch the current weather forecast
- A second tool that takes audio input (spoken user instructions) and transcribes it

In the docstrings of the functions that are turned into tools, briefly describe what the function does so the agent can map user questions to the correct tool. Do the same for the expected fields in the Pydantic model:

```
from langchain_core.tools import tool
from geopy.geocoders import Nominatim
from pydantic import BaseModel, Field
import requests

geolocator = Nominatim(user_agent="weather-app")

class SearchInput(BaseModel):
    location: str = Field(
        description="The city and state, e.g., San Francisco"
    )
    date: str = Field(
        description="the forecasting date for when to get "
        "the weather format (yyyy-mm-dd)"
    )
```

```

@tool("get_weather_forecast", args_schema=SearchInput, return_direct=True)
def get_weather_forecast(location, date):
    """Retrieves the weather using Open-Meteo API for a given
    location (city) and a date (yyyy-mm-dd).
    Returns a dict mapping time -> temperature for each hour.
    """
    location = geolocator.geocode(location)
    if location:
        try:
            url = "https://api.open-meteo.com/v1/forecast"
            params = {
                "latitude": location.latitude,
                "longitude": location.longitude,
                "hourly": "temperature_2m",
                "start_date": date,
                "end_date": date,
            }
            response = requests.get(url, params=params)
            data = response.json()
            hourly = data.get("hourly", {})
            times = hourly.get("time", [])
            temps = hourly.get("temperature_2m", [])
            result = {}
            for t, temp in zip(times, temps):
                result[t] = temp
            return result
        except Exception as e:
            return {"error": str(e)}
    else:
        return {"error": "Location not found"}

class AudioInput(BaseModel):
    audio_path: str = Field(
        description="Path to the audio file to transcribe"
    )

@tool("transcribe_audio", args_schema=AudioInput, return_direct=True)
def transcribe_audio(audio_path):
    """Transcribe audio file into text using OpenAI Whisper."""
    try:
        with open(audio_path, "rb") as audio_file:
            transcript = openai.Audio.transcribe(
                model="whisper-1",
                file=audio_file
            )
        return transcript.text
    except Exception as e:
        return {"error": str(e)}

tools = [get_weather_forecast, transcribe_audio]

```

The LLM will act as the orchestrator, which knows which tools the agent can access and decides which tool to call with which parameters.

In this example, you're binding the weather-forecast tool to the LLM. If you invoke the model with a query such as asking for the weather in Berlin on a particular date, the LLM can infer that the weather tool expects a location and a date as inputs.

For this Berlin example, you don't need an extra function to translate "Berlin" into a geolocation, because the model knows the coordinates of Berlin. The LLM will call the tool with the correct parameters and return the result. Use this code for initializing and binding:

```
from datetime import datetime
from langchain_openai import ChatOpenAI

# Create LLM class
llm = ChatOpenAI(
    model="gpt-5.2",
    temperature=1.0,
    max_retries=2,
)

# Bind tools to the model
model = llm.bind_tools([get_weather_forecast])

# Test the model with tools
query = f"What is the weather in Berlin on {datetime.today()}?"
res = model.invoke(query)

print(res)
```

In the next step, you define nodes and edges. Each node is defined as a Python function. In this example, one node calls the LLM, and another node calls the selected tool. Both nodes take the agent state as input.

You also define an edge function that determines whether the flow should continue. In this case, it checks the last message in the message list to see whether it contains `tool_calls`. If the last message is not a tool call, the run ends:

```
from langchain_core.messages import ToolMessage
from langchain_core.runnables import RunnableConfig

tools_by_name = {}
for _t in tools:
    tools_by_name[_t.name] = _t

# Define our tool node
def call_tool(state: AgentState):
    outputs = []
    # Iterate over the tool calls in the last message
    for tool_call in state["messages"][-1].tool_calls:
```

```

        # Get the tool by name and arguments in smaller steps
        # to keep lines short
        tool_name = tool_call["name"]
        tool_args = tool_call["args"]
        tool_obj = tools_by_name[tool_name]
        tool_result = tool_obj.invoke(tool_args)
        outputs.append(
            ToolMessage(
                content=tool_result,
                name=tool_name,
                tool_call_id=tool_call["id"],
            )
        )
    return {"messages": outputs}

def call_model(
    state: AgentState,
    config: RunnableConfig,
):
    # Invoke the model with the system prompt and the messages
    response = model.invoke(state["messages"], config)
    # Return a list, because this will get added to the
    # existing messages state using the add_messages reducer
    return {"messages": [response]}

# Define the conditional edge that determines whether to continue or not
def should_continue(state: AgentState):
    messages = state["messages"]
    # If the last message is not a tool call, then finish
    if not messages[-1].tool_calls:
        return "end"
    # Default to continue
    return "continue"

```

At this point, you have all the components needed to build the agent as a graph.

The following code creates a new graph, adds the nodes, and sets the entry point. It then adds the edge that is evaluated after the LLM step.

After each LLM call, the `should_continue` edge checks whether the LLM decided to call a tool next or whether it produced a final response. You also connect the tools back to the LLM by using a normal edge to continue the loop:


```

from langgraph.graph import StateGraph, END

# Define a new graph with our state
workflow = StateGraph(AgentState)

# 1. Add our nodes
workflow.add_node("llm", call_model)
workflow.add_node("tools", call_tool)
# 2. Set the entrypoint as `agent`, this is the first node called
workflow.set_entry_point("llm")
# 3. Add a conditional edge after the `llm` node is called.
workflow.add_conditional_edges(
    # Edge is used after the `llm` node is called.
    "llm",
    # The function that will determine which node is called next.
    should_continue,
    # Mapping for where to go next, keys are strings from the
    # function return, and the values are other nodes.
    # END is a special node marking that the graph is finish.
    {
        # If `tools`, then call the tool node.
        "continue": "tools",
        # Otherwise finish.
        "end": END,
    },
)
# 4. Add a normal edge after `tools` is called, `llm` node is called next.
workflow.add_edge("tools", "llm")

# Now you can compile and visualize the graph
graph = workflow.compile()

```

Because the agent is defined as a graph, you can also print it to get an overview of the execution flow and decision points:

```

from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))

```

As shown in [Figure 8-25](#), the graph contains two nodes: `llm` and `tools`. The `llm` node calls the LLM, and the `tools` node calls the selected tool. The edges determine whether execution ends or continues. If it continues, the tool result is fed back into the LLM.

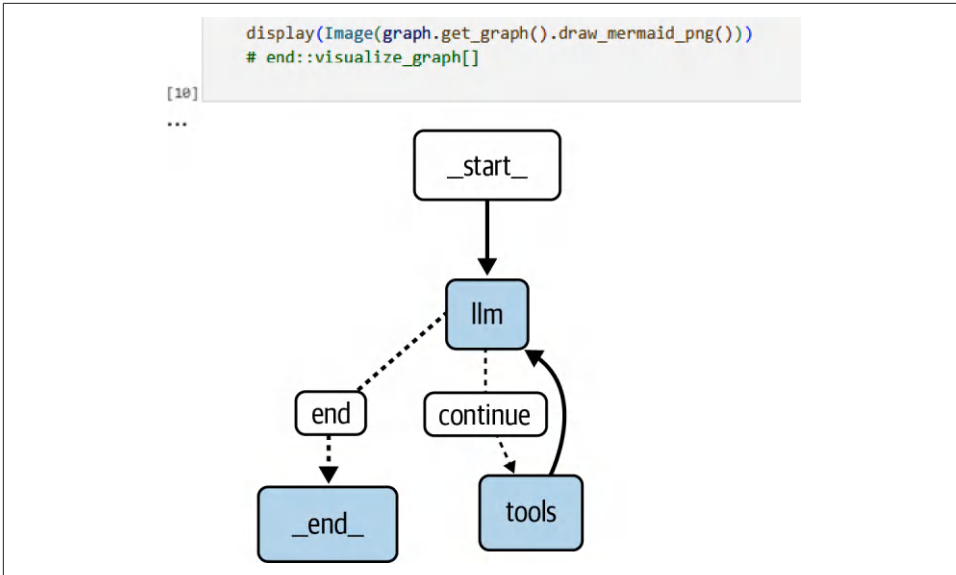


Figure 8-25. Visualizing the LangGraph agent

To run the agent, define an inputs dictionary and call the agent with streaming enabled to print intermediate steps:

```
from datetime import datetime

# Create our initial message dictionary
inputs = {
    "messages": [
        ("user", f"What is the weather in Berlin on {datetime.today()}?")
    ]
}

# Call our graph with streaming to see the steps
for state in graph.stream(inputs, stream_mode="values"):
    last_message = state["messages"][-1]
    last_message.pretty_print()
```

Figure 8-26 shows the printed steps of the agent run. First, the user asks for the weather in Berlin. The LLM decides to call the weather-forecast tool with the location Berlin and a date in October 2025. The tool executes, and the result is returned to the LLM, which interprets it and generates a final response. Because the final response does not contain another tool call, the run ends.

```
... ===== Human Message =====
What is the weather in Berlin on 2025-10-27 20:14:41.943957?
===== Ai Message =====
Tool Calls:
  get_weather_forecast (call_ioOweHeh6egt1sDcafLplsw4)
  Call ID: call_ioOweHeh6egt1sDcafLplsw4
  Args:
    location: Berlin
    date: 2025-10-27
===== Tool Message =====
Name: get_weather_forecast

{'2025-10-27T00:00': 8.0, '2025-10-27T01:00': 7.7, '2025-10-27T02:00': 7.3, '202
===== Ai Message =====

The weather in Berlin on 2025-10-27 at 20:00 will be approximately 8.1°C. The we
```

Figure 8-26. LangGraph agent run output

If you want the agent run to continue, append a follow-up user question to the state variable, which automatically triggers a rerun of the graph:

```
state["messages"].append(("user", "Would it be warmer in Munich?"))

for state in graph.stream(state, stream_mode="values"):
    last_message = state["messages"][-1]
    last_message.pretty_print()
```

Discussion

LangGraph defines agent control flow as a graph with three core components:

State

Tracks what is happening and holds shared data.

Nodes

Are functions that do the work by taking the current state, performing an action, and returning updated state.

Edges

Connect nodes and determine which node runs next.

Figure 8-27 shows the ReAct pattern. The router is an LLM that decides whether to call an external tool.

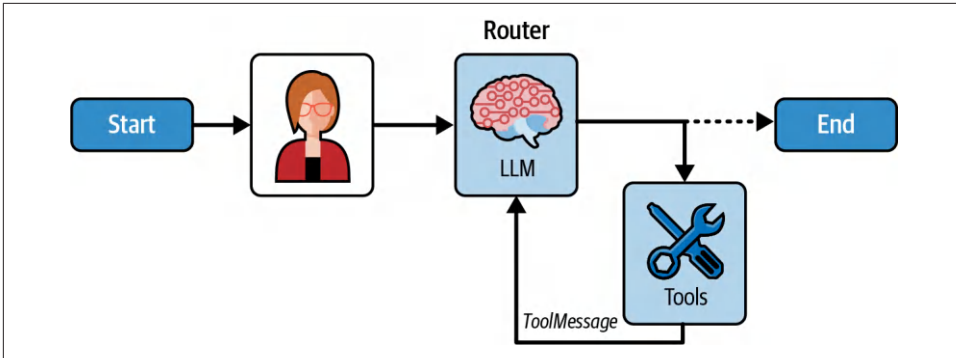


Figure 8-27. Chat with bound tools

LangGraph makes agent state and control flow explicit. You define exactly which node executes next based on the current state. Edges define valid transitions, preventing undefined states or infinite loops.

Use LangGraph when you need visibility into complex, multistep behavior. The graph visualization shows possible execution paths and which one the agent took. This helps debug unexpected decisions. State management becomes valuable when coordinating multiple agents that share context. LangGraph automatically maintains shared state.

The LangChain ecosystem includes LangGraph for workflows, LangChain for orchestration, and LangSmith for observability. **Figure 8-28** shows how these components relate.

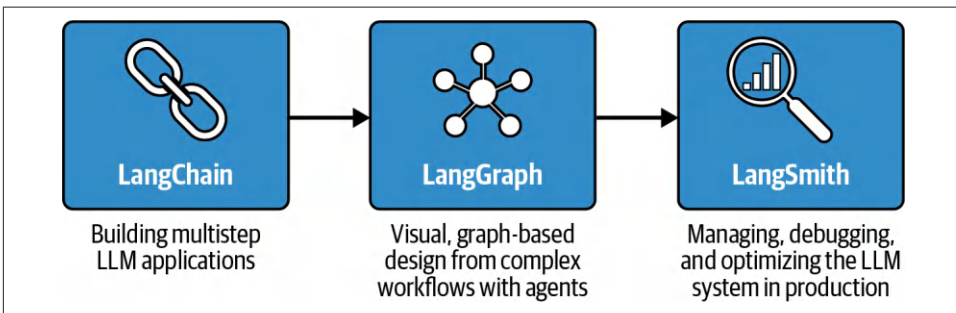


Figure 8-28. LangGraph versus LangChain versus LangSmith: key differences at a glance

Avoid LangGraph for simple linear workflows. If your agent executes a fixed sequence without branching logic or shared state, the graph abstraction adds complexity without benefit.

The trade-off is framework coupling. LangGraph's state management and graph execution become deeply embedded in your code architecture. Migrating means

reimplementing the entire state flow and transition logic. LangSmith provides bundled observability but increases ecosystem coupling.

See Also

- The [LangChain Academy LangGraph course](#) provides comprehensive modules on state, memory, and deployment.
- The [Gemini API LangGraph tutorial](#) offers a guide to using LangGraph with the Gemini API.

Graph RAG

Graph RAG extends basic retrieval with graph traversal, which lets you move through networks of entities and relationships instead of relying only on semantic similarity of isolated text embeddings.

Basic RAG systems split documents into chunks, embed them, and rely on vector search to surface relevant content. Vector search treats each chunk as an isolated unit, with no awareness of how pieces connect across a broader narrative. When information is spread across multiple parts of long documents or requires data from multiple sources, this approach misses relevant context. Dependencies, references, and relationships across sections get lost.

Graph RAG closes this gap. Instead of storing text only as embeddings, it extracts entities, forms explicit relationships between them, and combines this graph structure with the vector index. This produces context that is richer and more precise, and preserves explicit connections between entities.

Figure 9-1 uses contract document data to illustrate the difference between basic RAG and graph RAG. In a basic RAG setup, the system maintains a pool of isolated embedding vectors. In graph RAG, every text snippet is anchored to its surrounding structure. Each clause is linked to its clause type, its company, its address, and the service-level agreement (SLA) it originates from. This structural context helps the model not only find relevant text but also truly understand where it belongs.

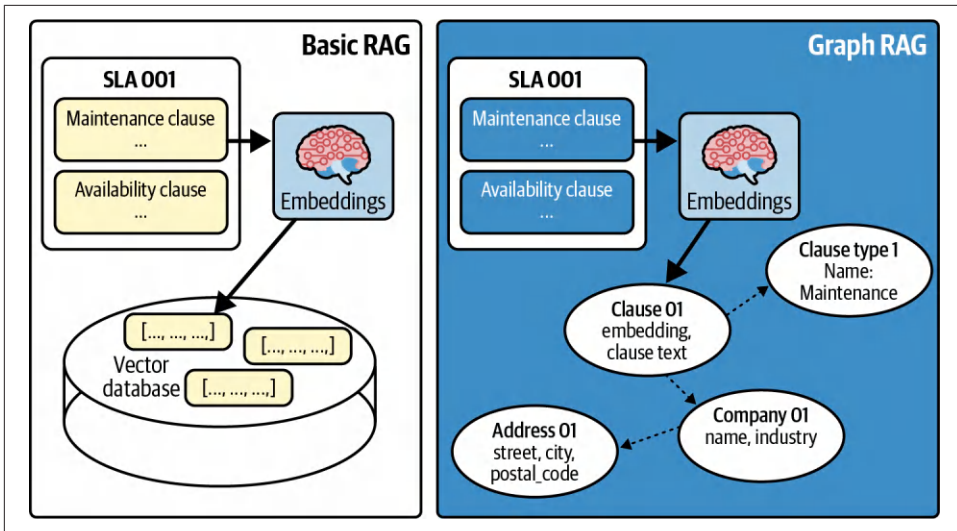


Figure 9-1. Graph RAG versus classic vector search

To build a graph RAG system, you replace the traditional retriever with a graph database. In this chapter, Neo4j is used for all graph examples.

Once the graph is populated, the retrieval flow typically follows four steps:

1. Initial search: You begin with either vector search or full-text search to identify relevant nodes. These act as your anchor points.
2. Graph expansion: Starting from the anchor nodes, you traverse the graph, usually through one or two hops, to gather related nodes and edges.
3. (Optional) Filtering and ranking: You refine the expanded result set and keep only the nodes that add meaningful context.
4. Context assembly and generation: You feed both the original anchor text and its connected context into the LLM.

This process combines semantic matching with structural reasoning, producing context that reflects both meaning and connections. Figure 9-2 shows the main retrieval process.

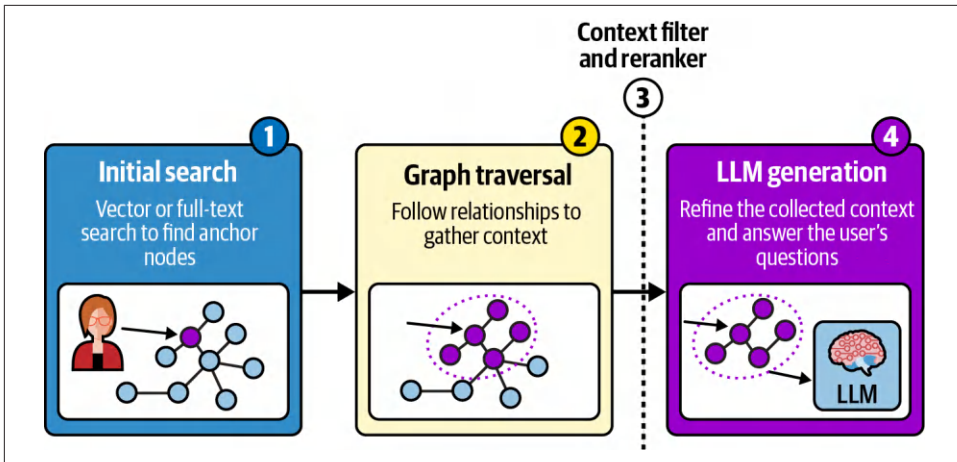


Figure 9-2. Phases of graph RAG retrieval

In this chapter, you will build your first knowledge graph, populate it with text from documents, and enrich it with structured data.



Graph databases deliver richer context by following connections between nodes. Graphs introduce additional complexity and cost compared to basic RAG systems. They require careful data modeling, structured ingestion, and more up-front work. When relationships matter and the final answer depends on how entities connect to each other, graphs unlock retrieval behavior that classic RAG systems cannot achieve.

The recipes that follow guide you through building a complete graph RAG system step by step, starting with graph construction and progressing to hybrid search. The first recipe establishes the foundation by creating a knowledge graph from SLA contract documents.

You can find all the code examples for this chapter in the [book's GitHub repository](#).

9.1 Creating Your First Neo4j Knowledge Graph and Feeding It with Text from Documents

Problem

You have never worked with Neo4j and want to build your first knowledge graph for your graph RAG system.

Solution

In this recipe, you will build a knowledge graph derived from service-level agreements (SLAs). SLAs are contracts between service providers and customers that define performance standards, responsibilities, and remedies. [Figure 9-3](#) shows the graph structure you'll create.

At the center is a Company node. Each company has an associated Address and one or more SLA nodes. Every SLA is divided into multiple Clause nodes, each representing an individual section of the document. Each clause is connected to its successor by using NEXT relationships to preserve document order. Additionally, each clause is linked to a ClauseType node, which groups similar thematic categories across suppliers.

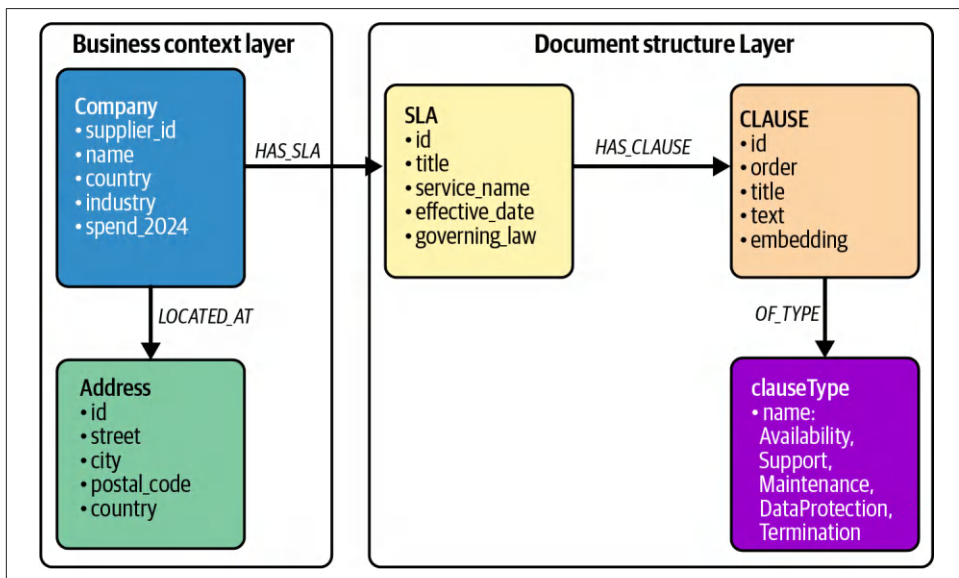


Figure 9-3. SLA knowledge graph schema

In this recipe, you'll build the right side of the graph diagram (SLA and Clause nodes). The left side (company data) is added in [Recipe 9.2](#).

[Figure 9-4](#) shows the structure. Create Clause nodes linked to their parent SLA by using `HAS_CLAUSE` relationships. Connect adjacent clauses with `NEXT` relationships to preserve document order. Create ClauseType nodes (Availability, Support, Maintenance, Termination, etc.) and link each clause to its type by using the `OF_TYPE` relationship.

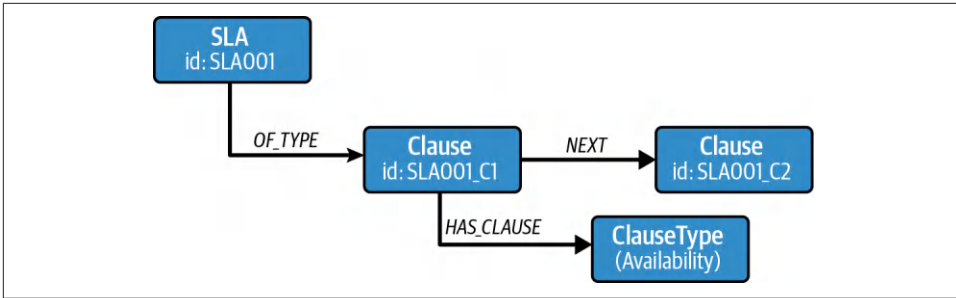


Figure 9-4. SLA knowledge graph structure

You need either Neo4j Desktop for local development or a Neo4j Aura cloud instance. For detailed installation steps, you can follow the [Neo4j installation guide](#). After installation, create a database instance. If you run Neo4j locally on your machine, the database is typically available at `localhost:7687`. There is always a predefined admin user named `neo4j`, and you choose the password when you create the instance. You can verify that everything works by starting the instance in Neo4j Desktop and connecting to it.

Next, you connect to the graph from your Python script so you can populate it with the prepared data. For this, you need the `neo4j` library and `python-dotenv` to manage your connection credentials in a safe and convenient way:

```
pip install neo4j python-dotenv
```

Store your connection details in an `.env` file. Make sure you never commit credentials to GitHub or share them publicly. In the `.env` file, set the connection URI, username, and password for your Neo4j instance:

```
NEO4J_URI=neo4j://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=testpassword
```

To avoid redefining the credentials each time, create a helper function `get_driver` that establishes connectivity whenever needed and returns a driver for executing Cypher queries in the knowledge graph:

```
def get_driver():
    """Create Neo4j driver from environment variables."""
    uri = os.getenv("NEO4J_URI", "neo4j://127.0.0.1:7687")
    user = os.getenv("NEO4J_USERNAME", "neo4j")
    pwd = os.getenv("NEO4J_PASSWORD", "testpassword")
    return GraphDatabase.driver(uri, auth=(user, pwd))
```

Before inserting data, define constraints to prevent duplicates. Without constraints, the `MERGE` command may create duplicate nodes if matching logic is ambiguous. The `id` property on `SLA` and `Clause` nodes, and the `name` property on `ClauseType` nodes need to be unique:

```

def create_constraints(driver):
    """Create uniqueness constraints for graph nodes."""

    constraints = [
        "CREATE CONSTRAINT sla_id IF NOT EXISTS "
        "FOR (s:SLA) REQUIRE s.id IS UNIQUE",

        "CREATE CONSTRAINT clause_id IF NOT EXISTS "
        "FOR (c:Clause) REQUIRE c.id IS UNIQUE",

        "CREATE CONSTRAINT type_name IF NOT EXISTS "
        "FOR (t:ClauseType) REQUIRE t.name IS UNIQUE",
    ]

    with driver.session() as session:
        for constraint in constraints:
            session.run(constraint)

driver = get_driver()
create_constraints(driver)

```

Parse the SLA document by splitting on section headings. Use a regex to separate the document at each second-level heading (lines starting with ##) and create a Clause object for each section:

```

from dataclasses import dataclass
import re

@dataclass
class Clause:
    id: str
    title: str
    text: str
    order: int
    clause_type: str = "Other"

def parse_sla_file(path, sla_id):
    with open(path, "r", encoding="utf-8") as f:
        content = f.read()

    sections = re.split(r"^##\s+", content, flags=re.MULTILINE)[1:]
    clauses = []

    for idx, section in enumerate(sections, start=1):
        lines = section.strip().splitlines()
        if not lines:
            continue

        title = lines[0].strip()

```

```

text = "\n".join(lines[1:]).strip()

clauses.append(
    Clause(
        id=f"{sla_id}_C{idx}",
        title=title,
        text=text,
        order=idx,
    )
)

return clauses

```

Figure 9-5 shows the resulting list of Clause objects.

```

[17] ✓ 0.0s Python
... [Clause(id='SLA001_C1', title='1. Service Description', text='The Alpha Compute service provi
Clause(id='SLA001_C2', title='2. Availability and Uptime Commitment', text='Alpha Cloud comm
Clause(id='SLA001_C3', title='3. Response Times and Incident Handling', text='Alpha Cloud pr
Clause(id='SLA001_C4', title='4. Maintenance Windows', text='Alpha Cloud may perform routine
Clause(id='SLA001_C5', title='5. Service Credits', text='If the monthly uptime percentage fe
Clause(id='SLA001_C6', title='6. Customer Responsibilities', text='The customer is responsit
Clause(id='SLA001_C7', title='7. Data Protection and Privacy', text='Alpha Cloud processes c
Clause(id='SLA001_C8', title='8. Security and Compliance', text='Alpha Cloud implements star

```

Figure 9-5. List of clauses extracted from an SLA

With the clauses extracted, transform the list into the graph structure. Create the SLA node with its title, create the Clause nodes and link them back to the SLA, and connect adjacent clauses with NEXT relationships:

```

def write_sla_and_clauses(driver, sla_id, sla_title, clauses):
    """Write SLA and Clause nodes to Neo4j with their relationships."""
    with driver.session() as session:
        # Create SLA node
        session.run(
            """
            MERGE (s:SLA {id: $id})
            SET s.title = $title
            """,
            id=sla_id,
            title=sla_title,
        )

        # Create Clause nodes and link to SLA
        for c in clauses:
            session.run(
                """
                MERGE (cl:Clause {id: $id})
                SET cl.title = $title,
                  cl.text = $text,
                  cl.order = $order
                """,

```

```

        id=c.id,
        title=c.title,
        text=c.text,
        order=c.order,
    )
    session.run(
        """
        MATCH (s:SLA {id: $sla_id})
        MATCH (cl:Clause {id: $cid})
        MERGE (s)-[:HAS_CLAUSE]->(cl)
        """,
        sla_id=sla_id,
        cid=c.id,
    )

    # Connect adjacent clauses to preserve document order
    for prev, nxt in zip(clauses, clauses[1:]):
        session.run(
            """
            MATCH (a:Clause {id: $p})
            MATCH (b:Clause {id: $n})
            MERGE (a)-[:NEXT]->(b)
            """,
            p=prev.id,
            n=nxt.id,
        )

```

```
write_sla_and_clauses(driver, sla_id, sla_title, clauses)
```

Each clause needs a type based on keyword matching. Here, you map keywords like availability or uptime to clause types, and clauses that don't match any keyword pattern are labeled Other:

```

def infer_clause_type(title):
    """Infer ClauseType based on keywords in the title."""
    title_lower = title.lower()
    keywords = {
        "Availability": ["availability", "uptime"],
        "Support": ["support", "response time", "incident"],
        "Maintenance": ["maintenance"],
        "DataProtection": ["data protection", "gdpr", "privacy"],
        "Liability": ["liability"],
        "Termination": ["termination"],
    }

    for clause_type, words in keywords.items():
        if any(word in title_lower for word in words):
            return clause_type
    return "Other"

```

```

for c in clauses:
    c.clause_type = infer_clause_type(c.title)

```

Once each clause has a type assigned, create the `ClauseType` nodes and connect each clause to its type with an `OF_TYPE` relationship:

```

def add_clause_types(session, clauses):
    # Create ClauseType nodes
    types = [{"clause_type": c.clause_type} for c in clauses]
    session.run(
        """
        UNWIND $rows AS row
        MERGE (t:ClauseType {name: row.clause_type})
        """,
        rows=types,
    )

    # Link clauses to their types
    links = [{"id": c.id, "type": c.clause_type} for c in clauses]
    session.run(
        """
        UNWIND $rows AS row
        MATCH (cl:Clause {id: row.id})
        MATCH (t:ClauseType {name: row.type})
        MERGE (cl)-[:OF_TYPE]->(t)
        """,
        rows=links,
    )

```

```

add_clause_types(driver.session(), clauses)

```

This completes the basic SLA knowledge graph structure. To verify that the import worked, run a simple Cypher query in Neo4j Browser. Open Neo4j Desktop, click Connect and Query, and run this query to find clauses with availability information:

```

MATCH (s:SLA {id: "SLA1"})-[:HAS_CLAUSE]->(cl:Clause)
RETURN s.id AS sla_id, cl.id AS clause_id,
        cl.order AS clause_order, cl.title AS clause_title
ORDER BY cl.order;

```

Figure 9-6 shows the completed graph with SLA contracts, clauses, and their relationships.

The screenshot shows a Neo4j query interface. At the top, a Cypher query is entered: `neo4j$ MATCH (s:SLA {id: "SLA1"})-[:HAS_CLAUSE]->(cl:Clause) RETURN s.id AS sla_id`. Below the query bar, there are two tabs: "Table" (selected) and "RAW". To the right of the tabs are icons for search, schema, and download. The results are displayed in a table with four columns: `sla_id`, `clause_id`, `clause_order`, and `clause_title`. A single row of data is shown.

sla_id	clause_id	clause_order	clause_title
"SLA1"	"SLA1_C1"	1	"1. Service Description"

Figure 9-6. First SLA knowledge graph

Discussion

This recipe models SLAs as a graph of contracts, clauses, and clause types, preserving both document structure and relationships. Constraints and indexes prevent duplicates and keep queries fast, while the clause order keeps the original document sequence intact.

Graph modeling works because it captures relationships that embeddings can't, such as which clause belongs to which contract or which clause comes next. This makes it possible to compare clauses across documents, follow reading order, or verify whether a contract is missing a required clause.

Use a graph when questions require traversing relationships—for example, comparing termination clauses across suppliers or checking which contracts lack a specific clause type. For simple semantic lookup inside a single document, vector search alone is usually enough.

The trade-off is schema and maintenance overhead. You must design node and relationship types up front and keep them in sync as data changes, but this structure enables powerful queries that pure vector RAG can't support.

The next recipe extends this foundation with supplier data to add business context to contract queries.

See Also

- The [Neo4j installation guide](#) provides instructions for installing Neo4j on various platforms.
- The [Neo4j Desktop installation guide](#) covers setup for the desktop development environment.

9.2 Extending the Knowledge Graph with Structured Data

Problem

You want to enrich the knowledge graph with structured business data.

Solution

Import four types of master data to enrich the knowledge graph:

Company information

Supplier name, country, and industry from *companies.csv*

Address data

Street, city, postal code, and country from *addresses.csv*

Spending information

Yearly spending and the spending category from *spend_2024.csv*

SLA metadata

Effective dates, service names, and governing law from *slas.csv*

After the import, your graph combines document structure with business context, as shown in [Figure 9-7](#).

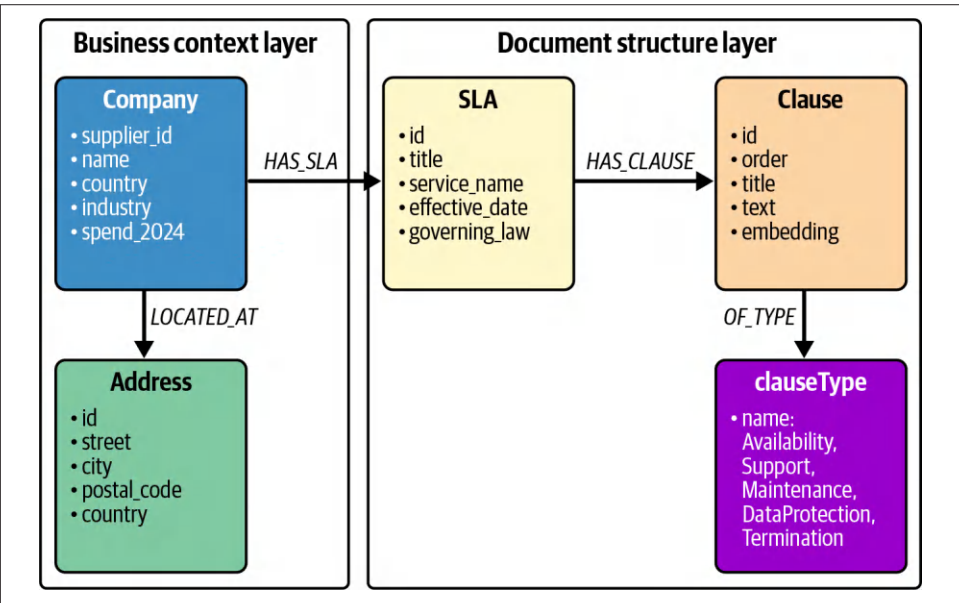


Figure 9-7. Extended SLA knowledge graph structure with company data

Import *companies.csv* and create one Company node per supplier with attributes such as name, country, and industry:

```
def load_companies(driver, companies_csv_path):
    """Load companies from CSV and create Company nodes."""
    df = pd.read_csv(companies_csv_path)
    print(f>Loading {len(df)} companies...")

    with driver.session() as session:
        session.run(
            """
            UNWIND $rows AS row
            MERGE (c:Company {supplier_id: row.supplier_id})
            ON CREATE SET c.name=row.name, c.country=row.country,
                c.industry=row.industry
            SET c.industry = row.industry
            """,
            {"rows": df.to_dict(orient="records")},
        )
```

Add the address nodes with street, city, postal code, and country as properties:

```
def load_addresses(driver, addresses_csv_path):
    """Load addresses from CSV and create Address nodes."""
    df = pd.read_csv(addresses_csv_path)
    print(f>Loading {len(df)} addresses...")

    with driver.session() as session:
        session.run(
            """
            UNWIND $rows AS row
            MERGE (a:Address {id: row.address_id})
            ON CREATE SET a.street = row.street,
                a.city = row.city,
                a.postal_code = row.postal_code,
                a.country = row.country
            """,
            {"rows": df.to_dict(orient="records")},
        )
```

Link companies to their addresses with LOCATED_AT relationships:

```
def connect_company_addresses(driver, companies_csv_path):
    """Create LOCATED_AT relationships between companies and addresses."""
    df = pd.read_csv(companies_csv_path)[["supplier_id", "address_id"]]
    print(f>Connecting {len(df)} company-address relationships...")

    with driver.session() as session:
        session.run(
            """
            UNWIND $rows AS row
            MATCH (c:Company {supplier_id: row.supplier_id})
            MATCH (a:Address {id: row.address_id})
```

```

MERGE (c)-[:LOCATED_AT]->(a)
"""
{"rows": df.to_dict(orient="records")},
)

```

The annual spending data for each supplier is stored in *spend_2024.csv*. Add this information as properties on the Company node:

```

def load_spend(driver, spend_csv_path):
    """Load spend data and update Company nodes with spend information."""
    df = pd.read_csv(spend_csv_path)
    print(f>Loading spend data for {len(df)} companies...")

    with driver.session() as session:
        session.run(
            """
            UNWIND $rows AS row
            MATCH (c:Company {supplier_id: row.supplier_id})
            SET c.spend_2024 = toFloat(row.spend_eur),
                c.spend_category = row.spend_category
            """
            , {"rows": df.to_dict(orient="records")},
        )

```

Finally, import the *slas.csv* file to link each company to its corresponding SLA by using the HAS_SLA relationship, while applying additional SLA metadata such as the title, service name, and effective date:

```

def load_slas(driver, sla_csv_path):
    """Load SLA metadata and link to Company nodes."""
    df = pd.read_csv(sla_csv_path)
    df["effective_date"] = df["effective_date"].replace({"N/A": None})
    df["effective_date"] = df["effective_date"].where(
        df["effective_date"].notna(), None
    )
    print(f>Loading {len(df)} SLA records...")

    with driver.session() as session:
        session.run(
            """
            UNWIND $rows AS row
            MATCH (c:Company {supplier_id: row.supplier_id})
            MERGE (s:SLA {id: row.sla_id})
            ON CREATE SET s.title = row.title,
                s.service_name = row.service_name,
                s.governing_law = row.governing_law,
                s.effective_date = CASE
                WHEN row.effective_date IS NOT NULL
                THEN date(row.effective_date)
                ELSE NULL END
            MERGE (c)-[:HAS_SLA]->(s)
            """
            ,
        )

```

```
    {"rows": df.to_dict(orient="records")},  
)
```

This import performs two key actions. First, it ensures that each SLA node exists and sets its metadata fields such as title, service name, governing law, and effective date. Second, it creates the HAS_SLA relationship that links each supplier to the contracts they hold.

This connection is crucial for queries that combine business context with contract content, such as “Which high-spend suppliers lack a termination clause?”

Discussion

Use structured enrichment when retrieval depends on business context. Questions like “Which high-spend suppliers lack termination clauses?” or “Show me data protection clauses for German healthcare companies” require both contract text and entity attributes in the same query. This pattern is central to procurement, compliance, vendor management, and any domain where documents make sense only in relation to the entities they describe.

Skip enrichment if your questions are purely document-centric. If you need only “What does the Acme Corp contract say about support?” a simple clause lookup is enough. Importing and maintaining business data pays off only when it actually changes what gets retrieved. If metadata rarely influences selection, keep it outside the graph.

Adding structured data increases operational complexity. Attributes like spending, addresses, or organizational structure change over time and must stay in sync with source systems. Stale data leads to wrong filters—for example, marking a supplier as low-spend when it isn’t. When those filters significantly narrow the search space before semantic ranking, the accuracy gains usually justify the effort.

The earlier recipe modeled only document structure, such as SLAs, clauses, and clause types. This recipe adds business entities and their attributes as a second dimension. The result is a hybrid knowledge graph that combines content structure with business context, which is what distinguishes enterprise-grade graph RAG from document-only graphs.

See Also

- The [Neo4j Python driver documentation](#) covers the official Python driver for connecting to Neo4j databases.
- The [Neo4j Cypher Manual](#) provides a comprehensive reference for the Cypher query language.

9.3 Building Your First Cypher Query

Problem

You built an SLA knowledge graph that connects suppliers, contracts, and clauses. Now you want a first practical example of how to query this graph by using Cypher.

Solution

Retrieve all clauses for a given SLA and return them in their original order:

```
def list_clauses_for_sla(sla_id):
    """Return all clauses for one SLA ordered by their original position."""
    cypher = """
    MATCH (s:SLA {id: $sla_id})-[:HAS_CLAUSE]->(cl:Clause)
    RETURN cl.order AS order,
           cl.title AS title,
           cl.text AS text
    ORDER BY order
    """
    driver = get_driver()
    with driver.session() as session:
        response = session.run(cypher, sla_id=sla_id)
        records = [r.data() for r in response]
    return records
```

Search all clauses of the type Termination across all SLAs. This makes it easy to compare how different suppliers phrase the same contractual topic:

```
def clauses_of_type(clause_type):
    """List all clauses of a specific ClauseType across suppliers."""
    cypher = """
    MATCH (c:Company)-[:HAS_SLA]->(s:SLA)-[:HAS_CLAUSE]->(cl:Clause)
    MATCH (cl)-[:OF_TYPE]->(t:ClauseType {name: $clause_type})
    RETURN c.name AS company,
           s.id AS sla_id,
           cl.order AS clause_order,
           cl.title AS clause_title,
           cl.text AS clause_text
    ORDER BY company, sla_id, clause_order
    """
    driver = get_driver()
    with driver.session() as session:
        return [r.data() for r in session.run(cypher, clause_type=clause_type)]
```

Now that you have seen basic lookups, you can move to more investigative queries that help you spot risk and gaps in your contracts.

Use the following function to return all companies above a spending threshold whose SLAs don't contain a termination clause. This lets you quickly find high-value suppliers missing a key protection:

```

def high_spend_missing_termination(min_spend):
    """Return companies above min_spend whose SLAs lack a termination clause."""

    cypher = """
MATCH (c:Company)-[:HAS_SLA]->(s:SLA)
WHERE c.spend_2024 > $min_spend
OPTIONAL MATCH (s)-[:HAS_CLAUSE]->(cl:Clause)
              -[:OF_TYPE]->(t:ClauseType {name: "Termination"})
WITH c, s, count(cl) AS num_termination
WHERE num_termination = 0
RETURN c.name AS company,
        c.spend_2024 AS spend_2024,
        s.id AS sla_id
ORDER BY spend_2024 DESC
    """

    driver = get_driver()

    with driver.session() as session:
        return [
            r.data()
            for r in session.run(cypher, min_spend=min_spend)
        ]

```

You can also zoom in on specific clause types and text patterns. The next example inspects availability clauses and filters them by a search phrase—for example, a specific uptime target such as 99.9:

```

def availability_clauses(search_phrase):
    """Inspect availability clauses per supplier."""

    cypher = """
MATCH (c:Company)-[:HAS_SLA]->(s:SLA)-[:HAS_CLAUSE]->(cl:Clause)
MATCH (cl)-[:OF_TYPE]->(t:ClauseType {name: "Availability"})
WHERE toLower(cl.text) CONTAINS toLower($phrase)
RETURN c.name AS company,
        s.id AS sla_id,
        cl.order AS clause_order,
        cl.text AS availability_text
ORDER BY company, sla_id, clause_order
    """

    driver = get_driver()
    with driver.session() as session:
        return [r.data() for r in session.run(cypher, phrase=search_phrase)]

```

Combine clause types with supplier metadata. The following query retrieves data protection clauses for suppliers in a given list of EU countries:

```

def eu_data_protection_clauses(countries: list[str]):
    """Retrieve data protection clauses for suppliers in given EU countries."""

    cypher = """
MATCH (c:Company)-[:LOCATED_AT]->(a:Address)
WHERE a.country IN $countries
MATCH (c)-[:HAS_SLA]->(s:SLA)-[:HAS_CLAUSE]->(cl:Clause)

```

```

MATCH (cl)-[:OF_TYPE]->(t:ClauseType {name: "DataProtection"})
RETURN c.name AS company,
       a.country AS country,
       s.id AS sla_id,
       cl.order AS clause_order,
       cl.text AS data_protection_clause
ORDER BY country, company, sla_id, clause_order
"""

driver = get_driver()
with driver.session() as session:
    return [r.data() for r in session.run(cypher, countries=countries)]

```

Discussion

Structural retrieval works by navigating the graph instead of searching text. You start from a known node and follow relationships to reach the information you need. If you know a specific SLA or company, you can go directly from that node to its clauses. If you want to compare the same kind of content across many contracts, you start from a `ClauseType` and collect all clauses of that type. If business rules matter first, you filter companies or contracts before looking at any text.

Use direct lookups when you know the exact document or entity you're interested in. Use type-based queries when you want to compare similar clauses across many documents. Use filtered searches when attributes like spending, region, or industry determine which data should be included.

Structural queries are fast and precise because they return only the nodes that match your labels and filters. Their limitation is that they depend on the quality of the graph structure. If your taxonomy is incomplete or inconsistent, relevant content can be missed. The next recipe adds semantic search to compensate for this by finding similar meaning even when labels don't line up exactly.

Compared to vector-only RAG, graph-based retrieval combines structure and semantics. Business rules and relationships can narrow the search space before semantic matching, or they can be used to validate and aggregate results after a semantic search.

See Also

- [“Get started with Cypher”](#) introduces core concepts of the Cypher query language.
- The [Cypher clauses reference](#) documents `MATCH`, `RETURN`, `WHERE`, and other query clauses.
- The [Cypher cheat sheet](#) provides a quick reference for common query patterns.

9.4 Enabling Semantic Search on a Neo4j Knowledge Graph

Problem

You want to search SLA clauses by explicit graph labels, but also by semantic meaning, and combine this with supplier metadata and graph relationships.

Solution

This recipe shows how to combine semantic vector search with structural graph filters in Neo4j. **Figure 9-8** illustrates the hybrid search approach.

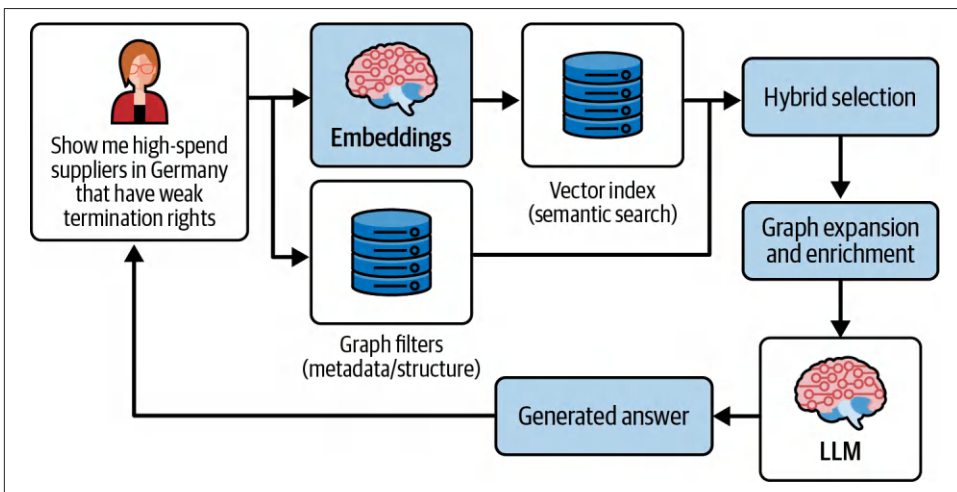


Figure 9-8. Hybrid semantic structural search in Neo4j

Let's start with the setup, which is run once after the graph creation:

Generate embeddings for every Clause node and write them to the graph:

```
def create_embedding(text):
    """Generate embeddings using OpenAI's text-embedding-3-small model."""
    return (
        client.embeddings.create(model="text-embedding-3-small", input=text)
        .data[0]
        .embedding
    )

def create_clause_embeddings():
    """Add embeddings to all Clause nodes in the graph."""
    driver = get_driver()
```



```

with driver.session() as session:
    clauses = list(session.run(
        "MATCH (cl:Clause) RETURN cl.id AS id, cl.text AS text"
    ))

    print(f"Creating embeddings for {len(clauses)} clauses...")

    for i, row in enumerate(clauses, 1):
        emb = create_embedding(row["text"])
        session.run(
            "MATCH (cl:Clause {id:$id}) SET cl.embedding=$emb",
            id=row["id"],
            emb=emb,
        )

        if i % 10 == 0:
            print(f"  Processed {i}/{len(clauses)} clauses")

driver.close()

```

Run `create_clause_embeddings` once after you create your graph. With the embeddings in place, create a vector index for clause embeddings:

```

def create_vector_index():
    """Create a vector index for semantic search on Clause embeddings."""
    driver = get_driver()
    with driver.session() as session:
        session.run(
            """
            CREATE VECTOR INDEX clause_embeddings IF NOT EXISTS
            FOR (c:Clause) ON c.embedding
            OPTIONS {
                indexConfig: {
                    `vector.dimensions`: 1536,
                    `vector.similarity_function`: "cosine"
                }
            }
            """
        )
    driver.close()

```

Now you can search the entire corpus with natural language queries. Return the top k most similar clauses based on their embeddings:

```

def semantic_search(query, top_k=5):
    """Find clauses semantically similar to the query."""
    driver = get_driver()
    emb = create_embedding(query)

    cypher = """
    CALL db.index.vector.queryNodes(
        "clause_embeddings", $top_k, $embedding
    )
    """

```

```

YIELD node, score
RETURN node.title AS title,
       node.text AS text,
       score
ORDER BY score DESC
"""

with driver.session() as session:
    rows = [r.data() for r in session.run(
        cypher, top_k=top_k, embedding=emb
    )]

driver.close()
return rows

```

Combine semantic search with graph filters to narrow the results. This example uses an industry filter:

```

def hybrid_search_by_industry(query, industry, top_k=5):
    """Combine semantic search with industry filtering."""
    driver = get_driver()
    emb = create_embedding(query)

    cypher = """
CALL db.index.vector.queryNodes(
    "clause_embeddings", $top_k, $embedding
)
YIELD node, score
MATCH (node)-[:HAS_CLAUSE]-(s:SLA)-[:HAS_SLA]-(c:Company)
WHERE c.industry = $industry
RETURN c.name AS company,
       s.id AS sla_id,
       node.title AS clause_title,
       node.text AS clause_text,
       score
ORDER BY score DESC
"""

    with driver.session() as session:
        rows = [r.data() for r in session.run(
            cypher,
            top_k=top_k,
            embedding=emb,
            industry=industry
        )]

    driver.close()
    return rows

```

This function returns semantically similar clauses filtered by industry. The results include company name, SLA ID, and clause title and text, along with similarity scores.

Discussion

This recipe combines vector search with graph filtering. Business rules such as industry or spending narrow the candidates first; then semantic similarity ranks the remaining clauses so the model sees only relevant text with company and SLA context.

Use hybrid search when queries mix meaning and business constraints. When labels like `ClauseType` are reliable, a pure Cypher query is simpler and faster.

The trade-off is overhead. You must maintain both embeddings and graph structure, and vector search adds cost and latency. In return, you get queries that neither graphs nor vectors can handle alone.

Compared to basic RAG, graph RAG does not return isolated chunks. It returns clauses together with their connected business context, which makes answers more precise and explainable.

See Also

- The [neo4j-graphrag Python package](#) provides prebuilt components for semantic and hybrid search in Neo4j.
- The [Neo4j vector index documentation](#) covers configuration and optimization of vector indexes.

9.5 Optimizing the Knowledge Graph for RAG Systems

Problem

You want to optimize your graph when you measure performance or cost bottlenecks such as slow queries, high token costs, or low retrieval precision.

Solution

Enhance the graph with optional components such as summaries, embeddings for entire SLAs, risk scores, domain ontologies, or retrieval shortcuts.

Use an efficient LLM like gpt-4o-mini to summarize each clause and attach it as a new attribute summary:

```
def summarize_clause(text):  
    """Generate a summary for a clause using GPT-4o-mini."""  
    prompt = f"Summarize this SLA clause:\n\n{text}"  
    return (  
        client.chat.completions.create(  
            model="gpt-4o-mini",  
            messages=[{"role": "user", "content": prompt}],
```

```

        max_tokens=120,
    )
    .choices[0]
    .message.content
)

def add_clause_summaries():
    """Add summaries to all Clause nodes that don't have one."""

    driver = get_driver()

    with driver.session() as session:
        result = session.run(
            """
            MATCH (cl:Clause)
            WHERE cl.summary IS NULL
            RETURN cl.id AS id,
                   cl.text AS text
            """
        )

        clauses = list(result)

        print(
            f"Generating summaries for {len(clauses)} clauses..."
        )

        for i, row in enumerate(clauses, 1):
            s = summarize_clause(row["text"])
            session.run(
                "MATCH (cl:Clause {id:$id}) SET cl.summary=$s", id=row["id"], s=s
            )
            if i % 5 == 0:
                print(f"  Processed {i}/{len(clauses)} clauses")
        driver.close()

```

You can register a vector index for SLAs, just as you did for clauses. Summaries can be indexed and searched separately.

To speed up queries, you can store useful aggregates such as these directly in the graph:

- Count of availability clauses per supplier
- Combined lists of support obligations
- Flags for whether a clause type exists

This Cypher query shows a basic example that counts the number of clauses of each type that exist per company and stores the result as a new property on the Company node:

```

MATCH (c:Company)-[:HAS_SLA]->(s:SLA)-[:HAS_CLAUSE]->(cl:Clause)
  -[:OF_TYPE]->(t:ClauseType)
WITH c, t, count(cl) AS num_clauses
SET c[t.name + '_count'] = num_clauses

```

Discussion

These optimizations reduce cost and latency by moving work to ingestion time instead of query time. Clause summaries let the model scan many clauses cheaply before reading full text. SLA-level embeddings make it possible to retrieve entire contracts without searching every clause. Aggregates avoid recomputing the same values on every query.

Each technique solves a different kind of bottleneck. Use summaries when you retrieve many clauses but only a few are relevant. Use aggregates when you keep calculating the same counts or flags. Use SLA embeddings when users ask about whole contracts rather than individual clauses.

Don't add these optimizations up front. Start with the baseline graph and measure how it performs. If queries are fast, token usage is acceptable, and answers are accurate, the simpler model is often enough.

The main trade-off is maintenance. Summaries, aggregates, and embeddings must stay in sync as data changes, and they increase storage and ingestion time. They are most valuable when they remove a proven cost or performance problem.

Always retrieve the full clause text for the final answer. Summaries and cached values are meant for filtering and ranking, not for replacing the original data.

See Also

- *Essential GraphRAG* by Tomaž Bratanič and Oskar Hane (Manning) covers comprehensive patterns for building graph RAG systems.
- “Knowledge Graphs for RAG Systems” by Andreas Kollegger teaches practical implementation techniques.
- The [Neo4j indexes documentation](#) is an official guide to creating and optimizing indexes for query performance.
- The [Neo4j query tuning guide](#) contains best practices for analyzing and improving query execution plans.
- The [graph RAG research paper](#) from Microsoft explains combining graph structures with retrieval-augmented generation.

Evaluating RAG Systems

You can't improve what you can't measure. As you tune retrieval parameters, adjust prompts, or switch models, you need metrics that tell you whether changes help or hurt. Without evaluation, optimization becomes guesswork.

RAG evaluation requires different approaches than traditional machine learning (ML). In traditional ML, you train models to learn patterns from labeled data, then test whether they generalize to unseen examples. RAG systems don't learn—they retrieve and generate. Foundation models already generalize across tasks. The question isn't whether the model learned patterns, but whether it retrieved the right information and generated useful answers.

This difference shapes how you evaluate. The key challenge is that test questions should cover realistic user intent without appearing word for word in the system's data, while remaining answerable from the available knowledge. Good generalization in RAG means handling different phrasings of the same underlying question. Intent matters more than exact wording. This affects test design—you reformulate real queries rather than split existing labeled examples.

Figure 10-1 illustrates the problem: asking philosophical questions to a system that only knows football (soccer) rules produces meaningless evaluation results.

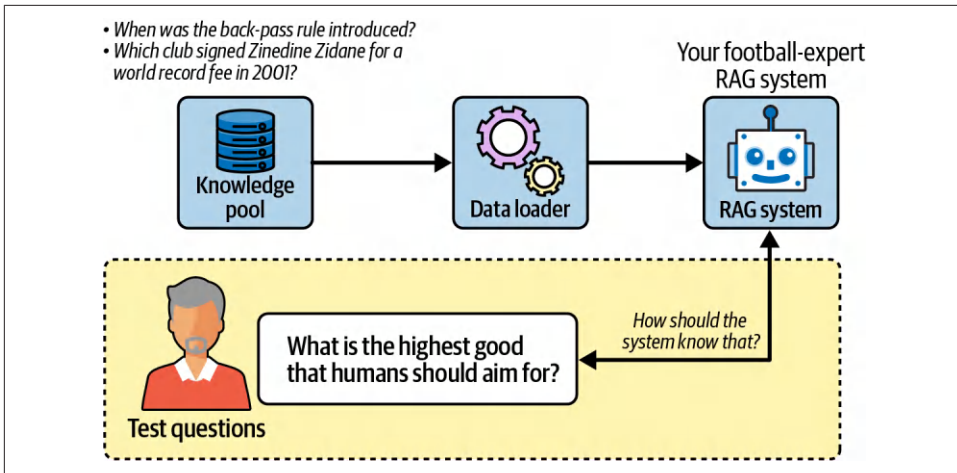


Figure 10-1. RAG systems should only be evaluated on questions that can be answered from their knowledge base

Figure 10-2 shows two dimensions for improving RAG performance: the RAG pipeline itself (retrieval, chunking, reranking) and the generation model (through fine-tuning or prompt engineering). With so many possible adjustments, you need evaluation metrics to determine which changes actually improve accuracy and relevance.

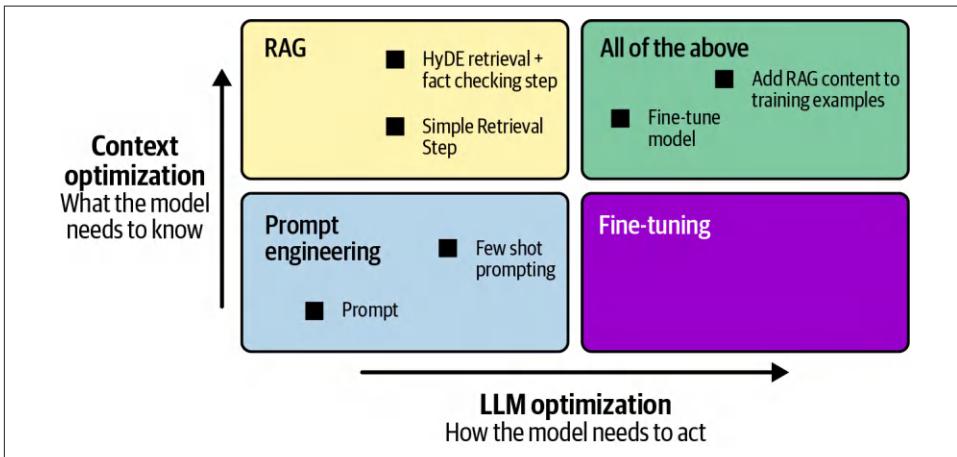


Figure 10-2. Ways to optimize your RAG system

Evaluating generation is challenging because language is subjective. The most scalable approach uses LLMs to evaluate other LLM outputs, similar to an author reviewing their own work. This approach works if you understand its limitations and design evaluations carefully.

For real-world RAG systems, the architecture often goes beyond a simple retriever-generator pipeline. You have different kinds of data sources and specialized retrieval components. Every additional module increases complexity and needs a tailored evaluation strategy.

Let's use a more tangible real-world example: imagine you're an online retailer building a chatbot that provides fashion advice based on articles from a fashion blog. Over time, you plan to evolve this chatbot into a full shopping assistant with access to the product catalog, item availability, order placement, and order tracking.

Your RAG system now has to work with two very different data sources:

- A SQL database backing the online store, which stores structured application data such as users and products
- A vector database containing text snippets extracted from the fashion blog

A possible architecture is shown in [Figure 10-3](#). An agent or orchestration layer decides which data source to query. Database-related questions are routed to a text-to-SQL component, while semantic questions are handled by a retriever connected to the vector database. A final generation step then combines the SQL results and retrieved context to produce a coherent answer.

Now your RAG system consists of multiple components:

1. An orchestration agent that decides which data source to query
2. A text-to-SQL translation step
3. A semantic retriever based on vector search
4. A generation module that turns retrieved context into a final answer

Every component can fail independently, so you need evaluation metrics for each part to identify bottlenecks and improve overall performance.

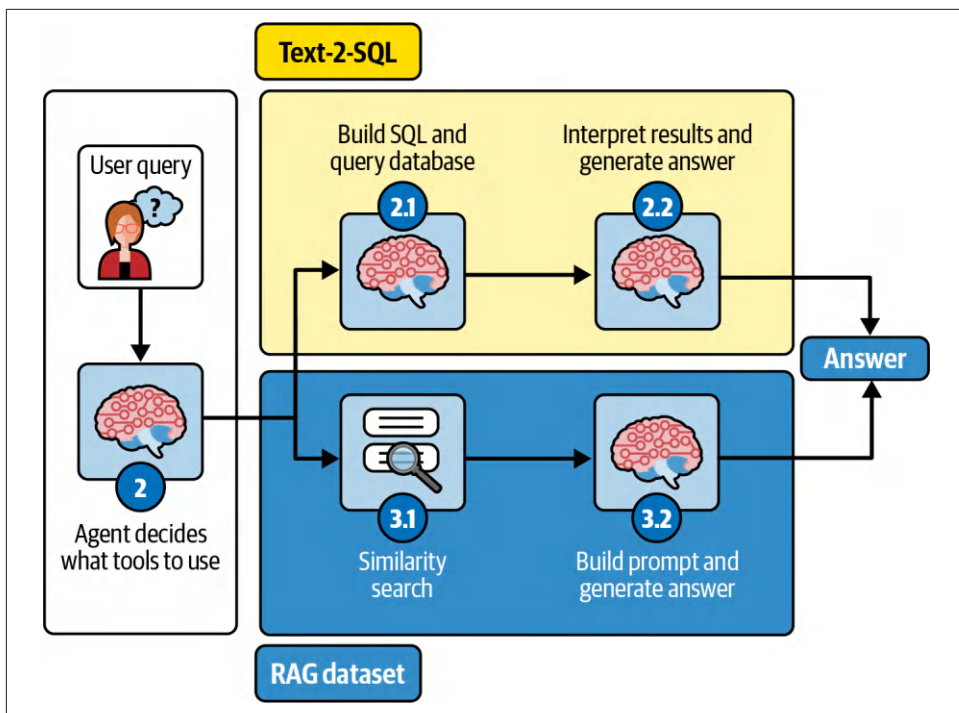


Figure 10-3. A slightly more complex RAG system with two possible retrieval paths

Fortunately, you don't have to start from scratch. The retriever step, implemented as a vector store, is conceptually similar to traditional search engines and classification systems. So there are well-established metrics such as precision and recall that you can reuse. This chapter covers both retrieval metrics adapted from search engines and generation metrics specific to evaluating LLM outputs.

The following recipes guide you through RAG evaluation, from selecting metrics to implementing them. Each recipe includes working code you can adapt to your system. The first recipe helps you choose appropriate metrics based on your system's architecture and available test data. Then you'll learn to generate synthetic test data, implement three core metrics that work without reference answers, and finally incorporate human evaluation for subjective quality dimensions.

You can find all the code examples for this chapter in the [book's GitHub repository](#).

10.1 Choosing the Right Evaluation Metrics for RAG Systems

Problem

You need to choose evaluation metrics that accurately measure the performance of the retrieval and generation step of your RAG system.

Solution

Evaluating a RAG system requires matching your metrics to the ground truth data you have available. This recipe walks you through a systematic approach to selecting evaluation metrics based on your data constraints, then focuses on three core metrics that work when reference answers are unavailable.

Match metrics to your ground truth

Start by identifying what labeled data you have. The answer determines which parts of your system you can measure reliably:

- If you have reference questions paired with correct answers, evaluate the system end to end by using answer-based metrics such as semantic similarity or LLM-based correctness scoring. These directly measure whether your system produces the right result.
- If you have labeled retrieval data—questions paired with the document chunks that should be retrieved—evaluate retrieval quality using metrics like context precision and recall. These measure whether your retriever finds the right information.
- If you have neither reference answers nor labeled retrieval data, evaluate generation quality by using intrinsic metrics such as faithfulness, answer relevancy, and coherence. These check whether the model stays grounded in its context and produces useful responses.

Regardless of your ground truth situation, choose a small, balanced set of metrics—typically two or three—that cover different parts of the pipeline. In most cases, this means at least one retrieval metric and one generation metric. This prevents you from optimizing one component while silently degrading another.

Use common metric categories

Figure 10-4 shows the full landscape of RAG evaluation metrics, organized into three groups:

Generation metrics

Evaluate the generated answer for qualities like faithfulness, correctness, coherence, fluency, and relevance. This category also includes safety metrics covering harmful content or protected material.

Retrieval metrics

Assess retrieval quality by using measures like recall, precision, and F1 score.

General metrics

Capture system behavior such as latency and token usage.

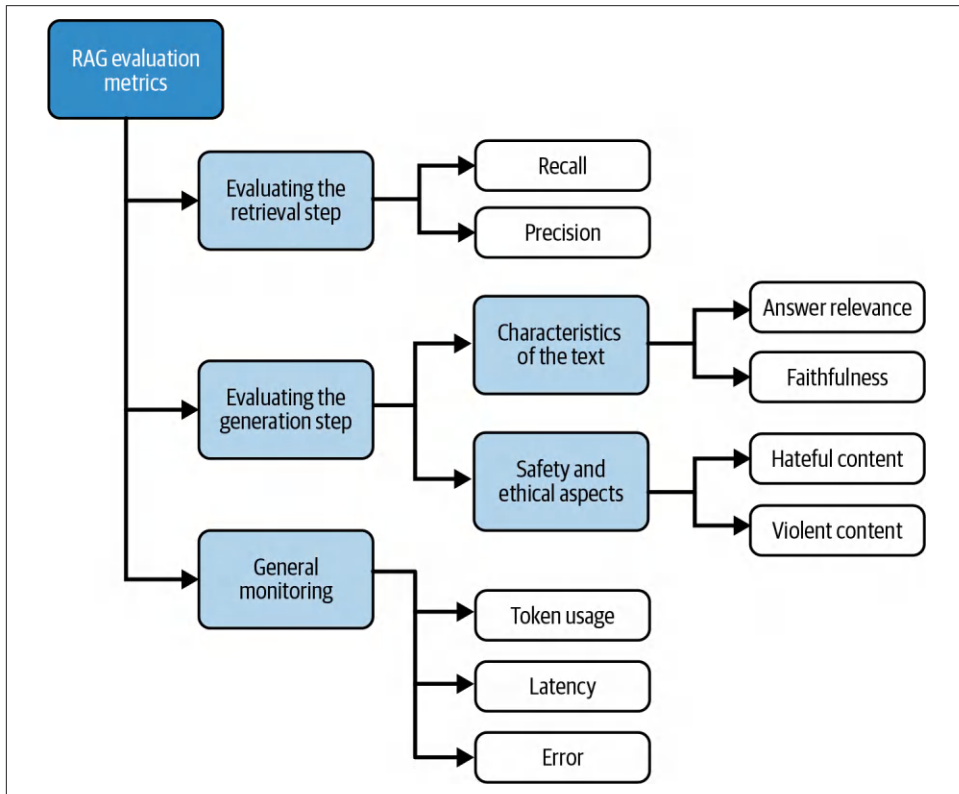


Figure 10-4. RAG evaluation metrics, divided into generation metrics, retrieval metrics, and general system metrics

Additionally, consider metrics tailored to specialized retrieval components. One example is text to SQL, where classic vector similarity metrics are not applicable. As a starting point, a [YouTube video](#) by Manas Singh provides practical insights into evaluating SQL generation by using LLMs as judges.

Focus on three core metrics

This chapter focuses on the common scenario in which you lack reference answers but need automated evaluation. The approach uses LLM-as-judge—where one language model evaluates the output of another—to assess three metrics that cover the most critical failure modes in standard RAG systems. These metrics assume that the retriever is a vector store returning similar text chunks.

For this case, the three metrics are as follows:

Context precision

Measures the proportion of retrieved chunks that are actually relevant to the query. This metric targets the retrieval step and detects noise in your search results (see [Recipe 10.4](#)).

Faithfulness

Measures whether the generated answer is strictly grounded in the provided context, avoiding hallucinations and unsupported claims. This metric targets the generation step and catches when the model invents information (see [Recipe 10.5](#)).

Answer relevancy

Measures whether the generated answer directly addresses the user’s question. This metric targets the generation step and identifies misdirection—cases where the answer is factual but doesn’t help the user (see [Recipe 10.6](#)).

[Figure 10-5](#) illustrates what the different metrics cover along the RAG pipeline.

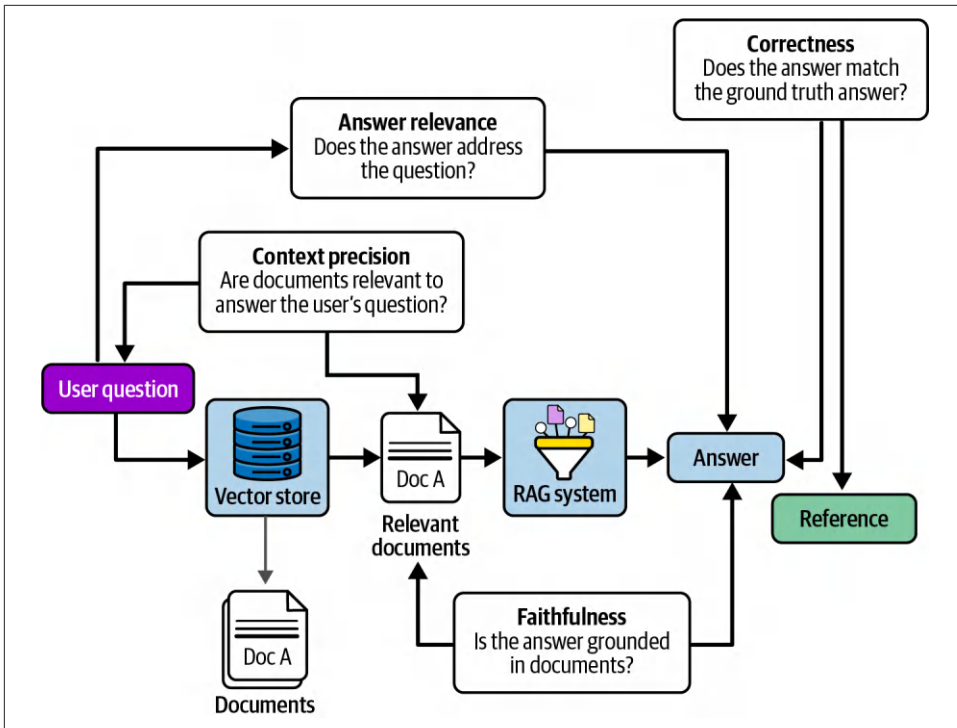


Figure 10-5. Evaluation metrics applied along the RAG pipeline

You can find all three metrics implemented in code in the following recipes.

Discussion

The three-metric approach works because each metric targets an independent failure mode. A system can retrieve precise chunks but still hallucinate, score high on faithfulness but low on answer relevancy, or answer the question while drawing on irrelevant context. By measuring all three dimensions, you catch issues that single metrics miss.

LLM-as-judge works well for these metrics because language models can compare semantic similarity and verify logical consistency without domain knowledge. The judge doesn't need to know the correct answer—it only determines whether claims are supported by context or whether answers address questions. This makes the approach practical when you lack reference answers but still need automated evaluation at scale.

Use this combination for regression testing after system changes and for continuous production monitoring.

Keep in mind that LLM-as-a-judge doesn't capture everything users care about. It struggles with subjective qualities like tone or brand voice, which require human evaluation. It can miss issues that frustrate users: overly verbose answers, confusing phrasing, or missing clarifying questions. These limitations worsen when using synthetic test data, which tends to produce easier questions than real users ask.



Balance automated metrics with human judgment. Use LLM-as-judge for continuous monitoring and high-frequency testing, but supplement it with periodic human review. Human evaluation helps calibrate your metrics, catches issues the automated approach misses, and ensures that your system improves on dimensions that users actually value.

See Also

- [The Ragas documentation](#) provides a comprehensive evaluation framework with detailed metric definitions and code examples for RAG systems.
- [The Hugging Face RAG evaluation cookbook](#) offers an end-to-end tutorial on evaluating RAG systems by using synthetic datasets.

10.2 Evaluating RAG Systems by Humans

Problem

You want to collect explicit user feedback, such as ratings, comparisons, or comments, to evaluate your RAG system based on real user judgments.

Solution

To collect user feedback for your RAG system, you can implement several practical approaches depending on your deployment scenario and user base size.

If you have a sufficiently large test user base, provide users with an old and a new version of your RAG system, let both answer the same set of questions, and ask users to rank the results. This pairwise comparison approach can even be used in production. From time to time, you can show users two alternative answers and ask which one they prefer.

In most real-world scenarios, users see only a single question-answer pair. Even then, you can collect valuable explicit feedback through simple signals such as thumbs-up or thumbs-down. To understand the root cause of poor responses, more detailed input is often required.

Figure 10-6 shows an example feedback form. When a user clicks thumbs down, a dialog appears with a short free-text question, optional quick labels, and a Submit button.

What are the main ingredients in a classic Margherita pizza?

A classic Margherita pizza typically features these simple yet essential ingredients:

- Dough: Usually a Neapolitan-style dough made with flour, water, salt, and yeast.
- Tomato Sauce: A simple sauce made from crushed or pureed San Marzano tomatoes.
- ...

Submit Feedback

Didn't like the response?

Give as much detail as you can

Why was the answer incorrect?

☐ Not correct ☐ Don't like the style ☐ No relevant documents found in the database

Figure 10-6. Online monitoring of RAG systems



In [Chapter 11](#), you'll find examples of how to build a chatbot with Streamlit. The Streamlit community provides ready-to-use modules for chat applications, including built-in user feedback components. Refer to the [Streamlit `st.feedback` documentation](#) for details on the supported feedback options.

You can collect passive monitoring signals:

- Timestamps reveal when users most frequently interact with your application.
- User identifiers help analyze which user groups interact with the system.
- Response times make it possible to identify bottlenecks along the pipeline and improve performance.
- Prompt logging allows you to trace how a response was generated.
- Retrieved documents can be tracked to assess retrieval accuracy.
- LLM responses can be analyzed to estimate answer relevancy and determine whether the model correctly interpreted the provided context.

Track response time by using Python's time module to collect timestamps throughout the RAG pipeline. [Figure 10-7](#) shows how latency is typically distributed.

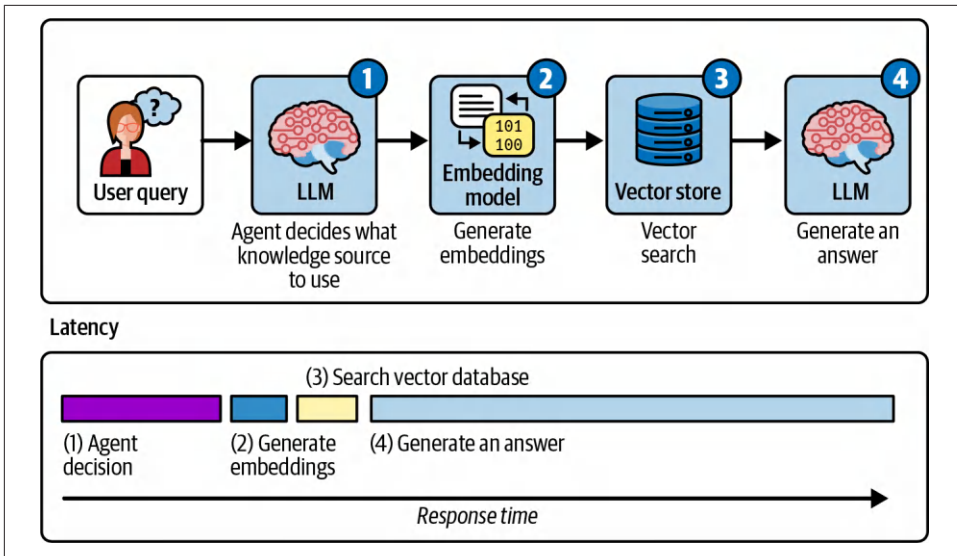


Figure 10-7. Techniques to improve the response time of RAG systems

Discussion

Human evaluators understand context, tone, and intent in ways automated metrics cannot. A thumbs-down often points to missing or outdated content in the knowledge base rather than a technical failure. Written feedback is especially valuable because it reveals gaps, unclear phrasing, or unmet expectations that metrics cannot detect.

Use human feedback when you need to measure user satisfaction and subjective qualities such as tone, clarity, and usefulness. This feedback is especially valuable when testing major changes like switching models or overhauling retrieval. Pairwise comparisons work well in these cases. Showing users two answers side by side and tracking preferences reveals improvements or regressions that automated metrics often miss.

Human evaluation is too slow and expensive for continuous monitoring. It's also inconsistent, since different people often rate the same answer differently. Use this approach for targeted reviews, not for every code change. Human feedback captures tone, completeness, and perceived usefulness. Automated metrics provide scale, consistency, and fast detection of technical failures such as hallucinations or irrelevant retrieval. A hybrid approach works best.

Use automated metrics for continuous monitoring and regression testing, and add periodic human reviews after major changes. A practical approach is to run automated metrics on every code change or deployment, but conduct human evaluation monthly or after significant system updates. Sample 20–30 responses per review session, involving two or three team members or beta users to reduce individual bias. Track both quantitative ratings (thumbs-up/down, 1–5 scales) and qualitative feedback (what was wrong, what would help).

See Also

- **Chatbot Arena** demonstrates pairwise evaluation workflows for comparing LLM outputs that can be adapted for RAG system evaluation.
- **Microsoft Copilot** provides design patterns for lightweight user feedback with ratings, explanations, and contextual logging.

10.3 Creating Synthetic Data for Automated Testing

Problem

You need a set of question-answer pairs that are suitable for testing your RAG solution.

Solution

The fastest and most practical approach is to use an LLM to automatically generate synthetic question-answer pairs. **Figure 10-8** visualizes this process. Select random text chunks from your vector store, then prompt an LLM to generate questions that can be answered from those chunks. You can use single chunks or combine multiple chunks as context. This creates test data at scale without manual labeling.

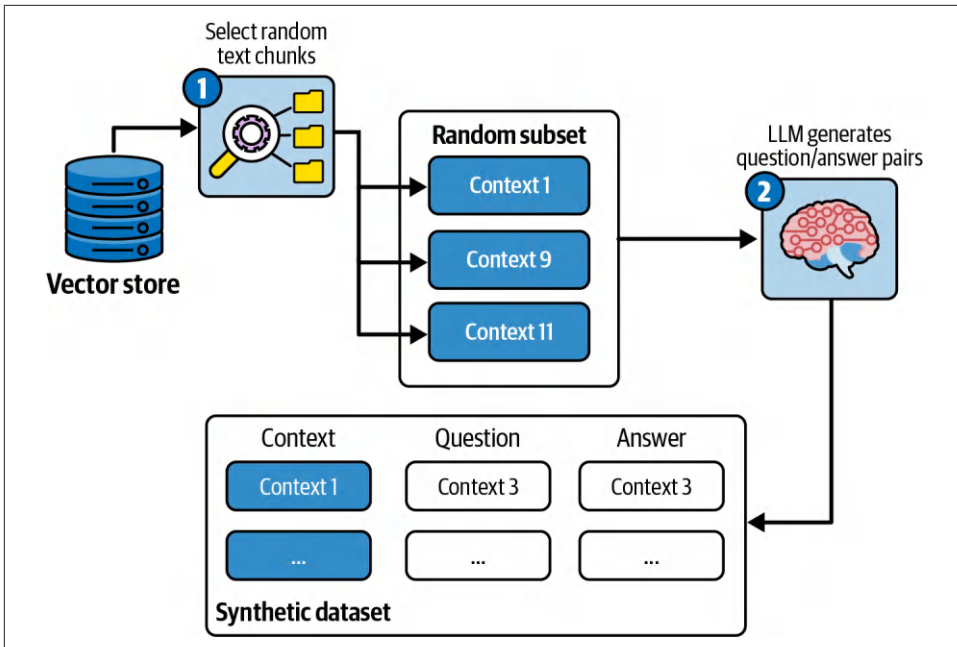


Figure 10-8. Creating a synthetic dataset

In this recipe, you'll use the RAG mini Wikipedia dataset from Hugging Face. The code snippets use the extracted text chunks from the Wikipedia articles so the LLM has context to generate synthetic questions and answers. In addition to the OpenAI SDK, pandas is used to manipulate and handle text data. Install the dependencies as follows:

```
pip install pandas openai pydantic
```

Start by loading the RAG mini Wikipedia dataset from Hugging Face to get a set of text chunks. You don't need to store these chunks in a vector store for this example; the snippet simply simulates that this content is part of your vector store. In other words, it represents the knowledge available to your RAG system's retriever:

```
# Example usage
import pandas as pd

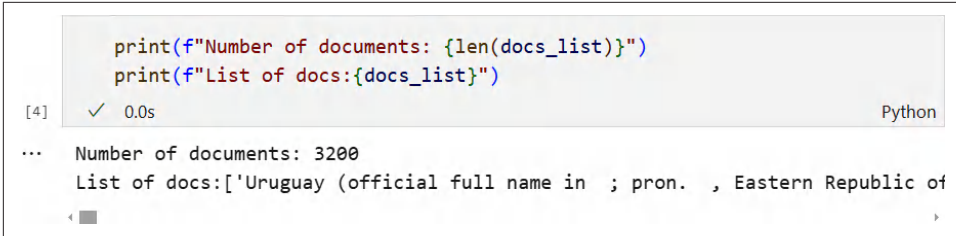
df = pd.read_parquet(
    "hf://datasets/rag-datasets/rag-mini-wikipedia/data/"
    "passages.parquet/part.0.parquet"
)

docs_list = df.passage.to_list()
```

If you check the dataset on Hugging Face, you'll notice it has two parts. The first is a raw text corpus from Wikipedia articles. The second is a set of prebuilt

question-answer pairs created from those Wikipedia snippets. Because this recipe shows how to automatically generate such question-answer pairs, you ignore the second part.

Figure 10-9 shows the loaded data structure from Hugging Face. You get a list of 3,200 strings. Each string is a text snippet from a Wikipedia article.



```
print(f"Number of documents: {len(docs_list)}")
print(f"List of docs:{docs_list}")
```

[4] ✓ 0.0s Python

```
... Number of documents: 3200
List of docs:['Uruguay (official full name in ; pron. , Eastern Republic of
```

Figure 10-9. Loaded data from Hugging Face

The next step is to randomly pick text chunks from the document list and instruct an LLM to generate realistic questions and answers based on the information in those chunks.

The prompt you use to instruct the LLM should have two parts:

- An instruction section telling the model what to do
- The context the model should use to generate questions and answers

In the instruction part, tell the model to think like a normal user and come up with natural queries that a real user would ask. The model should also generate an answer for each question. Because the question is derived from the context, the context should contain the answer. Here is the prompt, including all required components and a placeholder for the context:

```
from pydantic import BaseModel

QA_generation_prompt = """
Your task is to generate a question and its corresponding answer
based on the provided context.

The question should be:
- Direct and focused, seeking a specific factual piece of
  information present in the context.
- Formulated as a natural query a user might input into a
  search engine.
- Independent of the context, meaning it should not contain
  phrases like "according to the passage" or "in the context".

The answer should be:
- A concise and accurate response directly extracted or inferred
```

```

    from the context.

Present your output in the following format:

Output:
Question: (Your concise, fact-based question)
Answer: (The direct answer to the question)

Context: {context}\n
Output: ""

```

Next, define a response template as a Pydantic model. The returned Python object is a question-answer pair, with both fields represented as strings. The second part of the snippet then picks five random text chunks from the overall dataset, iterates over them, and generates question-answer pairs in the specified Pydantic format:

```

from openai import OpenAI
from pydantic import Field
import random

# Initialize the OpenAI client with your API key
client = OpenAI()

class QuestionAnswerPairs(BaseModel):
    question: str = Field(
        description="A factoid question about the context."
    )
    answer: str = Field(
        description="The answer to the factoid question."
    )

question_answer_pairs = []

random_text_chunks = random.sample(docs_list, 5)

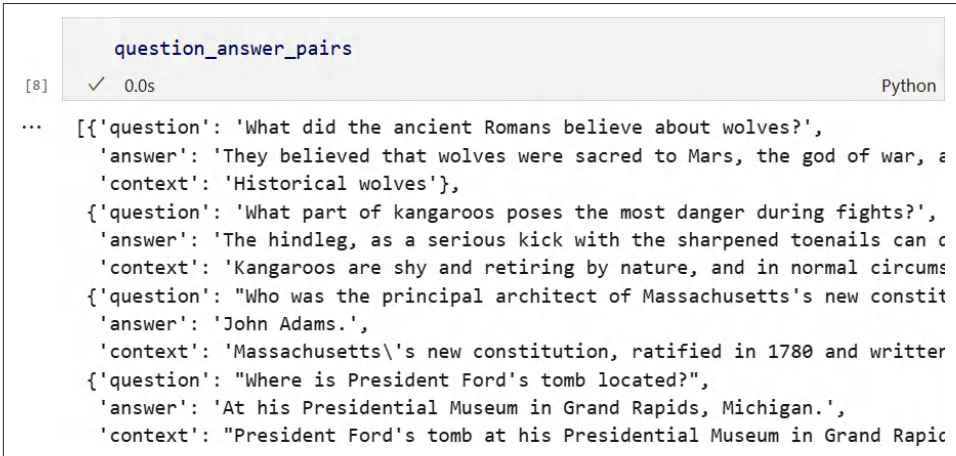
for text_chunk in random_text_chunks:
    # Generate hypothetical questions using the GPT-5.2 model
    completion = client.chat.completions.parse(
        messages=[
            {
                "role": "user",
                "content": QA_generation_prompt.format(context=text_chunk),
            }
        ],
        model="gpt-5.2",
        response_format=QuestionAnswerPairs,
    )

    question_answer_pair = (
        completion.choices[0].message.parsed.model_dump()
    )

```

```
question_answer_pair["context"] = text_chunk
question_answer_pairs.append(question_answer_pair)
```

Figure 10-10 shows the output. It contains five question-answer pairs for the randomly selected contexts. Each item is a dictionary containing the question, the answer, and the context used.



```
question_answer_pairs
```

```
[8] ✓ 0.0s Python
```

```
... [{"question": "What did the ancient Romans believe about wolves?",
      "answer": "They believed that wolves were sacred to Mars, the god of war, a",
      "context": "Historical wolves"},
     {"question": "What part of kangaroos poses the most danger during fights?",
      "answer": "The hindleg, as a serious kick with the sharpened toenails can c",
      "context": "Kangaroos are shy and retiring by nature, and in normal circum",
     {"question": "Who was the principal architect of Massachusetts's new constit",
      "answer": "John Adams.",
      "context": "Massachusetts\'s new constitution, ratified in 1780 and writer",
     {"question": "Where is President Ford's tomb located?",
      "answer": "At his Presidential Museum in Grand Rapids, Michigan.",
      "context": "President Ford's tomb at his Presidential Museum in Grand Rapi
```

Figure 10-10. Synthetic question-answer pairs

These questions can now be used to evaluate the retriever, the generator, and overall system performance. Because the questions were derived from the actual data, the RAG system should be able to find the answer in the existing content. If it cannot, that is a clear sign that something is wrong.

Discussion

LLM-generated questions are guaranteed answerable from the knowledge base because they derive directly from stored chunks. This creates a reliable baseline for testing system changes: if your RAG system can't answer synthetic questions, something is broken. The approach scales since you can generate hundreds of test cases automatically.

Use synthetic test generation for continuous testing, especially when your knowledge base changes frequently. It catches obvious failures after code changes such as model updates, prompt modifications, or parameter adjustments. Synthetic data measures technical correctness—whether the system retrieves relevant chunks and generates grounded answers. It's valuable in early development when you need fast feedback on basic functionality.

Don't use synthetic data as your only evaluation. Synthetic questions are systematically easier than real queries because they derive from clean, focused chunks. Real

users phrase questions ambiguously, combine intents, reference conversation context, or ask about edge cases. Synthetic questions don't capture query difficulty distribution—they test ideal conditions, not the long tail of ambiguous or poorly phrased questions.

Also avoid synthetic data for user experience dimensions like clarity, tone, or helpfulness. These require human judgment. Don't rely on it for multiturn conversations—generated questions are single-turn and miss conversation dynamics. Supplement with periodic analysis of production queries.

Generate at least 50–100 synthetic question-answer pairs to cover diverse topics in your knowledge base. Five examples, as shown in this recipe's code, are enough for prototyping but insufficient for reliable evaluation. Aim for coverage across the following:

- Different document types in your knowledge base (if you have articles, FAQs, and technical specs, sample from each)
- Various question complexities (factual lookups, comparisons, multihop reasoning)
- Edge cases near knowledge boundaries

Manually review a sample of 10–20 generated pairs to ensure quality. Check that questions sound natural and answers are accurate. If you find systematic issues—overly complex phrasing, incorrect answers, or trivial questions—adjust your generation prompt and regenerate.

Regenerate test data when you significantly update your knowledge base. Synthetic tests only catch issues if they reflect current content. If you add major new sections or restructure documents, old test questions may no longer align with retrieval patterns.

See Also

- [The Hugging Face RAG evaluation cookbook](#) demonstrates how to evaluate RAG systems by using synthetic datasets with step-by-step implementation examples.
- [The Ragas test set generation guide](#) provides techniques for generating synthetic questions and ground truth answers from your knowledge base.

10.4 Evaluating the Retriever Step by Calculating Context Precision@k

Problem

You want to evaluate the relevance of the retrieved text chunks by comparing them to the user's question.

Solution

To calculate context precision, follow these steps, as shown in **Figure 10-11**:

1. Pick a random question from your test dataset.
2. Let the question be answered by your RAG system. Capture the generated answer and the text chunks that were used.
3. Compare each retrieved text chunk to the answer and let an LLM decide whether the chunk was actually helpful in producing that answer or whether it was irrelevant.
4. Calculate context precision by dividing the number of relevant chunks by the total number of retrieved chunks. For example, if four chunks were included in the prompt and only two were relevant, the precision score is 0.5.

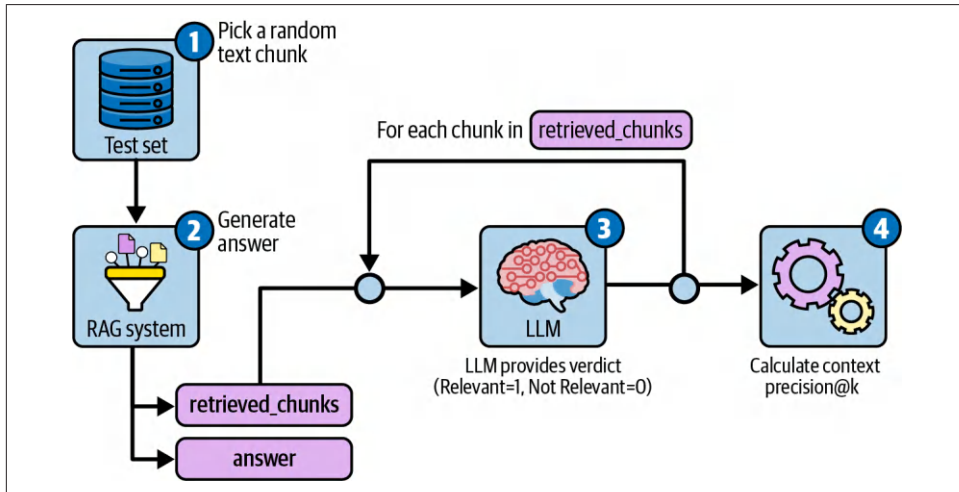


Figure 10-11. Calculating the context precision@k step by step

The following formula defines the context precision score. It calculates the proportion of the k retrieved text chunks that are actually helpful for generating the final answer.

$$\text{Context precision@k} = \frac{\sum_{k=1}^k (\text{precision@k} \times v_k)}{\text{Total number of relevant items in the top } k \text{ results}}$$

- k represents the total number of chunks in the retrieved_contexts.
- v_k is a binary flag indicating relevance at rank k (1 if the chunk is relevant, 0 otherwise).

To judge whether a text chunk was relevant for answering the question, several approaches exist.

One approach uses an LLM to compare the retrieved context to a reference context. In this case, you have a golden dataset that specifies, for each question, which context you expect to be retrieved and which context is relevant.

Another approach uses an LLM to compare the retrieved context to the generated answer. In this case, you don't need a golden reference dataset for the expected context. Instead, you compare the generated answer to the context and let the LLM decide whether any of the retrieved chunks were completely irrelevant to producing the answer. This provides an indication that some retrieved chunks were noise.

As a third approach, which is not LLM-based, you can use non-LLM methods to compare context with the answer or context with the question. Traditional techniques include string-similarity calculations or keyword-overlap metrics to estimate relevance without involving an LLM.

This recipe uses the second approach—using an LLM to compare retrieved context to the generated answer. Here's a simple example around the *Mona Lisa*:

Question

Who painted the *Mona Lisa*?

Context

The *Mona Lisa* is a 16th-century oil painting attributed to Italian polymath Leonardo da Vinci.

Answer

The *Mona Lisa* was painted by Leonardo da Vinci.

In this case, it's clear that the context helped the model produce a suitable answer. If you used k retrieved chunks and one chunk was about a completely different painting or a different artist, the LLM would likely not use that information. That chunk would be rated as not helpful (verdict = 0).

This recipe packages this concept into a few code snippets. To make the decision, the code needs access to an LLM, and you'll use one of OpenAI's models and the OpenAI SDK. You'll also use Pydantic to ensure that the model response contains only the following:

- A verdict (whether the context chunk was helpful)
- A reason (a short explanation as a string)

Install the required dependencies with the following command:

```
pip install openai pydantic pandas
```

The following code uses a sample question, answer, and retrieved context. Here's the sample data used for the following steps:

```
question = "What is the capital of France?"
answer = "The capital of France is Paris."
retrieved_contexts = [
    """France, a country in Western Europe, is known for
    its capital city, Paris, which is a major European city
    and a global center for art, fashion, and culture.""",
    """Paris is the capital city of France and is renowned
    for its historical landmarks such as the Eiffel Tower
    and the Louvre Museum.""",
    """The Amazon rainforest is the world's largest tropical
    rainforest, spanning across nine South American countries
    and supporting immense biodiversity."""
]
```

The next step evaluates how valuable each text chunk in the context was for answering the question.

The following code defines a Pydantic model called `Verification` that contains the following:

- `reason` (a string)
- `verdict` (an integer, either 1 or 0)

A verdict of 1 means the text chunk was helpful for the LLM; 0 means it was not.

For each text chunk, the code iterates over the retrieved chunks, sends a prompt, requests the response in the `Verification` format, and saves the results in a list of verifications:

```
from textwrap import dedent
from pydantic import BaseModel, Field
from openai import OpenAI

def context_relevance_verification(question, answer, retrieved_contexts):
    # Pydantic model for verification
    class Verification(BaseModel):
        reason: str = Field(..., description="Reason for verification")
        verdict: int = Field(
            ..., description="Binary (0/1) verdict of verification"
        )

    # Define the prompt for context relevance verification
    context_relevance_prompt = dedent(''
        Given a question, an answer, and a context, verify if the
        context was instrumental in deriving the given answer. Provide
        a detailed reason for your assessment and a binary verdict: "1"
        if the context is useful and "0" if it is not.
```

```

Input:
Question: "{question}",
Answer: "{answer}",
Context: "{context}"
'''

list_of_verifications = []

# Iterate through each question-answer pair and its context
for retrieved_context in retrieved_contexts:
    prompt = context_relevance_prompt.format(
        question=question,
        answer=answer,
        context=retrieved_context)

    client = OpenAI()
    completion = client.chat.completions.parse(
        messages=[
            {
                "role": "user",
                "content": prompt,
            }
        ],
        model="gpt-5.2",
        response_format=Verification,
    )

    verification = completion.choices[0].message.parsed.model_dump()
    verification["question"] = question
    verification["answer"] = answer
    verification["retrieved_context"] = retrieved_context

    list_of_verifications.append(verification)
return list_of_verifications

```

Figure 10-12 shows the output when applied to the sample data. In the sample, the user asks for the capital of France. The retrieved context consists of three text chunks: the first describes France and mentions its capital, the second describes Paris as the capital city, and the third discusses the Amazon rainforest—completely irrelevant to the question. The LLM assigns verdict values of 1 to the first two chunks and 0 to the irrelevant third chunk.

```

list_of_verifications = context_relevance_verification(
    question,
    answer,
    retrieved_contexts
)

list_of_verifications

```

[4] ✓ 7.4s Python

```

... [{"reason": "The context provides explicit and direct information that Paris is the capital of France",
      "verdict": 1,
      "question": "What is the capital of France?",
      "answer": "The capital of France is Paris.",
      "retrieved_context": "France, a country in Western Europe, is known for its capital city, Paris."},
     {"reason": "The context clearly mentions that Paris is the capital city of France",
      "verdict": 1,
      "question": "What is the capital of France?",
      "answer": "The capital of France is Paris.",
      "retrieved_context": "Paris is the capital city of France and is renowned for its culture, history, and landmarks."},
     {"reason": "The context provided focuses entirely on the Amazon rainforest and it does not mention Paris",
      "verdict": 0,
      "question": "What is the capital of France?",
      "answer": "The capital of France is Paris.",
      "retrieved_context": "The Amazon rainforest is the world's largest tropical rainforest, covering a vast area in South America."}]

```

Figure 10-12. Output of the context relevance verification

With a list of verdicts, you can now calculate context precision, which is the proportion of relevant chunks relative to the total number of retrieved chunks. Define a simple function that calculates this relevance score:

```

import pandas as pd

def calculate_context_relevance_score(list_of_verifications):
    verdict_list = pd.DataFrame(list_of_verifications)["verdict"].to_list()

    # Add small epsilon to avoid division by zero if no chunks are relevant
    denominator = sum(verdict_list) + 1e-10
    numerator = sum(
        [
            (sum(verdict_list[: i + 1]) / (i + 1)) * verdict_list[i]
            for i in range(len(verdict_list))
        ]
    )
    score = numerator / denominator # e.g., score = 0.835 for [1,0,1]
    return score

```

Discussion

Context precision evaluates whether your retriever surfaces relevant information. It treats retrieval as ranking: given a query, return chunks that help answer the question.

Irrelevant chunks waste tokens, increase cost, and confuse the generation model. The metric borrows from information retrieval because vector similarity search operates like traditional search engines—both rank documents by relevance. Metrics like precision@k, recall@k, and F1@k transfer naturally to RAG evaluation.

LLM-as-judge compares each retrieved chunk against the generated answer, determining whether that chunk contributed. This reveals retrieval noise—chunks that matched semantically but didn't provide useful information.

Use context precision when diagnosing retrieval problems. If users complain about wrong answers and you suspect the retriever isn't finding correct information, context precision reveals whether irrelevant chunks dominate. It's valuable when tuning retrieval parameters (top- k values, similarity thresholds, reranking strategies). Context precision matters when optimizing for cost and latency. Reducing k from 10 to 3 cuts token usage but only helps if those three chunks contain essential information. Track context precision while adjusting k .

Table 10-1 summarizes how to interpret context precision scores alongside answer quality.

Table 10-1. Interpreting context precision scores

Scenario	Interpretation
High precision + high answer quality	Retriever works well.
High precision + low answer quality	Generation problems (hallucination, poor reasoning).
Low precision + high answer quality	First chunk suffices, later chunks add noise—consider reducing k .
Low precision + low answer quality	Retrieval failure. Retrieved chunks don't contain needed information. Investigate embedding model, chunking strategy, or knowledge base content.

The following list provides an overview of common retrieval metrics, beyond context precision:

Precision@k

The fraction of top k results that are relevant

Recall@k

The fraction of relevant results retrieved in top k

F1@k

A balanced precision and recall score

Mean reciprocal rank (MRR)

How early the first relevant result appears

Average precision (AP)

Accounts for relevance and ranking position

Discounted cumulative gain (DCG@k)

Weights the relevant results higher when they appear earlier in the ranked results

Normalized discounted cumulative gain (NDCG@k)

Normalizes DCG for cross-query comparison

Use precision when you're concerned about irrelevant content. Use recall when you're worried about missing information. F1@k balances both but doesn't indicate which problem dominates. Ranking-aware metrics like MRR and NDCG@k matter when result order affects generation quality. If your model primarily uses the first few chunks, poor ranking hurts even if all relevant chunks exist.



ROUGE and BLEU metrics, commonly used in summarization and translation, aren't well suited for evaluating RAG retrievers. They measure n -gram overlap, which doesn't capture semantic relevance in the same way that precision and recall metrics do.

See Also

- [The Phoenix evaluation documentation](#) explains precision, recall, and F-score and provides guidance for evaluating retrieval and RAG workflows.
- [The Pinecone retrieval metrics guide](#) explains precision@k, recall@k, F1@k, MRR, AP, and NDCG@k with definitions and use cases.
- [Ragas context precision](#) implements context precision and other retrieval metrics with accessible source code.

10.5 Evaluating Faithfulness During Generation with LLM-as-a-Judge

Problem

You want to evaluate whether the RAG system response is grounded in the information returned by the retriever.

Solution

To calculate faithfulness, follow these steps, as shown in [Figure 10-13](#):

1. Use an LLM to break down the RAG system's answer into individual claims.
2. Iterate over all claims and check whether each claim is supported by any information in the retrieved context.

3. Calculate the faithfulness score as the proportion of supported claims divided by the total number of derived claims.

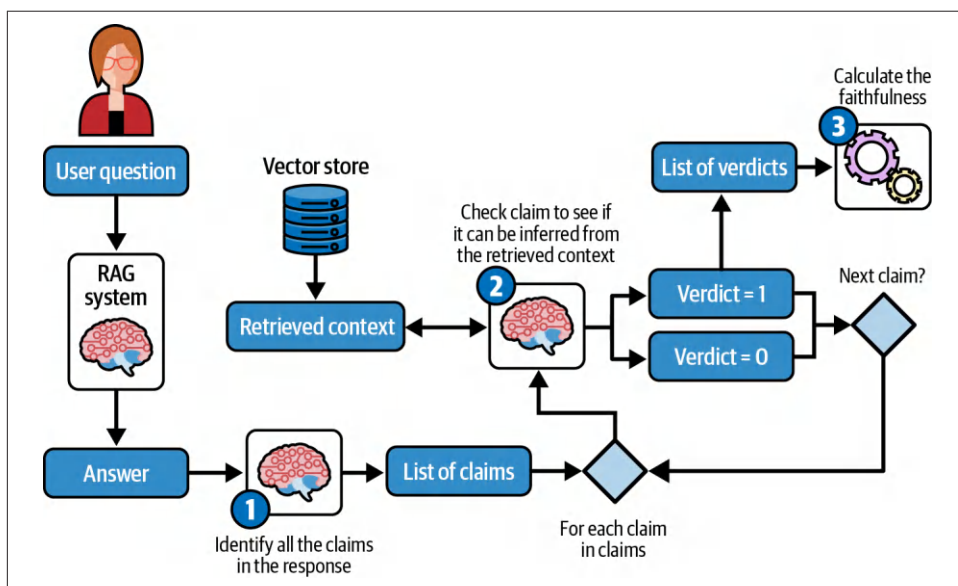


Figure 10-13. Calculating the faithfulness of a RAG system response

The first step provides the question and answer to the model and decomposes the answer into distinct claims.

The following is an example question and answer, and the statements an LLM would derive in this case:

Question

What are the main ingredients in a chocolate chip cookie?

Answer

They typically include flour, sugar, butter, eggs, and chocolate chips. Some recipes also call for vanilla extract or baking soda.

From this answer, the model derives standalone statements, seven in total:

- Chocolate chip cookies typically include flour.
- Chocolate chip cookies typically include sugar.
- Chocolate chip cookies typically include butter.
- Chocolate chip cookies typically include eggs.
- Chocolate chip cookies include chocolate chips.
- Some recipes call for vanilla extract.

- Some recipes call for baking soda.

Next, the process compares each claim with the retrieved context and determines which ones are supported. Claims not found in the context are considered unsupported.

The faithfulness score is calculated as follows:

$$\text{Faithfulness score} = \frac{\text{Number of claims in the response supported by the retrieved context}}{\text{Total number of claims in the response}}$$

For this recipe, you need access to an LLM. Here, it again uses an OpenAI model to break down answers into individual claims.

You'll also use the OpenAI SDK and the Pydantic library to define the model's response schema as a Pydantic model. Install both:

```
pip install openai pydantic pandas
```

Start by defining a function that instructs the model to decompose an answer into one or more fully understandable statements. The output is a Python object that follows a Pydantic model schema. In this case, it contains a `statements` attribute holding a list of strings:

```
from textwrap import dedent
from pydantic import BaseModel, Field
from openai import OpenAI

def decompose_answer(question, answer):
    # Initialize the OpenAI client
    client = OpenAI()

    statement_prompt = """
    Given a question and an answer, analyze the complexity of each
    sentence in the answer. Break down each sentence into one or more
    fully understandable statements. Ensure that no pronouns or
    ambiguous references are used in any statement. Output the
    decomposed statements as a list of strings.

    Question: {question}
    Answer: {answer}
    """

    prompt = statement_prompt.format(question=question, answer=answer)

    class Statements(BaseModel):
        """Structured response for statement extraction."""

        statements: list[str] = Field(
            description="List of decomposed statements."
        )

    completion = client.chat.completions.parse(
        messages=[
```



```

        {
            "role": "user",
            "content": prompt,
        }
    ],
    model="gpt-5.2",
    response_format=Statements,
)

statements = completion.choices[0].message.parsed.statements
return statements

```

Next, apply the function to a sample question/answer pair. The question is about the Amazon rainforest, and the answer describes it as one of the largest tropical rainforests, including its geographic extent:

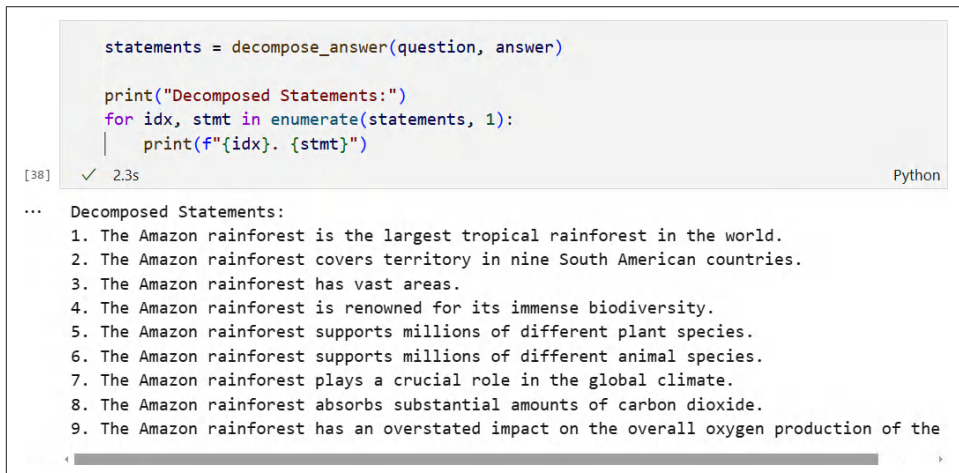
```

question = """Describe the geographic extent and ecological
significance of the Amazon rainforest."""

answer = dedent("""
The Amazon rainforest, the world's largest tropical rainforest, covers
significant territory across nine South American countries. This vast
area is renowned for its immense biodiversity, supporting millions
of different plant and animal species. While it plays a crucial role
in the global climate by absorbing substantial amounts of carbon dioxide,
its impact on the overall oxygen production of the Earth is often overstated.
""")

```

Figure 10-14 shows the output. In this example, nine statements have been extracted from the answer text.



```

statements = decompose_answer(question, answer)

print("Decomposed Statements:")
for idx, stmt in enumerate(statements, 1):
    print(f"{idx}. {stmt}")

```

[38] ✓ 2.3s Python

```

... Decomposed Statements:
1. The Amazon rainforest is the largest tropical rainforest in the world.
2. The Amazon rainforest covers territory in nine South American countries.
3. The Amazon rainforest has vast areas.
4. The Amazon rainforest is renowned for its immense biodiversity.
5. The Amazon rainforest supports millions of different plant species.
6. The Amazon rainforest supports millions of different animal species.
7. The Amazon rainforest plays a crucial role in the global climate.
8. The Amazon rainforest absorbs substantial amounts of carbon dioxide.
9. The Amazon rainforest has an overstated impact on the overall oxygen production of the

```

Figure 10-14. Decomposing an answer into claims

Then you compare each of those statements against the retrieved contexts. For each statement, you check whether it is directly supported by the information in the context provided to the LLM. This allows you to determine whether the answer is truly grounded in the retrieved documents or whether the model has introduced unsupported information.

You can use the LLM itself to perform these checks:

```
from pydantic import BaseModel, Field
from textwrap import dedent
from openai import OpenAI

def judge_faithfulness(statement, context):

    # Initialize the OpenAI client
    client = OpenAI()

    faithfulness_judge_prompt = dedent(
        """
        Your task is to judge the faithfulness of a statement based
        on the given context. You must return a verdict as 1 if the
        statement can be directly inferred from the context, or 0 if
        the statement cannot be directly inferred. Explain your
        reasoning.

        Context:
        {context}

        Statement:
        {statement}

        Answer::
        Reason: (Explain your reasoning)
        Verdict: (1 or 0)
        """
    )

    class StatementFaithfulness(BaseModel):
        """Structured response for statement extraction."""

        statement: str = Field(description="Decomposed statement.")
        reason: str = Field(
            description="Reasoning for the faithfulness judgment."
        )
        verdict: int = Field(
            description="1 if the statement is faithful, 0 otherwise."
        )

    prompt = faithfulness_judge_prompt.format(
        statement=statement, context=context
    )
```

```

completion = client.chat.completions.parse(
    messages=[
        {
            "role": "user",
            "content": prompt,
        }
    ],
    model="gpt-5.2",
    response_format=StatementFaithfulness,
)

statement_faithfulness = (
    completion.choices[0].message.parsed.model_dump()
)
return statement_faithfulness

```

Then run the previously defined faithfulness score judge:

```

statements_faithfulness = []

context = dedent(
    """
    The Amazon rainforest is the largest tropical rainforest in the
    world, covering parts of nine South American countries. It is
    known for its incredible biodiversity, housing millions of species
    of plants, insects, birds, and other animals. The forest plays a
    crucial role in regulating the Earth's climate by absorbing vast
    amounts of carbon dioxide.
    """
)

for statement in statements:
    statement_faithfulness = judge_faithfulness(
        statement=statement, context=context
    )
    # Append the statement faithfulness to the list
    statements_faithfulness.append(statement_faithfulness)

```

Figure 10-15 shows an example output. The code produces a list of dictionaries, each containing a statement, a verdict indicating whether the claim is supported by the context, and a brief explanation from the model in the `reason` field. The verdict shows which claims are grounded in the retrieved documents and why.

```

print("Number of statements:", len(statements_faithfulness))
statements_faithfulness

```

[42] ✓ 0.0s Python

```

... Number of statements: 9

... [{'statements': 'The Amazon rainforest is the largest tropical rainforest in the world.',
  'reason': 'The provided context explicitly states that the Amazon rainforest is the larg
  'verdict': 1},
 {'statements': 'The Amazon rainforest covers territory in nine South American countries.'
  'reason': 'The context mentions that the Amazon rainforest covers parts of nine South Am
  'verdict': 1},
 {'statements': 'The Amazon rainforest has vast areas.',
  'reason': 'The context specifies that the Amazon rainforest is the largest tropical rain
  'verdict': 1},
 {'statements': 'The Amazon rainforest is renowned for its immense biodiversity.',
  'reason': 'The statement claims that the Amazon rainforest is noted for its extensive bi
  'verdict': 1},
 {'statements': 'The Amazon rainforest supports millions of different plant species.',
  'reason': "The context mentions that the Amazon rainforest houses millions of species of
  'verdict': 0},
 {'statements': 'The Amazon rainforest supports millions of different animal species.'

```

Figure 10-15. Using the LLM to evaluate faithfulness

Finally, you use these verdicts to calculate the final faithfulness score. The faithfulness score is the ratio of supported claims to the total number of claims in the answer. Higher scores indicate that the RAG system stays grounded and provides information without adding unsupported claims. Here's the code:

```

import pandas as pd

statements_faith_df = pd.DataFrame(statements_faithfulness)

number_of_claims = len(statements_faith_df["verdict"]) # 7
number_of_claims_in_context = statements_faith_df["verdict"].sum() # 6
faithfulness_percentage = (
    number_of_claims_in_context / number_of_claims
) * 100

```

Aim for faithfulness scores above 80% in production systems. This indicates that the RAG system reliably grounds its answers in the retrieved context. Scores below 70% suggest potential hallucination issues, where the system generates claims that are not supported by the retrieved documents.

This 80% threshold balances reliability and usefulness. Scores of 85% or higher indicate strong grounding—the system rarely invents information. Scores of 70%–80% warrant investigation: review failed cases to identify whether issues are systematic (e.g., hallucinations on specific topics) or random. If you're consistently below 70%, consider adjusting your generation prompt to emphasize grounding, lowering temperature, or switching to a model with better instruction-following.

Discussion

LLM-as-judge fits well for the faithfulness metric because the task is clear: determine whether a claim is supported by text, requiring only reading comprehension rather than world knowledge.

Use faithfulness to detect hallucinations in high-stakes domains (medical, financial, legal) where unsupported claims create liability risk. Faithfulness also evaluates changes to the generation step—if you switch models, adjust prompts, or modify temperature, this metric reveals whether the new configuration stays grounded.

High faithfulness doesn't guarantee good answers. The system might return grounded but irrelevant information. Pair faithfulness with answer relevancy (does the answer address the question?) and context precision (did we retrieve the right information?). These three metrics move independently: improving retrieval doesn't guarantee better faithfulness if the model still hallucinates, and perfect faithfulness doesn't prevent low answer relevancy if the model focuses on wrong aspects of the context.

The trade-off is faithfulness versus informativeness. High faithfulness models are conservative, stating only what appears explicitly in context. For most RAG applications, prioritize faithfulness—users expect reliable information retrieval. Target scores above 80% in production; scores below 70% indicate frequent unsupported claims.

Understanding Framework Implementations

The most reliable way to understand how evaluation frameworks implement faithfulness is to inspect their source code. In VS Code, you can jump directly into the implementation by holding Ctrl and clicking the faithfulness function imported from Ragas, as shown in [Figure 10-16](#).



Figure 10-16. Browsing the framework code

By navigating through the code, you'll eventually find a class called `StatementGeneratorPrompt`. [Figure 10-17](#) shows how it's defined and provides insight into how these frameworks define their evaluation metrics.

```

34 class StatementGeneratorPrompt(
35     PydanticPrompt[StatementGeneratorInput, StatementGeneratorOutput]
36 ):
37     instruction = "Given a question and an answer, analyze the complexity of each
38     input_model = StatementGeneratorInput
39     output_model = StatementGeneratorOutput
40     examples = [
41         (
42             StatementGeneratorInput(
43                 question="Who was Albert Einstein and what is he best known for?",
44                 answer="He was a German-born theoretical physicist, widely acknowl
45             ),
46             StatementGeneratorOutput(
47                 statements=[
48                     "Albert Einstein was a German-born theoretical physicist.",
49                     "Albert Einstein is recognized as one of the greatest and most

```

Figure 10-17. The statement generator class in the Ragas library

See Also

- The [Ragas faithfulness metric](#) provides a detailed explanation and implementation of a faithfulness evaluation for RAG systems.
- The [OpenEvals repository](#) offers predefined evaluation prompts for metrics including correctness, conciseness, and groundedness.
- The [LangSmith evaluation guide](#) demonstrates how to evaluate RAG applications by using LangSmith and OpenEvals.

10.6 Evaluating the Response Relevancy of Your RAG System

Problem

You want an automated way to measure how well your RAG system's final answer addresses the user's original question.

Solution

The response relevancy score is calculated in five steps:

1. Run the user's question through your RAG system to generate a response.
2. Use an LLM to analyze the answer and derive a set of artificial questions that could be asked to elicit the same content reflected in the answer.

3. Generate embeddings for the original user question and each artificial question.
4. Compute the cosine similarity between the embedding of the original user question E_o and the embedding of each artificial question E_{g_i} .
5. Calculate the average of the cosine similarity scores across all artificial questions to get the final relevancy score.

The formula for answer relevancy is as follows:

$$\text{Answer relevancy} = \frac{1}{N} \sum_{i=1}^N \text{cosine similarity}(E_{g_i}, E_o)$$

where:

- E_{g_i} is the embedding of the i^{th} generated question.
- E_o is the embedding of the original user question.
- N is the number of generated questions (the default is 3).

Figure 10-18 visualizes the process for calculating answer relevancy in RAG systems.

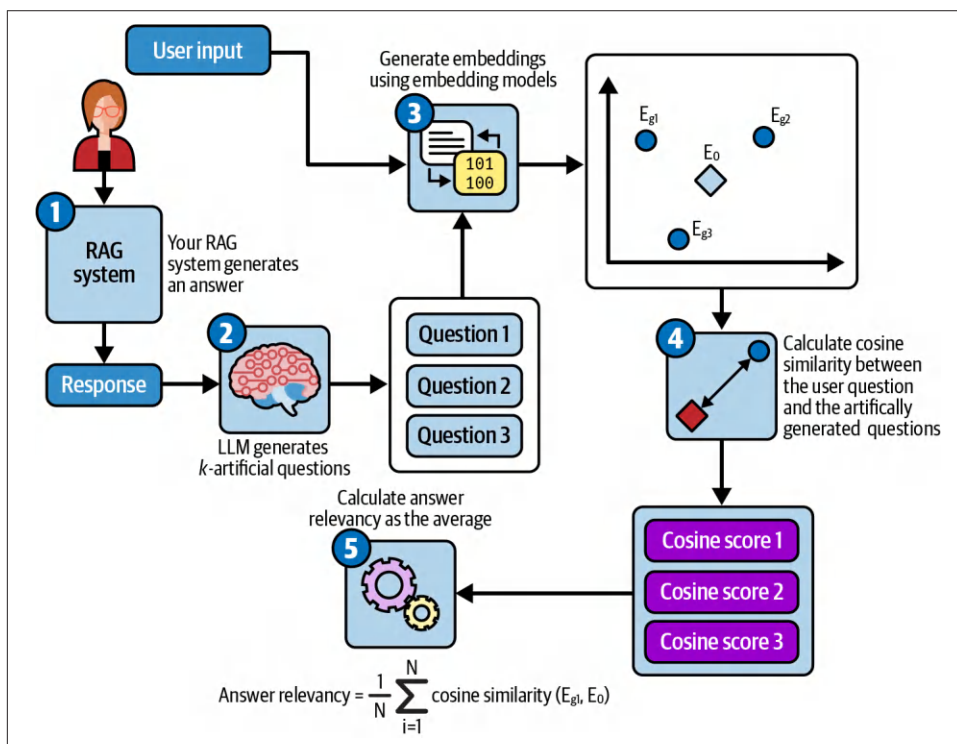


Figure 10-18. The process for calculating answer relevancy in RAG systems

For the technical implementation in this recipe, you need an embedding model and an LLM. In this example, both come from OpenAI and are accessed using the OpenAI SDK. To ensure that the artificially generated questions are returned as a list of strings, define a Pydantic model. You'll also use NumPy to calculate cosine similarity between embedding vectors. Install the required Python packages:

```
pip install openai pydantic numpy pandas
```

In the first two steps, you use an LLM to generate artificial questions derived from the RAG system's answer and rate each question as committal or noncommittal.

A question is noncommittal if the answer doesn't provide relevant, valuable information for it. A noncommittal value of 0 is good, because it means the answer contains relevant information for the question.

Table 10-2 shows some sample questions and responses with their noncommittal scores.

Table 10-2. Examples of noncommittal question assessment

Generated question	Response	Noncommittal score	Explanation
What is the capital of France?	The capital of France is Paris.	0	The answer directly addresses the question with specific information.
How are you feeling today?	As a large language model, I'm not capable of experiencing emotions.	1	The answer acknowledges the question but provides no relevant information.
What is the current price of Tesla stock?	I can't provide real-time stock prices.	1	The answer does not provide the requested information.

Here, you'll perform both steps in a single run. This code takes the answer, generates suitable questions based on the answer, and then assesses whether the answer is relevant and helpful in addressing each generated question:

```
from textwrap import dedent

response_relevance_prompt = dedent("""
    Generate three relevant questions for the following answer
    and indicate if the answer is noncommittal.

    Output:
    Question: [Generated Question]
    Noncommittal (1=Yes, 0=No): [0 or 1]

    An answer is considered noncommittal if it is evasive, vague,
    or ambiguous (e.g., "I don't know," "Maybe," "It depends").

    Answer: {response}
    """)
```


Define a Pydantic model to ensure that the code reliably returns a list of dictionaries in a consistent format. The model defines a `GenerateQuestions` class that contains a list of items; each item has a question and a noncommittal score, which gets the value 1 if the answer is noncommittal, 0 if it is committal:

```
from pydantic import BaseModel, Field

prompt = response_relevance_prompt.format(response=response)

class GeneratedQuestion(BaseModel):
    question: str = Field(description="Generated question.")
    noncommittal: int = Field(
        description="1 if the answer is noncommittal, 0 otherwise."
    )

class GeneratedQuestions(BaseModel):
    questions: list[GeneratedQuestion] = Field(
        description="List of generated questions and noncommittal status."
    )

client = OpenAI()

completion = client.chat.completions.parse(
    messages=[
        {
            "role": "user",
            "content": prompt,
        }
    ],
    model="gpt-5.2",
    response_format=GeneratedQuestions,
)

generated_questions = (
    completion.choices[0].message.parsed.model_dump()["questions"]
)
```

Next, define a function to create embeddings using an OpenAI embedding model:

```
from openai import OpenAI

client = OpenAI()

def create_embeddings(text_chunk, client):
    embed_model = "text-embedding-3-small"
    embedding = (
        client.embeddings.create(input=[text_chunk], model=embed_model)
        .data[0]
        .embedding
    )
    return embedding
```



A cosine similarity score close to 1 indicates that the two vectors are identical or point in the same direction, meaning the semantic content of both questions is very similar. Mathematically, the score can be negative, but this is rare with modern embedding models unless the texts are intentionally dissimilar or the embeddings are poor.

Next, use the previously defined function to generate embeddings for the artificial questions and the original user query.

In this example, the user asks for the capital of France, and the system answers that the capital of France is Paris.

Then calculate the cosine similarity between each artificial question and the user's question. This adds new attributes to each question object (or dictionary), such as the embedding vector and the computed cosine similarity:

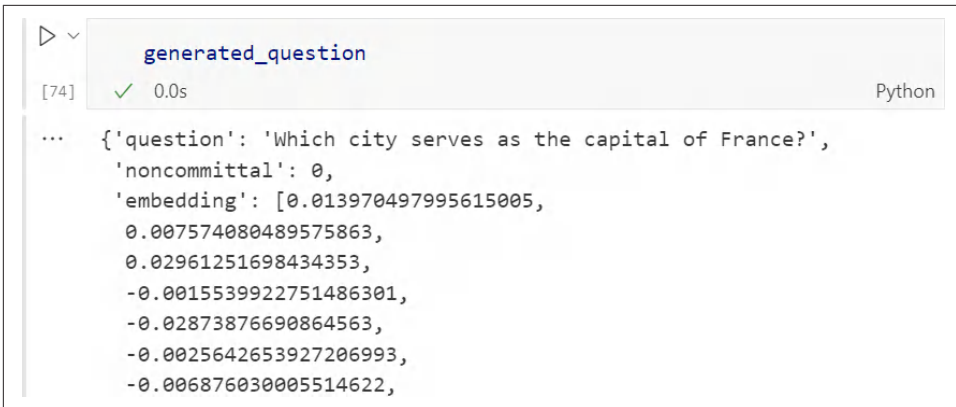
```
import numpy as np
import pandas as pd

user_question_embedding = create_embeddings(user_input, client)
cosine_sims = []

for generated_question in generated_questions:
    generated_question["embedding"] = create_embeddings(
        generated_question["question"], client
    )

    generated_question["cosine_sim"] = np.dot(
        user_question_embedding, generated_question["embedding"]
    ) / (
        np.linalg.norm(user_question_embedding)
        * np.linalg.norm(generated_question["embedding"])
    )
```

Figure 10-19 shows the example structure of a generated question object with embedding and cosine similarity.

A screenshot of a Jupyter Notebook cell. The cell is labeled '[74]' and has a green checkmark and '0.0s' indicating successful execution. The code in the cell prints a dictionary representing a generated question object. The dictionary has three keys: 'question', 'noncommittal', and 'embedding'. The 'question' value is 'Which city serves as the capital of France?'. The 'noncommittal' value is 0. The 'embedding' value is a list of 11 floating-point numbers.

```
generated_question

... {'question': 'Which city serves as the capital of France?',
     'noncommittal': 0,
     'embedding': [0.013970497995615005,
                   0.007574080489575863,
                   0.02961251698434353,
                   -0.0015539922751486301,
                   -0.02873876690864563,
                   -0.0025642653927206993,
                   -0.006876030005514622,
```

Figure 10-19. The structure of a generated question object with embedding and cosine similarity

The final step uses the cosine similarity score of each question to calculate the overall response relevancy score. The logic penalizes evasive answers: if any generated question was rated as noncommittal (meaning the answer doesn't provide useful information for that question), the overall score is set to 0. Otherwise, the score is the mean cosine similarity between all artificial questions and the user's question. Here's the code:

```
# Calculate the mean cosine similarity over all generated questions
cosine_sim_mean = pd.DataFrame(generated_questions)["cosine_sim"].mean()

# Check if any of the generated questions are noncommittal
has_noncommittal = any(pd.DataFrame(generated_questions)["noncommittal"])

# If any generated question is noncommittal, set score to 0
# This penalizes evasive answers that acknowledge but don't address questions
response_relevance_score = cosine_sim_mean * int(not has_noncommittal)
```

This strict approach treats any noncommittal response as complete failure. If you prefer a softer penalty that reduces the score proportionally instead of zeroing it out, use `response_relevance_score = cosine_sim_mean * (1 - noncommittal_ratio)`, where `noncommittal_ratio` is the fraction of generated questions marked noncommittal.

A response relevancy score of 0.87 is quite high. It indicates that the automatically generated questions are highly similar to the original user question.

Figure 10-20 shows the final response relevancy score.

```
[31] ✓ 0.0s Python
... np.float64(0.8739105774532437)
```

Figure 10-20. The final response relevancy score

Discussion

Answer relevancy uses reverse question generation to evaluate alignment. If the LLM generates questions from an answer, those questions should resemble the original—if the answer addresses what was asked. High semantic similarity indicates alignment; low similarity reveals misdirection. The noncommittal check catches evasive answers that acknowledge questions without providing useful information.

Use answer relevancy when diagnosing why users find technically correct answers unsatisfying. High faithfulness with low satisfaction often indicates relevancy problems—the system provides grounded facts that don’t address user intent. The metric also evaluates generation prompt changes.

The metric detects off-topic answers but not incomplete ones. A multipart question where only one part gets answered may still score high relevancy. Pair with correctness metrics or human evaluation for comprehensive assessment.

Faithfulness asks, “Does the answer stick to retrieved facts?” Answer relevancy asks, “Does the answer address the question?” Both are necessary: faithfulness ensures reliability, and answer relevancy ensures usefulness.

See Also

- **Ragas response relevancy** implements answer relevancy evaluation with detailed metric definitions and code examples.
- **The DeepEval answer relevancy metric** provides response relevancy evaluation along with faithfulness and contextual precision metrics.
- **The OpenEvals correctness metric** offers prompts for evaluating answer correctness to complement relevancy assessment.

RAG Web Apps

Most generative AI projects start as experiments, and most never reach production. Fast prototyping matters, especially when your team lacks full-stack engineering support. You need a setup that lets data scientists build RAG-powered apps that real users can test before committing to production infrastructure.

Streamlit is a Python framework for rapid web app development. It lets you build functional interfaces with a single script that covers the frontend and backend. The framework provides prebuilt components you can assemble without low-level web development, though you're limited to Streamlit's current offerings.

Streamlit reruns your entire script on every user interaction. This simplifies development but creates overhead that doesn't scale beyond a few dozen concurrent users. For validating whether a GenAI app provides value, this trade-off works in your favor.

For production applications with thousands or millions of users, alternative frameworks provide better scalability and performance. Django offers full-featured capabilities with authentication and complex workflows. Flask provides a lightweight and flexible foundation. FastAPI is optimized for API endpoints.

The recipes in this chapter build progressively. You'll start by creating a basic Streamlit app to understand the framework's core mechanics, then build a working RAG chatbot, and finally extend it to handle document uploads and queries.



In the early days of RAG, many applications defaulted to chatbots even when simpler interfaces would have worked better. Dashboards and search interfaces are often faster and more intuitive. Before choosing a chat interface, consider whether your use case actually benefits from conversational interaction.

You can find all the code examples for this chapter in the [book's GitHub repository](#).

11.1 Building Your First Streamlit App

Problem

You're not a web developer, but you still want to quickly turn your idea into a web app and deploy it.

Solution

Streamlit is a Python web framework that lets you build a fully functioning web app with a single script and a few lines of code. Install Streamlit to follow this recipe:

```
pip install streamlit
```

Build a basic Streamlit app that shows a title, a text input, and a section that prints the user's input to the screen:

```
import streamlit as st

st.title("Simple Chatbot")
user_input = st.text_input("You:")
if user_input:
    st.write(f"Bot: You said '{user_input}'!")
```

To run the app, execute the following command in the terminal:

```
streamlit run streamlit_app.py
```

This command starts a local web server that hosts the app. Unless you configure it differently, Streamlit uses port 8501 by default, so you can open the app at <http://localhost:8501>.

Figure 11-1 shows what the app looks like.

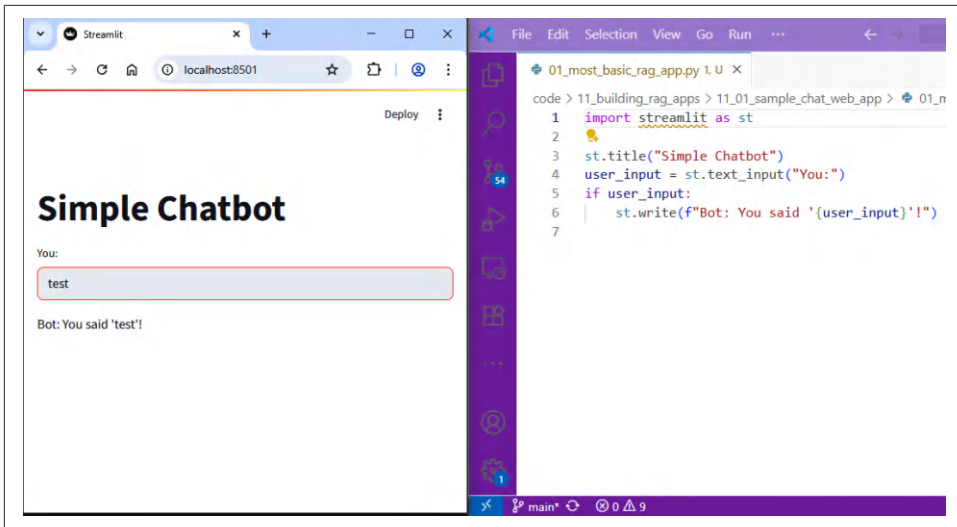


Figure 11-1. The most basic Streamlit app

This example demonstrates Streamlit’s core mechanics. The following recipes show you how to build more advanced RAG-specific features, including conversation history, session state management, and integration with external APIs.

Discussion

Streamlit’s simplicity is its main advantage. The script rerun model eliminates session handling, routing, and frontend-backend coordination. You write Python code that looks like a regular script, and Streamlit automatically handles the web interface. This makes state management trivial for prototypes.

The trade-off is scalability. Every user interaction reruns the entire script from top to bottom, creating CPU overhead for each user. Beyond 50 concurrent users, this model becomes a bottleneck.

Use Streamlit for internal tools, proof-of-concept demos, and rapid iteration during requirements gathering. Don’t use it for external apps with thousands of users (Fast-API with React scales better), apps requiring custom branding (Django offers more UI control), or apps needing fine-grained authentication (FastAPI integrates authentication middleware more naturally).

The critical trade-off is development speed versus scalability. You can prototype a working RAG app in hours, but refactoring to production infrastructure later takes weeks. For experiments that will likely not reach production, Streamlit is the better choice because it minimizes up-front effort.

You can extend Streamlit's reach by offloading compute-heavy tasks to microservices. Keep Streamlit as a lightweight frontend while delegating authentication, ML inference, or database operations to dedicated APIs. This hybrid approach works for 100–500 users before requiring a full rewrite.

See Also

- [The Streamlit documentation](#) provides comprehensive coverage of all framework concepts and components.
- [The Streamlit GitHub repository](#) contains source code and community examples.

11.2 Building a Chatbot App with Streamlit

Problem

You have designed a RAG workflow and want to integrate it into a chatbot app.

Solution

Streamlit provides features for building conversational UIs. In this recipe, you build a chatbot that lets users ask questions about the weather. The chatbot uses an external API to retrieve weather information and an LLM to interpret that data and answer the user's question.

After the user types a question into the Streamlit chat interface, the app performs the following steps ([Figure 11-2](#)):

1. After the user sends the request, the first step in the RAG workflow extracts the location from the user's question, like a mentioned city name or country.
2. An external weather API is called with the extracted location and returns today's weather information.
3. The weather information is sent back to the LLM, which interprets the user's question together with the provided context and formulates a response.

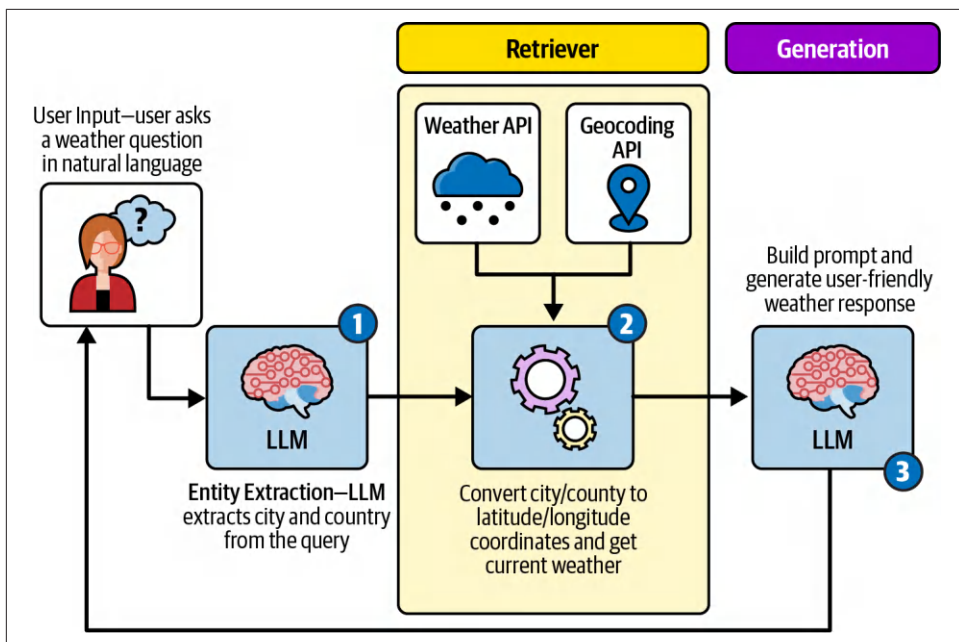


Figure 11-2. Process behind the RAG weather app

This recipe uses the Streamlit library to build the web app and the Requests library to perform calls to the external weather API. Because users typically ask for a city or country, but the API requires latitude and longitude, the app uses GeoPy to convert a place name into coordinates. Install these libraries as follows:

```
pip install streamlit openai requests geopy
```

Before you build and run the app, create a Python virtual environment by using the venv package. Virtual environments isolate your project dependencies, preventing conflicts with other Python projects on your system and making it easier to reproduce your setup on different machines.

Then create the initial frame of a basic chat interface with Streamlit. It's only a few lines, but it includes the key function for printing messages to the screen. Using Streamlit's chat message function and labeling each message as either user or assistant will automatically format the messages differently:

```
import streamlit as st

with st.chat_message("user"):
    st.write("Hello, I am a user message.")

with st.chat_message("assistant"):
    st.write("Hello, I am an assistant message.")
```

To run the app, use the following command in your app directory:

```
streamlit run my_app.py
```

Figure 11-3 shows how the basic formatting looks.

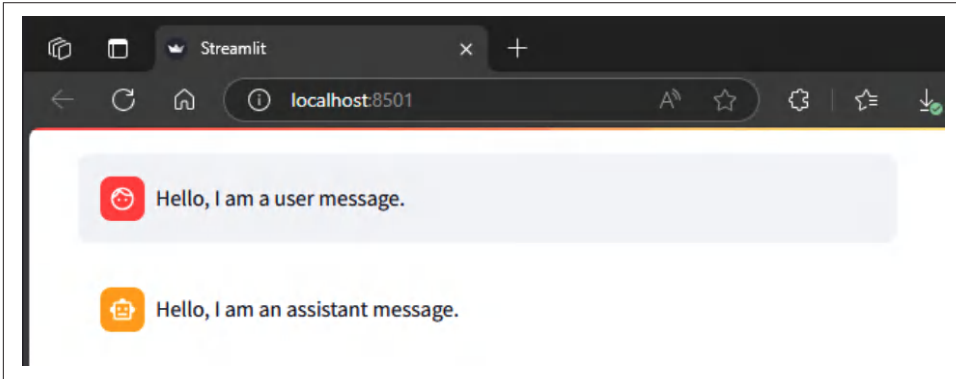


Figure 11-3. A basic Streamlit chat app

The next step is to implement logic that accepts user input and maintains chat history by using Streamlit session state. The code template for a basic chatbot interface stores the conversation in a messages list.

Each entry in `messages` is a dictionary with the keys `role` and `content`. The `role` is either `user` or `assistant`. The `content` is typically the user's question or the assistant's response. By continuously appending to `messages`, you build the session history of all user inputs and assistant replies.

You also use `st.chat_message` to alternately render the user's prompt and the assistant's response:

```
import streamlit as st

st.title("Simple RAG Chat App")

if "messages" not in st.session_state:
    st.session_state.messages = []

def simple_rag(query):
    if "hello" in query.lower():
        return "Hello! How can I help you?"
    elif "rag" in query.lower():
        return (
            "RAG combines retrieval and generation for better "
            "responses."
        )
    else:
```

```

    return (
        f"You asked: '{query}'. A real RAG system would "
        f"search documents and generate a response."
    )

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

if prompt := st.chat_input("Ask a question"):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    response = simple_rag(prompt)
    st.session_state.messages.append(
        {"role": "assistant", "content": response}
    )

    with st.chat_message("assistant"):
        st.markdown(response)

```

At this stage, the app logic is still a dummy function that contains some if/else logic. Next, you replace that dummy with the logic of a real RAG system.

In the basic RAG system built in this recipe, the retriever is represented by two API calls:

1. The first API call retrieves the latitude and longitude for a given location.
2. The second API call uses those coordinates to retrieve the weather for that location.

Before calling these APIs, you use an LLM to examine the user's query and determine whether it contains a location. A location is required for the weather API; without it, the question cannot be answered:

```

import openai
import json

def extract_the_city_and_country(user_query):
    client = openai.OpenAI()

    prompt = (
        f"Extract the city and country from: '{user_query}'. "
        "Return as JSON with 'city' and 'country' keys."
    )

    response = client.chat.completions.create(
        model="gpt-5.2",

```

```

        messages=[{"role": "user", "content": prompt}],
        response_format={"type": "json_object"},
    )

    try:
        return json.loads(response.choices[0].message.content)
    except:
        return None

```

The output looks like [Figure 11-4](#). The response follows the requested JSON structure: a dictionary with city and country keys.

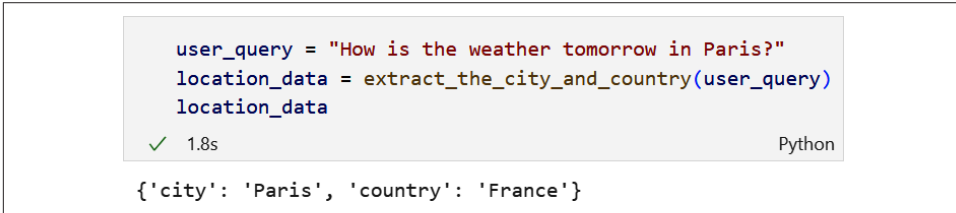


Figure 11-4. Extracting the city and country from the user query

Next, use the Nominatim API to resolve the city or country into exact coordinates:

```

from geopy.geocoders import Nominatim

def get_coordinates_for_city(city, country):
    geolocator = Nominatim(user_agent="city_coordinate_finder")
    location = geolocator.geocode(f"{city}, {country}")

    if location:
        return location.latitude, location.longitude
    else:
        print(f"Could not find coordinates for '{city}, {country}'.")
        return None, None

```

The output in [Figure 11-5](#) shows the latitude and longitude for the location; in this case, it's the coordinates for Paris.

```
latitude, longitude = get_coordinates_for_city(location_data["city"], location_data["country"])

print(f"Coordinates for {location_data['city']}, {location_data['country']}:")
latitude, longitude

✓ 1.5s Python

Coordinates for Paris, France:

(48.8588897, 2.320041)
```

Figure 11-5. The calculated geo location for city and country

Next, call the Open-Meteo API to fetch the current weather data for the given coordinates and return the full API response:

import requests

```
def get_current_weather_open_meteo(latitude, longitude):
    """
    Fetches current weather data from Open-Meteo for a given location.
    """
    base_url = "https://api.open-meteo.com/v1/forecast"
    params = {
        "latitude": latitude,
        "longitude": longitude,
        "current": (
            "temperature_2m,relative_humidity_2m,precipitation,"
            "weather_code,wind_speed_10m"
        ),
        # Automatically determine timezone based on location
        "timezone": "auto",
    }

    try:
        response = requests.get(base_url, params=params)
        # Raise HTTPError for bad responses (4xx/5xx)
        response.raise_for_status()
        weather_data = response.json()
        return weather_data
    except requests.exceptions.RequestException as e:
        print(f"Error fetching weather data: {e}")
        return None
```

Figure 11-6 shows the returned dictionary. It includes not only the temperature but also the humidity, time zone, elevation, and more.

```
weather_data = get_current_weather_open_meteo(latitude, longitude)
weather_data
```

✓ 21.2s

Python

```
{'latitude': 48.86,
 'longitude': 2.3199997,
 'generationtime_ms': 0.06663799285888672,
 'utc_offset_seconds': 7200,
 'timezone': 'Europe/Paris',
 'timezone_abbreviation': 'GMT+2',
 'elevation': 38.0,
 'current_units': {'time': 'iso8601',
 'interval': 'seconds',
 'temperature 2m': '°C'.
```

Figure 11-6. Weather data retrieved from the Open-Meteo API

The next step feeds the weather information into a prompt sent to the LLM. The model interprets the API results together with the user's question and generates a helpful answer.

In this example, you use a basic prompt template that includes the user's question and the weather data and asks the model to summarize the weather for the user:

```
import openai

def create_weather_prompt(user_query, weather_data):
    if not weather_data:
        return "No weather data available."

    prompt = (
        "You are a helpful assistant. Based on the weather "
        "data below, answer the user's question in a clear "
        "and simple way.\n\n"
        f"User question: {user_query}\n\n"
        f"Weather data (JSON):\n{weather_data}\n"
        "Summarize the weather information so that it is "
        "easy for anyone to understand."
    )
    return prompt

def send_prompt_to_llm(prompt):
    client = openai.OpenAI()

    response = client.chat.completions.create(
        model="gpt-5.2",
        messages=[
```

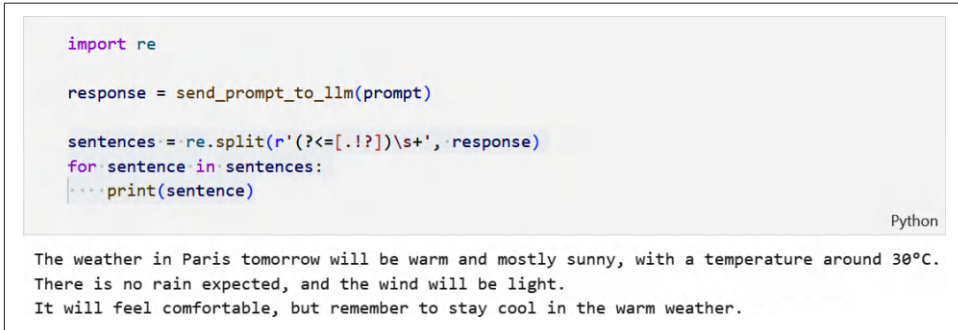
```

        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": prompt},
    ],
)

return response.choices[0].message.content.strip()

```

Figure 11-7 shows the final response generated by the LLM.



```

import re

response = send_prompt_to_llm(prompt)

sentences = re.split(r'(?<=[.!?])\s+', response)
for sentence in sentences:
    print(sentence)

```

Python

The weather in Paris tomorrow will be warm and mostly sunny, with a temperature around 30°C. There is no rain expected, and the wind will be light. It will feel comfortable, but remember to stay cool in the warm weather.

Figure 11-7. The final RAG weather app response

Finally, combine the previously defined functions into a single RAG workflow that the Streamlit app can call whenever needed. In this recipe, the helper functions are moved into a separate Python module `app_helper_functions` to keep the main app file clean and make the RAG flow easier to follow. The following code shows the `perform_rag` function that you should add to the `app_helper_functions` module:

```

# In app_helper_functions.py

def perform_rag(user_query):
    location_data = extract_the_city_and_country(user_query)
    latitude, longitude = get_coordinates_for_city(
        location_data["city"], location_data["country"]
    )
    weather_data = get_current_weather_open_meteo(latitude, longitude)
    prompt = create_weather_prompt(user_query, weather_data)
    response = send_prompt_to_llm(prompt)
    return response

```

Now integrate the `perform_rag` function into the Streamlit chat app. Here's the complete application that combines the RAG workflow with the Streamlit interface:

```

import streamlit as st
from app_helper_functions import perform_rag

st.title("Weather RAG Chat App")

if "messages" not in st.session_state:
    st.session_state.messages = []

```

```

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

if prompt := st.chat_input("Ask about the weather"):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    response = perform_rag(prompt)
    st.session_state.messages.append(
        {"role": "assistant", "content": response}
    )

    with st.chat_message("assistant"):
        st.markdown(response)

```

Figure 11-8 shows the app when asking a few sample questions—for example, about the weather in Berlin or Rome.

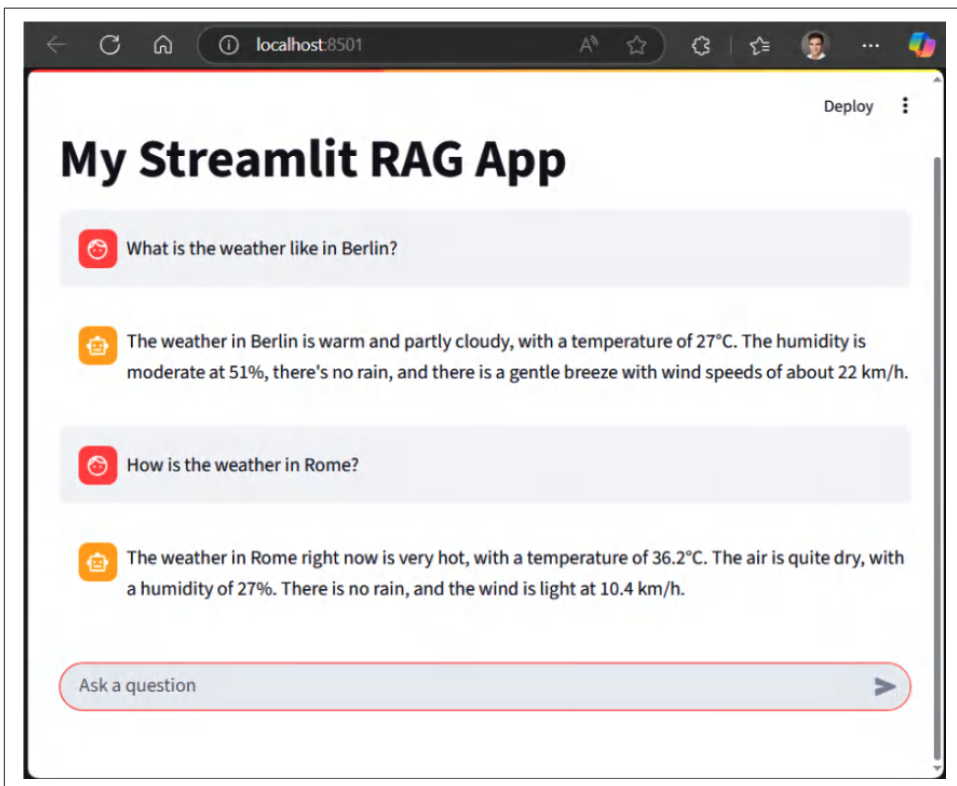


Figure 11-8. The RAG weather app interface

Discussion

Session state bugs stem from Streamlit's rerun behavior. Every user interaction triggers a script rerun, which resets all variables to their initial values. Data persists only if explicitly stored in `st.session_state`. Always check if "messages" not in `st.session_state` before accessing it to avoid `KeyError` exceptions.

Use session state for chatbots when conversation history matters. Don't use it for stateless apps like search interfaces or dashboards when each query is independent—session state adds unnecessary complexity.

The main risk is state leaking between users. Streamlit creates one session per browser tab, but single-tab testing misses multiuser bugs. If user A uploads a document and user B refreshes, does user B see user A's document? Test with two browser windows to catch this early.

For complex workflows, separate RAG logic from UI code: keep the Streamlit script under 100 lines, move API calls and prompt assembly to separate modules, and deploy compute-heavy operations (embeddings, large model calls) as Lambda functions or microservices. Use single-script structure only for demos. Use module separation when multiple developers work on the codebase or when the RAG logic will be reused outside Streamlit.

Debugging Streamlit apps is challenging because print statements appear out of order during reruns. Use the VS Code debugger instead. [Figure 11-9](#) shows the setup: create a configuration with `streamlit` as the module, run as the first argument, and your app filename as the second. Set breakpoints to inspect `st.session_state` at each rerun.



Figure 11-9. Creating a new debugger configuration in VS Code

See Also

- [The Streamlit API reference](#) documents chat UI components including `st.chat_message` and `st.chat_input`.
- [The Streamlit session state documentation](#) explains state management in multi-user applications.

11.3 Adding PDF Analyzer Functionality to Your Chatbot

Problem

You want to enable your chatbot to accept PDF uploads and analyze them by using LLMs and multimodal models.

Solution

This recipe shows how to add upload functionality to your chatbot and then use multimodal models to extract and analyze the document text.

Figure 11-10 shows the high-level process. Because multimodal models work with images, such as JPEGs or PNGs, the first step is to split the PDF into individual page images. Then you iterate over all pages and use a multimodal model to extract text from each page. Finally, you concatenate the extracted page text into a single document text and run an entity/value extraction step over the full text.

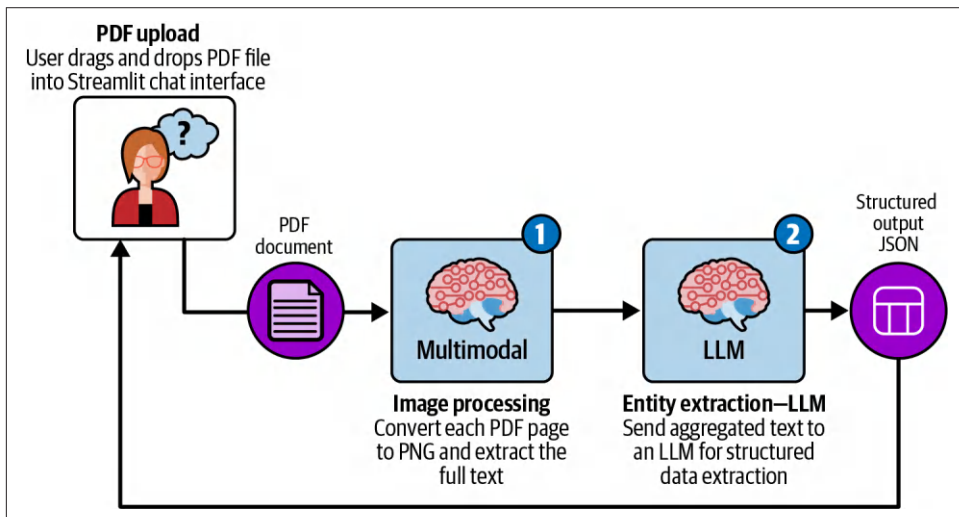


Figure 11-10. Process for the entity extraction web app

The app uses multimodal models from OpenAI, along with pdf2image and Pillow, to preprocess PDFs and convert each page into an image. Install them as follows;

```
pip install streamlit openai pdf2image pillow pydantic
```

The recipe builds a chatbot with drag-and-drop PDF upload. Start by defining a function that converts a PDF into images via the pdf2image library, saves each page as a PNG, and stores the resulting image paths in a list:

```
from pdf2image import convert_from_bytes

def convert_pdf_to_images(pdf_file):
    # Read the PDF file and convert each page to an image
    # (PIL Image objects)
    images = convert_from_bytes(pdf_file.read())

    # Save each image to a temporary PNG file and return the file paths
    image_paths = []
    for idx, image in enumerate(images):
        temp_path = f"images/temp_page_{idx}.png"
        image.save(temp_path, "PNG")
        image_paths.append(temp_path)
    return image_paths
```

Next, you send each image to a multimodal model, which extracts the text. For this OCR/extraction step, you use a simple prompt that turns a multimodal model into an OCR assistant focused on text:

```
from openai import OpenAI
import base64
from io import BytesIO

client = OpenAI()

def perform_ocr_and_extract_entities(image_paths):
    extracted_text = []

    # Prepare the system message for GPT-5.2 vision
    system_message = {
        "role": "system",
        "content": (
            "You are an OCR assistant. Extract all text from the "
            "provided images. Do not summarize or skip any content."
        ),
    }

    for image_path in image_paths:
        with open(image_path, "rb") as image_file:
            # Read image data and base64 encode it
            base64_image = base64.b64encode(
                image_file.read()
```

```

        ).decode("utf-8")

        # Create the image_url dictionary directly from the base64 string
        image_content = {
            "type": "image_url",
            "image_url": {"url": f"data:image/png;base64,{base64_image}"},
        }

        messages = [
            system_message,
            {
                "role": "user",
                "content": [image_content],
            },
        ]

        response = client.chat.completions.create(
            model="gpt-5.2",
            messages=messages,
            max_tokens=4096,
        )

        full_text = response.choices[0].message.content
        extracted_text.append(full_text)

    return extracted_text

```

From the extracted text, you can now run an entity extraction step. Here, you extract person entities with fields like Name, Age, and City from the full document text. You can adapt the prompt to extract any other type of entity or value as needed:

```

from openai import OpenAI
import json

client = OpenAI()

def extract_entities_from_text(extracted_text):
    system_message = (
        "Extract all people entities from the text as a JSON array "
        "of objects with fields: ID (int), Name (str), Age (int), "
        "City (str)."
    )

    text = "\n".join(extracted_text)
    response = client.chat.completions.create(
        model="gpt-5.2",
        messages=[
            {"role": "system", "content": system_message},
            {"role": "user", "content": text},
        ],
        max_tokens=1024,
    )
    extracted_data = response.choices[0].message.content

```

```
    return extracted_data
```

The last step is to combine these functions into a single chatbot application:

```
st.title("Chat with File Upload")

uploaded_pdf = st.file_uploader(
    "Upload a PDF (optional)",
    type=["pdf"],
)

prompt = st.chat_input("Ask a question about the uploaded PDF")

if prompt and prompt.files:
    with st.spinner("Processing PDF..."):
        for uploaded_file in prompt.files:
            if uploaded_file.type == "application/pdf":
                images = convert_pdf_to_images(uploaded_file)
                extracted_text = perform_ocr_and_extract_entities(images)

                st.write("Extracted Text from PDF Pages:")
                for idx, page_text in enumerate(extracted_text, 1):
                    st.markdown(f"***Page {idx}***")
                    st.write(page_text)

                extracted_data = extract_entities_from_text(extracted_text)
                st.write("Extracted Entities:")
                st.write(extracted_data)
```

With uploads enabled, you'll see a paperclip icon in the lower-left corner of the chat UI, indicating that you can attach a file via drag-and-drop. In **Figure 11-11**, you can also see the first result from a sample file. This file contains information about survey participants, including age, city of origin, and first name. The app extracts these as entities and returns a list of dictionaries.

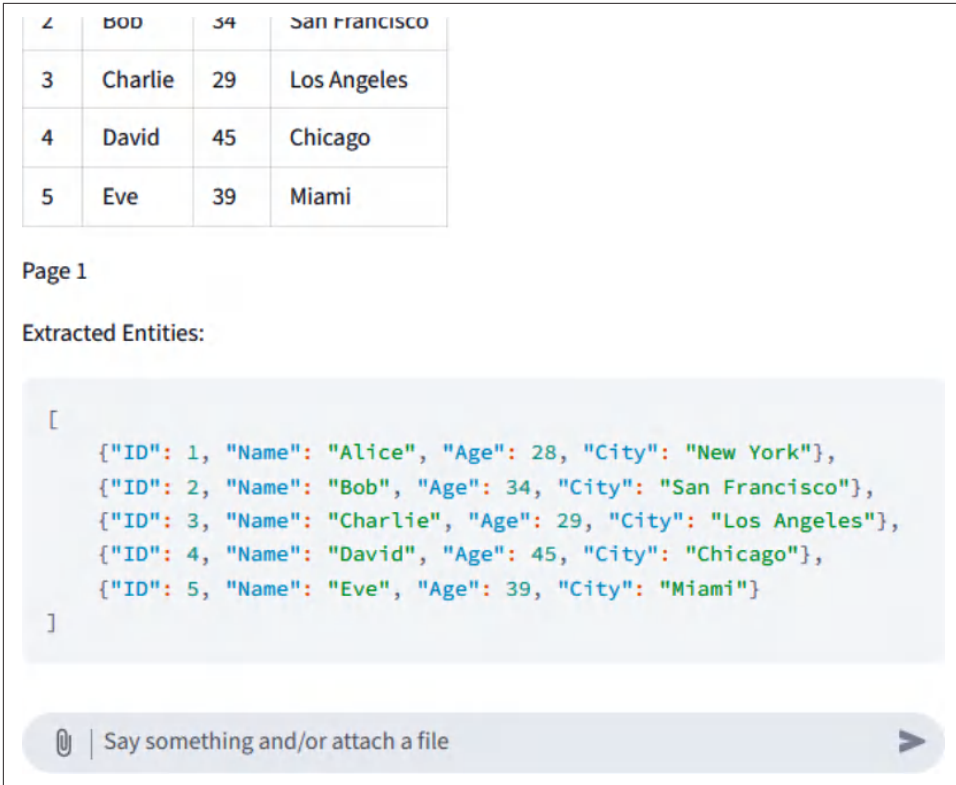


Figure 11-11. The entity extraction web app

Discussion

Multimodal models process images through vision transformers that recognize text, tables, and layout simultaneously. They handle scanned documents and forms better than traditional OCR because they understand spatial relationships like “this number is in the Total column” or “this signature appears below the approval line.”

Converting PDFs to images creates a uniform processing pipeline. Both scanned and digital PDFs become images, so one code path handles everything. This simplifies development but increases cost because you process pixels instead of text layers.

Use this approach when documents mix text, tables, and images (invoices, forms, research papers), when scanned documents have no text layer, or when layout context affects meaning. Don’t use it for plain-text PDFs (10 to 100 times more expensive than PyMuPDF extraction), for real-time workflows requiring subsecond latency, or when processing thousands of pages daily (costs scale linearly with page count).

The critical trade-off is extraction quality versus cost. For a 100-page document, text-layer extraction costs \$0.05 compared to \$0.50–\$2.00 for multimodal OCR.

Digital-native documents (PDFs created from Word, LaTeX, or web browsers) containing only paragraphs and headings don't benefit from multimodal processing.

Multimodal models occasionally hallucinate field values or skip sections when pages are rotated or low-quality. Always validate extracted data against expected schemas by using Pydantic before writing to downstream systems. Production systems need field presence validation, value range checks, and manual review queues for low-confidence extractions.

You can extend the prompt to extract richer structure: table data in JSON format, image descriptions for figures and diagrams, heading hierarchies for document navigation, or form field mappings for structured data entry.

See Also

- [pypdf](#) provides text extraction from digital PDFs when OCR is not needed.
- [LlamaIndex SimpleDirectoryReader](#) offers multiformat document parsing including PDFs.
- [LangChain PDF loaders](#) support various PDF parsing approaches from simple to advanced.

11.4 Connecting Your RAG App to a SQL Database

Problem

You want to connect your RAG chatbot to a SQL database so users can ask questions about the data.

Solution

To let users ask natural language questions about SQL data, the system must generate valid SQL queries. This capability is called text to SQL.

Figure 11-12 shows the process when the RAG system receives a new user request. First, all available database documentation and data models are embedded into a vector store. When a user asks a question, the system performs a semantic similarity search to find relevant parts of the database documentation, and uses it to let an LLM generate a suitable SQL query. The SQL query is then executed against the database. Finally, the retrieved data is combined with a prompt and instructions for the generation model, which uses the data as the basis for answering the user's question.

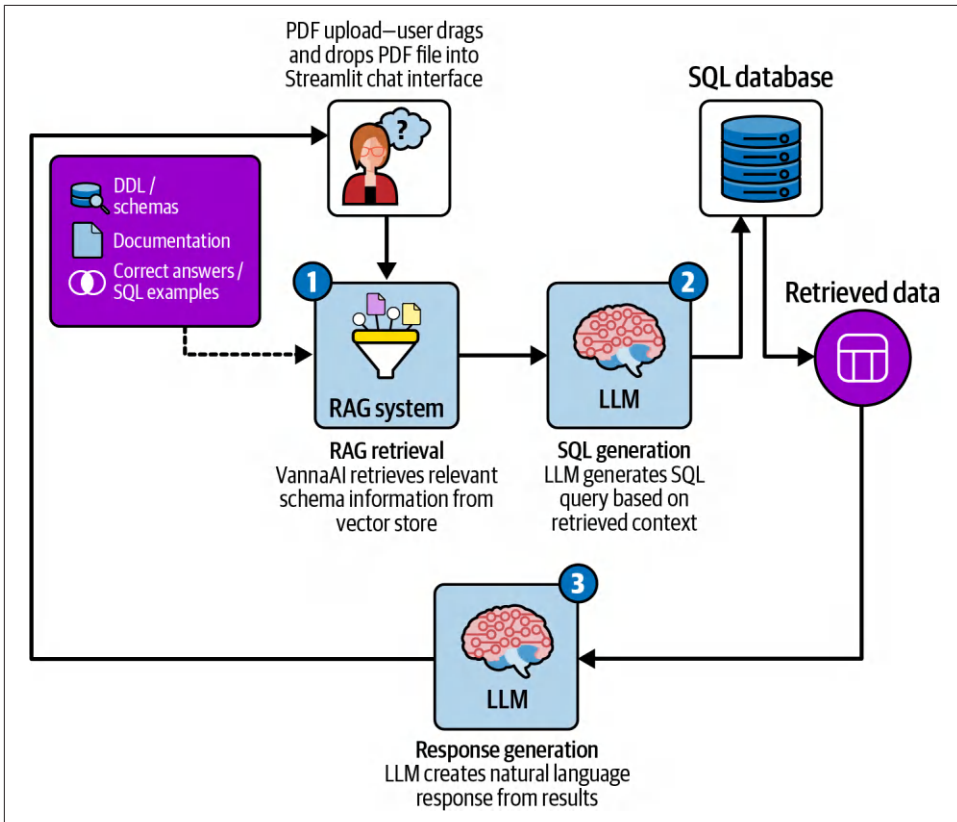


Figure 11-12. The steps Vanna AI takes to answer a user question

This recipe uses OpenAI LLMs for SQL query generation and final answer generation, with the Vanna AI framework handling text-to-SQL functionality and Chroma as the vector store backend. For the sample SQL database, a new SQLite database is generated and filled with sample data. Install these components as follows:

```
pip install streamlit openai vanna chromadb sqlite3
```

The first step is to set up the SQL database and fill it with sample data. SQLite is a lightweight, in-file database well suited for small projects. Because it's just a file, it's cost-efficient and requires little maintenance. However, it doesn't cover more advanced database features such as session and user management.

In the [book repository](#), you'll find a Python script that includes SQL statements to create the SQLite database and populate it with sample data. The sample database represents an online bookstore and includes tables for genres, clients, staff, deliveries, authors, books, and sales items.

The next step is to initialize Vanna AI. Create a new class that combines the required Vanna functionality, which includes the OpenAI chat model and the Chroma vector store:

```
class MyVanna(ChromaDB_VectorStore, OpenAI_Chat):
    def __init__(self, config=None):
        ChromaDB_VectorStore.__init__(self, config=config)
        OpenAI_Chat.__init__(self, config=config)
```

To connect Vanna to the SQLite database, you only need the following line of code:

```
vn.connect_to_sqlite('bookstore.db')
```

The next step “trains” the Vanna engine. This does not train an ML model; it preprocesses documentation and stores it in the vector store.

Table 11-1 shows examples of Vanna functions you can use to process different types of documentation.

Table 11-1. Vanna AI training functions with examples

Documentation type	Description	Example
DDL statements	The schema definitions (tables, columns, keys) that describe the database structure.	<code>vn.train(ddl="CREATE TABLE customers (id INT PRIMARY KEY, name TEXT)")</code>
Documentation strings	Human-written descriptions of tables, columns, business meaning, constraints, and conventions.	<code>vn.train(documentation="Revenue is calculated as sales minus returns.")</code>
SQL statements	Example SQL queries that reflect common access patterns and business logic.	<code>vn.train(sql="SELECT id, name FROM employees WHERE department = 'Sales'")</code>
Question-SQL pairs	Example user questions paired with the correct SQL query. This is the most effective training method.	<code>vn.train(question="Show me all active users", sql="SELECT * FROM users WHERE status = 'active'")</code>

Next, you load DDL statements from the database and use them to train Vanna. The first part retrieves the DDL directly from the database, and the second part iterates over those DDL statements and calls Vanna’s training function for each one:

```
# Initialize Vanna
if "vn" not in st.session_state:
    st.session_state.vn = MyVanna(
        {
            "api_key": os.environ.get("OPENAI_API_KEY"),
            "model": "gpt-5.2",
            "path": "./chroma_db",
        }
    )
    st.session_state.vn.connect_to_sqlite("bookstore.db")
```

```
# Train on database schema
ddl_df = st.session_state.vn.run_sql(
    "SELECT sql FROM sqlite_master WHERE sql IS NOT NULL"
)
for ddl in ddl_df["sql"]:
    st.session_state.vn.train(ddl=ddl)
```

For this straightforward example, the DDL statements alone are sufficient because the bookstore database is self-explanatory. For more complex data models, you need additional documentation such as table relationships, column meanings, and value formats.

Next, test whether Vanna behaves as expected. It asks a sample question, generates the SQL query, and executes it. The question asks the system to find the five best-selling books:

```
user_input = "What are the top 5 best-selling books?"

sql_query = vn.generate_sql(user_input)
result = vn.run_sql(sql_query)
```

In [Figure 11-13](#), you can see the SQL query generated by the LLM and the data returned from the database.

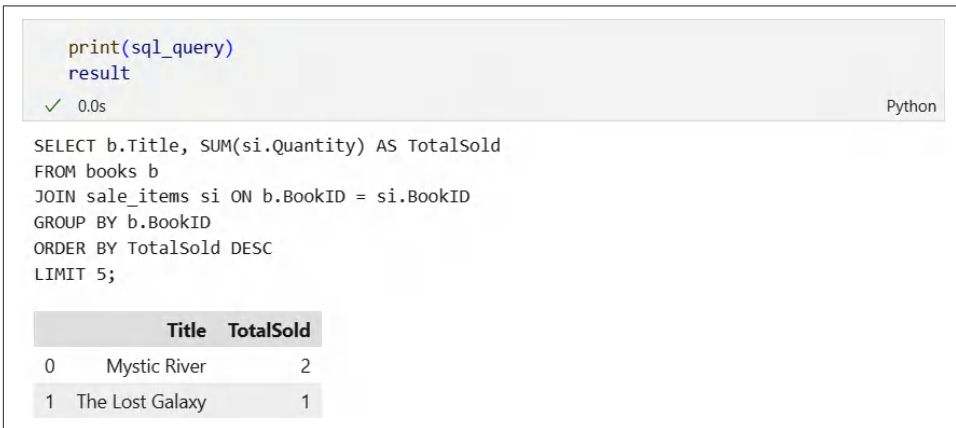


Figure 11-13. SQL query for top five best-selling books

At this point, you have a working retrieval step and data you can feed into the generation step. Next, embed the functions into a Streamlit chatbot. The following code shows the complete implementation of the chatbot app, including the call to Vanna:

```
import streamlit as st
import os
from vanna.openai import OpenAI_Chat
from vanna.chromadb import ChromaDB_VectorStore
```

```

from openai import OpenAI

class MyVanna(ChromaDB_VectorStore, OpenAI_Chat):
    def __init__(self, config=None):
        ChromaDB_VectorStore.__init__(self, config=config)
        OpenAI_Chat.__init__(self, config=config)

# Initialize Vanna
if "vn" not in st.session_state:
    st.session_state.vn = MyVanna({
        "api_key": os.environ.get("OPENAI_API_KEY"),
        "model": "gpt-5.2",
        "path": "./chroma_db"
    })
    st.session_state.vn.connect_to_sqlite("bookstore.db")

# Train on database schema
ddl_df = st.session_state.vn.run_sql(
    "SELECT sql FROM sqlite_master WHERE sql IS NOT NULL"
)
for ddl in ddl_df["sql"]:
    st.session_state.vn.train(ddl=ddl)

st.title("SQL Chat App")

# Chat interface
if prompt := st.chat_input("Ask about the bookstore"):
    # Generate and run SQL
    sql = st.session_state.vn.generate_sql(prompt)
    result = st.session_state.vn.run_sql(sql)

    # Generate answer
    client = OpenAI()
    answer = client.chat.completions.create(
        model="gpt-5-mini",
        messages=[{
            "role": "user",
            "content": (
                f"Answer based on this data: {result}\n"
                f"Question: {prompt}"
            )
        }]
    ).choices[0].message.content

    # Display
    st.chat_message("user").write(prompt)
    st.chat_message("assistant").write(answer)
    with st.expander("SQL Result"):
        st.write(result)

```

Figure 11-14 shows what the chatbot looks like. The SQL query returns only two results in this example because the test database contains only two entries. It's still just a sample database.

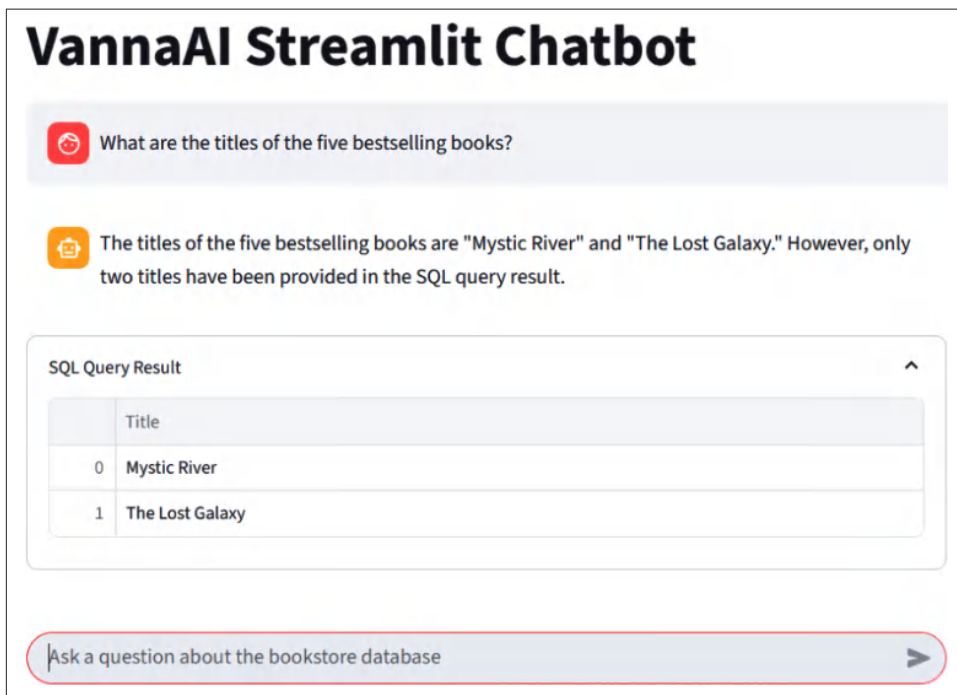


Figure 11-14. Vanna AI Streamlit chatbot interface

Discussion

Text-to-SQL democratizes database access by letting nontechnical users query data through natural language. When the database is well documented with 5–20 tables and clear relationships, users can quickly get answers to common questions like top *N* queries, filters, and aggregations without writing SQL themselves.

The approach works best when users can review generated SQL before execution. Showing the query lets users catch errors and builds trust in the system. This transparency also helps users learn SQL patterns over time.

Text-to-SQL has predictable failure modes. Models guess wrong metrics for ambiguous questions like “Show me trends” because they lack business context. Complex joins across 5+ tables often time out or return incorrect results. Poor documentation leads to hallucinated column names, with the model inventing plausible but nonexistent fields like `customer_lifetime_value` when the actual column is `clv`.

Don't use text-to-SQL when users aren't SQL-literate and ask vague questions (they can't validate generated queries), when queries need business logic not in the database, or when you need deterministic behavior for compliance or audit trails (LLM output varies).

The agentic approach ([Chapter 8](#)) handles these cases better. Define 10–20 SQL query templates as tools with parameters. The model chooses which template to use and fills in parameters. This gives you deterministic queries with clear audit trails while allowing natural language input.

For production systems, start with the agentic approach. Add free-form text-to-SQL only if users explicitly request it and your database documentation is solid. The hybrid model—agentic templates for common queries, text-to-SQL for ad hoc exploration—works well for analytics teams.

See Also

- [The LangChain SQL agent documentation](#) explains how to build agents that translate natural language questions into SQL queries and execute them against relational databases.
- [The LlamaIndex SQL Query Engine](#) enables natural language queries against SQL databases.
- [The Vanna AI GitHub repository](#) contains additional examples and configuration options for text-to-SQL implementations.

11.5 Deploying Your Streamlit App with Docker and AWS

Problem

You need to deploy your app in a robust, reproducible, and scalable way.

Solution

This recipe shows how to deploy your Streamlit app using Docker containers on AWS. Docker is a containerization platform that packages your application with all its dependencies into a portable container image. For more information about Docker, see [Docker's official website](#).

On AWS, you'll use Amazon Elastic Container Registry (ECR) to store your Docker images and Amazon Elastic Container Service (ECS) to run the containers. ECS manages the container lifecycle, including scaling and health monitoring.

To follow this recipe, you need the following:

- Docker Desktop installed on your local machine to build and test images. Docker Desktop is free for personal use, but enterprise users may need a paid license.
- An account with a cloud provider to deploy the container.

This recipe describes AWS briefly, but you can use other providers as well. Major cloud providers (AWS, Google Cloud, and Azure) offer free tiers with limited allowances that are often sufficient for small side projects.

To reproduce the container build and deployment, use a folder structure that includes the following:

- Dockerfile
- *requirements.txt* (Python dependencies)
- The Streamlit app code
- *entrypoint.sh* (starts the app)

The first step is to create the Dockerfile, which specifies all dependencies, with the following content:

```
# Dockerfile (example)
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8501

ENTRYPOINT ["bash", "entrypoint.sh"]
```

The Dockerfile defines the steps which are executed within the Docker container. It sets the working directory to *./app*. When copying files into the image, ensure that you create this working directory (in this case, the *app* folder) and then copy your repository files into it. After the files are copied, the image installs the requirements via pip and exposes port 8501.

The last command executes the *entrypoint.sh* script whenever the container starts. This script starts the Streamlit app and sets the server port so the app can be reached at a known address.

Now you can use the Dockerfile to create the actual Docker image. Open your terminal or command prompt, navigate to the directory where your Dockerfile is located, and execute the following command:

```
docker build -t rag-streamlit-app:latest .
```

This builds your image. Test it locally before deploying:

```
docker run -p 8501:8501 rag-streamlit-app:latest
```

Open <http://localhost:8501> to verify that the app works. If you see the Streamlit interface, the container is ready for deployment.

To deploy to AWS, follow these steps:

1. Install the AWS CLI.
2. Create the ECR repository: `aws ecr create-repository --repository-name rag-streamlit-app`.
3. Push the image (follow the authentication commands from the ECR console).
4. Create the ECS task definition pointing to your ECR image.
5. Deploy the task on ECS Fargate (serverless container runtime).

To deploy to Google Cloud, use Cloud Run. Upload the image to the Google Container Registry and deploy with one command.

To deploy to Azure, use Container Apps. The workflow is similar to that of GCP Cloud Run.



Fargate charges per vCPU-hour and memory. A single container running 24/7 with 1 vCPU and 2 GB RAM costs approximately \$30 to \$40 per month. Enable auto-scaling carefully. Unrestricted scaling during a traffic spike can generate unexpected costs. Set maximum task count limits in your ECS service configuration.

Discussion

Docker packages the application with all its dependencies into a portable container. This makes deployment platform-independent and reproducible. Any developer can build the same image from the Dockerfile, ensuring consistency across development, testing, and production environments.

This recipe covers container creation and manual deployment. Production systems need three additional layers:

Infrastructure as code (IaC)

IaC tools like Terraform or AWS Cloud Development Kit (CDK) define your infrastructure in version-controlled files. Without IaC, debugging production issues means manually clicking through consoles to remember configurations. IaC prevents configuration drift. When you manually update production through

the console, your dev environment slowly diverges. IaC enforces that both environments are built from the same source files.

Continuous integration and continuous deployment (CI/CD) pipeline

Use GitHub Actions or GitLab CI to rebuild and redeploy on every commit to `main`. The pipeline runs tests, builds the Docker image, pushes it to ECR, and updates the ECS task definition. If tests fail, the pipeline blocks deployment. Manual deployment introduces human error. You might forget to run tests, build from the wrong branch, or skip a migration script. CI/CD pipelines make these mistakes impossible.

Secrets management

Don't hardcode API keys in the Docker image. Images are stored in registries where they're visible to anyone with access. Use AWS Secrets Manager or environment variables injected by ECS at runtime. This keeps credentials out of version control and container registries. If an image leaks (through misconfigured registry permissions or compromised CI/CD credentials), attackers gain your API keys. Rotating credentials means rebuilding and redeploying every image. With secrets management, you rotate in one place.

For prototypes, manual deployment is fine. Add these layers when you're confident the app will run long-term, when multiple developers need to deploy, or when compliance requires audit trails.

See Also

- [The AWS ECS container image guide](#) provides step-by-step instructions for creating container images.
- [The Docker Get Started guide](#) covers Docker fundamentals including building and running containers.
- [The Streamlit Docker deployment tutorial](#) demonstrates containerizing Streamlit applications.

Symbols

@function_tool decorator (OpenAI Agents SDK), 238

@tool decorator (LangChain), 253

A

abbreviations, replacing to enhance data quality, 90-94

ACID transactions, 164

Agent class (OpenAI Agents SDK), 239

agent hand-offs, 244

agentic chunkers, 111-115

agentic chunking implementation guide (Murallie), 115

agentic RAG

 asyncio for accelerating agents, 231-236

 choosing agentic frameworks, 221-223

 custom tools in Python, 215

 function calling

 discussion and trade-offs, 230

 iterative tool calls in loop, 227-230

 tool definitions with docstrings, 224-227

 LangGraph

 agent execution and visualization, 256

 discussion and comparison, 257-259

 graph nodes and edges, 253-255

 state and tool definitions, 250-253

 MCP tools, 245-250

 sales negotiation agent

 agent workflow architecture, 236

 multiagent negotiation setup, 239-242

 tool definitions and vector store, 238

 tracing and hand-offs discussion, 243

 workflow patterns, multiagent systems, 216-220

Agents SDK (OpenAI) (see OpenAI Agents SDK)

all-MiniLM-L6-v2 embedding model, 119-120

Amazon Aurora, 57

Amazon Bedrock Agents, 196

Amazon Elastic Container Registry (ECR), 347

Amazon Elastic Container Service (ECS), 347

Amazon Lightsail, 57

Amazon Transcribe, 60

anchor nodes, 262

ANN Benchmarks, 151

Annoy, 148, 173

answer correctness, 322

answer relevancy metric

 cosine similarity scoring, 320-321

 discussion and relevancy interpretation, 322

 noncommittal answer detection, 319

 reverse question generation method, 316-318

answers, grounded, 1, 25

Anthropic

 guide to building effective agents, 221, 224

 Interactive Prompt Engineering Tutorial, 26

 Messages API reference, 37

 prompt engineering guide, 37

 quickstart guide, 37

 tool use documentation, 44, 216

 vision documentation, 67

Anthropic API, 19, 35-37

anthropic library, 36

AP (average precision), 307

API keys, storing with .env files, 10-12

- Arena leaderboard, 29
- AssemblyAI, 60
- asyncio, accelerating agents with, 231-236
- "Attention Is All You Need" (Vaswani et al.), 85
- audio files, loading via speech-to-text models, 59-60
- auto-merging retriever, designing, 2, 197-200
 - (see also sentence window retriever)
- automated testing, creating synthetic data for, 296-300
- automation, process, 4
- average precision (AP), 307
- AWS Cloud Development Kit (CDK), 349
- AWS ECS container image guide, 350
- AWS Secrets Manager, 12, 350
- AWS, deploying Streamlit apps with, 347-350
- Azure AI Agent Service, 196
- Azure Container Apps, 349
- Azure Key Vault, 12
- Azure OpenAI, 30

B

- BaseModel class (Pydantic), 42
- Beautiful Soup 4 library, 19
- benchmark leakage, 41
- BLEU metrics, 308
- BM25, 2, 206
 - (see also hybrid search)
- boundary effects, 203
- breakpoint threshold, 109-110

C

- Cassandra, 151
- CDK (AWS Cloud Development Kit), 349
- centroids (IVF), 171
- character splitting, 99-101
- Chatbot Arena, 296
- chatbots, 3
 - adding PDF analyzer functionality to, 336-339
 - building with Streamlit
 - chat UI and session state, 326-328
 - discussion and debugging, 335
 - entity extraction from queries, 329
 - weather API and LLM integration, 331-333
- ChatOpenAI class (langchain_openai), 253
- Chroma, 2, 14-16
 - (see also FAISS)

- building sales negotiation agent using, 236-244
 - agent workflow architecture, 236
 - tool definitions and vector store, 238
 - tracing and hand-offs discussion, 243
- metadata filtering documentation, 89
- storing embeddings in, 155-158
- chromadb library, 15, 156, 237, 342
- chunk overlap, 100
- chunking
 - character splitting, 99-101
 - recursive text splitters, 101-104
 - semantic-aware chunkers, 108-111
- CI/CD (continuous integration and continuous deployment), 350
- classification (see text classification)
- Claude (Anthropic), API access for coding workflows, 8
- Claude Code, 7-8
- Claude Console, 35
- Claude Cookbook, 221
- Claude Haiku 4.5, 27
- Claude Sonnet 4.5, 27
- clause summaries, 282
- CLIP (OpenAI), 70, 133-136
- code examples, running from book repository, 21-22
- Code Llama, 38
- coding agents, setup, 6-7
- coherence (metric), 290
- ColBERT, 206
- common table expression (CTE), 167
- comparison, pairwise, 293
- complex queries, decomposing into multiple subqueries, 206-210
 - end-to-end process overview, 206
 - Pydantic and prompt implementation, 207
 - when to use and skip, 209
- complex requests, via query routing system, 192-196
- configuration drift, 349
- context precision@k, evaluating retriever step
 - by calculating
 - discussion and score interpretation, 306-308
 - LLM relevance verification code, 303-306
 - precision formula and steps, 301
- context relevance, 306
- context windows, 1

- continuous integration and continuous deployment (CI/CD), 350
- Contrastive Language-Image Pre-training (CLIP), 70
- coroutines, 235
- cosine similarity, 2, 115, 127-130, 182
 - (see also Euclidean distance)
 - in initial retrieval scoring, 206
 - vs. L2 distance, 154
 - vs. other distance metrics, 130
- cross-attention, 206
- cross-encoder models, 206
- cross-modal retrieval, 70, 133
- CSV files, loading tabular data from, 52-56
- CTE (common table expression), 167
- Cursor (code editor), 7-8
- custom tools, designing in Python, 215
- custom vocabularies, 60
- Cypher, 275-277

D

- DAIR.AI Prompt Engineering Guide, 26
- data, 2
 - (see also loading data)
 - synthetic, for automated testing, 296-301
 - tabular, from Excel/CSV, 52-57
- data extraction, 4
- data ingestion, 13
- data preparation
 - agentic chunkers, 111-115
 - character splitting, 99-101
 - data quality enhancement, 90-93
 - document-aware splitting, 105-108
 - hypothetical questions for text chunks, 95-99
 - metadata filtering
 - discussion and semantic overlap, 88
 - extracting existing PDF metadata, 84-86
 - LLM-generated metadata fields, 86
 - recursive text splitters, 101-104
 - semantic-aware chunkers, 108-111
- data preprocessing, 83, 95, 99, 114
- database (see PostgreSQL; SQL database)
- DataGrip, 58
- DBeaver, 58
- DCG@k (discounted cumulative gain), 308
- DDL statements, 343
- DeepEval, 322
- Deepgram, 60

- DeepSeek, 68
- Demonstrate-Search-Predict, 210
- deployment, of Streamlit app with Docker and AWS, 347-350
- diarization, speaker, 60
- dimensionality reduction techniques, visualizing semantic relationships via, 123-127
- discounted cumulative gain (DCG@k), 308
- distance calculation, between embeddings, 127-130
- Django, 19, 323, 325
- Docker, 159-160, 347-350
- Dockerfile, 348
- Docling library, 49
- docstrings, 216
- docTR, 64
- document-aware splitting, 105-108
- documents
 - chunking with document-aware splitting, 105-108
 - feeding text into Neo4j
 - Cypher queries and clause types, 268-269
 - discussion and graph modeling, 270
 - Neo4j setup and clause extraction, 265
 - SLA knowledge graph schema, 263-265
 - splitting via character splitting, 99-101
 - splitting with recursive text splitters, 101-104
- dot product, 130

E

- EasyOCR, 63, 64
- ECR (Amazon Elastic Container Registry), 347
- ECS (Amazon Elastic Container Service), 347
- ECS Fargate, 349
- ef_construction parameter (HNSW), 171
- ef_search parameter (HNSW), 171
- Elasticsearch, 151
- embedded tables, text summaries via multimodal models, 70-72
- embedding classifier (routing approach), 195
- embeddings
 - calculating distance between, 127-130
 - choosing embedding models, 131-132
 - CLIP for images and text, 133-136
 - dimensionality reduction techniques, 123-127
 - for images and text (CLIP), 133-136

- hybrid search approach
 - combining keyword and semantic search, [141](#)
 - discussion and when to use, [144-145](#)
 - reciprocal rank fusion implementation, [143](#)
- mapping text to numerical representation, [118-123](#)
- performing text classification with, [137-141](#)
- storing and searching with FAISS, [151-155](#)
- storing in Chroma, [155-158](#)
- storing in PostgreSQL (pgvector), [159-164](#)
- text classification, [137-141](#)
- entity extraction, [336, 338](#)
- .env files, storing secrets and API keys, [10-12](#)
- environment variables, [11-12](#)
- "Essential GraphRAG" (Bratanić and Hane), [283](#)
- Euclidean distance, [130](#)
- evaluating RAG systems
 - choosing evaluation metrics, [289-292](#)
 - context precision@k
 - discussion and score interpretation, [306-308](#)
 - LLM relevance verification code, [303-306](#)
 - precision formula and steps, [301](#)
- human evaluation, [293-296](#)
- LLM-as-a-judge for faithfulness
 - claim decomposition approach, [308](#)
 - discussion and Ragas source code, [315](#)
 - extracting claims from answers, [311](#)
 - verifying claims against context, [312-314](#)
- response relevancy
 - cosine similarity scoring, [320-321](#)
 - discussion and relevancy interpretation, [322](#)
 - noncommittal answer detection, [319](#)
 - reverse question generation method, [316-318](#)
 - synthetic data for testing, [296-301](#)
- evaluation metrics, [289-292](#)
- evaluator-optimizer (workflow pattern), [220](#)
- Excel files, loading tabular data from, [52-56](#)
- expansions
 - graph expansion, [262](#)
 - query expansion, [188](#)
- extraction, data, [4](#)

F

- F1@k (metric), [307](#)
- Facebook AI Similarity Search (FAISS), [19](#)
- FAISS, storing and searching embeddings with, [2](#)
 - (see also Chroma)
- faiss-cpu library, [152](#)
- faithfulness metric
 - claim decomposition approach, [308-309](#)
 - discussion and Ragas source code, [315](#)
 - extracting claims from answers, [310-311](#)
 - verifying claims against context, [312-314](#)
- Falcon 40B (TII), [39](#)
- false positives, [88](#)
- FastAPI, [323](#)
- filtering, metadata (see metadata filtering)
- fine-tuning, [40, 286](#)
- "5 Levels of Text Splitting" tutorial (Kamradt), [111, 115](#)
- Flask, [19, 323](#)
- foundation models
 - Anthropic API, [35-37](#)
 - Gemini models (Google), [33-34](#)
 - Ollama, open source models, [37-40](#)
 - OpenAI API, [29-32](#)
 - prompt templates, [24-26](#)
 - selecting language models, [26-29](#)
 - structured outputs with Pydantic, [41-43](#)
- framework lock-in, [223](#)
- frameworks and libraries, choosing for RAG, [18-21](#)
- function calling, building agentic system via
 - discussion and trade-offs, [230](#)
 - iterative tool calls in loop, [227-229](#)
 - tool definitions with docstrings, [224-226](#)

G

- GCP (Google Cloud Platform), [60, 64, 349](#)
- Gemini 2.5 Flash, [27, 34](#)
- Gemini 2.5 Pro, [33-35](#)
- Gemini API, [259](#)
- Gemini models, generating content with, [33-34](#)
 - API Quickstart guide, [35](#)
 - JSON mode guide, [44](#)
 - model documentation, [35](#)
 - vision API documentation, [67](#)
- GenAI & LLM System Design repository, [6](#)
- generation metrics, [290](#)

- generation, evaluating faithfulness during with LLM-as-judge, 308-315
- generator, 1
- GeoPy library, 327
- GitHub Actions, 350
- Goodhart's law, 41
- Google AI Studio, 33
- Google Cloud Platform (GCP), 60, 64, 349
- Google Cloud Run, 349
- Google Cloud Speech-to-Text, 60
- Google Cloud Vision OCR guide, 64
- Google Colab, 22
- Google Gemini (see Gemini models)
- GPT OSS (OpenAI), 39
- GPT-4.1, 32
- gpt-4o-transcribe model, 31
- GPT-5, 30
- GPT-5 mini, 27, 30
- GPT-5 nano, 27, 30
- GPT-5.2, 64
- Gradio, 19
- graph databases, 263
- graph expansion, 262
- graph RAG
 - Cypher queries, 275-277
 - extending with structured data, 271-274
 - Neo4j knowledge graph, creating
 - Cypher queries and clause types, 268-269
 - discussion and graph modeling, 270
 - Neo4j setup and clause extraction, 265
 - SLA knowledge graph schema, 263-265
 - optimizing for RAG systems, 281-283
 - semantic search on Neo4j, 278-281
- graph traversal, 261
- graphs, knowledge (see knowledge graphs)
- ground truth data, 2, 289
 - (see also synthetic data)
- grounded answers, 1, 25
- guardrails, 243

H

- hallucination, 2, 25
 - (see also faithfulness)
 - as limitation of foundation models, 1
 - faithfulness metric for detecting, 314
 - in multimodal models, 341
 - in text-to-SQL, 346
- hand-offs, agent, 244

- HashiCorp Vault, 12
- Haystack, 22
 - auto-merging implementation, 200
 - cross-encoder ranking, 206
 - HyDE implementation, 99
 - query decomposition guide, 210
- Hegselmann, Stefan, 57
- hierarchical retrieval, 49
- HNSW (Hierarchical Navigable Small Worlds), 171-173
- HNSWlib, 173
- Hugging Face, 118, 120, 293, 297, 301
- human evaluation of RAG systems, 293-296
- human-in-the-loop, 4
- hybrid search
 - improving search results with
 - combining keyword and semantic search, 141
 - discussion and when to use, 144-145
 - reciprocal rank fusion implementation, 143
 - in Neo4j knowledge graph, 278-281
 - keyword and similarity in PostgreSQL, 173-176
- hypothetical document embeddings (HyDE), 2, 99, 184-187
 - (see also multiquery retrieval)
- hypothetical questions, creating for text chunks, 95-98

I

- IaC (infrastructure as code), 349
- IDE (integrated development environment), 6-8
- images
 - extracting text via multimodal models, 65-67
 - extracting text via Tesseract OCR, 61-64
 - generating embeddings for with CLIP, 133-136
 - generating text descriptions, 68-70
- IndexFlatL2 (FAISS), 152
- indexing techniques, accelerating vector searches with
 - accuracy-speed trade-offs, 172
 - HNSW index and hyperparameters, 171-173
 - IVFFlat index creation and benchmarking, 167-170

- infrastructure as code (IaC), 349
- ingestion phase, 23
- ingestion, data, 13
- inner product, 166
- integrated development environment (IDE), 6-8
- ISO standards, 4
- IVFFlat index, 167-171

J

- JSONB data type (PostgreSQL), 181
- Jupyter Notebook
 - getting started with in VS Code, 8-10
 - VS Code documentation, 10
- JupyterLab, 10

K

- Kamradt, Greg, 111
- keyword search, 141, 173-176
- knowledge graphs
 - creating in Neo4j
 - Cypher queries and clause types, 268-269
 - discussion and graph modeling, 270
 - Neo4j setup and clause extraction, 265
 - SLA knowledge graph schema, 263-265
 - extending with structured data, 271-274
 - optimizing for RAG systems, 281-283
- Kollegger, Andreas, 283

L

- LangChain, 19, 20, 258
 - character text splitter documentation, 101
 - documentation, 22
 - Excel loader, 57
 - HyDE integration, 188
 - multiagent router, 196
 - multimodal RAG tutorial, 70, 77
 - MultiQueryRetriever, 191
 - ParentDocumentRetriever, 203
 - PDF loaders, 341
 - RAG tutorial, 18
 - RankLLM Reranker, 206
 - recursive text splitter documentation, 105
 - SemanticChunker source code, 111
 - SQL agent documentation, 347
 - text splitters, 102
 - Text Splitters API reference, 108

- LangChain Academy, 259
- LangChain Hub, 112
- langchain-experimental package, 109
- langchain-openai package, 109, 253
- langchain-text-splitters library, 102
- LangGraph, 19
 - building agentic system with
 - agent execution and visualization, 256
 - discussion and comparison, 257-259
 - graph nodes and edges, 253-255
 - state and tool definitions, 250-253
- LangSmith, 19, 258, 316
- language models, selecting, 26-29
- large language models (LLMs), 1
- latency, 29, 195
- leakage, benchmark, 41
- libraries, choosing for RAG applications, 18-21
- linguistic meaning, mapping to numerical representation, 118-123
- LJ Speech Dataset, 31
- Llama 2 (Meta AI), 38, 39
- Llama 3 Vision, 64
- LlamaIndex, 19, 20
 - auto-merging retriever, 200
 - BM25 retriever example, 145
 - ColBERT reranking, 206
 - documentation, 22
 - Excel loader, 57
 - HyDE implementation, 188
 - multimodal retrieval guide, 70
 - multimodal retrieval with CLIP, 136
 - multiquery retriever, 191
 - node parsers documentation, 108
 - query transform cookbook, 210
 - query transformations documentation, 99
 - RAG examples repository, 18
 - router query engine, 196
 - semantic chunker examples, 111
 - sentence window retriever, 203
 - SimpleDirectoryReader, 341
 - SQL Query Engine, 347
 - starter example, 18
- LLM-as-a-judge
 - evaluating faithfulness during generation, 308-315
 - evaluating metrics, 291-293
 - revealing retrieval noise, 307
- LLM-generated metadata, 86-87
- LLMs (large language models), 1

- loading data
 - audio files via speech-to-text, 59-60
 - from PostgreSQL database, 57-58
 - parsing PDFs, multimodal content, 73-76
 - PDF files, 50-52
 - tabular data from Excel/CSV, 52-57
 - text descriptions for images, 68-70
 - text extraction from images, 65-67
 - text summaries for embedded tables, 70-73
 - via Tesseract OCR, 61-64
 - videos via multimodal models
 - audio transcription with Whisper, 81
 - discussion and segmentation trade-offs, 81
 - frame extraction and processing, 79-81
 - video partitioning strategy, 77-79
 - Word files, 47-49
- long-term memory (agent), 212

M

- m parameter (HNSW), 171
- majority vote (routing approach), 195
- Maliugina, Dasha, 6
- management, secrets, 350
- Manhattan distance, 130
- MarkdownHeaderTextSplitter class, 106
- Massive Text Embedding Benchmark (MTEB), 131
- matplotlib library, 124
- MCP (Model Context Protocol)
 - documentation, 250
 - tools, enriching agent capabilities with, 245-250
- MCP Servers directory, 250
- MCPServerStdio class (OpenAI Agents SDK), 247
- mean reciprocal rank (MRR), 307
- metadata filtering
 - adding metadata to enable
 - discussion and semantic overlap, 88
 - extracting existing PDF metadata, 84-86
 - LLM-generated metadata fields, 86
 - optimizing query results in PostgreSQL, 180-184
 - vs. routing, 195
 - vs. text classification routing, 140
- Microsoft Azure Speech, 60
- Microsoft Copilot, 296
- Microsoft Copilot Studio, 223

- Milvus, 19, 173
- MIRACL (Multilingual Information Retrieval Across a Continuum of Languages), 131
- Mistral 7B (Mistral AI), 38, 39
- modal conversion, 76
- Model Context Protocol (see MCP)
- models
 - open source, running with Ollama, 37-41
 - tiers, 27
- MongoDB, 151
- MoviePy library, 19
- MRR (mean reciprocal rank), 307
- MTEB (Massive Text Embedding Benchmark), 131
- multiagent systems, workflow patterns in, 216-220
- Multilingual Information Retrieval Across a Continuum of Languages (MIRACL), 131
- multimodal models, 23-24, 336, 340
 - extracting text from images via, 65-67
 - loading videos via speech-to-text
 - audio transcription with Whisper, 81
 - discussion and segmentation trade-offs, 81
 - frame extraction and processing, 79-81
 - video partitioning strategy, 77-79
 - speech-to-text as input pathway, 23
 - text descriptions for images, 68-70
 - text summaries for embedded tables, 70-73
- multiquery retrieval, improving search results with, 2, 188-191
 - (see also hypothetical document embeddings (HyDE))
- Murallie, Thuwarakesh, 115

N

- n8n, 222
- natural language processing (NLP), 19
- NDCG@k (normalized discounted cumulative gain), 308
- nearest neighbor search, 150
- Neo4j, 281
 - creating knowledge graph
 - Cypher queries and clause types, 268-269
 - discussion and graph modeling, 270
 - Neo4j setup and clause extraction, 265
 - SLA knowledge graph schema, 263-265
 - Cypher Manual, 274

- indexes documentation, 283
 - installation guide, 270
 - Python driver documentation, 274
 - query tuning guide, 283
 - semantic search on knowledge graph, 278-281
 - Neo4j Aura, 265
 - Neo4j Desktop, 265, 270
 - neo4j library, 265
 - neo4j-graphrag Python package, 281
 - NLP (natural language processing), 19
 - NLTK library, 19
 - nodes, anchor, 262
 - Nominatim API, 330
 - noncommittal answers, 319
 - normalized discounted cumulative gain (NDCG@k), 308
 - numerical representation, mapping text chunks to, 118-123
 - NumPy library, 19, 127, 318
- ## O
- OCR (optical character recognition), 51, 337
 - Ollama, running open source models locally, 37-41
 - 1Password, 12
 - one-hot encoding, 120
 - online monitoring, 294
 - Open LLM Leaderboard, 40
 - open source models, running locally with Ollama, 37-41
 - Open-Meteo API, 331
 - OpenAI Agents SDK
 - building sales negotiation agent with agent workflow architecture, 236
 - multiagent negotiation setup, 239-242
 - tool definitions and vector store, 238
 - tracing and hand-offs discussion, 243
 - documentation, 224, 245
 - MCP integration guide, 250
 - tool creation guide, 216
 - OpenAI API, 19, 29-33
 - Chat API reference, 33
 - embeddings guide, 33
 - model selection guide, 29
 - PDF parsing guide, 73, 77
 - prompt engineering guide, 95
 - speech-to-text API documentation, 61
 - structured outputs guide, 89
 - vision documentation, 67
 - OpenAI function calling guide, 216, 230
 - openai library, 31, 87, 119, 152, 181
 - OpenAI SDK, 41-44, 227, 297
 - openai-agents package, 237
 - OpenAIEmbeddings class, 109
 - opencv-python library, 19
 - OpenEvals, 316, 322
 - openpyxl library, 19
 - OpenSearch, 151
 - optical character recognition (OCR), 51
 - orchestration frameworks, 20-21
 - orchestrator-workers (workflow pattern), 219
 - outputs, structured, 41-44
 - overlap, chunk, 100
- ## P
- PaddleOCR, 63
 - pairwise comparison, 293
 - pandas library, 19, 124, 297
 - parallel tasks (workflow pattern), 218
 - patterns, workflow, 216-220
 - PCA (principal component analysis), 123-127
 - pdf2image library, 19, 337
 - PDFs
 - analyzing in chatbots, 336-339
 - extracting text via Tesseract OCR, 61-64
 - loading, 50-52
 - parsing with multimodal content, 73-76
 - pgAdmin, 58, 182
 - pgvector extension, 58, 159-164
 - phase, ingestion, 23
 - Phoenix (evaluation framework), 19, 308
 - Pillow library, 19, 134, 337
 - Pinecone, 19, 150
 - hybrid search introduction, 145
 - metadata filtering documentation, 89
 - multimodal RAG example, 73
 - retrieval metrics guide, 308
 - Playwright MCP server, 246
 - Poppler, 62, 74, 237
 - positives, false, 88
 - PostgreSQL
 - accelerating vector searches
 - accuracy-speed trade-offs, 172
 - HNSW index and hyperparameters, 171-173
 - IVFFlat index creation and benchmarking, 167-170

- keyword and similarity search, 173-176
- loading structured data from, 57-58
- metadata filtering for queries, 180-184
- performing similarity search in, 164-167
- storing embeddings with pgvector extension, 159-164
- PQ (product quantization), 173
- precision@k (metric), 206, 307
- process automation, 4
- processing, query, 13
- product quantization (PQ), 173
- prompt chaining (workflow pattern), 217
- prompt templates, defining, 24
- propositions (agentic chunking), 111-115
- Psycopg 2 library, 19
- psycopg2-binary library, 58, 161, 181
- PyCharm, 6
- Pydantic library, 297, 299, 303, 310, 318
- Pydantic model
 - LangGraph agent state, 251
 - metadata schema generation, 86
 - retrieval response templates, 189
 - structured outputs with OpenAI SDK, 41-43
- PyMuPDF, 340
- pypdf library, 19, 341
- PyPDF2 library, 19, 85
 - image extraction guide, 52
 - text extraction guide, 52
- Pytesseract library, 19, 64
- Python
 - asyncio documentation, 236
 - designing custom tools in, 215
 - loading Word files in, 47-50
- python-docx library, 19, 47-50
- python-dotenv library, 11, 12
- PyTorch, 134

Q

- Qdrant, 58, 150
- queries
 - decomposing into multiple subqueries, 206-210
 - Pydantic and prompt implementation, 207
 - when to use and skip, 209
 - designing query routing system, 192-195
 - expanding, 188
 - processing, 13

- results of, optimizing via metadata filtering, 180-184
- Qwen (Alibaba Cloud), 39
- Qwen-VL, 64
- qwen3:4b language model, 38

R

- RAG mini Wikipedia dataset, 297
- RAG web apps
 - building first Streamlit app, 324
 - chatbot apps with Streamlit
 - chat UI and session state, 326-328
 - discussion and debugging, 335
 - entity extraction from queries, 329
 - weather API and LLM integration, 331-333
 - connecting to SQL database
 - Streamlit chatbot interface, 344
 - training functions and SQL generation, 343-344
 - Vanna AI architecture overview, 341
 - deploying with Docker and AWS, 347-350
 - PDF analyzer for chatbot, 336-339
- RAG, getting started with
 - building first RAG app
 - Harry Potter chatbot example, 14-17
 - three essential components, 13
 - when basic pattern works, 17
 - choosing frameworks and libraries, 18-21
 - IDE and coding agent setup, 6-8
 - identifying high-value use cases, 3-6
 - Jupyter notebooks in VS Code, 8-10
 - running code samples, 21-22
 - storing secrets with .env files, 10-12
- Ragas, 2, 19, 315
 - (see also faithfulness)
 - context precision implementation, 308
 - documentation, 293
 - faithfulness metric, 316
 - response relevancy implementation, 322
 - test set generation guide, 301
- random forest classifier, 139
- rank BM25, 19, 141-143, 206
- ReAct pattern, 213
- "ReAct: Synergizing Reasoning and Acting in Language Models" (Yao et al.), 213
- Real Python, 236
- recall (metric), 191
- recall@k (metric), 307

- reciprocal rank fusion (RRF), 143
- recursive text splitters, 101-104
- RecursiveCharacterTextSplitter class, 102
- Redis, 151
- regression testing, 292
- Requests library, 46, 327
- reranking methods, improving retrieval relevancy with, 203-206
- response relevancy, evaluating
 - cosine similarity scoring, 320-321
 - discussion and relevancy interpretation, 322
 - noncommittal answer detection, 319
 - reverse question generation method, 316-318
- responses.parse method, 42
- retrieval
 - auto-merging retriever, 197-200
 - HyDE, 184-187
 - metadata filtering in PostgreSQL, 180-184
 - multiquery retrieval, 188-191
 - overview, 177-179
 - query decomposition into subqueries, 206-210
 - Pydantic and prompt implementation, 207
 - when to use and skip, 209
 - query routing system, 192-195
 - reranking methods, 203-206
 - sentence window retriever, 200-203
- retrieval accuracy
 - combining keyword and similarity search, 173-176
 - enhancing with HyDE, 184-187
 - evaluating by calculating context precision@k
 - discussion and score interpretation, 306-308
 - LLM relevance verification code, 303-306
 - precision formula and steps, 302
- retrieval metrics, 290
- retrieval-augmented generation (see RAG)
- retriever, 1
- reverse question generation, 322
- ROUGE metrics, 308
- routing (workflow pattern), 217
- row-to-text conversion, 56
- RRF (reciprocal rank fusion), 143
- rule-based systems, 5

- Runner class (OpenAI Agents SDK), 239

S

- safety metrics, 290
- sales negotiation agent, with OpenAI Agents SDK/Chroma
 - agent workflow architecture, 236
 - multiagent negotiation setup, 239-242
 - tool definitions and vector store, 238
 - tracing and hand-offs discussion, 243
- scikit-learn library, 19, 124
- scikit-llm library, 141
- scipy.spatial.distance module, 130
- script rerun model (Streamlit), 325
- search results
 - improving with hybrid search approach
 - combining keyword and semantic search, 141
 - discussion and when to use, 144-145
 - reciprocal rank fusion implementation, 143
 - improving with multiquery retrieval, 188-191
 - via hypothetical questions, 95-99
- secrets
 - managing, 350
 - storing with .env files, 10-12
- semantic alignment, 98-99
- semantic relationships, visualizing between text chunks, 123-127
- semantic search, enabling on Neo4j knowledge graph, 278-281
- semantic-aware chunkers, 108-111
- SemanticChunker class, 109
- Sentence Transformers library, 19, 119
- sentence window retriever, 2, 200-203
 - (see also auto-merging retriever)
- session state (Streamlit), 328
- similarity search
 - combining with keyword search, 173-176
 - performing in PostgreSQL, 164-167
- similarity, cosine, 115, 127-130, 182
- Singh, Manas, 291
- speaker diarization, 60
- speech-to-text models
 - as multimodal input pathway, 23
 - loading audio files via, 59-60
 - loading videos via
 - audio transcription with Whisper, 81

- discussion and segmentation trade-offs, 81
- frame extraction and processing, 79-81
- video partitioning strategy, 77-79
- transcribing audio with gpt-4o-transcribe, 31
- Spyder, 6
- SQL database, 287
- SQL database, connecting RAG app to
 - Streamlit chatbot interface, 346
 - training functions and SQL generation, 343-344
- Vanna AI architecture overview, 342
- SQLAlchemy library, 19, 59
- SQLite, 19, 342
- Starmer, Joshua, 123
- state leaking, 335
- StateGraph class (LangGraph), 254
- Streamlit, 19, 294
 - API reference for chat components, 336
 - building chatbot apps with
 - chat UI and session state, 326-328
 - discussion and debugging, 335
 - entity extraction from queries, 329
 - weather API and LLM integration, 331-333
 - building first app, 324
 - deploying with Docker and AWS, 347-350
 - Docker deployment tutorial, 350
 - documentation, 326
 - GitHub repository, 326
 - session state documentation, 336
- structural retrieval, 277
- structured data
 - extending knowledge graphs with, 271-274
 - loading from PostgreSQL database, 57-58
- structured outputs, 41-44, 87
- subqueries, decomposing complex queries into, 206-210
 - end-to-end process overview, 206
 - Pydantic and prompt implementation, 207
 - when to use and skip, 209
- Swarm (OpenAI), 244
- synthetic data, creating for automated testing, 296-300

T

t-SNE (t-Distributed Stochastic Neighbor Embedding), 126

- tabular data, loading from Excel and CSV, 52-56
- technical terms, replacing to enhance data quality, 90-94
- templates, prompt, 24
- Terraform, 349
- Tesseract OCR, extracting from images and PDFs, 61-64
 - documentation, 64
- testing, creating synthetic data for automated, 296-300
- text
 - classification of, with embeddings, 137-141
 - descriptions for images, 68-70
 - generating embeddings for with CLIP, 133-136
 - splitting with agentic chunkers, 111-115
 - splitting with semantic-aware chunkers, 108-111
 - summaries, for embedded tables, 70-73
- text chunks
 - creating hypothetical questions for, 95-99
 - mapping to numerical representation, 118-123
 - via sentence window retriever, 200-203
 - visualizing semantic relationships, 123-127
- text-embedding-005 embedding model, 118
- text-embedding-3-small embedding model, 16, 118, 278
- text-embedding-ada-002 embedding model, 132
- text-to-SQL, 55, 287, 291, 341, 346
- threshold, breakpoint, 109-110
- tiers, model, 27
- time-to-first-token, 34
- tokenization, 156
- tokens, 27
- Transformer architecture, as foundation for modern AI, 1
- Transformers library (Hugging Face), 19, 134
- traversal, graph, 261
- TrOCR, 64
- truth data, ground, 289
- tsvector data type (PostgreSQL), 174
- ts_rank function (PostgreSQL), 175

U

UMAP (Uniform Manifold Approximation and Projection), 126

- unstructured data, 45
- Unstructured library, 19, 47-50, 52
- use cases, high-value, identifying, 3-5

V

- Vanna AI, 342-347
- variables, environment, 11-12
- vector arithmetic, with embeddings, 121
- vector databases and similarity searches
 - accelerating with indexing techniques
 - accuracy-speed trade-offs, 172
 - HNSW index and hyperparameters, 171-173
 - IVFFlat index creation and benchmarking, 167-170
 - choosing vector databases, 148-151
 - Chroma vector database, 155-158
 - FAISS, 151-155
 - keyword and similarity search combined, 173-176
 - pgvector extension for PostgreSQL, 159-164
 - similarity search in PostgreSQL, 164-167
- vector index (Neo4j), 279
- vector store, 1
- vector_cosine_ops (pgvector), 170-171
- videos, via speech-to-text and multimodal
 - audio transcription with Whisper, 81
 - discussion and segmentation trade-offs, 81

- frame extraction and processing, 79-81
- video partitioning strategy, 77-79

- Vim/Neovim, 6
- virtual random access memory (VRAM), 40
- vision model, 23
- vision transformers, 340
- vocabulary mismatch, 187
- voyage-large-2-instruct embedding model, 118
- VRAM (virtual random access memory), 40
- VS Code, Jupyter notebooks in, 8-10

W

- Weaviate, 19, 149
- Whisper (OpenAI), 59-60, 253
- windows, context, 1
- Word files, loading in Python, 47-50
- Word2Vec, 123
- workflow patterns, in multiagent systems, 2, 216-220
 - (see also function calling)

X

- xAI, 68

Z

- Zapier, 222

About the Author

Dominik Polzer is AI lead for procurement at Siemens Energy, where he leads transformative machine learning and agentic AI projects that will change how procurement departments work tomorrow.

With nearly a decade of experience deploying machine learning solutions in production, Dominik has built AI applications across diverse business functions—from manufacturing and finance to procurement. His expertise spans the full lifecycle of industrial AI systems, with a particular focus on retrieval-augmented generation (RAG) and foundation models in recent years.

Through his popular [Medium blog](#), Dominik has reached millions of readers by explaining RAG architectures, vector databases, and AI agent frameworks. You can connect with him on [LinkedIn](#).

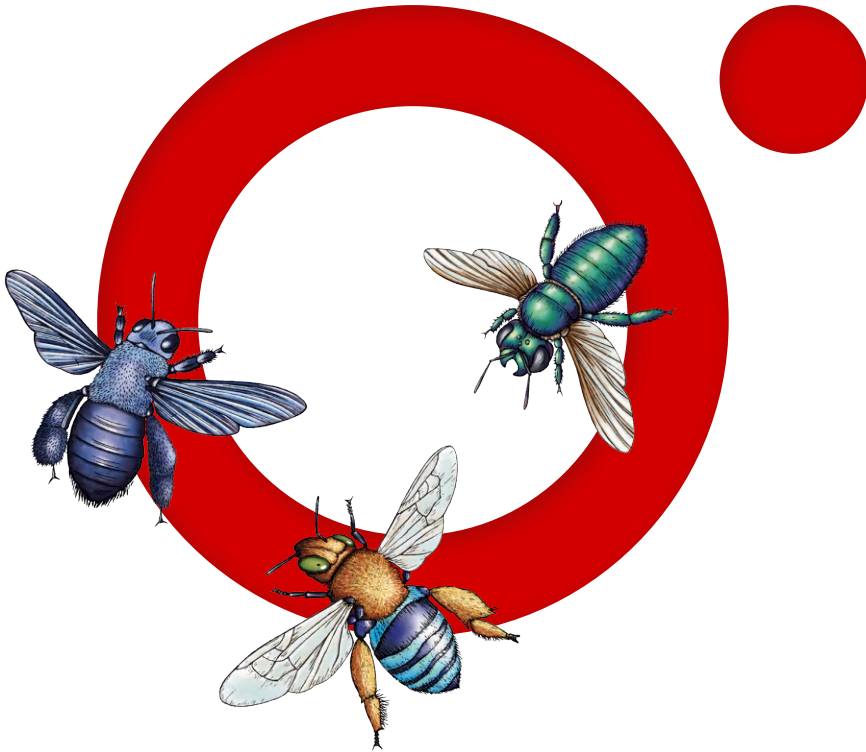
Colophon

The animal on the cover of *RAG with Python Cookbook* is a striped keelback (*Xenochrophis vittatus*), a medium-sized snake native to western Indonesia. Females reach about 70 cm in length, while males reach about 50 cm. They are yellow to brown with black stripes running along the top and sides of the body.

The striped keelback is diurnal and primarily lives around moist lowlands or aquatic areas such as ponds and wetlands. Its diet consists of fish, amphibians, and lizards. It is mildly venomous but is considered harmless to humans.

It is considered a species of least concern and easily adapts to agricultural areas like rice paddies and fields. The striped keelback has also reportedly been introduced to Singapore and Puerto Rico. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by José Marzan Jr., based on an antique line engraving from Lydekker's *Royal Natural History*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.



O'REILLY®

**Learn from experts.
Become one yourself.**

60,000+ titles | Live events with experts | Role-based courses
Interactive learning | Certification preparation | Verifiable skills

Try the O'Reilly learning platform free for 10 days.

